O'REILLY®

2nd Edition



Compliments of CONFLUENT

Early Release

RAW & UNEDITED

Building Event-Driven Microservices

Leveraging Organizational Data at Scale

Adam Bellemare

Confluence

CONFLUENT

Kafka cannot

autoscale

guarantee resilience

minimize operational overhead

secure your data

save you money

Learn More



Building Event-Driven Microservices

Leveraging Organizational Data at Scale

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

Adam Bellemare



Building Event-Driven Microservices

by Adam Bellemare

Copyright © 2025 Adam Bellemare. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (http://oreilly.com). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Melissa Duffield

Development Editor: Corbin Collins

Production Editor: Clare Laylock

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

August 2020: First Edition

September 2025: Second Edition

Revision History for the Early Release

- 2025-02-19: First Release
- 2025-03-26: Second Release
- 2025-05-30: Third Release

See http://oreilly.com/catalog/errata.csp?isbn=9781492057895 for release details.

The O’Reilly logo is a registered trademark of O’Reilly Media, Inc. *Building Event-Driven Microservices*, the cover image, and related trade dress are trademarks of O’Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher’s views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

This work is part of a collaboration between O’Reilly and Confluence. See our statement of editorial independence.

979-8-341-62214-2

[LSI]

Brief Table of Contents (*Not Yet Final*)

Preface (unavailable)

Chapter 1: Why Event-Driven Microservices? (unavailable)

Chapter 2: Fundamentals of Events and Event Streams (available)

Chapter 3: Fundamentals of Event-Driven Microservices (available)

Chapter 4: Schemas and Data Contracts (available)

Chapter 5: Designing Events (available)

Chapter 6: Integrating Event-Driven Architectures with Existing Systems (available)

Chapter 7: Denormalization and Eventification (available)

Chapter 8: Stateful Event-Driven Microservices (available)

Chapter 9: Deterministic Stream Processing (available)

Chapter 10: Building Workflows with Microservices (unavailable)

Chapter 11: Basic Producer and Consumer Microservices (available)

Chapter 12: Heavyweight Framework Microservices (available)

Chapter 13: Lightweight Framework Microservices (available)

Chapter 14: Streaming SQL (available)

Chapter 15: Microservices Using Function-as-a-Service (unavailable)

Chapter 16: Eventual Consistency and Convergence (available)

Chapter 17: Integrating Event-Driven and Request-Response Microservices (unavailable)

Chapter 18: Handling Bad Data in Event Streams (unavailable)

Chapter 19: Supportive Tooling (unavailable)

Chapter 20: Testing Event-Driven Microservices (unavailable)

Chapter 21: Deploying Event-Driven Microservices (unavailable)

Chapter 22: Conclusion (unavailable)

Part I. Introduction to Event-Driven Microservices

Chapter 1. Fundamentals of Events and Event Streams

> **A NOTE FOR EARLY RELEASE READERS**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 2nd chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

Event streams served by an *event broker* tends to be the dominant mode for powerful event-driven architectures, though you'll find that queues and ephemeral messaging also have a place. The second half of this chapter will cover each of these modes in more detail.

For now, let's now take a closer look at events, records, and messages, as well as the relationship between an event stream and an event broker.

What's an Event?

An event can be *anything* that has happened within the scope of the business communication structure. Receiving an invoice, booking a meeting room, requesting a cup of coffee (yes, you can hook up a coffee machine to an event stream), hiring a new employee, and successfully completing arbitrary code are all examples of events that happen within a business. It is important to recognize that events can be anything that is important to the business. Once

these events start being captured, event-driven systems can be created to harness and use them across the organization.

An event is a *recording* of what happened, much like how an application's information and error logs record what takes place in the application. Unlike these logs, however, events are *also* the single source of truth, as covered in [[Link to Come]]. As such, they must contain all the information required to accurately describe what happened.

Events are immutable. You can't modify an event once it is published to the event stream. Immutability is an essential property for ensuring that consumers can access to exactly the same data. Mutating data that has already been read by several consumers is of no benefit, since there is no way to easily notify them that they must make a change to data that they're already read. You can, however, create and publish a new events containing the necessary corrections or updates.

Events use schemas, which is covered in more detail in Chapter 3. For now, consider events to have well-defined field names, types, and default values.

I avoid using the term *message* when discussing event streams and event-driven architectures. It's an overloaded term that has different meanings for different people, and is also heavily influenced by the technology you're using. Think about what messages mean in our day-to-day life, particularly if you use instant messaging. One person sends a message *to* a specific person or *to* a specific private group. Additionally, the messages may or may not be durable - some chat applications delete the messages after read, others after a period of time.

Events in an event-driven architecture are more akin to a post that you publish to a social media platform or message board. The posting is public to all who have access, and everyone is free to read it and use it as they see fit. More than one person can read the post, and of course the post can be read again and again. It isn't deleted just because it's old. Event streams are effectively a *broadcast* to share important data, letting others subscribe to it and use it however they see fit.

Just to be clear, you can use event streams to send messages. *All messages are events, but not all events are messages.* But for clarity's sake, I'll use the terms *event* and *record* instead of *message* for the remaining chapters of this book. But before we dig further into events, let's take a brief look at the event stream.

What's an Event Stream?

An event stream is a durable and append-only immutable log. Records are added to the end of the log (the tail) as they are published by the producer. Consumers begin at the start of the log (the head) and consume records at their own processing rate.

In its most basic form, an event stream is a time-stamped sequence of business facts pertaining to a domain. Events form the basis for communicating important business data between domains reliably and repeatably.

Event streams have several critical properties that enable us to rely on them for event-driven microservices, and as the basis for effective inter-domain data communication as a whole. For clarity, and with a bit of repetition, these properties include:

Immutability

> Events cannot be modified once written to the log. The contents cannot be altered, nor can their offset position, timestamp, or any other associated metadata. You may only add new events.

Partitioned

> Partitions provide the means for supporting massive datasets. A consumer can subscribe to one or more paritions from a single event stream, allowing multiple instances of a single microservice to consume and process the stream in parallel.

Indexed

Events are assigned an immutable index when written to the log. The index, also often called an *offset*, uniquely identifies the event.

Ordered

Records in an event stream partition are served to clients in the exact same order that they were originally published.

Durability and replayability

Events are durable. They can be consumed either immediately or in the future. Events can be replayed by new and existing consumers alike, provided the event broker has sufficient storage to host the historical data. Events are not deleted once they are read, nor are they simply discarded in the case of an absence of consumers.

Indefinite storage support

You can retain all events in your stream for as long as necessary. There is no forced expiry or time-limited retention, allowing you to consume and reconsume events as often as you need.

Figure 1-1 shows an event stream with three partitions. New events have just been appended to partition 0 (offset 5) and partition 1 (offset 7). The microservice consuming these events has two instances. Instance 0 is consuming only partition 0, which instance 1 is consuming both partition 1 and partition 2.

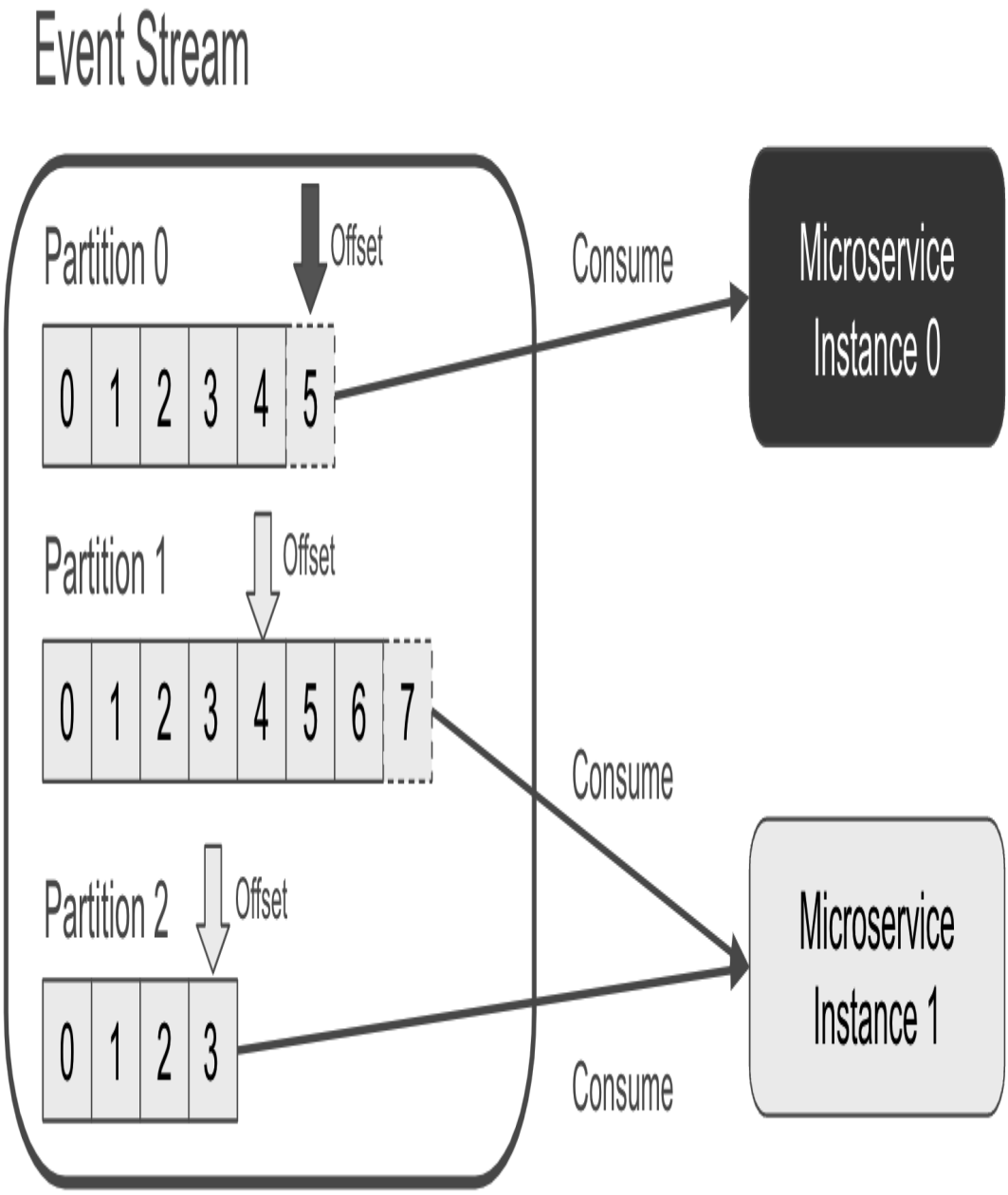


Figure 1-1. An event stream with two microservice instances consuming from a three partitions

With sufficient processing power, the consuming service will remain up to date with the event stream. Meanwhile, a new consumer beginning at an

earlier offset (or the head) will need to process all events to catch up to current time.

Event streams are hosted on an *event broker*, with one of the most popular (and de-facto standard) being Apache Kafka. The event broker, such as in the case of Kafka, provides a structure known as a *topic* that we can write our events to. It also handles everything from data replication and rebalancing to client connections and access controls. Publishers write events to the event stream hosted in the broker, while consumers subscribe to event streams and receive the events.

Unfortunately, due to a long and often messy history, event brokers have often been confused with *ephemeral messaging* and *queues*. Each of these three options is different from the others. Let's take a deeper look at each, and why event streams form the backbone of a modern event driven architectures.

Ephemeral Messaging

A *channel* is an ephemeral substrate for communicating a *message* between one producer and one or more subscribers. Messages are directed to specific consumers, and they are not stored for any significant length of time, nor are they written to durable storage by the broker. In the case of a system failure or a lack of subscribers on the channel, the messages are simply discarded, providing at-most-once delivery. NATS.io Core (not JetStream) is an example of this form of implementation.

Figure 1-2 shows a single producer sending messages to the ephemeral channel within the event broker. The ephemeral messages are then passed on to the currently subscribed consumers. In this figure, `Consumer 0` obtains messages `7` and `8`, but `Consumer 1` does not because it is newly subscribed and has no access to historical data. Instead, `Consumer 1` will receive only message 9 and any subsequent messages.

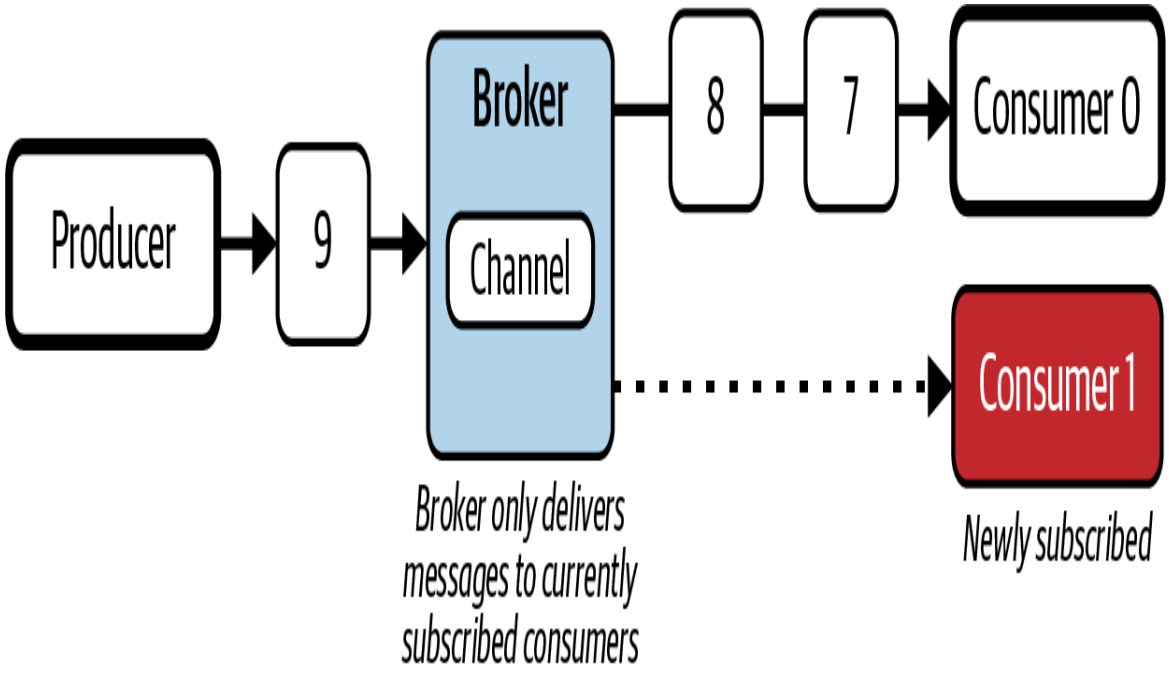


Figure 1-2. An ephemeral message-passing broker forwarding messages

Ephemeral communication lends itself well to direct service-to-service communication with low overhead. It is a message-passing architecture, and is not to be confused with a durable publish-subscribe architecture as provided by event streams.

Message-passing architectures provide point-to-point communication between systems that don't necessarily need at-least-once delivery and can tolerate some data loss. As an example, the online dating application Tinder uses NATS to notify users of updates. If the message is not received, not a big deal—a missed push notification to the user only has a minor (though negative) effect on the user experience.

Ephemeral message-passing brokers lack the necessary indefinite retention, durability, and replayability of events that we need to build event-driven data products. Message-passing architectures are useful for event-driven communication between systems for current operational purposes but are completely unsuited for providing the means to communicate data products.

Queuing

A *queue* is a durable sequence of stored records awaiting processing. It is fairly common to have multiple consumers that asynchronously (and competitively) select, process, and acknowledge records on a first-come, first-served basis. This is one of the major differences when compared to event streams, which use partition-exclusive subscriptions and strict in-order processing.

Two common and free to use open-source queue brokers include RabbitMQ and Apache ActiveMQ. Both of these provide robust queueing semantics and support common queueing protocols, including both Advanced Message Queuing Protocol (AMQP) and Message Queuing Telemetry Transport (MQTT). These protocol standards enable interoperability with other frameworks, clients, and services.

One common use of a queue is as a *work queue*. The producer publishes records representing work to do. The consumers cooperatively dequeue the records, process them, and then signal the queue broker that the work is complete on a per record basis. The broker then typically deletes the processed records, which is the second major difference when compared to the event stream, as the latter retains the records as long as specified (including indefinitely).

Figure 1-3 shows two subscribers consuming records from a queue in a round-robin manner. Note that the queue contains records currently being processed (dashed lines) and those yet to be processed (solid lines).

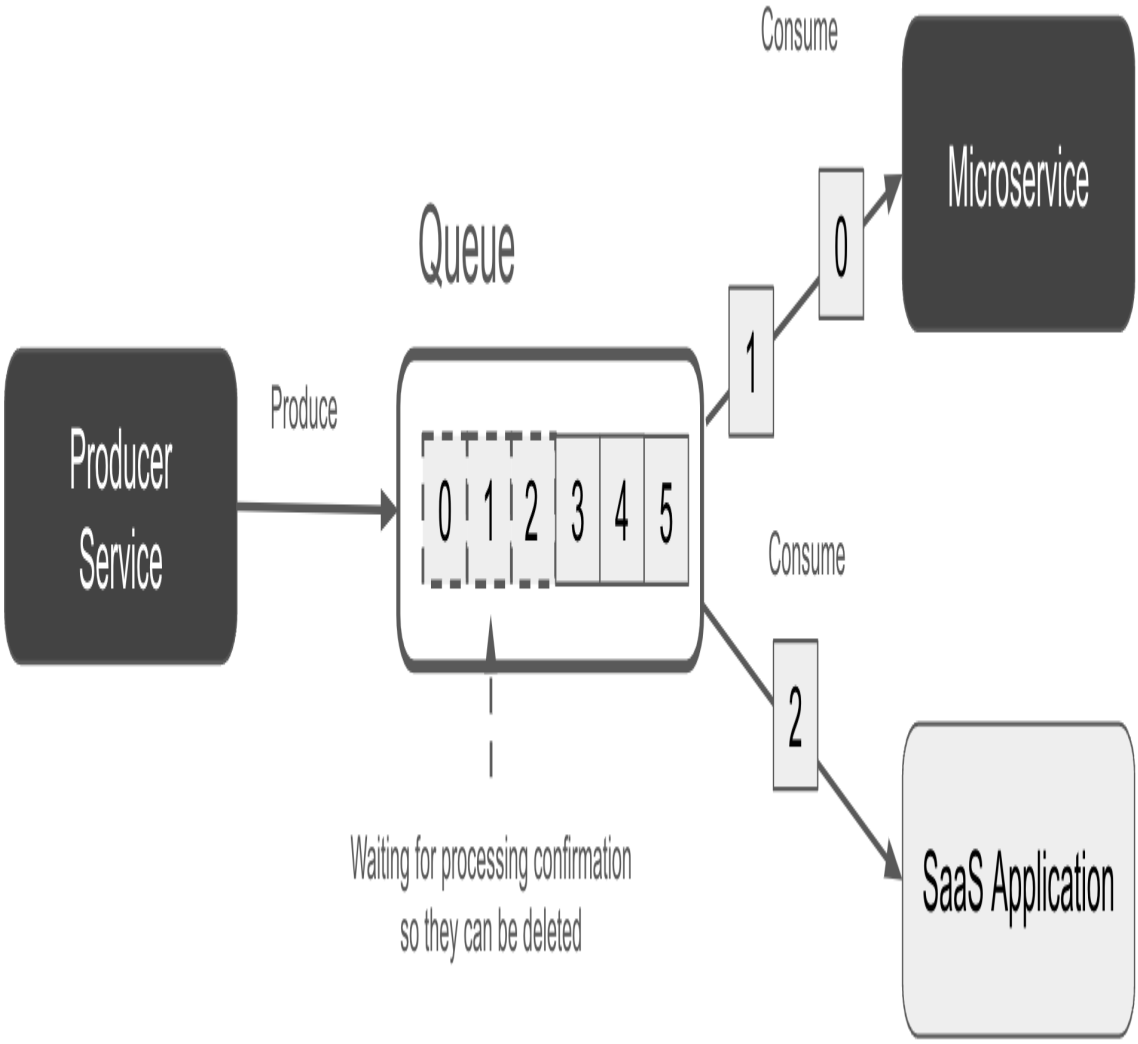


Figure 1-3. A queue with two subscribers each processing a subset of events

Queues typically provide *at-least once* processing guarantees. Records may be proccessed more than once, particularly if a subscriber fails to commit its progress back to the broker, say due to a crash, after processing a record. In this case, another subscriber may pick up the record and process it again.

Queue brokers can also provide *priority-based record ordering* for their consumers, so that high priority records get pushed to the front of lower priority records. Queues provide an ideal data structure for priority ordering, since they do not enforce a strict first-in, first-out ordering like an event stream.

If multiple independent consumers (e.g. microservice applications) each need access to the records in the queue, then you have to make a decision. One option is to use a per-consumer queue, ensuring that each consumer obtains their own copy of the record through their own queue. A second option is to use a durable event stream, where each consumer gets a copy of each record all sourced from a single logical queue. Let's take a look at each of these options in turn.

Queues via Modern Queue Brokers

Modern queue brokers provide *publish-subscribe* mechanism that allows for loose coupling between a producer and its interested consumers. Instead of publishing directly to each and every queue that may need the record, the producer sends its message to an *exchange* hosted on the broker. The exchange then routes *a copy* of the record to each subscribing queue that is registered at the time the exchange receives the record. Figure 1-4 shows an example of this behavior.

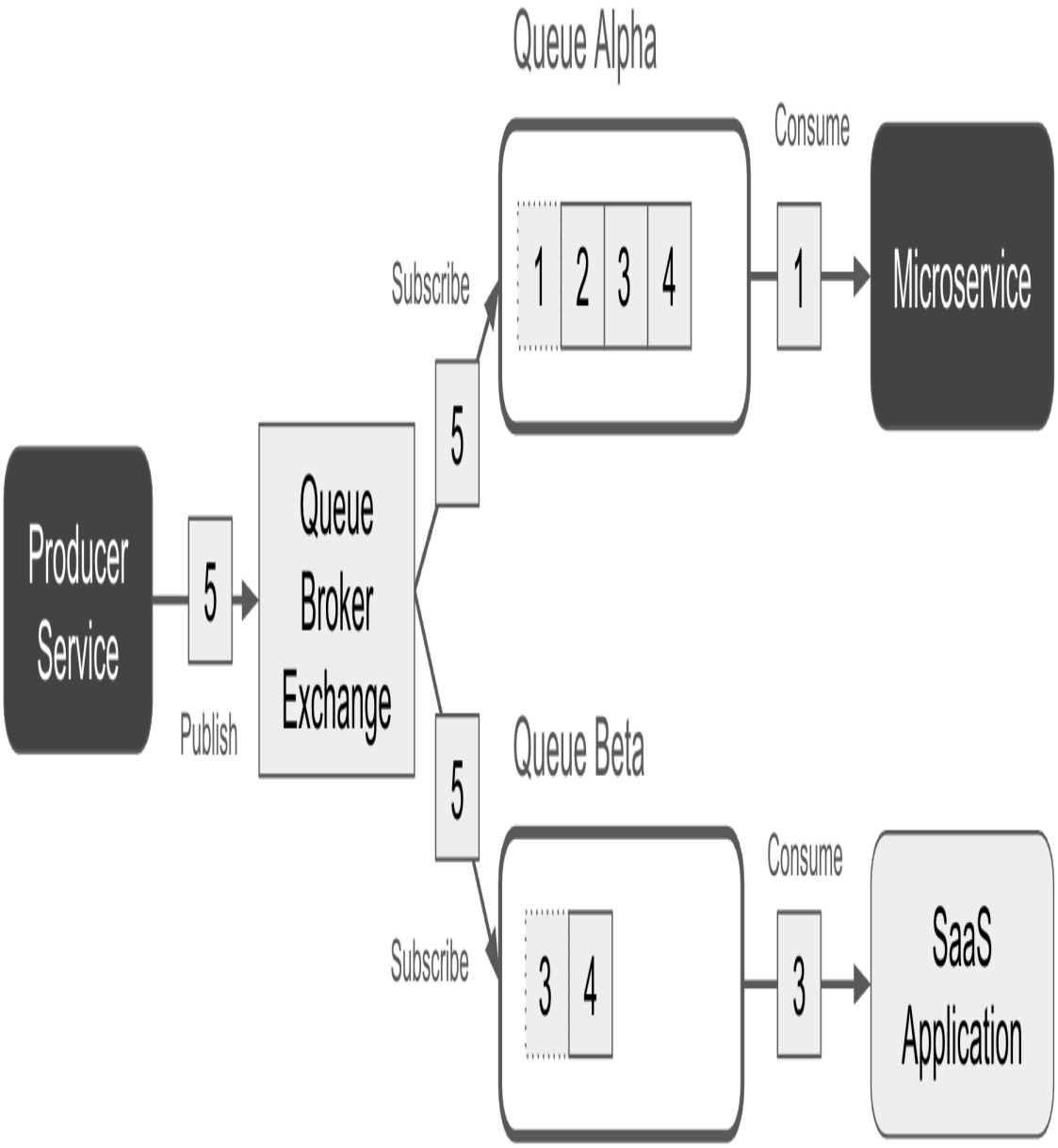


Figure 1-4. An exchange distributing records to each subscribed queue

The producer service publishes its records to the exchange, which in turn fans provides a copy to each of the Alpha and Beta queues.

Historically, queue brokers have limited the time-to-live (TTL) for storing records in the queue. Records not processed within a certain time frame are marked as dead, purged, and no longer delivered to the subscribers. Similar to ephemeral communications, time-based retention and non-replayable data

has influenced the false notion that brokers (queue and event alike) cannot be used to retain data indefinitely.

With that being said, it is important to note that low TTLs are no longer the norm, at least for the more modern queue technologies. For example, both RabbitMQ and ActiveMQ let you set unlimited TTL for your records, letting you keep them in the queue for as long as is necessary for your business.

> **NOTE**
>
> Modern queue brokers may also support replayability and infinite retention of records via durable append-only logs - effectively the same as an event stream. For example, both Solace and RabbitMQ Streams allow for individual consumers to replay queued records as though they were event streams.

Queues provide excellent input buffer capabilities for parallelizable work, particularly where order of the events is not important. You can rely on a queue to buffer records that need processing by another system, allowing the producer application to get on with its other tasks. The queue will durably store all the records until your consumer service can get to working on them. Additionally, your consumer can scale up its processing by registering new consumers on the queue, and sharing processing in a round-robin manner.

Queues via Apache Kafka

One of the big changes since the first edition of this book is the introduction of queues for Kafka (KIP-932) as an early access release for Kafka 4.0. Historically, Kafka has not provided any queue functionality, so if you wanted to implement a queue (and not an event stream) you were forced to go use something else.

But what's wrong with using an event stream as a queue? Here are a few of the original problems that the KIP-932 contributors resolved:

One Consumer Per-partition limitation

When consuming a stream of records as an event stream, Kafka will only allow one consumer instance to consume per partition. You cannot add other instances to process that partition under the same consumer group.

Head of line blocking

Given that there is only one consumer instance per partition, if a record is taking a very long time to process, no additional forward progress can be made until it is finished. It is blocked by the record at the head of the line.

Per-record failures and retries

There is no per-record retry mechanism, nor any way to mark if a record is failed (perhaps it has been retried 3 times already). With the Kafka consumer protocol for event streams, you can only acknowledge the entire batch of consumed records, or redo the entire batch. Any per-record tracking would need to be implemented in custom code within the consumer.

Queues for Kafka KIP-932 introduces the concept of a *share group* . It is similar to the Kafka consumer group, but it treats the append-only log of a Kafka topic as a *queue* instead of as an event stream. The same data, but the consumer instances read, process, and commit progress back to the broker as though they were consuming from a queue, and not an event stream.

KIP-932 provides the comprehensive details about exactly how it works. In short, you create a KafkaShareConsumer that registers its own share group, which enables the following features:

Multiple instances per partition

Consumer instances can share processing of a single partition for much higher throughput. Record batches are

distributed to each consumer instance and can be processed out of order.

Individual record acknowledgement

Records are acknowledged individually. There is no more head of line blocking as the consumers can process and acknowledge the records in any order.

Retries

The failure to process a record is tracked by the share group, and can be retried multiple times until reaching a maximum count (default 5) or until success.

Processing Timeouts

Share groups can release records that have timed out during processing for other instances to process, say due to an individual instance failure.

Rewindable Progress

Share groups can be rewound to replay data from a specific offset. Unlike traditional queues, the data in a Kafka topic is not deleted after processing.

Queues for Kafka add extra flexibility to the event broker architecture, allowing you to use queue semantics without having to spin up and manage a dedicated queue broker.

The Structure of an Event

Events, as written to an event stream, are typically represented using a key, a value, and a header. Together, these three components form the record representing the event. An example of the record structure is shown in

Figure 1-5, containing a minimal set of details pertaining to an e-commerce order.

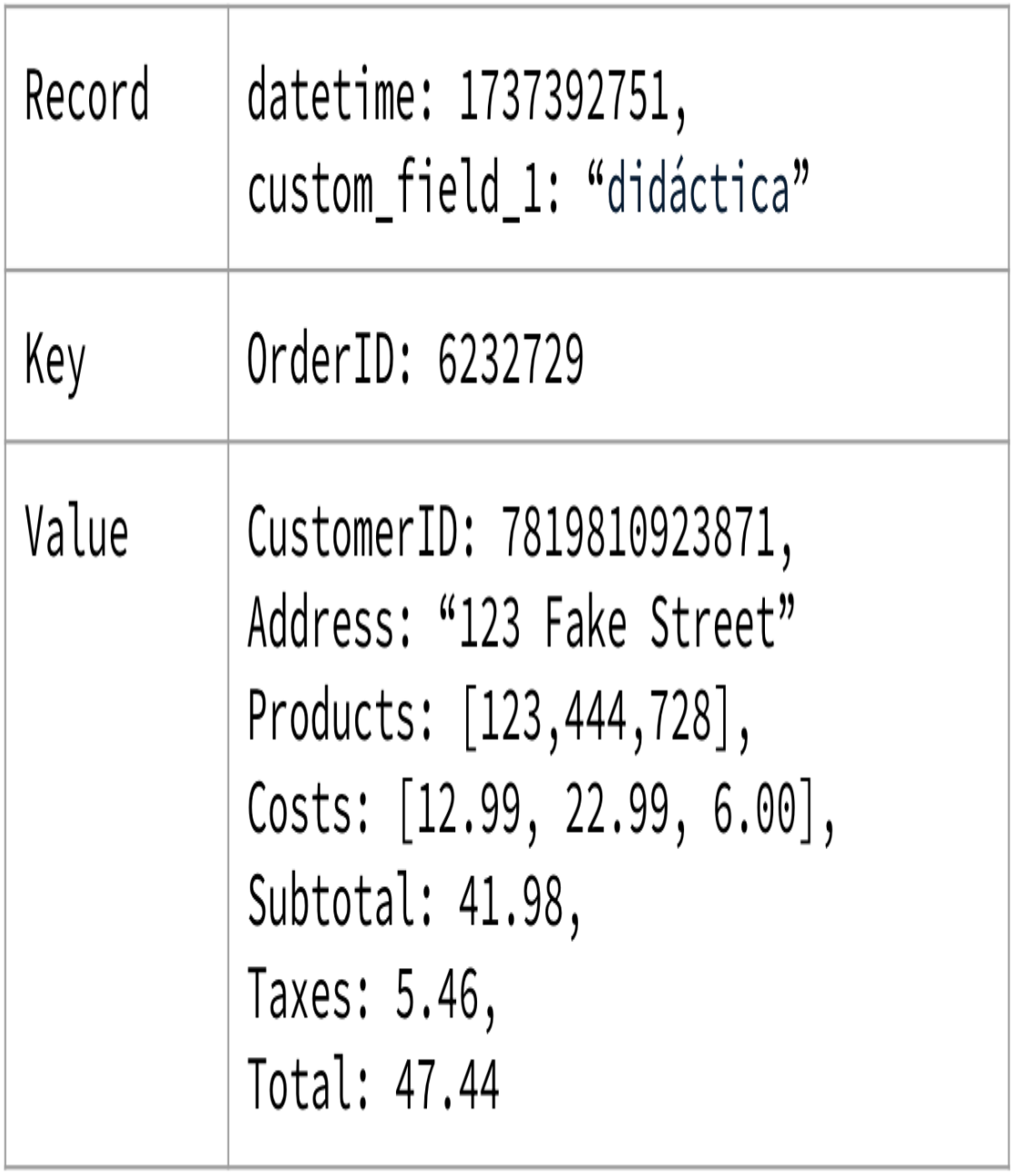


Figure 1-5. A simple e-commerce order showing the items purchased by a user, along with the total cost

The key

The key is optional but extremely useful. It is most commonly used to route the event to a specific partition of the event stream, so that all records of the same key are co-located. It can also be used to represent a unique *entity*, which is covered in more detail later in this chapter ("Entity Events").

The value

Contains the bulk of the data relating to the event. If you think of the event key as the primary key of a database table's row, then think of the value as all the other fields in that row. The value carries the majority of an event's data.

The header (also known as "record properties")

Contains metadata about the event itself, and is often a proprietary format depending on the event broker. The record is usually used to record information such as datetimes, tracking IDs, and user-defined key-value pairs that aren't suitable for the value.

The record's exact structure will vary with your technology of choice. For example, queues and ephemeral-messaging tend to use similar yet different conventions and components, such as header keys, routing keys, and binding keys, to name a few. But for the most part, this following three-piece record format is generally applicable for all events.

Events tend to fall into three main classifications: unkeyed events, keyed events, and entity events. Let's take a look closer look at each.

Unkeyed Events

Unkeyed events do not contain a key. They're generally considered to be fully independent of one another. There is no special treatment for routing to a particular partition.

Unkeyed events are commonly some form of raw measurement. For example, a camera that photographs an automobile running through a red light may creates an unkeyed event, as shown in Figure 1-6

Record	datetime: 1736322412
Key	null
Value	camera_id: 2734578334, photo_uri: “S3://…”, local_time: 1736297212, measured_speed: “23 m/s”

Figure 1-6. An unkeyed red light traffic camera event

Could the red light camera have produced the event with the `camera_id` as the key? Sure! But it didn’t, because it only records the data in that specific format, and this specific camera doesn’t support custom post-processing. You get the event in the format that it specifies in the user manual.

If we want to add a key, we’re going to have to do some additional processing and emit a new event stream, as you’ll see in this next section.

Keyed Events

Keyed events contain a non-null key related to something important about the event. The event key is specified by the producer service when it creates the record, and remains immutable once written to the event stream.

If we were to apply a key to the red light traffic camera data, we may choose to use the driver's license plate. You can see an example of that in Figure 1-7.

Record	datetime: 1736322412
Key	license_plate: "FAST 321"
Value	camera_id: 2734578334, photo_uri: "S3://…", local_time: 1736297212, Vehicle_color: "red", vehicle_type: "4-door sedan" vehicle_speed: "23 m/s"

Figure 1-7. A red light traffic camera event keyed on the driver's license plate

A key enables the producer to partition the records deterministically, with all records of the same key going to the same event stream partition, as per Figure 1-8. This is known as *data locality*, and is an essential property for building scalable event-driven microservices.

Data locality is guarantee to the consumers that all data of a given key will be in just a single partition. In turn, consumers can easily divide up the work on a per-partition basis, knowing that any key-based work they perform will only rely on reading a single partition, and not all the events from all partitions.

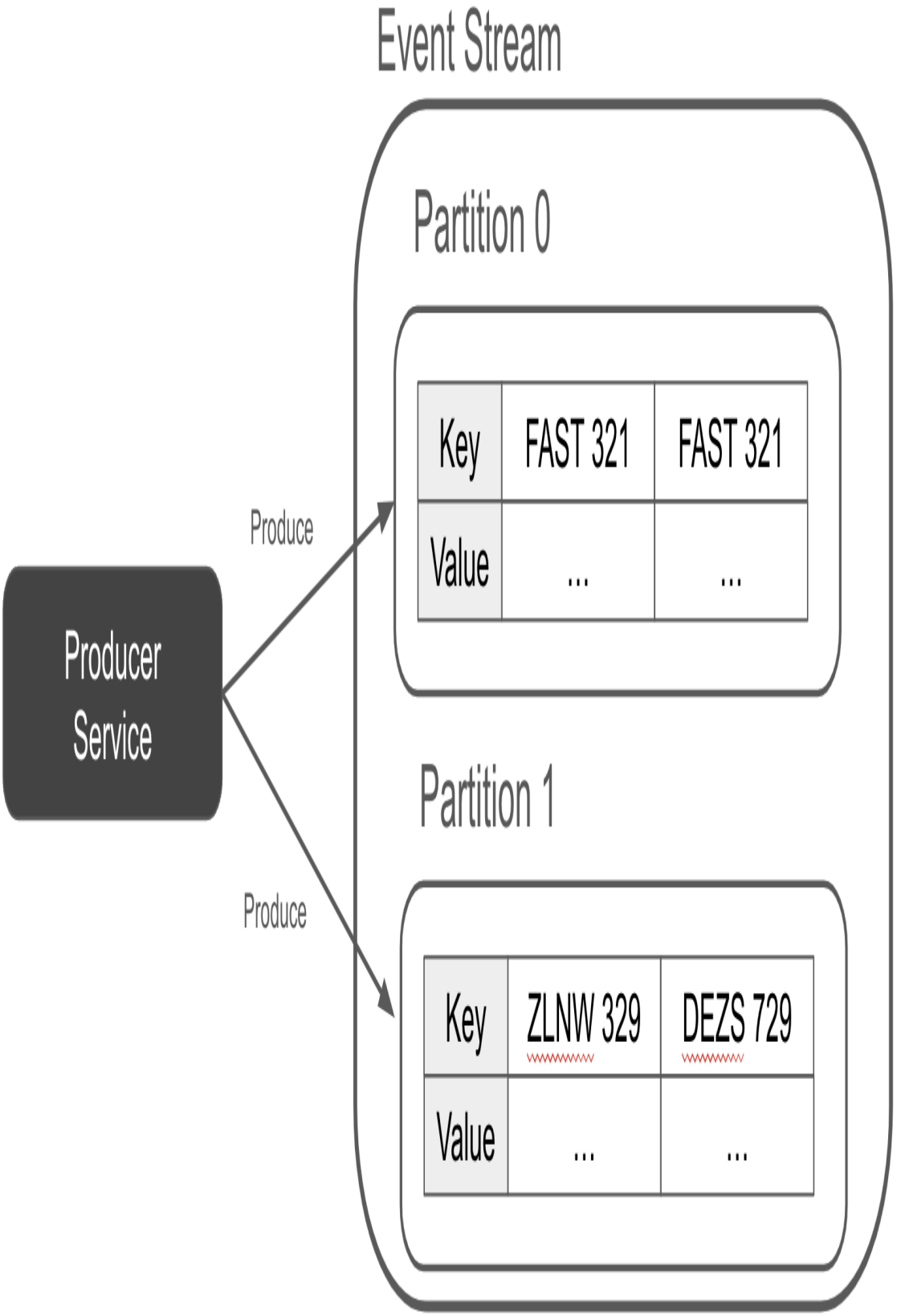

Event Stream
Partition 0
Key
FAST 321
FAST 321
Value
...
...
Produce
Producer Service
Produce
Partition 1
Key
ZLNW 329
DEZS 729
Value
...
...

Figure 1-8. A red light traffic camera event keyed on the driver's license plate

As illustrated in the partitioned topic, you can see that there are at least two events for `FAST 321`. Each of these events represents an instance of the car running through a red light. Keep this in mind as we go to our final event classification, the entity event.

Entity Events

An entity event represents a *unique thing*, and is keyed on a unique ID that represents that thing. It describes the properties and state of the entity at a specific point in time. Entity events are also sometimes called *state events*, as they represent the state of a given thing at a given point in time.

For something a bit more concrete, and to continue the red light camera analogy, you could expect to see a `Car` as an entity event. You may also see entity events for `Driver`, `Tire`, `Intersection`, or any other number of "things" involved in the scenario. A car entity is featured in Figure 1-9.

Record	datetime: 1737333331
Key	VIN: “WBAYE4C59DD136996”
Value	make: “Dodge” model: “Charger” year: 1969, color: “red”, classification: “sports car”

Figure 1-9. A Car entity describing the sportscar that keeps running the red lights

You may find it helpful to think of an entity event like you would think of a row in a database table. Both have a primary key, and both represent the data for that primary key *as it is at the current point in time*. And much like a database row, the data is only valid for as long as the data remains unchanged. Thus, if you were to repaint the car to blue, you could expect to see a new event with the color updated, as in Figure 1-10.

Record	**datetime: 1737829481**
Key	VIN: “WBAYE4C59DD136996”
Value	make: “Dodge” model: “Charger” year: 1969, **color: “blue”,** classification: “sports car”

Figure 1-10. The car has been repainted blue

You may also notice that the `datetime` field has been updated to represent when the car was painted blue (or at least when it was reported). You'll also notice that the entity event also contains all the data that *didn't* change. This is intentional, and it actually allows us to do some pretty powerful things with entity events - but Chapter 4 will cover this subject in more detail.

Similarly to how *keyed events* each go to the same partition, the same is true for *entity events*. Figure 1-11 shows an event stream with two events for the `FAST 321` - one when it was red (the oldest event), and one while it is blue (the latest event, appended to the tail of the stream).

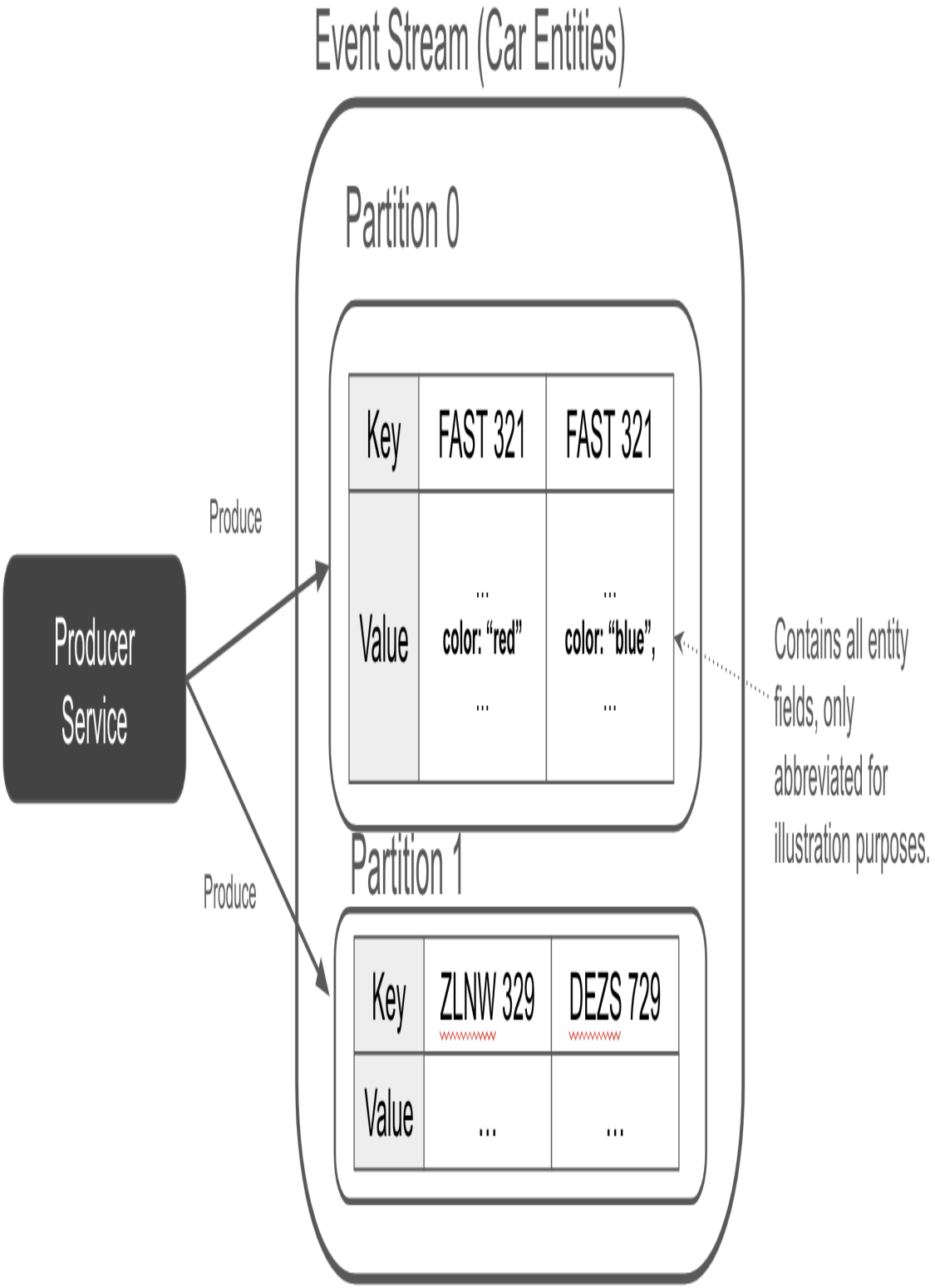

Event Stream (Car Entities)
Partition 0
Key
FAST 321
FAST 321
Value
...
color: "red"
...
...
color: "blue",
...
Produce
Producer Service
Produce
Partition 1
Key
ZLNW 329
DEZS 729
Value
...
...
Contains all entity fields, only abbreviated for illustration purposes.

Figure 1-11. The producer appends a full entity event whenever an entity is created, updated, or deleted

Entity events are particularly important in event-driven architectures. They provide the basis for *event-carried state transfer (ECST)*, as the state of the entity is carried by the event to all consumers. ECST allows for the *materialization* of entity event streams into the consumer's bounded context, which is covered later in this chapter in "Materializing State from Entity Events".

There is more to event design than what has been covered in this chapter so far, but that will have to wait until Chapter 4. Instead, let's look at how we might use these events that I've just introduced to build real world event-driven microservices.

Repartitioning Event Streams

Repartitioning is when a service reads events from one stream and writes them to a new stream with a different key, a different key-to-partition mapping strategy, and / or a different partition count. Repartitioning is a key component of event-driven microservices, as it lets you join, group, and aggregate data from streams of varying partition counts, partition strategies, and differing keys. Figure 1-12 shows a microservice reading from an event stream with 2 partitions and writing the same data back to a 4 partition output stream.

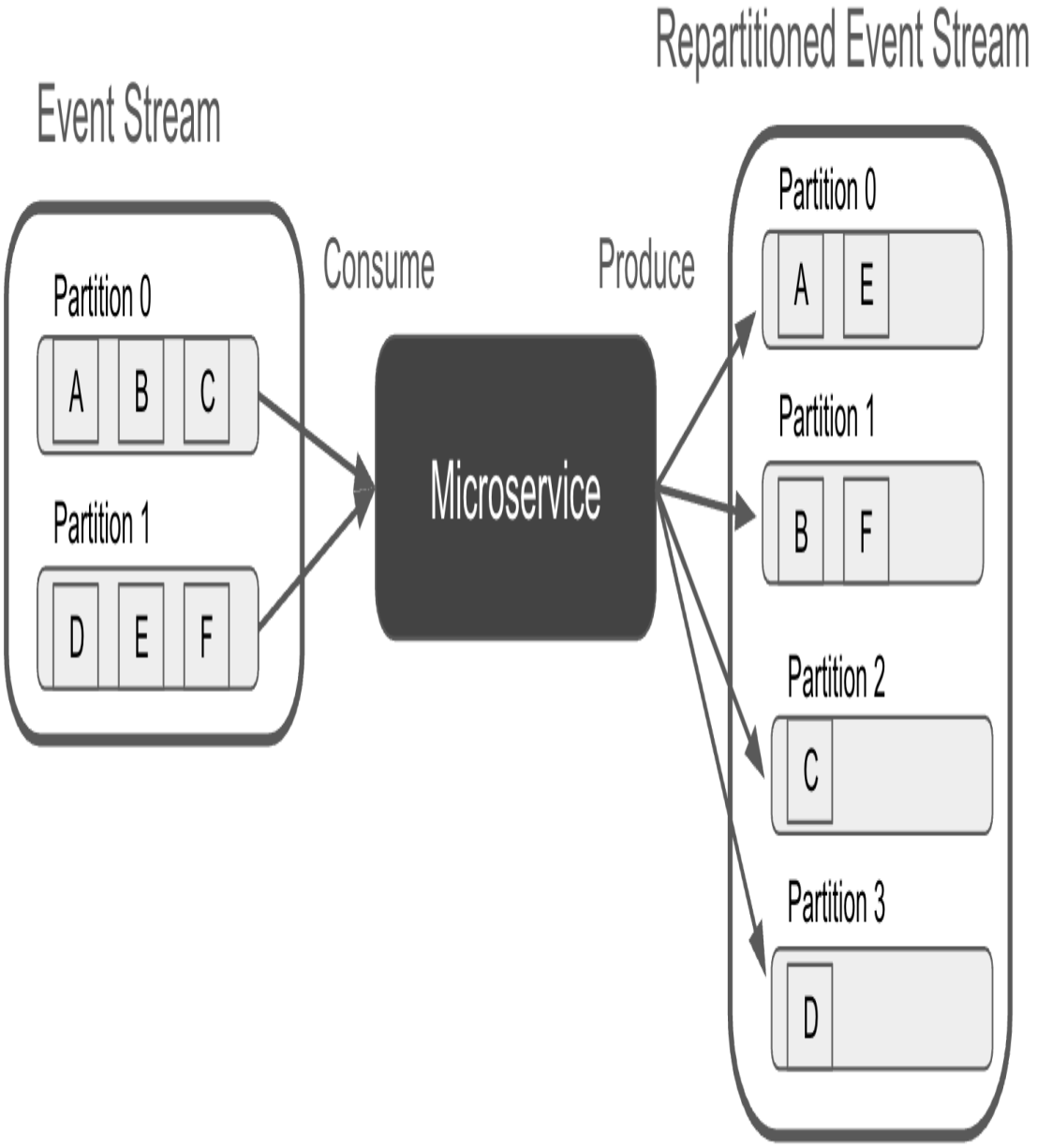


Figure 1-12. Repartitioning a stream from 2 to 4 partitions

Repartitioning event streams may introduce out-of-order data, though this is explored in more detail in "Multiple producers to multiple partitions".

Copartitioning Event Streams

Two event streams are *copartitioned* when they have the same key, the same key-to-partition mapping strategy, and the same partition count. Copartitioning is required for joins on primary keys, aggregations, and reduce operations, by assigning the same keyed data to the same processing instances for the purposes of data locality. Figure 1-13 shows two microservice instances each consuming from two copartitioned streams.

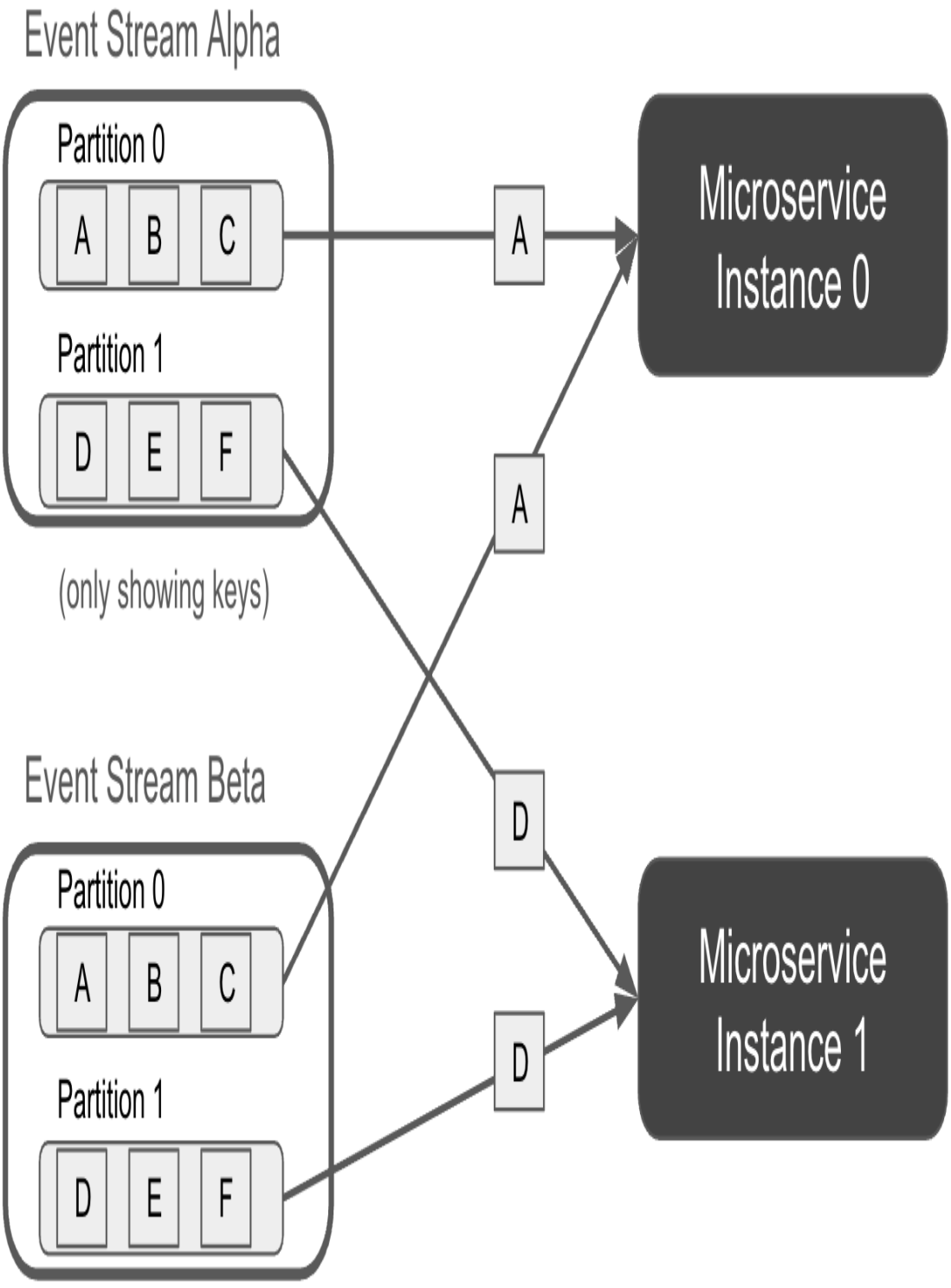


Figure 1-13. Copartitioned Event Streams Alpha and Beta consumed by two microservice instances

Microservice instance 0 is assigned partition 0 from both copartitioned streams, while instance 1 is assigned partitions 1. All events with the key of `A` are routed to instance 0, while all events of key `D` are routed to instance 1.

Aggregating State from Keyed Events

An *aggregation* is the process of consuming two or more events and combining them into a single result. Aggregations are a common data processing primitive for event-driven architectures, and are one of the primary ways to generate state out of a series of events. Building an aggregation requires storing and maintaining durable state, such that aggregation progress is persisted if the service fails. State and recovery are covered in chapter Chapter 7.

The keyed event plays an important role in aggregations, since all the data of the same key is in the same partition. Thus, you can simply aggregate a single key by reading a single partition. If you're reading multiple topics, then you'll need to ensure that they're partitioned identically - otherwise you're going to have to *repartition* the data so that the streams match one another. We'll cover this in the next section.

An aggregation may be as simple as a sum, as shown in Figure 1-14. For example, summing up the traffic infraction tickets that have been issued to the owner of a speeding sportscar, and computing the total of the amount owed.

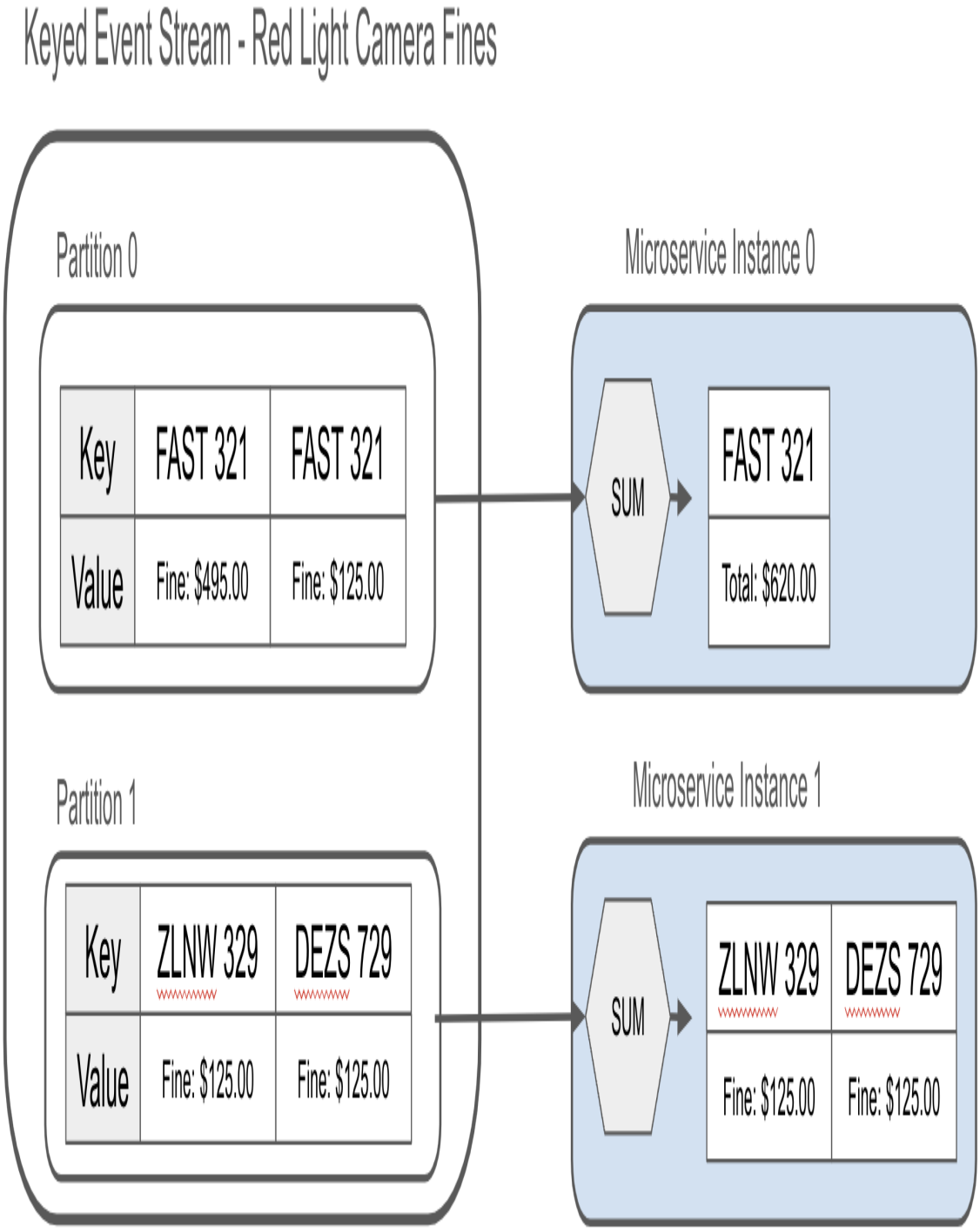


Figure 1-14. Aggregating the keyed events of the red light camera fines

Aggregations may also be more complex, incorporating multiple input streams, multiple event types, and internal state machines to derive more

complex results. We'll revisit aggregations throughout the book, but for now, let's take a look at materializations.

Materializing State from Entity Events

A *materialization* is a projection of a stream into a table. You materialize a table by applying entity events, in order, from an entity event stream. Each entity event is upserted into the table, such that the most recently read event for a given key is represented. This is illustrated in Figure 1-15, where `FAST 321` and `SJFH 372` both have the newest values in their materialized table.

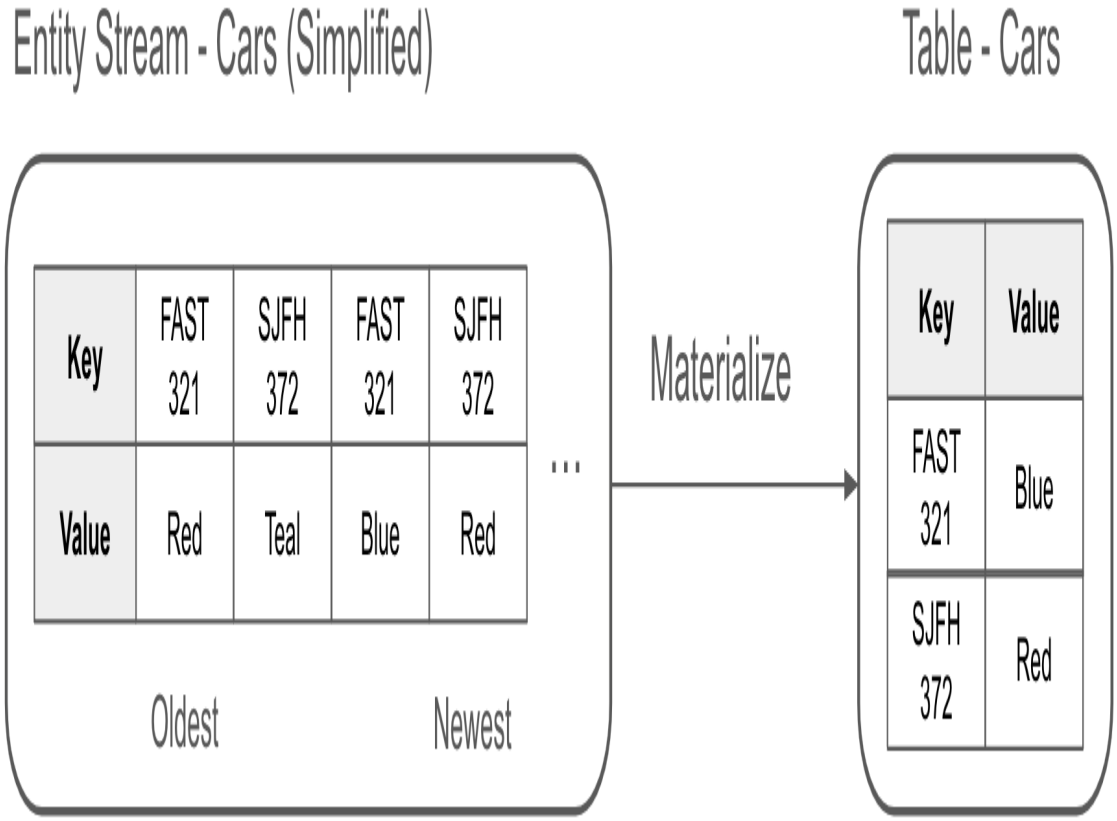


Figure 1-15. Materializing an event stream into a table

You can also convert a table into a stream of entity events by publishing each update to the event stream.

> **TIP**
>
> *Stream-table duality* is the principle that a stream can be represented by a table, and a table can be represented as a stream. It is fundamental to the sharing of state between event-driven microservice, without any direct coupling between producer and consumer services.

In the same way, you can have a table record all updates and in doing so produce a stream of data representing the table's state over time. In the following example, BB is upserted twice, while DD is upserted just once. The output stream in Figure 1-16 shows three upsert events representing these operations.

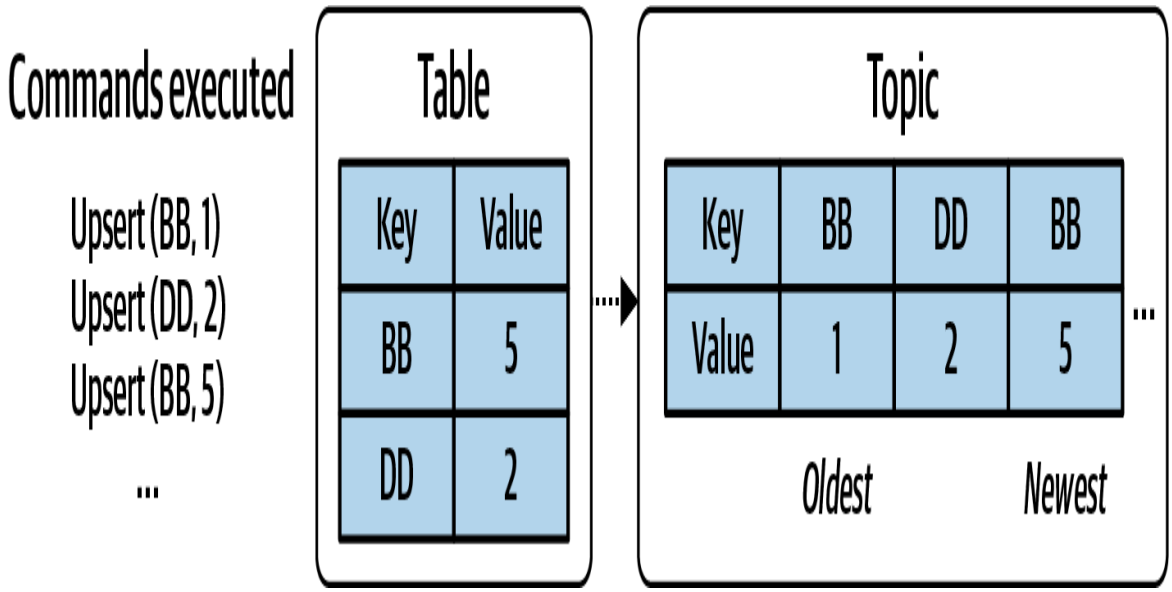


Figure 1-16. Generating an event stream from the changes applied to a table

A relational database table, for instance, is created and populated through a series of data insertion, update, and deletion commands. These commands can be produced as events to an immutable log, such as a local append-only file (like the binary log in MySQL) or an external event stream. By playing

back the entire contents of the log, you can reconstruct precisely the table and all of its data contents.

Deleting Events and Event Stream Compaction

First, the bad news. You can't delete a record from an event stream as you would a row in a database table. A major part of the value proposition of an event stream is its immutability. But you can issue a *new* event, known as a *tombstone*, that will allow you to delete records with the same key. Tombstones are most commonly used with entity event streams.

A tombstone is a keyed event with its value set to null, a convention established by the Apache Kafka project. Tombstones serve two purposes. First, they signal to the consumer that the data associated with that event key should now be considered deleted. To use the database analogy, deleting a row from a table would be equivalent to publishing a tombstone to an event stream for the same primary key.

Secondly, tombstones enable *compaction*. Compaction is an event broker process that reduces the size of the event streams by retaining *only the most recent events for a given key*. Events older than the tombstone will be deleted, and the remaining events are compacted down into a smaller set of files that are faster and easier to read. Event stream offsets are maintained such that no changes are required by the consumers. Figure 1-17 illustrates the logical compaction of an event stream in the event broker.

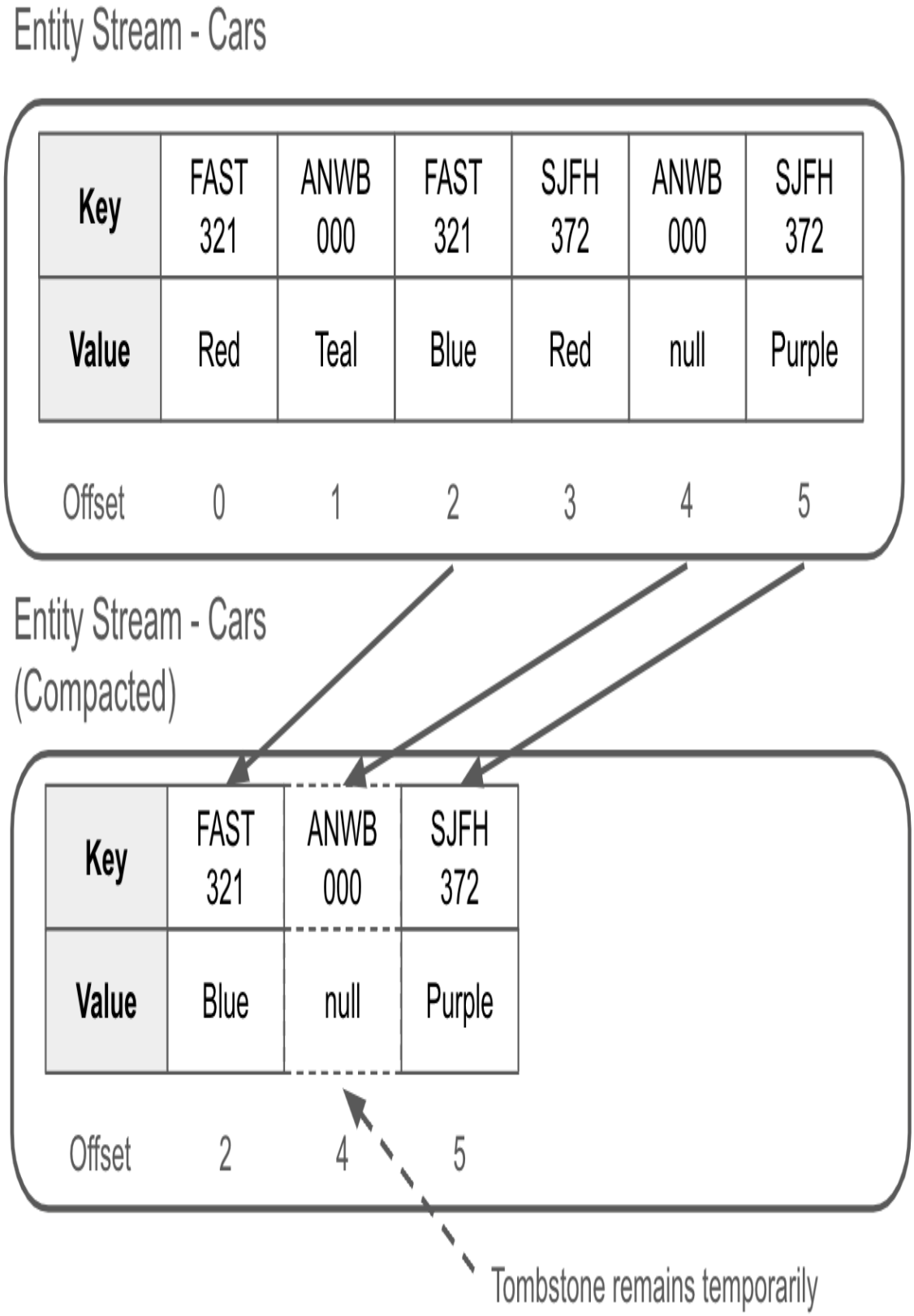

Entity Stream - Cars
Key
FAST 321
ANWB 000
FAST 321
SJFH 372
ANWB 000
SJFH 372
Value
Red
Teal
Blue
Red
null
Purple
Offset
0
1
2
3
4
5
Entity Stream - Cars (Compacted)
Key
FAST 321
ANWB 000
SJFH 372
Value
Blue
null
Purple
Offset
2
4
5
Tombstone remains temporarily

Figure 1-17. After a compaction, only the most recent record is kept for a given key — all predecessors records of the same key are deleted

The tombstone record typically remains in the event stream for a short period of time. In the case of Apache Kafka, the default value of 24 hours gives consumers a chance to read the tombstone before it too is (asynchronously) cleaned up.

Compaction is an asynchronous process performed only when a certain set of criteria is met, including dirty ratio, or record count in inactive segments. You can also manually trigger compaction. The precise mechanics of compaction vary with event broker selection, so you'll have to consult your documentation accordingly.

Compaction reduces both disk usage and the quantity of events that must be processed to reach the current state, at the expense of eliminating a portion of the event stream history. Compaction typically provides several useful configurations and guarantees, including:

Minimum compaction lag

> You can specify the minimum amount of time a record must live in the event stream before it is eligible for compaction. For example, Apache Kafka provides a `min.compaction.lag.ms` property on its topics. You can set this to a reasonable value, say 24 hours or 7 days, to ensure that your consumers can read the data before it is compacted away.

Offset guarantees

> Offsets remain unchanged before, during, and after compaction. Compaction will introduce gaps between sequential offsets, but there remains no consequences for consumers sequentially consuming the event stream. Trying to read a specific offset that no longer exists will result in an error.

Consistency guarantees

> A consumer reading from the start of a compacted topic can *materialize* exactly the same table as a consumer that has been running since the beginning.

> **CAUTION**
>
> While you may be able to find tools that allow you to delete records from an event stream manually, *be very careful*. Manually deleting records can lead to unexpected results, particularly if you have consumers that have already read the data. Simply deleting the records won't fix the consumers derived state. Prevention of bad data is essential, and is covered more in [[Link to Come]].

Stream-table duality and materialization allow services to communicate state between one another. Compaction lets us keep the event streams to a reasonable size, in line with the domain of the data.

The Kappa Architecture

The Kappa architecture was first presented in 2014 by Jay Kreps, co-creator of Apache Kafka and co-founder of Confluent. The Kappa architecture relies on event streams as the sole record for both current and historical data. Consumers simply start consuming from the start of the stream to get a full picture of everything that has happened since inception, eventually reaching the head of the stream and the latest events as per Figure 1-18.

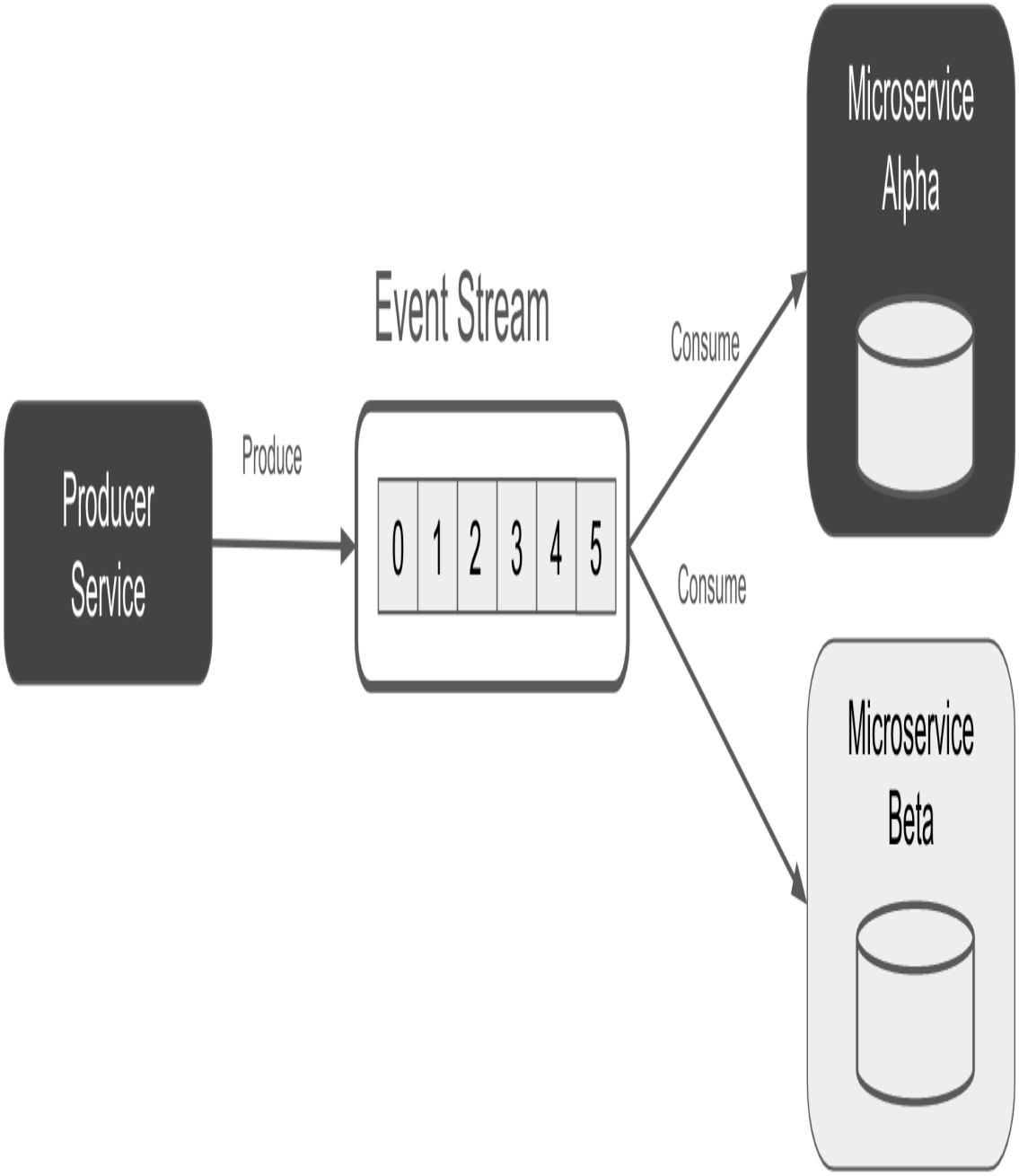


Figure 1-18. Kappa architecture, with each service building its state from just an event stream

Kappa architectures do not use a secondary store for historical data, as is the case in the Lambda architecture (more on that in the next section). Kappa relies entirely on the event stream for storage and serving.

Kappa architectures have only been realized with modern event brokers, in combination with cheap storage ushered in with the cloud computing revolution. It is affordable and easy to store as much data as you need in the stream, and you are no longer limited by broker technologies that applied mandatory TTLs, deleting your records after a few hours or days.

Event-driven microservices simply materialize their own state from the streams as they need. There is no need to access data from a secondary store somewhere else, nor any of the complexity overhead in managing extra permissions. It's clean, simple, and easy to code.

There are some trade-offs with Kappa. Each service must build its own state, and for extremely large data sets it may take some time to materialize from the event stream. A low partition count and insufficient parallelization may see your service take many hours or days to materialize. This is often called *hydration time*, and you'll need to plan for it in your service life cycle.

You can mitigate the hydration time problem by maintaining snapshots or backups of your materialized and computed state. Then, when loading your application, your service simply loads from its backed-up data and restores processing from where it left off. While this is a responsibility of the application itself, snapshots and backups come with leading stream-processing technologies such as Apache Kafka Streams, Apache Flink, and Apache Spark Streaming, just to name a few of the open source industry leaders.

The Kappa architecture is key to building decoupled event-driven microservices as it provides your services with a single powerful guarantee - that your event stream can act as a single source of truth for a given set of data. There is no need to go elsewhere to a secondary or tertiary store. All the data you need is in that event stream, ready to go for your services.

What does the Kappa architecture look like in code? Example 1-1 shows a Kafka Streams application with two `KTables`, which is just a stream materialized into a table using ECST. Next, the `products KTable` and the `productReviews KTable` are joined using a non-windowed `INNER` join to create a `KTable` of denormalized and enriched product data. Stream-

processing frameworks make it very easy to handle event streams, build up internal state using ECST, and merge and join data from various data products, in just a few lines of code:

Example 1-1. Showcasing joins with Kafka Streams

```
StreamsBuilder builder = new StreamsBuilder();

//Materializes the tables from the source Kafka topics
KTable products = builder.table("products")
KTable productReviews = builder.table("product_reviews")

//Join events on the primary key, apply business logic as needed
KTable productsWithReviews = products.join(productReviews, businessLogic(..), ...)
builder.build();
```

Apache Flink can provide something similar, as seen in Example 1-2. Say we wanted something more SQL-like:

Example 1-2. Showcasing joins with Flink SQL

```
CREATE TABLE PRODUCTS (
  product_id BIGINT,
  name VARCHAR,
  brand VARCHAR,
  description VARCHAR,
  timestamp TIMESTAMP(3),
  PRIMARY KEY (product_id) NOT ENFORCED,
) WITH (
  'connector' = 'kafka',
  'topic' = 'products',
  'format' = 'protobuf',
  'properties.bootstrap.servers' = 'localhost:9092',
  'properties.group.id' = 'foobar'
);

CREATE TABLE PRODUCT_REVIEWS (
  product_id BIGINT,
  reviews VARCHAR,
  timestamp TIMESTAMP(3),
  PRIMARY KEY (product_id) NOT ENFORCED,
) WITH (
  'connector' = 'kafka',
  'topic' = 'product_reviews',
  'format' = 'protobuf',
  'properties.bootstrap.servers' = 'localhost:9092',
  'properties.group.id' = 'foobar'
```

```
);

CREATE TABLE PRODUCTS_WITH_REVIEWS AS
  SELECT *
  FROM PRODUCTS
  INNER JOIN PRODUCT_REVIEWS
  ON PRODUCTS.product_id = PRODUCT_REVIEWS.product_id;
```

Both the Flink SQL and the Kafka Streams code samples are simple, clear, and concise. This book will go deeper into each of these options, and more, in Part III. But for now, as we go to look at the Lambda architecture, just keep in mind how easy it is to leverage the Kappa architecture. It just takes a few lines of code to transform a stream of events into a self-updating table capable of driving your microservice code.

The Lambda Architecture

The *Lambda architecture* relies on both an event stream for real-time data and a secondary repository for storage of historical data. The Lambda architecture is predicated on the outdated notion that you cannot store events in an event stream indefinitely. It is, however, consistent with the outdated technologies that enforced mandatory TTLs on event streams. It remains stubbornly persistent in the minds of many, and so I include it in this book for the sake of historical context, but not as an endorsement for use.

Consumers in a lambda architecture must obtain their historical data from the historical repository first, loading it into their respective state stores. Then, the consumers must swap over to the event stream for further updates and changes.

There are two main versions of this architecture. In the first, the historical repository and the stream are built independently. In the second, the historical repository is built from the stream. Let's take a look at the first, first.

The historical repository contains the results of aggregations, materializations, or richer computations. It's not just another location to store events. Historically speaking, the historical repository has often simply been just the internal database of the source system. In other words, you'd ask the

source system for the historical data, load it in to your system, then swap over to the event stream. This is a bit of a anti-pattern as it puts the entire consumer load on the producer service, but it's common enough that it's worth mentioning.

Figure 1-19 shows a simplified implementation of the Lambda architecture. The producer writes new data to both the event stream and the historical data store.

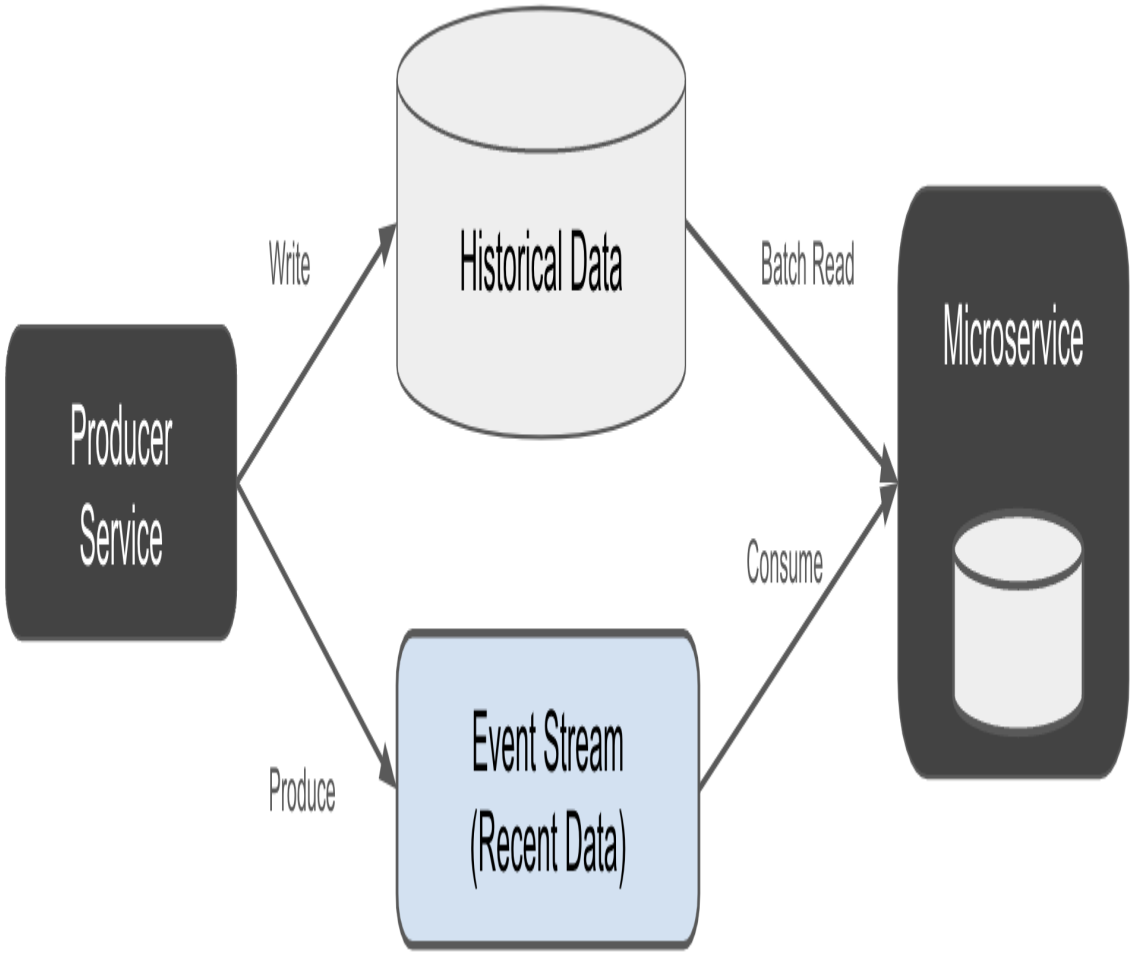


Figure 1-19. Lambda architecture, writing to both the stream and the historical data table at the same time

A major flaw in this plan is that the data is not written atomically. The reality is that it's very difficult to get high performance distributed transactions

across multiple independent systems. What tends to happen *in reality* is that the producer updates one system first (say, the historical data store), then the other (the event stream). An intermittent failure during the writes may see the event written to the historical store but not the stream - or vice versa, depending on your code.

The problem is that your stream and historical data set will *diverge*, meaning that you get different results if you build from the stream than you would from the historical data. An old consumer reading solely from the stream may compute a different result than a new consumer bootstrapping itself from the historical data. This can cause serious problems in your organization, and it can be very difficult to track down the reason why - particularly since the event stream data is time-limited, and evidence of its divergence is deleted after just a few days.

In the second version of Lambda architecture, the historical data is populated directly from the initial event stream, as shown in Figure 1-20.

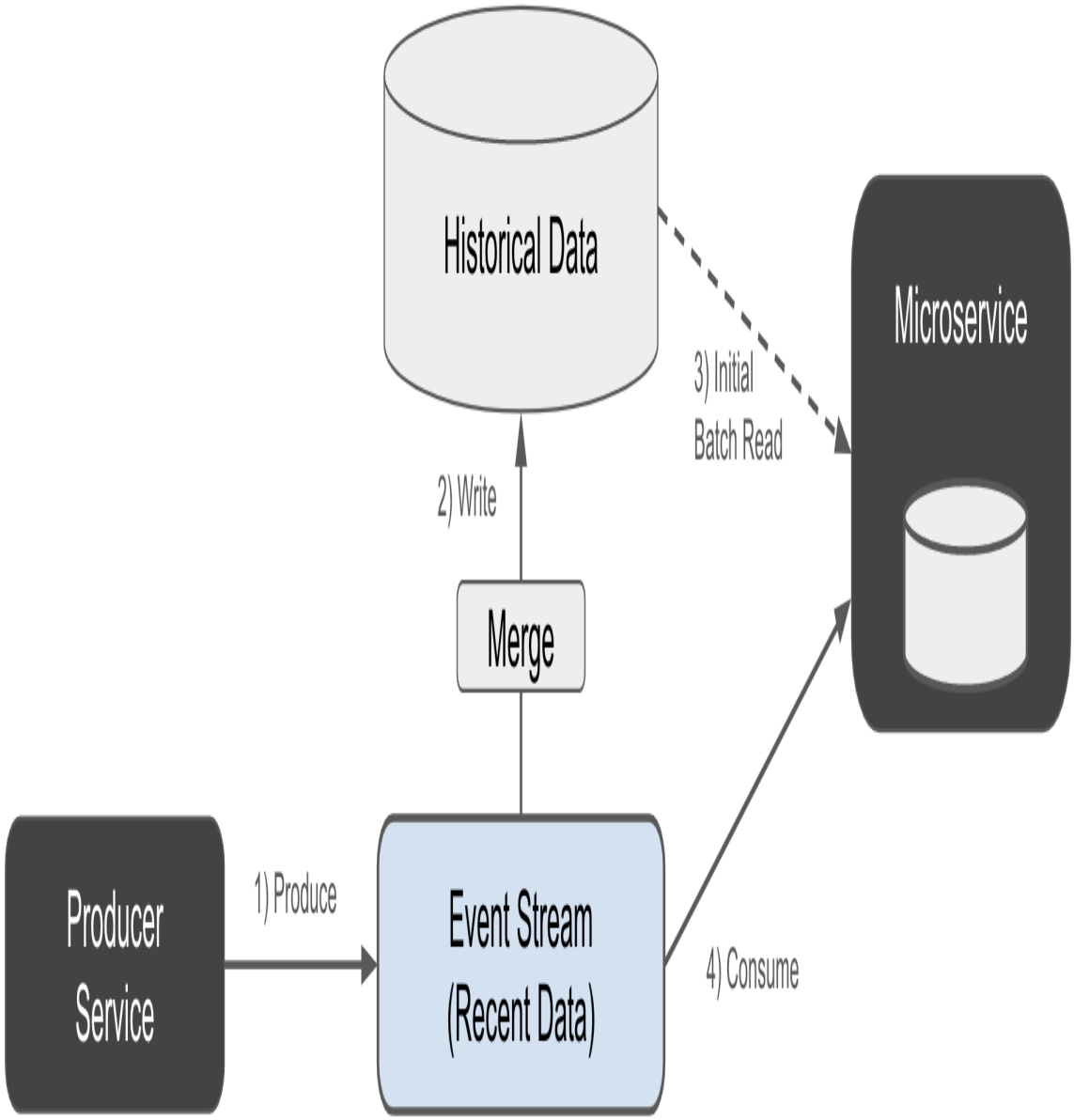


Figure 1-20. Historical lambda data build from the event stream

The producer writes directly to the event stream (1). The historical data is populated by a secondary process that merges it into the data store (2). The

consumer in turn reads the historical data first (3), then switches over to the event stream (4).

If you squint a little, you may find that this second version looks an awful lot like the Kappa architecture - except that the service is building the state store *outside* of the microservice. The only complication is that we've introduced this awkward split between the event broker and the historic data store - an artifact due to the now-invalid notion that an event broker cannot store events indefinitely.

Overall, the Lambda architecture may seem simple in theory, but it ends up being very difficult to do well *in practice*. Why? Here are a few of the major obstacles:

The producer must maintain extra code

> The code that writes to the stream, and the code that writes to the historical store.

The consumer must maintain two code paths

> One path reads from the historical store, and one path reads from the stream. The consumer must write code that seamlessly switches from one to the other, without missing any data or accidentally duplicating data. This can be quite challenging in practice, particularly with distributed systems and intermittent failures.

Streamed data may not converge to the same results as the historical data

> Consumers may not get *exactly the same results* if reading from the stream as the table. The producer must ensure that the data between the two does not diverge over time, but this is challenging without atomic updates to both the stream and the historical data store.

The stream and historical data models must evolve in sync

> Data isn't static. If you have to update the data format, it will require code changes in two places - the historical store and the stream.

Merging multiple Lambda-powered data sets is almost impossible

> It's easy to merge multiple Kappa data sets, as illustrated in Example 1-1. There is no crossover from the historical offline data store to the event stream. In contrast, with Lambdas, you must reconcile the offline historical data sets with each other *and* with the event streams. With two streams there is just one reconciliation (stream-stream), whereas in a Kappa mode there are four reconciliations - one for each combination of stream and offline data store.

The long and the short of it is that Lambda architecture is simply too difficult to manage at any reasonable scale. It puts the entire onus of accessing data onto the consumer, and increases significantly in complexity when you move beyond just a single source of data. It is far easier, both cognitively and coding-wise, to rely on the Kappa architecture with the single stream source for providing historical data.

Event Data Definitions and Schemas

Event data serves as the means of long term and implementation agnostic data storage, as well as the communication mechanism between services. Therefore, it is important that both the producers and consumers of events have a common understanding of the meaning of the data. Ideally, the consumer can interpret the contents and meaning of an event without having to consult with the owner of the producing service. This requires a common data specification between producers and consumers, and is analogous to an API definition between synchronous request-response services.

Schematization selections such as Apache Avro and Google's Protobuf provide two features that are leveraged heavily in event-driven

microservices. First, they provide an evolution framework, where certain sets of changes can be safely made to the schemas without requiring downstream consumers to make a code change. Second, they also provide the means to generate typed classes (where applicable) to convert the schematized data into plain old objects in the language of your choice. This makes the creation of business logic far simpler and more transparent in the development of microservices. Chapter 3 covers these topics in greater detail.

Powering Microservices with the Event Broker

Event broker systems suitable for large-scale enterprises all generally follow the same model. Multiple, distributed event brokers work together in a cluster to provide a platform for the production and consumption of event streams and queues. This model provides several essential features that are required for running an event-driven ecosystem at scale:

Scalability

> Additional event broker instances can be added to increase the cluster's production, consumption, and data storage capacity.

Durability

> Event data is replicated between nodes. This permits a cluster of brokers to both preserve and continue serving data when a broker fails.

High availability

> A cluster of event broker nodes enables clients to connect to other nodes in the case of a broker failure. This permits the clients to maintain full uptime.

High-performance

> Multiple broker nodes share the production and consumption load. In addition, each broker node must be highly performant to be able to handle hundreds of thousands of writes or reads per second.

Though there are different ways in which event data can be stored, replicated, and accessed behind the scenes of an event broker, they all generally provide the same mechanisms of storage and access to their clients.

Selecting an Event Broker

Though this book makes frequent reference to Apache Kafka as an example, there are other event brokers that could make suitable selections. Instead of comparing technologies directly, consider the following factors closely when selecting your event broker.

Support Tooling

Support tools are essential for effectively developing event-driven microservices. Many of these tools are bound to the implementation of the event broker itself. [[Link to Come]] covers tooling in more detail, but for now, some of the things we'd be looking for when selecting an event broker include:

- Browsing event stream records and schemas
- Quotas, access control, and topic management
- Monitoring, throughput, and lag measurements

Hosted Services

Hosted services allow you to outsource the creation and management of your event broker.

- Do hosted solutions exist?
- Will you purchase a hosted solution or host it internally?
- Does the hosting agent provide monitoring, scaling, disaster recovery, replication, and multizone deployments?
- Does it couple you to a single specific service provider?
- Are there professional support services available?

Client Libraries and Processing Frameworks

There are multiple event broker implementations to select from, each of which has varying levels of client support. It is important that your commonly used languages and tools work well with the client libraries.

- Do client libraries and frameworks exist in the required languages?
- Will you be able to build the libraries if they do not exist?
- Are you using commonly used frameworks or trying to roll your own?

Community Support

Community support is an extremely important aspect of selecting an event broker. An open source and freely available project, such as Apache Kafka, is a particularly good example of an event broker with large community support.

- Is there online community support?
- Is the technology mature and production-ready?
- Is the technology commonly used across many organizations?
- Is the technology attractive to prospective employees?
- Will employees be excited to build with these technologies?

Indefinite and Tiered Storage

Indefinite storage lets you store events forever, provided you have the storage space to do so. Depending on the size of your event streams and the retention duration (e.g. indefinite), it may be preferable to store older data segments in slower but cheaper storage.

Tiered storage provides multiple tiers (or layers) for storing events. The fastest and most expensive tier is served directly from the broker nodes themselves, either from in memory or on locally-attached solid state drives (SSDs). A subsequent lower cost and lower performance tier typically includes a dedicated large-scale storage service, such as Amazon's S3, Google Cloud Storage, and Azure Storage. Data in the high performance tier is moved asynchronously to a lower tier as it ages, but remains fully addressable and replayable through the event broker's consumer API.

- Is tiered storage automatically supported?
- Can data be rolled into lower or higher tiers based on usage?
- Can data be seamlessly retrieved from whichever tier it is stored in?

Summary

Event streams provide durable, replayable, and scalable data access. They can provide a full history of events, allowing your consumers to read whatever data they need via a single API. Every consumer is guaranteed an identical copy of the data, provided they read the stream as it was written.

Your event broker forms the core of your event-driven architectures. It's responsible for hosting the event streams, and providing consistent, high-performance access to the underlying data. It's responsible for durability, fault-tolerance, and scaling, to ensure that you can focus on building your services, not struggling with data access.

The producer service publishes a set of important business facts, broadcasting the data via the event stream to subscribed consumer services. The producer is no longer responsible for the varied query needs of all other services across the organization.

Consumers do not query the producer service for data, eliminating unnecessary point-to-point connections from your architecture. Previously a team may simply have written SQL queries or used request/response APIs to access data stored in a monolith's database. In an event-driven architecture they instead access that data from an event stream, materializing and aggregating their own state for their own business needs.

The adoption of event-driven microservices enables the creation of services that use only the event broker to store and access data. While local copies of the events may certainly be used by the business logic of the microservice, the event broker remains the single source of truth for all data.

In the next chapter, we'll take a look at the basics of event-driven microservices, and how they relate to event streams.

Chapter 2. Fundamentals of Event-Driven Microservices

> **A NOTE FOR EARLY RELEASE READERS**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 3rd chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

An *event-driven microservice*, as introduced in [[Link to Come]], is an application like any other. It requires the exact same type of compute, storage, and network resources as any other application. It also requires a place to store the source code, tools to build and deploy the application, and monitoring and logging to ensure healthy operation. As an event-driven application, it reads events from a stream (or streams), does work based on those events, and then outputs results - in the form of new events, API calls, or other forms of work.

[[Link to Come]] briefly introduced the main benefits of event-driven microservices. In this chapter, I'll cover the fundamentals of event-driven microservices, exploring their roles and responsibilities, along with the requirements, rules, and recommendations for building healthy applications.

The Basics of Event-Driven Microservices

Event-driven means that the events *drive* the business logic, just as water from a stream turns the water wheel of a mill (Figure 2-1). Event-driven applications, be they micro or macro, typically only do work when there are events coming through it (or a timer expires - also an event). Otherwise, they sit idle until there are new events to process.


10,197. - I.O.M. LAXEY WHEEL

Figure 2-1. The water stream powers the wheel, as event streams power the microservice Source - wikimedia

The consumer microservice reads events from the stream. Each consumer is responsible for updating its own pointers to previously read indices within the event stream. This index, known as the *offset*, is the measurement of the current event from the beginning of the event stream. Offsets permit multiple consumers to consume and track their progress independently of one another, as shown in Figure 2-2.

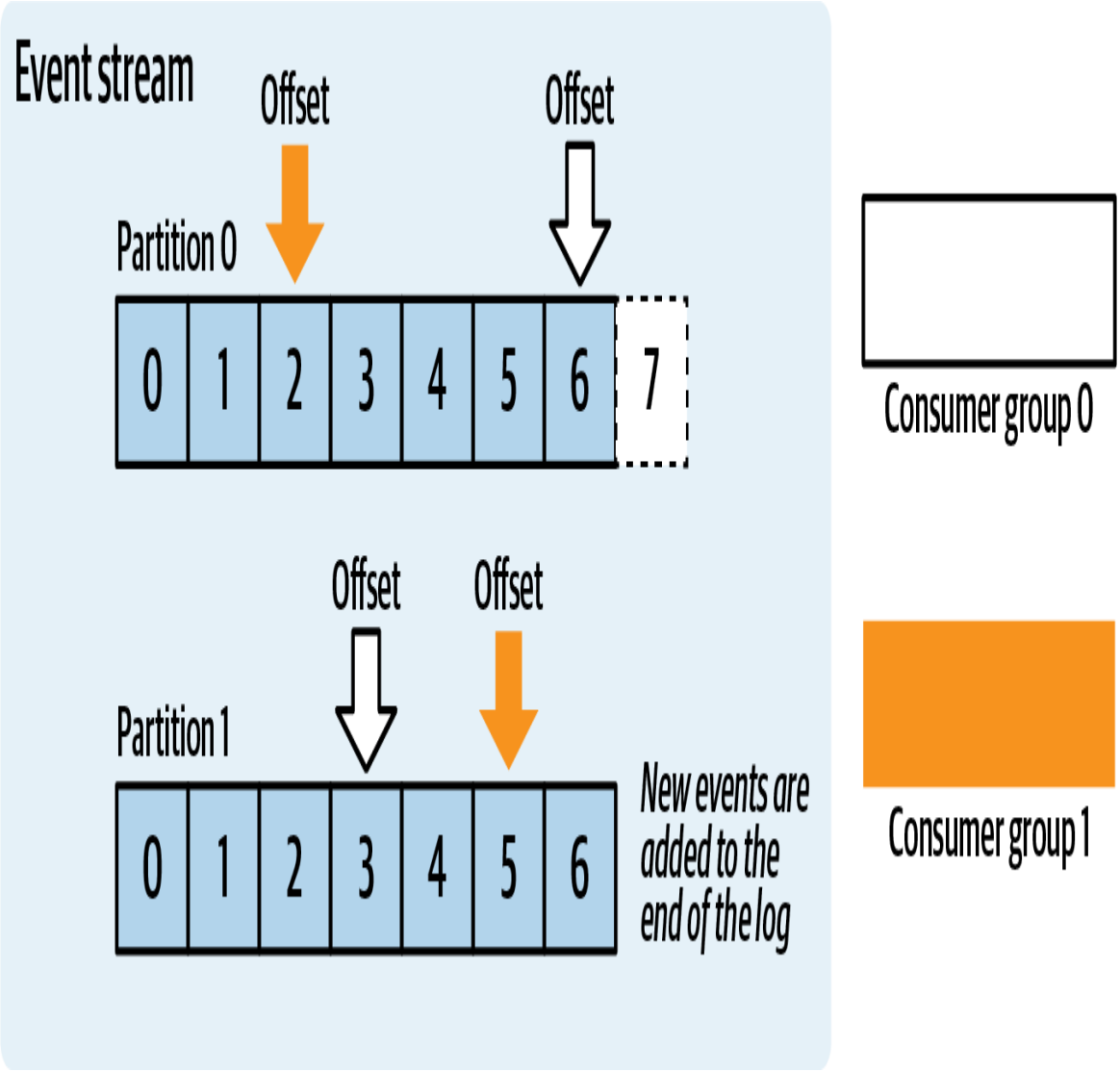


Figure 2-2. Consumer groups and their per-partition offsets

The *consumer group* allows for multiple consumers to be viewed as the same logical entity and enables horizontal scaling of message consumption.

When a new consumer instance joins a consumer group, partitions are typically automatically redistributed among the active instances. This balances the load of incoming events across the active consumers. State, especially *local state*, can make this rebalancing more challenging, but this is discussed further in Chapter 7.

"So what makes up an event-driven microservice?" you may ask. Perhaps the best thing to do is just look at a few examples first, and work backwards from there. Let's take a look at a few examples of microservice implementations.

The Basic Producer/Consumer

This example is entitled a _basic producer/consumer because that's all it really is - the producer produces events, the consumer consumes events, and you, the software dev, must write all the operations to stitch together the business logic. Chapter 9, will go deeper into this process, but for now it's enough to just start with a basic Python microservice example.

This service consumes events from a Kafka topic (event stream) named `input_topic`, tallies a sum, and emits the updated total to a new event stream if the value exceeds `1000`.

The application creates the `KafkaConsumer` (1) and `KafkaProducer` (2). The aptly named `KafkaConsumer` will read events from the specified topic, convert the events from serialized bytes to objects that application understands - in this case, plain text keys and JSON value. For a refresher on keys and values, you can look back to "The Structure of an Event".

```
from kafka import KafkaConsumer, KafkaProducer
import json
import time

# 1) Initialize Kafka consumer
consumer = KafkaConsumer(
    'input_topic_name',
    bootstrap_servers=['localhost:9092'],
    value_deserializer=lambda x: json.loads(x.decode('utf-8')),
    key_deserializer=lambda x: x.decode('utf-8'),
```

```
    group_id='ch03_python_example_consumer_group_name',
    auto_offset_reset='earliest'
)

# 2) Initialize Kafka producer
producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=lambda x: json.dumps(x).encode('utf-8'),
    key_serializer=lambda x: x.encode('utf-8')
)

# 3) The state store is a simple dictionary
key_sums = {}

# 4) Polling loop
while True:
    # 5 Poll for new events
    event_batch = consumer.poll(timeout_ms=1000)

    # 6 Process each partition's events
    for partition_batch in event_batch.values():
        for event in partition_batch:
            key = event.key
            number = event.value['number']

            # 7) Update sum for this key
            if key not in key_sums:
                key_sums[key] = 0
            key_sums[key] += number

            # 8) Check if sum exceeds 1000
            if key_sums[key] > 1000:
                # Prepare and send new event
                output_event = {
                    'key': key,
                    'total_sum': key_sums[key]
                }
                # 9) Write to topic named "key_sums"
                producer.send('key_sums',
                              key=key,
                              value=output_event)
                # 10) Flushes the events to the broker
                producer.flush()
```

Next, the code creates a simple state store (3). You will most likely want to use something other than an in-memory dictionary, but this is a simple

example and you could swap this for a RDBMS, a fully managed key-value store, or some other durable state store.

> **TIP**
>
> Choose the state store that's most suitable for your microservice's use case. Some are best served with high performance key-value stores, while other use cases are best served via RDBMS, graph, or document, for example.

The fourth step is to enter an endless loop (4) that polls the input topic for a batch of events (5) and processes each event on a per-partition basis (6). For each event, the business logic updates the key sum (7), and if it's > 1000 (8), then it creates an event with the sum to write to the output topic `key_sums` (9 and 10).

This is a very simple application. But it showcases the key components common to the vast majority of event-driven microservices: event stream consumers, producers, state stores, and a continual processing loop driven by the arrival of new events on the input streams. The business logic is embedded within the processing loop, and though this example is very simple, there are far more powerful and complex operations that we can perform.

Let's take a look at a few more examples.

The Stream-Processing Event-Driven Microservice

The stream-processing event-driven microservice is built using a stream-processing framework. Popular examples include Apache Kafka Streams, Apache Flink, and Apache Spark, just to name a few of the top contenders. Some older examples that I cited in the first edition of this book aren't as common anymore, and included Apache Storm and Apache Samza.

The streaming frameworks provide a tighter integration with the event broker, and reduce the amount of overhead you have to deal with when building your EDM. Additionally, they tend to provide more powerful

computations with higher level constructs, such as letting you `join` streams and materialized tables with very little effort on your part.

A key differentiator of stream-processing frameworks from the basic producer/consumer is that stream-processors *require* either a separate standalone cluster (e.g. Flink and Spark), or a tight implementation with a *specific* event broker (e.g. Kafka Streams).

Flink and Spark (and others like it) use their own proprietary processing clusters to manage state, scaling, durability, and the routing of data internally. Kafka Streams, on the other hand, relies on just the Kafka cluster to store their own durable state, provide topic repartitioning, and provide application scaling functionality.

> **TIP**
>
> Stream-processing frameworks tend to use MapReduce programming (a form of functional programming), where you declare the transforms for the data using functions like `map`, `reduce`, `aggregate`, `join`, and `filter`.

Chapter 10 and Chapter 11 cover both these types of frameworks in more detail. For now, let's turn to a practical (and concatenated) example using Apache Flink. This example uses Java, as it's one of the major languages supported by the Flink framework.

```
// 1) TableEnvironment manages access and usage of the tables in the Flink
application
TableEnvironment tableEnv =
  TableEnvironment.create(EnvironmentSettings.inStreamingMode());

// 2) Create the output table for joined results.
//    Connects to a Kafka broker at localhost:9092
tableEnv.executeSql(
    "CREATE TABLE PaymentsInnerJoinOrders (" +
    "   order_id STRING," +
    "   /* Add all columns you want to output here */" +
    ") WITH (" +
    "   'connector' = 'kafka'," +
    "   'topic' = 'PaymentsInnerJoinOrders'," +
```

```
    "   'properties.bootstrap.servers' = 'localhost:9092',"
    ")"
);
//Other table definitions removed for brevity. They are very similar.

// 3) Reference the Sales and Payment tables
//   (assumes they have already been declared)
Table sales = tableEnv.from("Sales");
Table payments = tableEnv.from("Payments");

// 4) Perform inner join between Sales and Payments tables on order_id
Table result = sales
    .filter { sale -> sale.total >= 100 }
    .join(payments)
    .where($("Sales.order_id")
    .isEqual($("Payment.order_id")));
     //5) Declare your output schema in the JoinFunction
    .apply(new JoinFunction<> ...)

// 6) Insert the joined results into the output Kafka topic
result.executeInsert("PaymentsInnerJoinOrders");
```

This is a fairly terse application, but it packs a lot of power. First (1), we create the `tableEnv` connection that will manage our table environment work. Next, we declare the output table (2), which is connected to the Kafka topic `PaymentsInnerJoinOrders`. You'll only have to create this once. I've omitted the other declarations for brevity.

Third (3), you create references to the `Sales` and `Payments` tables so that you can use them in your application. These tables are *materialized streams*, as covered in "Materializing State from Entity Events".

Fourth (4), the code *declares the transformations* to perform on the events populating the materialized tables. Note that the first transformation that this application performs is to `filter` *in* any records that have a `sales.total >= 100`. Anything with `sales.total < 100` will be filtered *out* and discarded from the rest of the declared transformations and joins.

These few lines perform a *lot* of important work. Not only does the code perform the operations of joining these two tables together, but it also handles fault-tolerance, load balancing, in-order processing, and big-data scale loads. It can also easily join streams and tables that are partitioned and

keyed completely differently,something that you are likely to encounter on your own.

Fifth (5), I have left a placeholder where you would add in your logic to merge the matching `Sales` and `Payment` events together. It's not particularly difficult to write, but it would take up a lot of space and would be predominantly boilerplate code. The important part is that this function is where you declare your output event schema, and is part of the data contract discussed in the next chapter.

Sixth (6) and finally, the application writes the data to the output table. The machinations of the Flink streaming framework will handle the rest for committing the data to the Kafka topic.

Streaming frameworks provide very powerful capabilities, but typically require a larger upfront investment into supporting architecture. They also typically only provide limited language support (Python and JVM being the most popular), though some have added support for other languages since the first edition of this book was published. In fact, SQL (or SQL-like) languages have been the fastest growing of the bunch. Let's take a look at those next.

The Streaming SQL Query

Should my microservice actually just be a SQL query?

It's a good question, particularly with the rise of streaming SQL as managed services. The very same things that make SQL popular in the database world make it popular in the streaming world. Its declarative nature means that you just declare the results you're looking for. You don't have to specify *how* to get it done, as that's the job of the underlying processing engine.

Additionally, SQL lets you write very clear and concise statements about what it is you want to do with your event streams. Example 2-1 shows the very same Flink application, but written using Flink SQL:

Example 2-1.

```
--Assumes we have already declared the materialized
--input tables and the output table
```

```
INSERT INTO PaymentsInnerJoinOrders
SELECT *
FROM Sales s
INNER JOIN Payment p
ON s.order_id = p.order_id;
```

That's it. Simple but powerful. There are more nuances that you may need to consider, but that's covered in more detail in Chapter 12.

Streaming SQL queries like this one result in long-running processes that continually execute the business logic, just like any other microservice. It'll run indefinitely, scaling up and down as needed.

Streaming SQL is not standardized, and it depends heavily on the frameworks you choose to build your microservices. It requires a robust lower-level set of APIs to function beyond just toy examples. The Flink project, for example in Figure 2-3, has several layers of APIs, each which depends on the ones below it

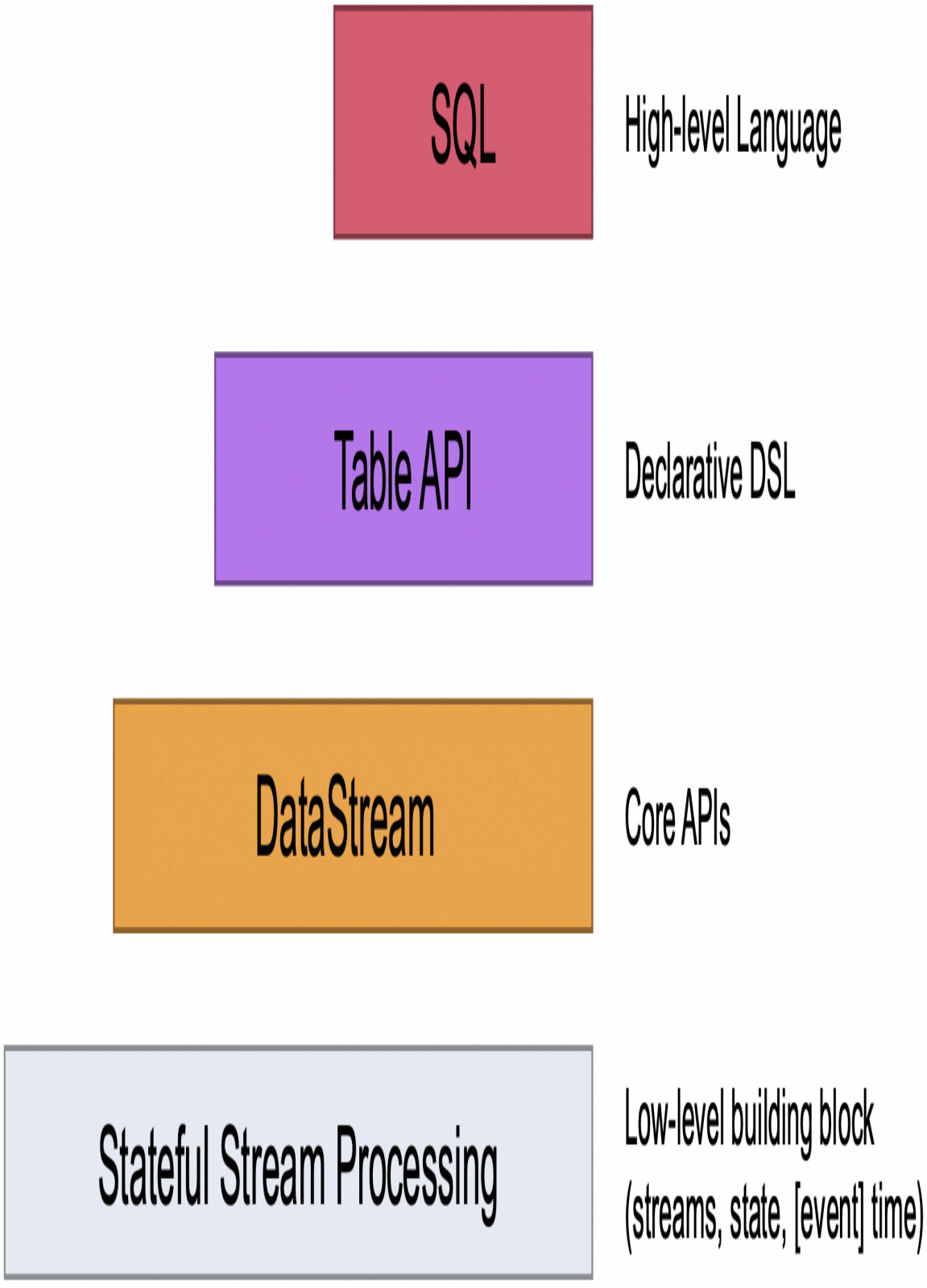


Figure 2-3. The four levels of Flink APIs - Source - apache.org

> **TIP**
>
> Keep on the lookout for opportunities to use streaming SQL within your event-driven architectures. It can save you a ton of time and effort, and let you get on with other work.

There are many different flavors and types, and so you'll have to do your own due diligence to find out what is and what isn't supported per streaming framework. Streaming SQL is covered in more detail in Chapter 12.

The Legacy Application

Legacy applications typically aren't written with event-driven processing in mind. They're usually old but important systems that serve critical business functions, but that aren't under active development anymore. Changes to these services are rare, and are only performed when absolutely necessary. They're also often the gatekeepers to important business data, siloed away inside the database or file stores of the system.

The legacy application is basically a rigid structure that you're unlikely to be able to change. But you *can* still integrate it into your overarching event-driven architecture through the use of *connectors*, as shown in Figure 2-4.

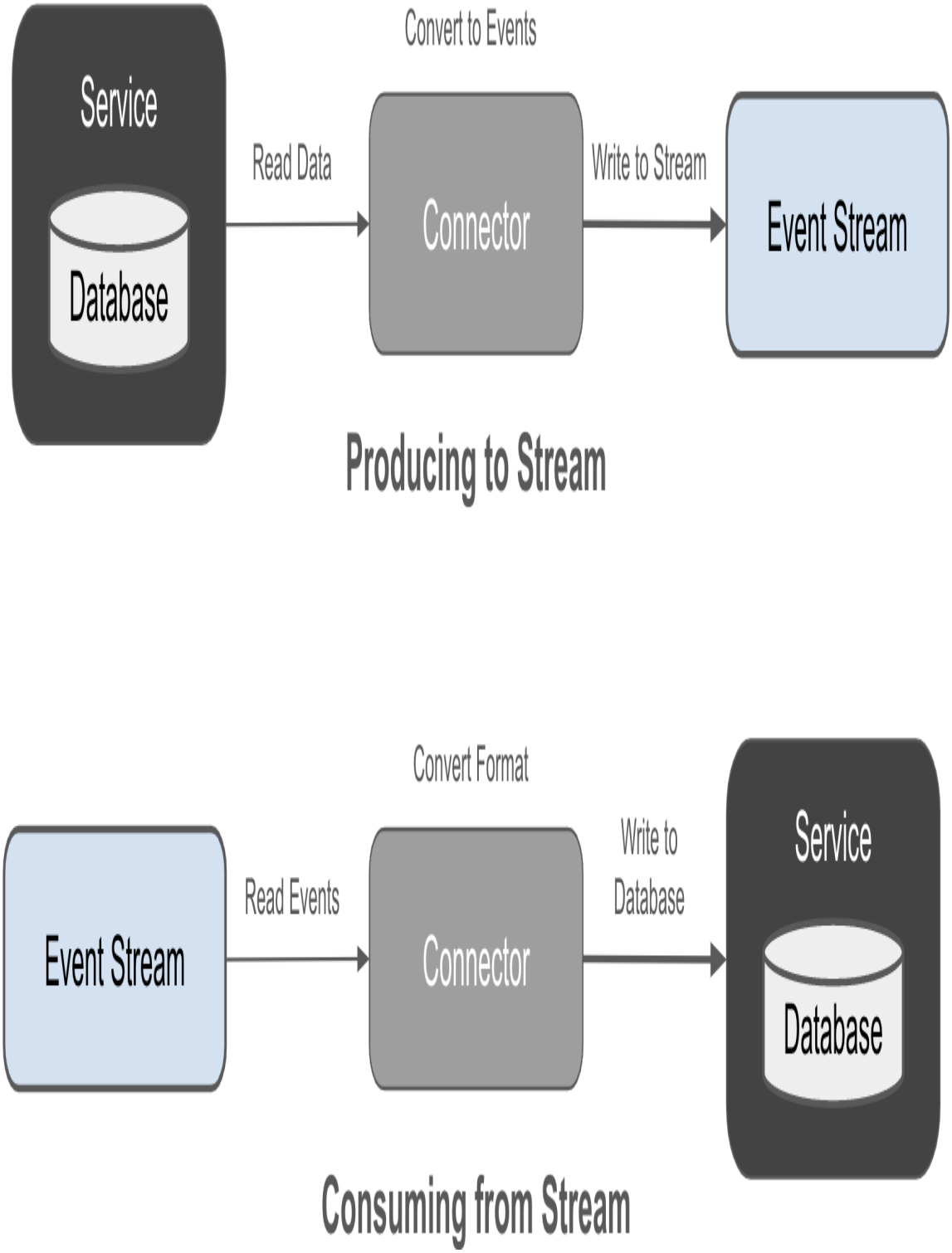


Figure 2-4. Producing and consuming event streams with connectors

Connectors can read events from a source system or database, convert the data to events, and write it into an event stream. Similarly, connectors can also read events from a stream, convert them into a suitable format, and write them to a legacy system's API or database. They provide the means for integrating these existing applications into your event-driven architecture without having to redesign the whole system as a native event-driven architecture.

Connectors enable you to get started with event streams without having to reinvent your entire architecture. They make it easy to get data into streams, so that you can start getting value from your event-driven microservices as soon as possible. Chapter 5 covers connectors in more detail.

Event-Driven Microservice Responsibilities

All microservices share a common set of responsibilities, independent of how you build them and the technologies you use. You'll need to account for these responsibilities when you build your own services:

Service boundaries and scope

> A well-defined set of boundaries. What is this application responsible for? And what is it not? The latter becomes more important when you have multiple microservices working together to fulfil a more complicated workflow, where it can be a bit difficult to discern the responsibilities of each service. By mapping microservices as cleanly as possible to bounded contexts, we can avoid much of the guesswork and ambiguity that may otherwise crop up.

Scalability

> The microservice is responsible for ensuring its own scalability. Specifically, it must be written in such a way that allows it to scale horizontally (more instances) or vertically (a more powerful instance), depending on its requirements.

EDM frameworks that provide scaling out of the box tend to have far greater appeal due to the seamless built-in scaling capabilities. The underlying processing power that runs the microserivce, however, can either be provided independently on a per-microservice basis, or via a shared pool of compute resources. "Managing Microservices at Scale" introduces some options for scaling microservices.

State management

The microservice is solely responsible for the creates, reads, updates, and deletes made to its data store. Any operations that modify the state in the microservice remain entirely within its boundary of control. Any problems with the state store, such as running out of disk or failing to test applications changes also fall within the microservice's problem space. Chapter 7 goes into greater detail on how to build and manage state for event-driven microservices.

Keep track of stream progress

Each microservice must keep track of its progress in reading the input event streams. For example, Apache Kafka tracks this progress using a `consumer group`, one per logical microservice. Consumer groups are also used by many other leading event brokers, though a consumer can still choose to manually manipulate its own offsets (say to replay some records) or store them elsewhere, such as in their own data store.

Failure recovery

Microservices are responsible for ensuring that they can get themselves back to a healthy state after a failure. The application must take into consideration its stored state in relation to its stream input progress. In the case of a crash,

the consuming service will resume from its last known good offsets, which could mean duplicate processing and duplicate state store updates. Database snapshots, restoration from event streams, and replaying data are all commonly used state preservation and restoration techniques, which are explored further in Chapter 7.

The single writer per stream principle

Each event stream has one and only one producing microservice. This microservice is the owner of each event produced to that stream. This allows for the authoritative source of truth to always be known for any given event, by permitting the tracing of data lineage through the system. Access control mechanisms, as discussed in [[Link to Come]], should be used to enforce ownership and write boundaries.

Event keys and partitioning

The microservice is responsible for choosing the output record's primary key (see "The Structure of an Event"), and is also responsible for determining which partition to send that record to. Records of the same key typically go to the same partition, though you can choose other strategies (e.g. round-robin, random, or custom). Ultimately, it is your microservice that is responsible for selecting which partition to write the event to.

Event schemas and data contracts

Just as it's your microservice's choice for choosing the record key, it's also its choice for choosing the schema format for writing the event. For example, do you write `product_id` as a `string`? Or do you write it as a `Integer`? Chapter Chapter 3 covers this in far more detail, but for now, plan to use a well-defined Avro, Protobuf, or JSON schema to write your

> records. It will make your event streams much easier to use, provide clarity for your consumers, and enable a much healthier event-driven ecosystem.

A microservice should also be *reasonably sized*. Does that sound ambiguous to you? Well, read on.

How Small Should a Microservice Be?

First up, the goal of a microservice architecture isn't to make as many as possible. You won't win any awards for having the highest service count, nor would you even find the experience rewarding. In fact, you'd probably end up writing a blog about how you made a 1000 microservices and everything was awful.

The reality is that every service you build, micro or otherwise, incurs overhead. However, we also want to avoid the pains that come with a single monolith. Where's the balance?

A microservice should be manageable by a single team, and the team should be able to have others within it work on the service as needed. How big is that team? The two-pizza team is a popular measurement, though it can be a bit of an unreliable guide depending on how hungry your developers are.

A single developer on this team should be able to fit the space of duties that the service has in their own head. A new developer coming to the microservice should be able to figure out the duties of that service, and how it accomplishes those duties, in just a day or two.

Ultimately, the real concern is that the service *serves a particular bounded context* and satisfies a business need. The size, or how *micro* it really is, comes as a secondary concern. You may end up with microservices that push the upper boundary of what I've prescribed here, and that's okay - the important part is that you can identify the specific business concern it meets, the boundaries of the service, and be able to maintain a good mental list of what the service *should* do and *should not*.

Here are a few quick tips:

- Only build as many services as are necessary. More is not better.
- One person should be able to understand the whole service.
- Your team will need to be responsible for owning and maintaining the service and the output event streams.
- Look to add functionality to an existing service first. It reduces your per-application overhead, as you'll see next in the next section.
- For the microservices you do build, focus on building *modular components*. You may find that as your business develops that you need to break off a module to convert to its own microservice.

In the next section we'll take a look at how to manage microservices, including how to manage them at scale.

Managing Microservices at Scale

Managing microservices can become increasingly difficult as the quantity of services grows. Each microservice requires specific compute resources, data stores, configurations, environment variables, and a whole host of other microservice-specific properties. Each microservice must also be manageable and deployable by the team that owns it, which requires streamlining developer operations (DevOps) to reduce overhead and complexity.

Containerization, container management, and infrastructure as code (IaC) have transformed how applications are deployed, monitored, and scaled. Docker, Kubernetes, and Terraform are popular examples of technology choices and each plays a crucial role in modern DevOps practices.

Containerization provides isolation of dependencies of the application, allowing you to create custom environments to suit your microservice's needs. Containers ensure consistency across development, testing, and production environments, reducing dependency conflicts and simplifying the

complexity of managing different environments. The lightweight nature of containers, as compared to traditional virtual machines, allows for more efficient use of underlying resources, rapid startup times, and isolated environments for each application component.

[[Link to Come]] takes a closer look at how container management services and infrastructure as code contribute to microservice management. For now, keep in mind that you can isolate your microservices via containerization, and by using virtual machines. Let's look into each in more detail.

Putting Microservices into Containers

Containers revolutionized software deployment by allowing the packaging of an application along with its dependencies into an isolatable logical container. Containers leverage the host operating system (OS) via a shared kernel model. This provides basic separation between containers, while the container itself isolates environment variables, libraries, and other dependencies. Containers provide most of the benefits of a virtual machine (covered next) at a fraction of the cost, with fast startup times and low resource overhead.

Containers' shared operating system approach does have some tradeoffs. Containerized applications must be able to run on the host's OS. If an application requires a specialized OS, then an independent host will need to be set up. Security is also a major concern, since containers share access to the host machine's OS. A vulnerability in the kernel can put all the containers on that host at risk. With friendly workloads this is unlikely to be a problem, but popular shared tenancy models in cloud computing make it a very important consideration.

Putting Microservices into Virtual Machines

Virtual machines (VMs) address some of the shortcomings of containers, though they aren't nearly as common for microservice deployments. Traditional VMs provide full isolation with a self-contained OS and virtualized hardware specified for each instance. Although this alternative

provides higher security than containers, it has historically been much more expensive. Each VM has higher overhead costs compared to containers, with slower startup times and larger system footprints.

> **TIP**
>
> Google's gVisor, Amazon's Firecracker, and Kata Containers, are examples of popular frameworks for running lightweight virtual machines.

MicroVMs are much more similar to containers in their operations and handling. They have very low memory and CPU overhead and start much faster (less than a second) than classic VMs. They provide hardware-level isolation for multi-tenant services, resulting in a more secure and trustworthy ecosystem.

Yet while options and frameworks for running microservices have improved in the five years since the first edition of this book, the reality is that VMs, micro or otherwise, are still relatively rare in comparison to containers. Container isolation has improved and security risks have decreased through iterative improvements to container management systems and the host technologies themselves. While virtual machines remain an option for running your microservice, it is unlikely to be your first choice unless you have some very strict. Time will tell if microVMs come to replace containers, if containers adopt all the features of microVMs, or if the two continue to coexist as a duality for hosting services.

Managing Containers and Virtual Machines

Containers and VMs are managed through a variety of purpose-built software known as *container management systems* (CMS). These control container deployment, resource allocation, and integration with the underlying compute resources. Popular and commonly used CMSes include Kubernetes, Docker Engine, Mesos Marathon, Amazon ECS, and Nomad.

Microservices must be able to scale up and down depending on changing workloads, service-level agreements (SLAs), and performance requirements. Vertical scaling must be supported, in which compute resources such as CPU, memory, and disk are increased or decreased on each microservice instance. Horizontal scaling must also be supported, with new instances added or removed.

Each microservice should be deployed as a single unit. For many microservices, a single executable is all that is needed to perform its business requirements, and it can be deployed within a single container. Other microservices may be more complex, requiring coordination between multiple containers and external data stores. This is where something like Kubernetes's pod concept comes into play, allowing for multiple containers to be deployed and reverted as a single action. Kubernetes also allows for single-run operations; for example, database migrations can be run during the execution of the single deployable.

VM management is supported by a number of implementations, but is currently more limited than container management. Kubernetes and Docker Engine support Google's gVisor and Kata Containers, while Amazon's platform supports AWS Firecracker. The lines between containers and VMs will continue to blur as development continues. Make sure that the CMS you select will handle the containers and VMs that you require of it.

> **TIP**
>
> There are rich sets of resources available for Kubernetes, Docker, Mesos, Amazon ECS, and Nomad. The information they provide goes far beyond what I can present here. I encourage you to look into these materials for more information.

Paying the Microservice Tax

The *microservice tax* is the sum of costs, including financial, manpower, and opportunity, associated with implementing the tools, platforms, and components of a microservice architecture.

The tax includes the costs of managing, deploying, and operating the event broker, CMS, deployment pipelines, monitoring solutions, and logging services. These expenses are unavoidable and are paid either centrally by the organization or independently by each team implementing microservices. The former results in a scalable, simplified, and unified framework for developing microservices, while the latter results in excessive overhead, duplicate solutions, fragmented tooling, and unsustainable growth.

Other costs include reduced productivity while developers learn new skills to build, manage, and maintain microservices. Learning how to manage asynchrony, recover from failures, and service modularization will all take time, especially for junior developers who may have little experience in event-driven architectures. That said, there exists a wealth of blogs, videos, books, articles, how-to guides, interactive development environment plugins, and the emergence of effective AI copilots that make it easier to adopt event-driven architectures.

Paying the microservice tax is not a trivial matter, and it is one of the largest impediments to getting started with event-driven microservices. Small organizations would do well to stick with an architecture that better suits their business needs, such as a modular monolith, and only expand into microservices once their business runs into scaling and growth issues.

The good news is that the tax is not an all-or-nothing thing. You can invest in your event-driven microservices platform incrementally, adding new functions and features over time and as necessity demands.

Fortunately, self-hosted, hosted, and fully managed services are available for you to choose from. The microservice tax is being steadily reduced with new integrations between CMSes, event brokers, and other commonly needed tools.

Cloud services are worth an explicit mention for greatly reducing the cost of the microservices tax. Cloud services and fully managed offerings have significantly improved since the first version of this book was released in 2020, and their progressive uptake shows no sign of abating. For example, you can easily find fully managed CMSes, event brokers, monitoring

solutions, data catalogs, and integration/deployment pipeline services without having to run any of them yourself. It is much easier to get started with using cloud services for your microservice architecture than building your own, as it lets you experiment and trial your services before committing significant resources.

Service Contracts

Service contracts are an important component for establishing clear and concise microservice boundaries. They're also critical for formal documentation, API management, and establishing exactly which services you have access to, and which ones you don't.

While there are many ways to implement service contracts, specifications like OpenAPI or implementations like Swagger tend to be the most popular choices.

Service contracts establish:

Service APIs

> The API calls that the service supports, including name, required and optional parameters, return types, and protocols supported.

Code Generation

> Create stubbed out functions based on the specification. Then, edit them to contain the business logic necessary to fulfil the service contract.

Documentation

> You can also generate documentation from the specification, which is extremely helpful in saving you the time and effort of manually updating it. By generating the code and documentation together, you can ensure that the docs will match the actual available function calls.

Namespacing

> You can specify the namespace of the service, providing organizational information and removing ambiguity for similarly named services.

Versioning

> The version of the service contract, along with a history of contract changes.

Ownership

> Specify the team or person who owns the service.

Service Level Objectives (or Agreements)

> The owner is responsible for ensuring that adequate service levels are kept. While a site reliability engineer (SRE) may be on the hook for keeping the service instances running, the owner is responsible for everything else.

Breaking Change Management

> Service APIs can change over time. The service contract enables you to mark APIs as deprecated, indicating to service users how to migrate off the soon-to-be deleted APIs.

Event-driven microservices are a bit different than the request-response microservices communicating via direct API calls. Many of the service contract notions may not apply at all, as communication occurs via event streams and not point-to-point. The next chapter covers the use of *data contracts* for event streams, which, while similar to service contracts, differ in a few key ways.

That being said, microservices that provide a mixture of request-response based APIs and event stream data will require both service contracts and data contracts.

Summary

Event-driven microservices are applications like any other, relying on incoming events to drive their business logic. They can be written in many different languages, though the functionality you have available to you will vary accordingly. Basic producer/consumer microservices may be written in a wide range of languages, while purpose-built streaming frameworks may only support one or two languages. SQL queries and connectors also remain options, but their use requires further integration with application source code than what was covered in this chapter.

Microservices have a host of responsibilities, including managing their own code, state, and runtime properties. The service creators have the freedom to choose the technologies best suited to solving the business problem, though they of course need to ensure that the service can be maintained, updated, and debugged by others. Microservices are typically deployed in containers or virtual machines, relying on container/VM management systems to help monitor and manage their life cycles.

Finally, consider the microservice tax. It is the total sum of non-business work you need to do to make using microservices a reasonable choice. There isn't a clear recipe for what *is* and *isn't* in the microservice tax. Every organization has a different technology stack, and what could be good advice for one organization may be a poor choice for another. That being said, try to use what you already have before buying something new, and work incrementally. It's not an all or nothing thing. Work on getting your first microservice up and running, learn from the process, then iterate and improve from there.

Part II. Events and Event Streams

Chapter 3. Schemas and Data Contracts

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 4th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

NOTE

An earlier version of this chapter previously appeared in Building an Event-Driven Data Mesh.

Schemas are essential for getting an event-driven architecture off the ground. A schema is an explicit declaration of the data structures, names, types, defaults, and limitations. They facilitate a common understanding of the data for both producers and consumers alike. Schemas eliminate ambiguity, support both discovery and self-service, and significantly reduce parsing and interpretation errors.

Data contracts, which are covered in more detail in the second half of this chapter, rely on schemas as a fundamental component. They also incorporate

further aspects, such as ownership, service-level agreements, encryption, data evolution rules, and more.

> **WARNING**
>
> Use schemas! While technically optional, they're effectively essential for making event-driven architectures that actually work.

This chapter is a *prescriptive and opinionated* look at schemas and data contracts in an event-driven architecture, and how to set yourself up for success. There are many different schema technologies and many different ways to communicate data between systems through events. However, some methods and technologies are better than others—they're more common, they're more flexible, and they also reflect the ways most businesses use and communicate events.

Before we get too far into it, let's step back and take a look at how a producer creates, serializes, and transmits an event across a network into an event stream. We'll also take a look at the reverse of this process, where a consumer consumes, deserializes, and processes an event from an event stream.

A Brief Introduction to Serialization and Deserialization

Serialization is the process of converting an event object in the application's memory space into a sequence of bytes. The sequence of bytes is then sent across the network and written into the event stream. The *schema* provides the specification for converting the data from the application object space into the byte-based representation. For example, a schema that specifies that the field named `length` must be an `Integer` will throw an exception when attempting to serialize with `length` set to a `String` value of `"six feet"`.

Figure 3-1 shows a producer converting an object into a sequence of bytes that is then written to an event stream. Note that the schema is also attached to the serialized event, written together into the event stream. The consumer relies on the schema to deserialize the event from a sequence of bytes back into an object or struct that the consumer understands. There are some tricks we can use to avoid sending a schema with every event, which we'll look at later in this chapter.

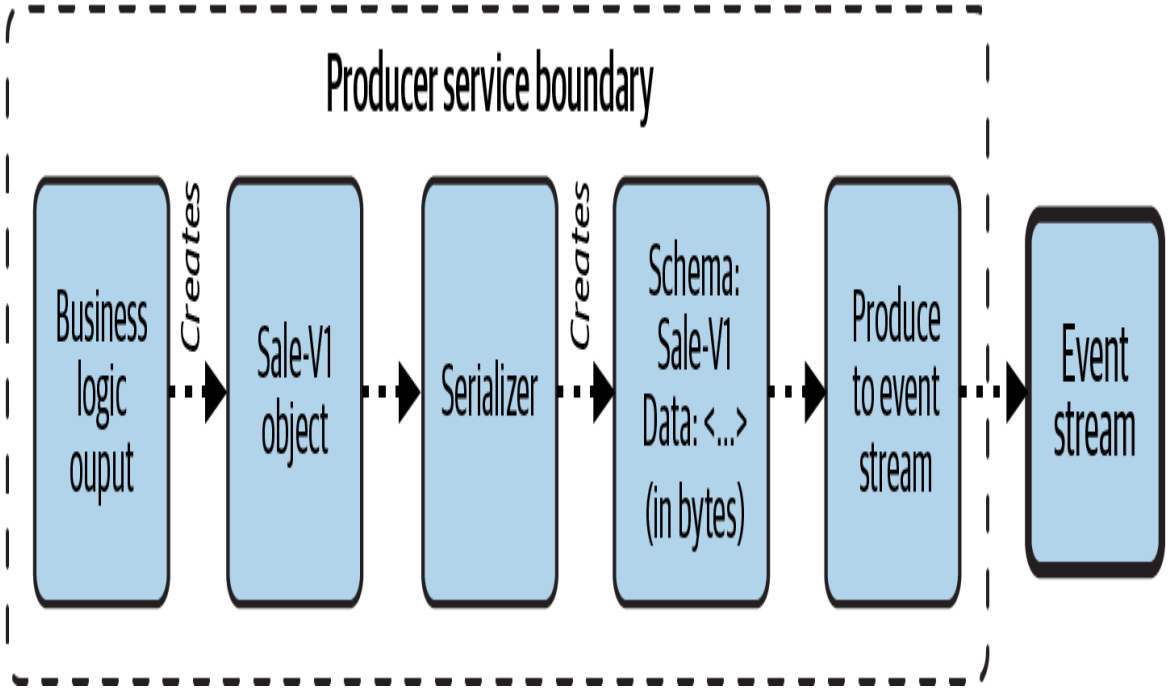


Figure 3-1. Producer serializing the event object into bytes and writing it to the event stream

On the other end of the event stream, a consumer reverses this process and deserializes the byte sequence into an object. It requires the schema to convert the sequence of bytes into an object that is legible by the consumer process. Figure 3-2 illustrates the process of consuming, deserializing, and converting the data into a representation that can be processed by the consumer's business logic code.

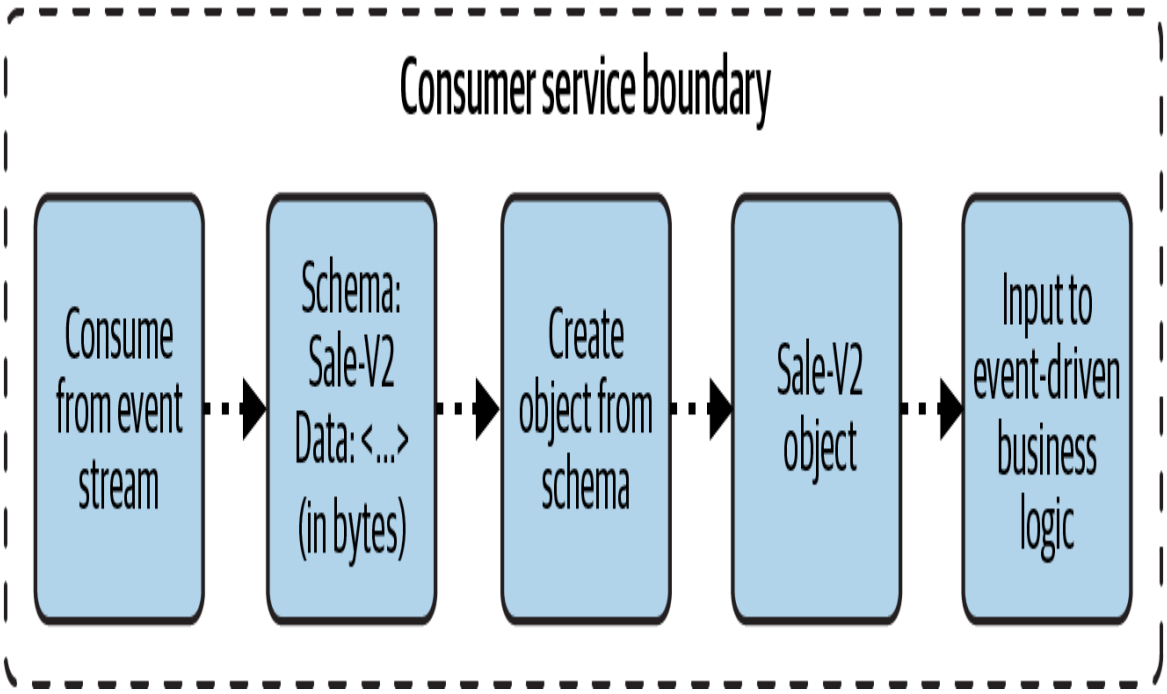


Figure 3-2. A consumer reading from the event stream and deserializing the bytes back into an event representation

What is a Schema?

Schemas provide structure and definition. They're synonymous with the definition of a database table: at a minimum, they specify names, types, restrictions, and default values. Again, to re-emphasize, *schemas ensure that both the event data producer and all of its consumers have a shared common understanding of the data*.

A failure to use schemas leaves data interpretation up to your consumers. If you only have one consumer, then it *may* work (though it is NOT advisable). In reality, the most important data in your organization will have many consumers, all across different business units, each using their own preferred programming languages and databases. Figure 3-3 shows just two streams consumed by five services, resulting in ten unique interpretations of data.

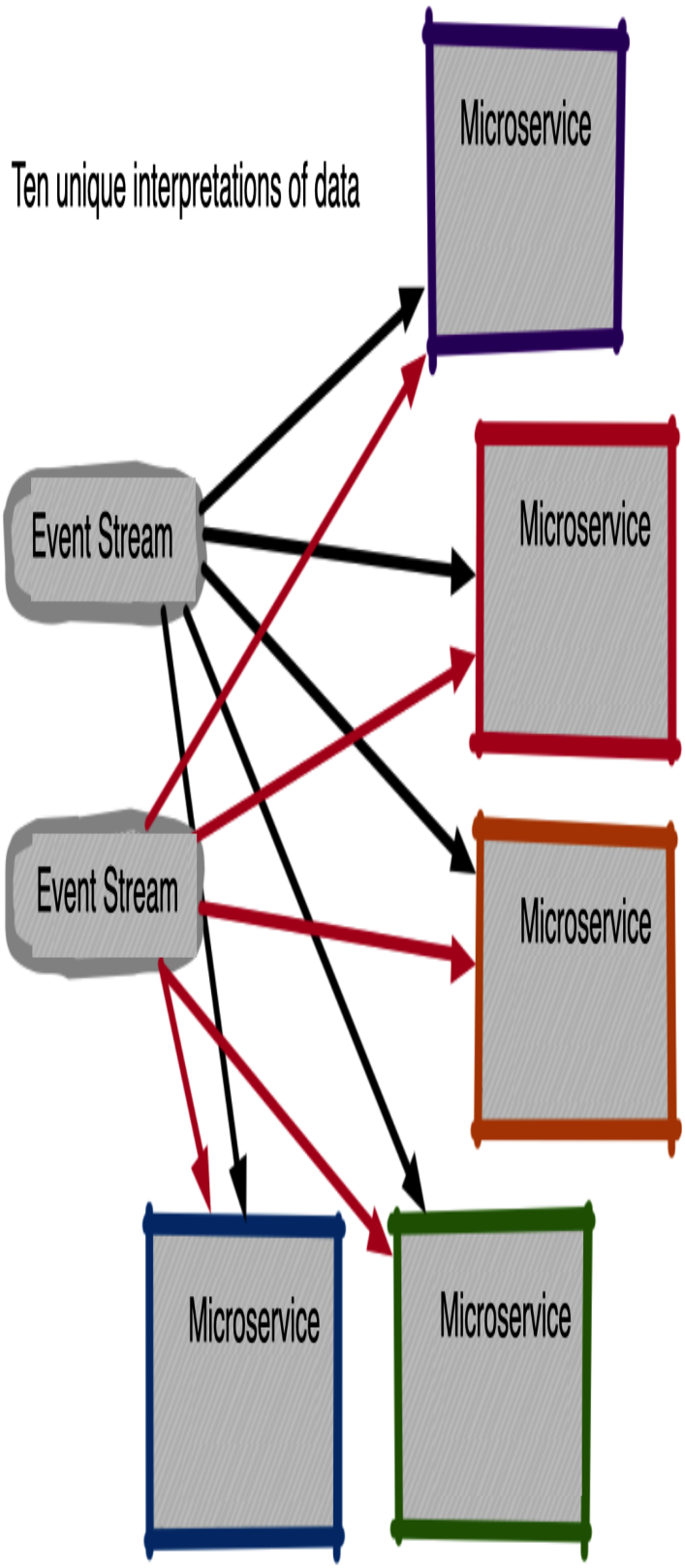

Ten unique interpretations of data
Microservice
Event Stream
Microservice
Event Stream
Microservice
Microservice
Microservice

Figure 3-3. Two event streams with five consumers will have ten unique interpretations of the data

The reality is that the less structure your data has, the more likely your consumers are to misinterpret it. They will get divergent results, their data will not match those of their peers, and you will find yourself fighting a losing battle against easily preventable mistakes. It is essential that you use schemas.

Here's a simple Protobuf-powered example to illustrate some of the more compelling benefits. Example 3-1 showcases a schema for the `Person` entity.

Example 3-1. `Person` schema with Protobuf.

```
message Person {
  //The person's unique ID.
  //The field number = 1, indicating its position in the wire format.
  //You cannot change the field number once it is established.
  int32 id = 1;
  //The person's full legal name
  string name = 2;
  //Measured in centimeters, rounded to the nearest centimeter
  int32 height = 3;

  enum CountryCode {
    ABW = 0;
    AFG = 1;
    ...
    ZWE = 248;
  }
  //ISO3166-1-alpha-3 standard. AAA=OTHER
  CountryCode country = 4;
}
```

The event schema details the `id`, `name`, and `height` of a person, along with their ISO 3166 three-letter Latin-script `country` code.The producer microservice adheres to this format when creating `Person` events, while the consumer expects that all the `Person` event stream data is formatted in this way. There is no need to interpret or guess at what the data is supposed to mean, though there are still places where errors can creep in. For example, a well-meaning developer may accidentally record the height as inches instead of centimeters.

Figure 3-4 shows two clients: a Java producer that writes to the event stream and a C++ consumer that reads the events out of the stream and into its own memory space. The producer can use the Protobuf code generator for Java to automatically create classes and client code for the `Person` schema. The consumer can similarly generate its own structs and client code using the C++ code generator.

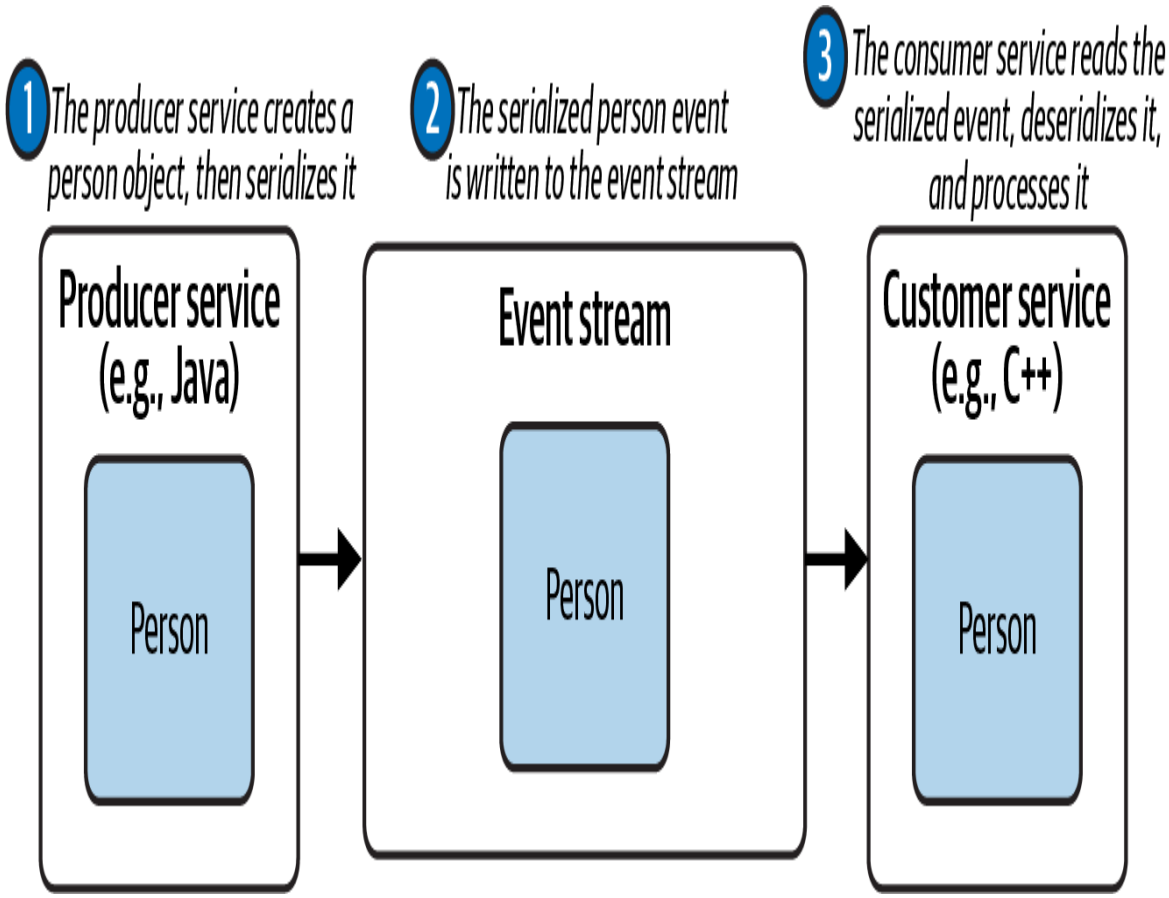


Figure 3-4. The producer to event stream to consumer workflow of an event serialized with a schema

> **NOTE**
>
> Both Avro and Protobuf have many code generators that support a wide variety of languages. You can find support and code generators for Rust, Go, Java, Kotlin, Python, C++, C#, Dart, and Javascript, just to name a few.

Example 3-2 shows a producer Java client with which we create a `Person` named "Jerimiah Wasabullfrog" and the associated properties. Next, we serialize the object and write it to a file named *ProtoPerson.data* (we're using a file as a placeholder for the event broker client, simply to avoid bogging this example down with the proper event-broker client produce, retry, and error-handling code).

Example 3-2. Using Java to populate a `Person` object, generated from the Protobuf schema

```
Person jeremiah = Person.newBuilder()
    .setId(8675309)
    .setName("Jerimiah Wasabullfrog")
    .setHeight(167)
    .setCountryCode("CAD")
    .build();
output = new FileOutputStream("ProtoPerson.data");
jeremiah.writeTo(output);
```

The consumer reads the `Person` data from the file (i.e., the event), deserializes it using the schema, and finally converts it into an object or structure native to the consumer's language. Example 3-3 shows a C++ consumer using the `ParseFromIstream` function to convert the serialized bytes into a well-structured `Person` class object.

Example 3-3. Using C++ to parse the serialized Protobuf `Person` object

```
Person jeremiah;
fstream input(argv[1],
    ios::in | ios::binary);
jeremiah.ParseFromIstream(&input);

//jeremiah is now populated with the id, name, and height received in the event
id = jeremiah.id();
name = jeremiah.name();
height = jeremiah.height();
cc = jeremiah.countryCode();
```

Schemas provide structure to data and enable easy serialization and deserialization. Other main benefits include:

Schemas as code

Your schemas can act as code just like any other in your application. Code reviews and commit histories make it so you can see who changed what, when, and why.

Data protection

The producer service will fail to serialize the data if it doesn't adhere to the expected schema. Instead of publishing bad data, the producer will throw an exception. While you can still *choose* to publish the malformed data, it requires a deliberate decision to circumvent the schema safeguards.

Embedded Documentation

Schemas provide inline commenting capabilities for embedding documentation. It is easy to keep the documentation up to date when it's inside the schema itself.

Provides a foundation for discussion

Schemas provide a basis for discussing (and arguing) over the content and form of the data. Explicit schemas, in conjunction with pull request reviews, enable productive discussions over what should and should not be in an event.

Code generation

Code generation lets the application operate on the events as objects in its native language, handling the parsing, conversion, and object creation for you. It greatly simplifies business logic by letting you write code against well-defined classes in your programming language instead of against a map of generic object types.

Schema evolution

Data can change over time. The best schema options provide a safe path for schema evolution, including rules,

restrictions, and safeguards that prevent you from inadvertently violating your data contract with your consumers. Schema evolution is covered in more detail later in this chapter.

Test data event generators

Event generators let you create data that matches your schemas, including specific parameter constraints such as foreign-key and primary-key relationships. You can use these events to test your event-driven service code and use it to produce sample events for consumers to try in their services. Kafka Connect Datagen is an example of such a generator, where you can specify constraints and ranges on the data being generated. Example 3-4 shows a snippet of a schema that contains a `userid`, where the range of valid data output will vary from `User_1` to `User_9`.

Example 3-4. Kafka Connect Datagen specification

```
{"name": "userid",
 "type": {
    "type": "string",
    "arg.properties": {
        "regex": "User_[1-9]"
    }
}}
```

Schemas bring significant benefits and are necessary for developing event-driven microservices at scale. The next question is: what schema technology should you use?

What Are Your Schema Technology Options?

While there are many schema techologies that you could use to build your event streams, a few stand above the rest. In terms of features, commonality, community development, ease of use, and supportive tooling. Google's Protobuf and Apache Avro are your two best choices. JSON Schema is

somewhat popular for those who favor JSON, but please do not confuse it with the anti-pattern mess that is schemaless JSON (more on this in a bit).

> **TIP**
>
> Martin Kleppman has written an excellent breakdown and analysis of JSON, XML, Protobuf, Thrift, and Avro in *Chapter 4 of _Designing Data-Intensive Applications* (O'Reilly). Consider giving it a read if you would like to learn more about how these schema technologies work under the hood.

While there are other schema technologies out there, the reality is that Avro and Protobuf still stand among the top. They were at the top when the first edition of this book was written, and they're still at the top five years later. They work, and they do their job well. JSON Schema has slow uptake, but it remains a valid third contender and behaves similarly to Avro and Protobuf.

Let's take a look at each.

Google's Protocol Buffers, aka Protobuf

Protobuf became open source in 2008 and has long been popular for its gRPC format. It has also proven to be a strong competitor to Avro for the top schema format. Here are a few notable points about Protobuf's data types and schema management:

- Protobuf supports support for scalar (e.g., double, float, bool, string, etc.) and complex data types.
- The schema is not stored as part of the event but is maintained in a separate file. Protobuf files are most commonly shared via a schema registry, as covered later in this chapter in "The Role of the Schema Registry".
- You can compose complex schemas by referring to schema declarations stored in other files.

- There is no support for dynamic types. However, this is not a significant shortcoming, as strongly-typed schemas remain your best option for defining your events.

Example 3-5 shows the `Person` object again as defined using the Proto3 version of Protobuf:

Example 3-5. Proto3 example of Person

```
message Person {
  //The person's unique ID
  int32 id = 1;
  //The person's full legal name
  string name = 2;
  //Measured in centimeters, rounded to the nearest centimeter
  int32 height = 3;

  enum CountryCode {
    ABW = 0;
    AFG = 1;
    ...
    ZWE = 248;
  }
  //ISO3166-1-alpha-3 standard. AAA=OTHER
  CountryCode country = 4;
}
```

Protobuf has multiple versions. In the first edition, Protobuf really only had v2 and v3. In 2024, Protobuf released a "protobuf editions" version, that brought inter-compatibility to v2 and v3, along with replacing the future versioning sequence. More on that in a moment.

Protobuf v2 enabled:

- Marking fields as either `required` or `optional`
- Setting custom default values.

This standard allowed you to determine if a field is included in the message or if it is missing. Additionally, you could specify default values that could be populated by the consumer in the case that the data was indeed missing. Protobuf v2 was more similar to Apache Avro (next section) than Protobuf v3 is today.

Protobuf v3 *removed* some of the functionality of v2 and replaced it with a different set of behavior:

- Fields can no longer be marked as `required`. All fields are technically optional, either explicitly (uses `optional` tag) or implicitly (no tag).
- Implicit default values can only be one of empty string, empty bytes, false, or 0. The problem is that it is not possible to tell if the default value was applied or if record was actually sent with that data (eg: `name = ""` - is this accurate data, or).

The long and the short of it is that the evolution of Protobuf from v2 to v3 had a lot of dialogue, discourse, arguments, and disagreements. The eventual design decisions led to v3 diverging from the functionality that v2 provided, resulting in an ongoing schism. Some developers prefer v2, while others prefer v3.

Protobuf editions is a rethink of the relationship between v2 and v3. Instead of putting `syntax = "proto2"` or `syntax="proto3"` at the top of your file to indicate the protobuf version, you can now specify it as `edition="2023"` - and presumably as `edition="XXXX"` as more editions become available in time.

One of the major improvements of Protobuf editions over the older v2/v3 releases is that it allows you to *choose* how your defaults, optionals, and other features will work. It gives you a lot of flexibility to enable some features but not others. If you're starting out with protobuf schemas, the protobuf website documentation covers everything in great detail. Start there.

Apache Avro

Apache Avro is another extremely common schema and serialization format, created under the Apache Software Foundation. Initially released in 2009, Avro has been commonly associated with Apache Kafka over the years, as well as a row-based, big data storage format. Here are a few notable points about Avro's data types and schema management:

- There is support for primitive (e.g., string, integer, boolean, etc.) and complex data types.
- By default, the schema is stored alongside the event data during serialization. The schema can also be decoupled from the data and stored independently in a schema registry (discussed further in "The Role of the Schema Registry").
- By including the schema with the event, Avro offers dynamic deserialization. A consumer can deserialize the event into a `GenericRecord` object built and populated dynamically from the schema and data. This feature is typically used when a consumer client does not have a code generator.
- You can compose complex schemas by referring to schema declarations stored in other files.
- While schema evolution is possible, *it is not explicitly defined* as part of the standard.

Avro is more verbose than Protobuf, but semantically very similar. Example 3-6 shows an Avro schema of the same `Person` object from the previous code snippet:

Example 3-6. Avro example of Person

```
{
  "type": "record",
  "name": "Person",
  "namespace": "com.event.driven.datamesh",
  "doc": "Example of a Person record",
  "fields": [
    {
      "name": "id",
      "type": "integer",
      "doc": "The person's unique ID"
    },{
      "name": "name",
      "type": "string",
      "doc": "The person's full legal name"
    },{
      "name": "height",
```

```
      "type": "integer",
      "doc": "Measured in centimeters, rounded to the nearest centimeter"
    },{
       "name": "countryCode",
       "type": "enum",
       "symbols": ["AAA", "ABW", ... "ZWE"],
       "doc": "ISO3166-1-alpha-3 standard. AAA=OTHER"
    }
  ]
}
```

In Avro, all properties, such as `type` and `doc`, are contained entirely within the field definition. In contrast, comments in Protobuf are simply added with C/C++ style syntax. Semantically, they are quite similar.

Avro is an all-around robust option for building your event-driven microservice architecture. It's compatible with many programming languages and technologies, and has a proven track record alongside Protobuf.

The next and final schema technology that we'll look at in this chapter is JSON Schema.

JSON Schema

JSON Schema format allows you to annotate and validate JSON documents. Unlike Avro and Protobuf, you can use JSON documents *without* a schema. A schemaless JSON means that there is really no definition of what *should* and *should not* be in the schema nor of any typing or defaults.

> **CAUTION**
>
> Don't use Schemaless JSON. It is not suitable for event streams as it leaves too much room for error, misinterpretation, missing fields, and missing data.

Here are a few notable points about JSON Schema's data types and schema management:

- There is support for six primitive types (null, boolean, object, array, number, string) as well as some more complex typing.

- You can compose complex schemas by using references to schemas stored in other files or locations.
- Data validation is similar to that of Protobuf and Avro. Producers validate their data against their schema prior to writing it to the event stream.
- There is support for adding validation keywords for data quality enforcement to numbers, strings, arrays, and objects.

Example 3-7 shows a JSON Schema representation of the same `Person` object from the previous code snippet:

Example 3-7. JSON Schema example of Person

```
{
  "$id": "https://example.com/person.schema.json",
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Person",
  "type": "object",
  "properties": {
    "id": {
      "type": "number",
      "description": "The person's unique ID"
    },
    "name": {
      "type": "string",
      "description": "The person's full legal name"
    },
    "height": {
      "type": "number",
      "description": "Measured in centimeters, rounded to the nearest centimeter",
      "minimum": 1,
      "maximum": 300
    },
    "countryCode": {
      "type": "string",
      "enum": ["AAA", "ABW", ... "ZWE"],
      "description": "ISO3166-1-alpha-3 standard. AAA=OTHER"
    }
  }
}
```

JSON Schema is the only technology discussed so far that contains both a *language* for specifying schemas and *validation parameters* that restrict the

ranges of certain properties. As exemplified in the `height` property, the height of a person is restricted to between `1` and `300 cm`, with the `description` including further information about rounding to the nearest number. These constraints will prevent a system from accidentally inputting the height as millimeters (unless your person was 30cm = 300mm tall!), but would unfortunately fail to prevent it from writing it as inches (72 inches would still fit within the 1 to 300 unit range).

Schema Evolution: Changing Your Schemas Through Time

Even if you create the perfect data model, there's a good chance that it may need to change anyways over time. New business opportunities, expansion of existing lines, and unforseen data demands are all potential causes for changing your schemas.

While new systems may need access to the latest data format, existing systems that have no need for the new data require a guarantee that their data contract won't change. *Schema evolution* allows us to change our schemas such that you can meet the use cases of new consumers without breaking compatibility for existing consumers.

There are a few properties of schema evolution that are common between Avro, Protobuf, and JSON Schemas. The first is compatibility modes, where events can be converted forward or backward depending on the changes made between schema versions. Compatibility modes are essential for change management, for guarding against (unintentional) breaking changes, and for alleviating consumers of the need to write custom code for each version of the schema (e.g., if `schema.version==1` do this, if `schema.version==2` do that, etc.).

The main compatibility modes are as follows:

Backward compatibility

Consumers using the new schema can read data produced with the old schema. This compatibility mode is important for ensuring that consumers using the latest schema can still read and process older data encoded under earlier versions. For example, deleting a field is backward compatible. Say you have the record shown in Example 3-8:

Example 3-8.

```
{
  "type": "record",
  "name": "Example",
  "doc": "This is Version 1",
  "fields": [
    { "name": "id", "type": "integer" },
    { "name": "foobar", "type": "string" }
  ]
}
```

You could remove the `foobar` field for Version 2, as shown in Example 3-9:

Example 3-9.

```
{
  "type": "record",
  "name": "Example",
  "doc": "This is Version 2",
  "fields": [
    { "name": "id", "type": "integer" }
  ]
}
```

Note that the schema converter (as part of the Schema framework) can take Version 1 data and convert it to Version 2 simply by dropping the `foobar` field. Thus, Version 2 is backward compatible with events written using Version 1.

Forward compatibility

Consumers using an old schema (Version 1) can still read new events written with a newer schema (Version 2). They will only be able to access data that matches their Version.

This compatibility mode is important for when an existing consumer coded against a current schema version rewinds its consumer offset to read historical data in the event stream.

From the previous example, you'll note that you can't convert Version 2 to Version 1, so it is *not* forward compatible. Why? Version 1 lacks a *default value* for the field `foobar`, which is needed by the consumer to fill in the missing value during conversion.

You would need to modify our Version 1 schema to add a default value *before* you published it (good thing you always create compatibility test cases as part of our pull request, right?). In Avro, specifying a default is relatively simple, as shown in `foobar` in Example 3-10:

Example 3-10.

```
{
  "type": "record",
  "name": "Example",
  "doc": "This is Version 1, but with a default value for foobar",
  "fields":
[
{
  "name": "id",
  "type": "integer"
},{
  "name": "foobar",
  "type": "string",
  "default": "DEFAULT_VALUE_STRING"
}
]
}
```

The term *default value* can be a bit misleading. Unlike a relational database table, where the default value is populated at *write time*, these default values are used at *read time*. Furthermore, the default value is only applied if the record itself does not contain the field. To further our previous example: if you try to convert a record written with schema Version 2 to schema Version

1, the converter will notice that the field `foobar` does not exist in the original event. Thus, it will set `foobar="DEFAULT_VALUE_STRING"` upon creating the converted Version 1 record instead of throwing an exception.

> **NOTE**
>
> Avro, JSON Schema, and Protobuf v2 each enable custom default values. This can be a powerful option for ensuring that records remain compatible through multiple changes. Developers often use default values to flag the fact that data is missing due to a compatibility conversion (e.g., `DEFAULT_VALUE_STRING`) and not because the payload was actually received with `foobar=null`.
>
> Google's Protobuf v3 removed custom default values as a deliberate design decision, so you may find Protobuf v3 a bit more difficult to use for evolutionary purposes.

As with all things schema-related, ensure that you check out the specifics of your schema selection for more details. Each of these four standards has much more content than can comfortably fit into this single chapter.

Full compatibility

: When an event can be converted both immediately forward and immediately backward.

Full-transitive compatibility

: When an event can be converted both forwards and backward to *any other version in the event stream*. This is the strongest guarantee, and it means that every single schema evolution is fully compatible with previous schemas. A Version 3 schema would be able to be converted to Version 2 and Version 1, and vice versa.

> **TIP**
>
> Full-transitive compatibility is the best palce to start. It provides the strongest guarantees for consumers and ensures that they need to update their code only when business use cases change and not because the schema was broken. While you can loosen the compatibility level later, it's extremely difficult to tighten it back up.

While schema evolution is extremely helpful, there will inevitably come a time when your domain shifts significantly enough that it is insufficient. A breaking change will need to occur, and it must be navigated carefully.

What is a Data Contract?

Chapter 3 briefly discussed service contracts as a mechanism for formalizing the APIs, documentation, ownership, and lifecycle ("Service Contracts"). Data contracts are similar in many ways, though a key difference is that while a service contract refers to a service, a data contract refers to the data guarantees of the data in the event stream itself.

A data contract is the compact between a producer of data, and all services past, present, and future that consume and use that data. Data contracts, as a concept, have featured in multiple architectures and systems of thinking, including data mesh, data lakes, and data warehouses. Conceptually they're mostly the same, answering the key question of "what are the guarantees relating to *this specific data*?"

> **WARNING**
>
> Not all event streams *require* a data contract. Event streams that belong to clusters of tightly coupled microservices, such as in the case of a highly-orchestrated workflow, may forgo establishing a data contract. In these cases, the producer and consumer are often owned by the same team, and no other service(s) consume or use the data.

Data contracts are essential for establishing reliable access to your core event streams. At it's core, a data contract is the definition of social, security,

and management expectations of the data. Components of a data contract include:

The Schema(s)

An event stream may have just a single schema. It may also have multiple schemas, either due to schema evolution or if you are co-locating multiple types of events in a single stream (discussed further in Chapter 4).

Service-level objectives (SLOs)

While not all data contracts require SLOs, they're remain very important for critical service functionality. Event-driven microservices rely on ever updating streams of business facts. If those streams stop, then the business stops. Just as you're expected to provide a SLO for request-response services, the same remains true for event streams and the services that drive them..

Ownership

The team and/or person who owns the event stream. They are also responsible for handling change requests, providing support, and ensuring service-level objectives are met.

Access Restrictions

It's important to keep track of which people and services jave access to an event stream. You need to know who your consumers are so you can alert them to upcoming changes. Issuing per-service access credentials is a good starting point, and is covered in more detail in [[Link to Come]].

Field-Level Encryption (FLE)

FLE allows you to encrypt only specific fields of your event stream. For example, you may want to encrypt the name and

address of a shipment, but leave the list of item information unencrypted. FLE is covered in more detail later in this chapter.

Schema Evolution Rules

The data contract also specifies rules for evolving the schema, as covered in "Schema Evolution: Changing Your Schemas Through Time".

Negotiating a Breaking Schema Change

Breaking changes most commonly occur due to shifting boundaries of a source domain model, often due to the expansion of the business model. Adding new product lines or services may necessitate rethinking and redefining data ownership, and leads to boundaries that don't map 1:1 with your existing event streams. As a simple example, a `User` data model may have previously modeled `address` with a simple `string`, as shown in Example 3-11:

Example 3-11.

```
{
  "type": "record",
  "name": "User",
  "namespace": "user.namespace",
  "fields": [
    { "name": "first_name", "type": "string" },
    { "name": "last_name", "type": "string" },
    { "name": "address", "type": "string" }
  ]
}
```

But new locality features require the creation of an `Address` object containing `home_address`, `work_address`, and `phone_number`. Example 3-12 is an Avro schema of the new standardized `User_v2` object. Note that both `home_address` and `work_address` are `Address` objects, and that the `Address` object allows for optional inclusion of `phone_number`.

Example 3-12.

```
{
  "type": "record",
  "name": "User_v2",
  "namespace": "user.namespace",
  "fields": [
    { "name": "first_name", "type": "string" },
    { "name": "last_name", "type": "string" },
    { "name": "home_address",
      "type": {
          "type" : "record",
          "name" : "Address",
          "namespace": "user.namespace.inner",
          "fields" : [
              {"name": "phone_number", "type": ["null", "string"]},
              {"name": "address", "type": "string"},
              {"name": "city", "type": "string"},
              {"name": "country", "type": "string"}
          ]
      }
    }, {
      "name": "work_address",
      "type": "user.namespace.inner.Address"
    }
  ]
}
```

The old, singular `string address` field will no longer be available to downstream consumers. But do all of them know about this change? What happens to systems that still rely on the `string address` field?

Coordinating a change that concerns multiple teams can be challenging. However, there is no need to reinvent the wheel, and you can draw on existing precedents for navigating this process.

It is common policy for a well-maintained API, be it a library, framework, or REST API, to maintain a degree of backward compatibility with legacy clients. New clients can use the latest calls to power their business logic, while older clients can continue to use older API calls. Eventually, the API calls are deprecated and subsequently removed, giving maintainers of the application time to migrate to the newer APIs. We can adopt this same process for addressing breaking changes with event stream schemas.

Step 1: Design the New Schema

The team that owns the schema must come up with the new candidate schema (or schemas). Sometimes breaking changes are relatively easy to navigate, such as a single breaking type change. Other times they may be more involved, perhaps renegotiating several existing event schemas and changing multiple boundaries.

In either case, the existing consumers are responsible for providing feedback and reviewing the proposed changes. They'll be involved in migrating their services to the new schema, after all.

Step 2: Consult Your Existing Consumers and Gain Approval

Consumers are the most valuable source of feedback for your schema changes. Show them the proposed changes, ideally via a code review, and ensure that they have viewed, understand, and can accommodate the changes before moving forward.

These discussions usually yield rich feedback. You may find that you need several iterations of edits before everyone is satisfied, though you may also just get it right the first time. Once the new schemas are approved you can move on to creating a release guide and a deprecation plan.

Step 3. Plan the Release, Migration, and Deprecation

The release schedule should reflect the estimated time it will take to migrate all consumers, along with some extra padding for safety's sake. Your consumers may need to rebuild their state stores during the migration, or them may be able to simply swap over to the new event stream(s).

Migrating historical stream data is a significant consideration for a breaking change. Maintaining a full history in the event stream will require either migrating the previous event data model to the new one, or recreating the events from the source data. The former can work when the breaking change

is simply a remodeling of existing fields, but tends to fall short when creating net-new data.

In the example provided, `address` is changed to both a `work_address` and a `home_address`. There isn't sufficient information about whether the old `address` is for work or home. Even if you were to figure that out, the source system will not yet be able to populate the other field.

You could backfill the `User_v2` event stream by creating new records to match the new schema. However, these new records would only represent the `User` at the current point in time. Historical events in the original `User` stream would not be migrated without additional work.

Finally, deprecation. You'll need to support both `User` and `User_v2` event streams for a period of time (such as 8 to 12 weeks) so that consumers can migrate from the old one to the new one.

Step 4. Execute the Release

Release the new event stream(s) alongside the existing ones, such that consumers have time to migrate over. Ideally, you'll also be using a central data portal (more information in [[Link to Come]]) where your users discover and register as event stream consumers.

Mark the original `User` stream as *deprecated* to block new consumer registration, and instead redirect them to use the latest version, `User_v2`. Both `User` and `User_v2` will co-exist for the duration of the migration timeline.

> **TIP**
>
> Make sure you have a hard date cutoff for migrating event to new streams. Send lots of warning messages and emails indicating the cutoff. If you do not, there's a good chance you'll never get your consumers to complete the migration, often due to lack of resources and competing priorities.

It is very important to follow through on your migration plan. It is unreasonable to maintain two versions of your stream indefinitely, and it will eventually lead to problems such as inconsistent data, partial outages, and ever-accumulating technical debt.

Managing schemas well requires keeping track of previous versions and ensuring compatibility rules are followed. Consumers additionally need the ability to discover which schemas belong to which event streams as part of self-service tooling requirements.

A schema registry can provide a solution for these needs, among others, and it is an essential part of any event-driven architecture.

The Role of the Schema Registry

A *schema registry* is a service that allows us to register schemas in association with their event streams. One of the main roles of the schema registry is to reduce the number of bytes sent over the network. Back in "A Brief Introduction to Serialization and Deserialization", we saw that the basic serialization process appends a full copy of the schema to each event record, which results in a significant amount of duplicated data sent over the network. Figure 3-5 shows a producer writing events with a full copy of each schema, reducing throughput, increasing data replication costs, and increasing the load on consumers.

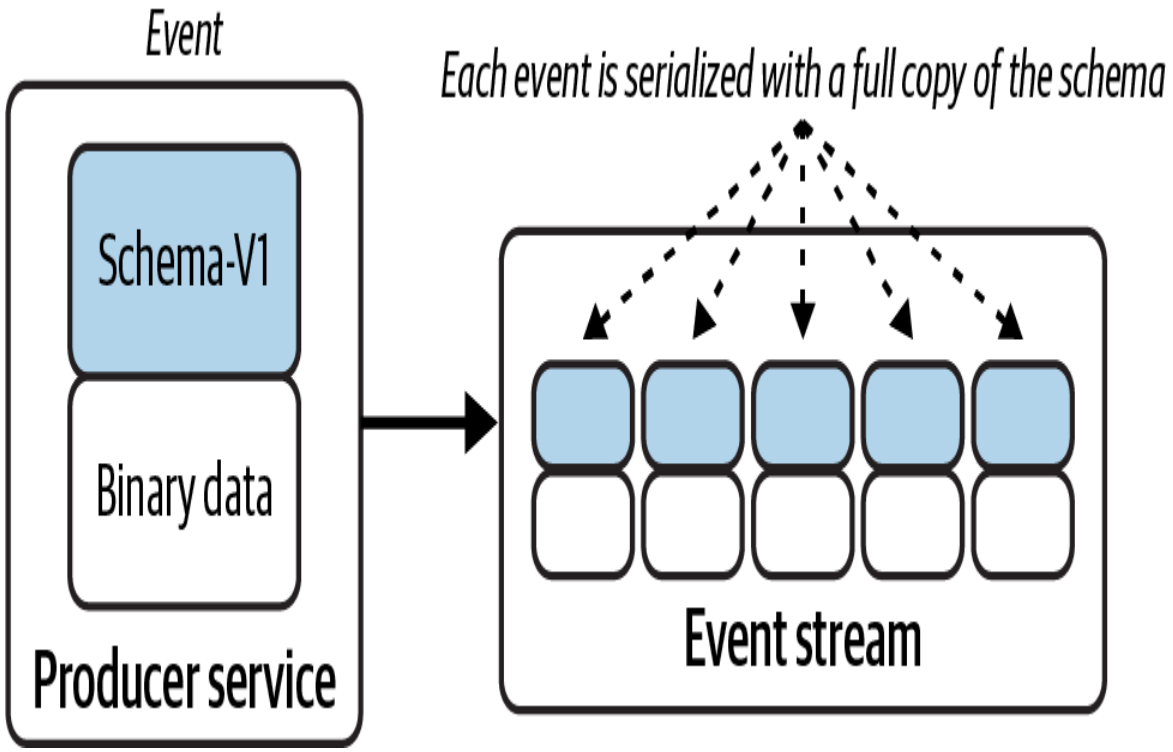


Figure 3-5. Each serialized event published to the event stream with the entire schema, resulting in excessive network utilization and storage overhead

A schema registry can absolve us of the need to write the schema with each event. Instead, we can store the schema *external* to the event and just use a unique ID in its place to track the associated schema. Thus, whenever a consumer reads an event, it can simply reference the schema registry to obtain the schema associated with that ID.

Figure 3-6 shows the entire end-to-end process. Instead of serializing the schema along with the event, the producer service queries the schema registry (1), registers the schema (2), then replaces the schema with a short unique ID (3 and 4). This event is then produced to the event stream (5). The process is performed in reverse on the consumer side: the event is read from the stream (6), the schema registry is queried using the unique ID (7 and 8). Once the schema is obtained (9), the record can then be deserialized (10 and 11) for processing by the consumer's business logic.

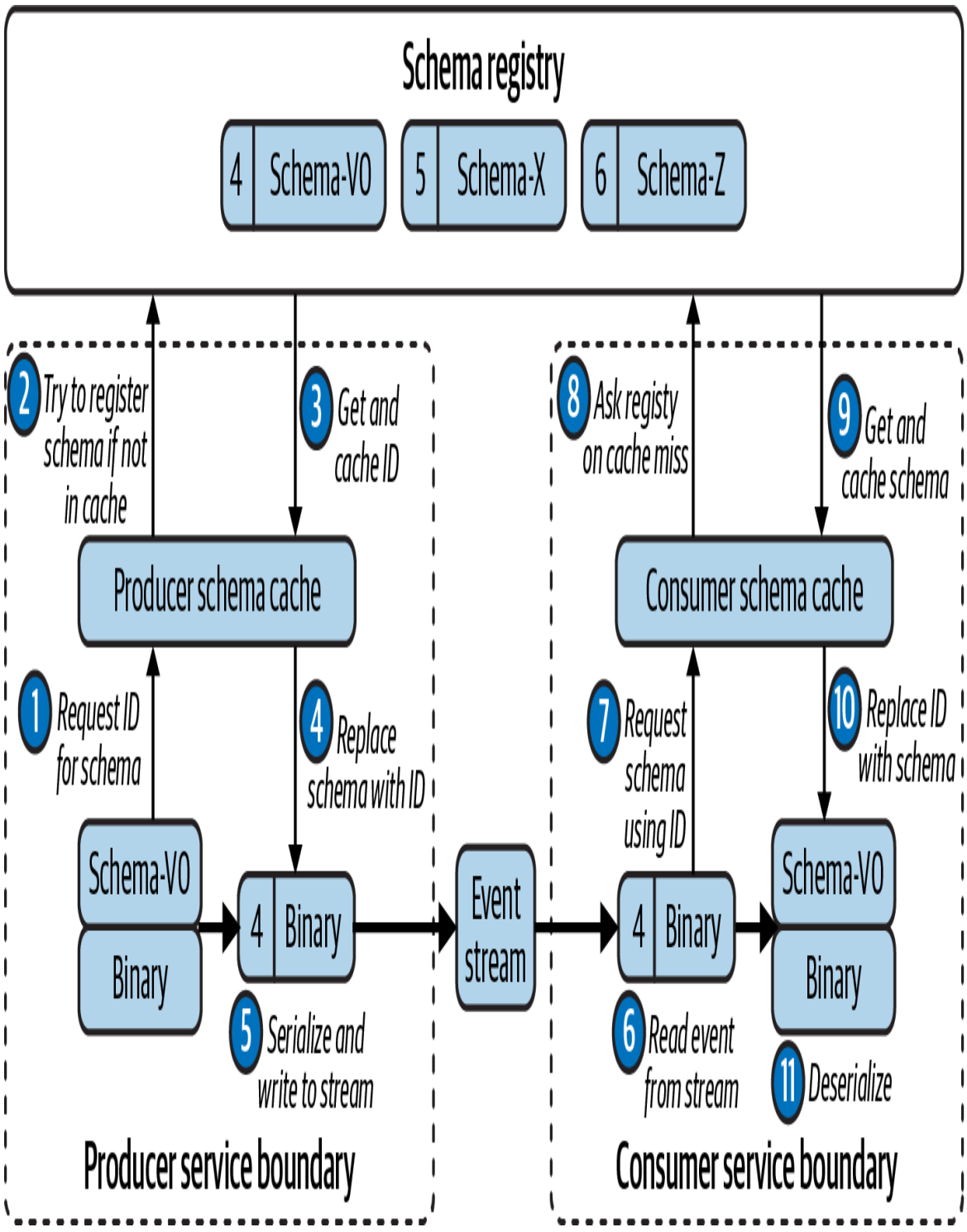


Figure 3-6. Leveraging a schema registry in an event-driven producer/consumer workflow

Precisely *how* the schema is replaced by the unique ID depends on both the producer's serialization logic and any limitations in the format of events in

the schema registry. For example, Confluent's Kafka Schema Registry replaces the schema with a 5-byte prefix.

Using a schema registry requires serializers and deserializers to adhere to the proprietary format. And while Confluent's schema registry is fairly widely used with Kafka, it's not the only schema registry, and not all schema registries will use the same format.

A schema registry provides other benefits in addition to network and disk I/O savings. Here are a few other significant benefits provided by a schema registry:

Data discovery

> The schema registry provides a mapping of event streams to its registered schemas. Prospective consumers can examine the schemas to see if the stream contains the data they're interested in. Self-service tooling can provide search functionality on top of the schema registry API to search for specific fields, documentation, or metadata tags.

Schema evolution validation

> Registering a schema with the schema registry is a mandatory part of the write path of an event to a stream. This action provides a hook for validating the producer's current schema against that stored in the registry. Did the user evolve the schema in a way that is unauthorized by the schema registry (e.g., the new schema does not support backward compatibility)? Throw an exception. Is the schema completely invalid compared to what was registered under that ID before? Throw an exception. Is the event malformed? Throw an exception. The schema registry provides a safeguard against unintentional and unauthorized changes, protecting the event stream's data integrity.

Automatically updated documentation

> The schema registry provides a minimal form of automatically updated documentation for each event stream. Users can view all the registered schemas, including the names, types, and doc fields. Embedded schema docs are very useful for highlighting any idiosyncrasies or corner cases in the data. They are also far more likely to be up to date because they are embedded, as opposed to documentation maintained independently by an outside party.

Downloadable schemas to generate code

> A consumer can download schemas to generate class definitions and test events for their unit tests.

TIP

You can write a custom registration script to evaluate the schema during the registration process. A simple and common check is to verify that every value in a schema has a "`doc`" string of nonzero length to ensure that there is *some* documentation for each field. Another option is to scan for personally identifiable information (PII) patterns and request an additional verification step if something suspicious is found. Similarly, you can automatically generate class definitions and run validation tests prior to deployment.

Schema registries provide many benefits for reducing both human and computer overhead. The savings on network and storage costs alone make them a valuable choice, with the remaining functionality providing the icing on the cake.

Before we exit this chapter, there's still a few things left to cover. In the interval since the first edition of this book, *field-level encryption* has become a popular subject. And for good reason, since it's directly related to data security and ensuring compliance with legal data requirements. Let's turn our attention now to the security and compliance portion of data contracts.

Field-Level Encryption

Field-level encryption (FLE) enables a producer to encrypt specific fields in its schema. Only consumers who have been granted access to that field can decode the data. PII, financial, health, and legal information commonly require special handling for compliance and security reasons. FLE can significantly simplify your event streaming topologies.

Consider a bank transfer. You may use field-level encryption on the fields that you don't want just anyone to use, while leaving others unenecrypted. Table 3-1 shows the encryption of the `email`, `user`, and `account` fields of an event, while `amount` and `datetime` remain unencrypted.

Table 3-1. Using field-level encryption to partially encrypt an event

Field name	Original event	Partially encrypted event
email	adam@bellemare.com	n2Zl@p987NhB4.L0P
user	abellemare	9ajkpZp2kH
account	VD8675309	0PlwW81Mx
amount	$777.77	$777.77
datetime	2022-02-22:22:22:22	2022-02-22:22:22:22

Consumers with decryption permissions can access the decrypted information to settle account balances, while an analytical system without decryption permissions can still track how much money is moving around during a period of time. One of the major benefits is that *you do not need to create separate event streams for each consumer based on security needs*, as was commonly the case before FLE became more widely adopted.

You can also use *format-preserving encryption* to maintain the format of the event data. In this case, the `email`, `user`, and `account` are format-preserving encrypted, having the same alphanumeric characters, spacing, and character count of the original fields. This allows for the schema to accurately represent the underlying data.

Format-preserving encryption is particularly useful for applying encryption to data after the schema has already been negotiated - for example, in legacy event streams. You won't need to renegotiate the schemas as the encypted types remain the same as the unencrypted types. In contrast, using non-format-preserving encryption often results in malformation, such as converting a `long` bank account ID into a 64-character `string` or encrypting a complex nested object into an array of hashed bytes.

> **TIP**
>
> Field-level encryption permits finer-grained access controls and independent per-field decryption capabilities. Consumers can request decryption keys for only the data they need, reducing the surface area risk for leaking sensitive data.

Encryption of sensitive data, whether end-to-end or field-level, can also help with legal requirements, such as the right to be forgotten, or to have ones data deleted. This is covered more in [[Link to Come]].

Integration with Data Catalogs

A *data catalog* is a repository that stores schemas, location, and other business metadata pertaining to a data source. Additionally, it can store

information such as event stream ownership, schema evolution restrictions, sample data, metadata tags, and control-related information, such as PII, financial, or other restricted information. Data engineers, data scientists, and data analysts have long made use of data catalogs for sorting and organizing their data sets.

Historically, data catalogs have primarily documented data at rest, such as in a relational database or as a set of Apache Parquet files in cloud storage. While a schema registry offers operational support for schema management and evolution, a data catalog goes a step further by providing a one-stop shop for browsing, searching, discovering, and publishing new data sets. Event streams compatibility demand has substantially increased since the first edition of this book was printed, primarily in the form of support for Avro, Protobuf, and JSON Schemas. Modern day data catalogs support data in multiple forms and formats, including data at rest in cloud storage, databases, and event streams.

Popular catalogs for the major cloud providers include AWS Glue, Azure Data Catalog, and Google's BigQuery universal catalog. Open source offerings include Apache Atlas, OpenMetadata, and DataHub, to name just a few. Many data catalogs also offer additional functionality pertaining to data governance, dependency modeling, and access controls, but you'll need to evaluate those offerings on their own to decide if they're right for your organization.

Data catalogs make it easy to find and discover data. While they're not necessarily *required* for a successful event-driven microservice architecture, I suspect you'll discover that their utility will lead you to adopting one. You'll need to carefully consider the schema compatibility, vendor lock-in risk, and the age-old debate of build-versus-buy.

Summary

Schemas are essential for succeeding with an event-driven architecture. They provide structure and clarity for both the producers and consumers of the event stream. They also form the foundation of the data contracts, containing

additional information such as ownership, SLOs, evolution rules, and additional documentation.

Code generators bridge the gap between the schema itself and the business logic and provide a benefit to both the producer and the consumer. The former benefits from the strict type definitions and distinction between optional and mandatory fields, ensuring that no data is accidentally malformed or excluded. Subsequently, the latter benefits from the same well-defined type-system, absolving it of the need to interpret and standardize the data. Schemas provide the means to impose quality controls as close to the source as possible.

Schema evolution provides the ability to evolve and change schemas over time, with explicit up-front rules as to which changes are allowed given compatibility requirements. While breaking changes can still occur, schemas provide the common framework for determining what is and is not allowed. And since they're commonly integrated as part of the codebase, schema changes can follow the same review processes as standard business application code changes.

While there are many options available for you to choose from, Apache Avro and Google's Protobuf remain your best options, with JSON Schemas as a reasonable third choice. Your investment into one of these will vary depending on your pre-existing technology choices, but remains best discussed and decided centrally by your federated governance team.

In the next chapter, we'll take a look at leveraging what we've learned with schemas and apply it to the problem of event design. There are many different ways to model and design events. We'll explore the best ways to do it and the pitfalls and gotchas that are best avoided.

Chapter 4. Designing Events

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 5th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

NOTE

An earlier version of this chapter previously appeared in Building an Event-Driven Data Mesh.

There are many ways to design events for event-driven architectures. This chapter covers the best strategies for designing events for your event streams, including how to avoid the numerous pitfalls that you will encounter along the way. It also provides guidelines for when to use certain types and when to avoid using others, plus some illustrations as to why this is the case.

Introduction to Event Types

There are two main types of events that underpin all event design: the *state* event, introduced in "Entity Events", and the *delta* event.

Figure 4-1 shows a simple square wave in steady state, periodically altering from one state to another based on a delta. Similar to this square wave, we model our events to either capture the state itself or the edge that transitions from one state to another.

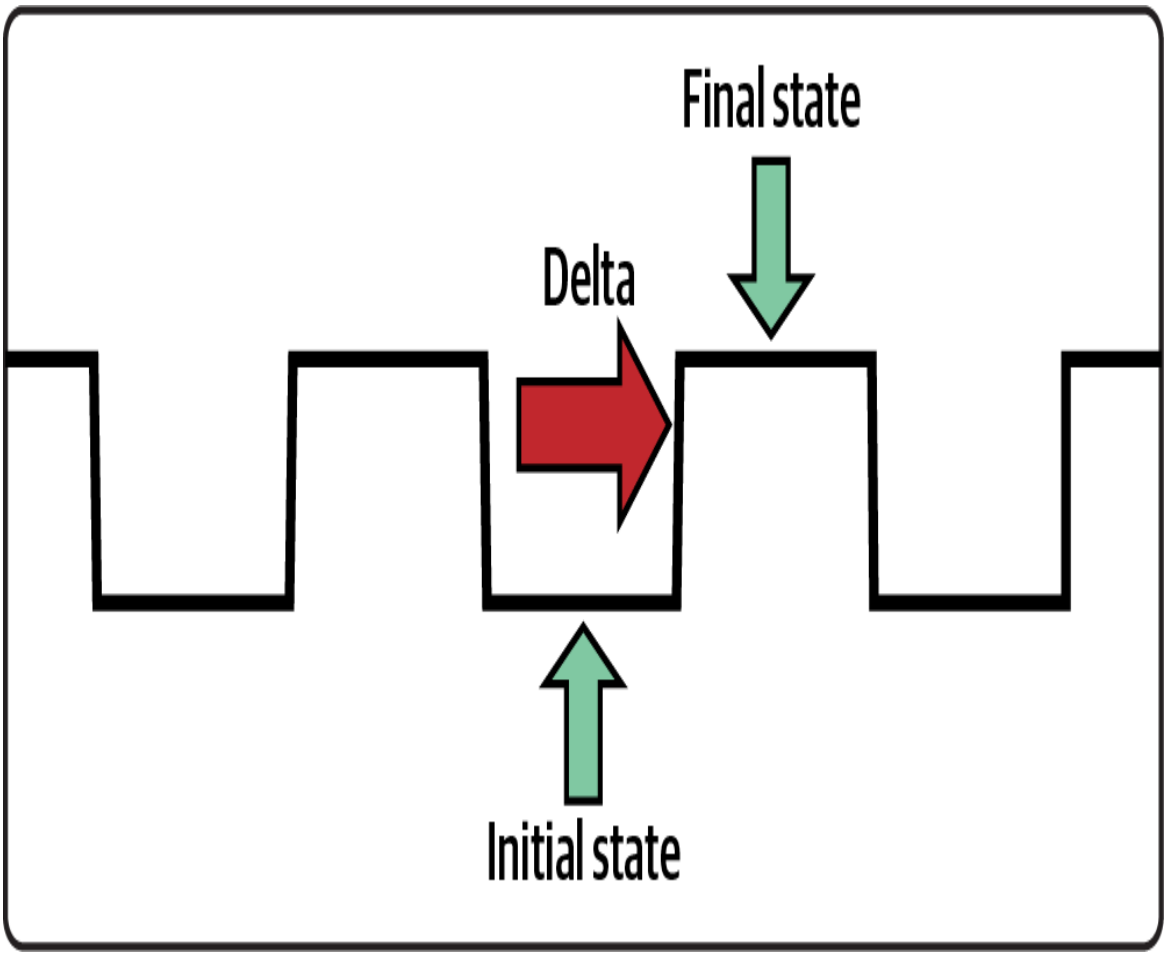


Figure 4-1. State and delta during a change

There are three stages to any occurrence in a system:

1. The initial state
2. The delta that alters the initial state to produce the final state
3. The final state (which is also the initial state for the next change cycle)

The majority of events we encounter can be fully categorized as either state or delta. Looking at events in this way helps separate concerns and focus

design efforts:

State events

> State events fully describe the state of an entity at a given point in time. They are akin to a row in a database, chronicling the entire current state of affairs for the given data. State events are typically the most flexible and useful event type for sharing important business data between teams, people, and systems. State events are sometimes called level, noun, or fact type events.

Delta events

> These describe the transition between states and typically only contain information about what has changed. Delta events are more commonly used in and between systems with very high degrees of coupling, as there is a close relationship between the definition of the event and how it is interpreted. Deltas are also sometimes called edge, verb, or action type events.

You may also encounter *hybrid* events that have characteristics of each, though these tend to be less common because they can cause an undesirable strong coupling effect.

Hybrid events

> These events describe both a state and transition. They are usually a bit of a kludge, implemented when a deeper rework isn't possible due to deadlines such as trying to make the quarter's profit goal. But since people will still use them, they are covered in more detail later in this chapter.

Let's take a look at state events first.

State Events and Event-Carried State Transfer

A state event provides the current state of an entity at a specific moment in time. The record's key details the unique identifier of the entity, while the value contains the entire set of data made available to all consumers. State events can commonly be thought of as how you'd store data in a database table - a full accounting of the data, for a given primary key, at the time it was last updated. Examples of state events include the names of the very entities that you model and use in your business domain. For example:

- `Item`, `Order`, and `User` in an commerce domain.
- `Truck`, `Package`, and `Driver` in a package shipping domain.
- `Payment`, `Debit`, and `Account` in a banking domain.

State events provide several critical benefits, including:

Event-carried state transfer

> State events are commonly used for *event-carried state transfer* (ECST). As the name suggests, the event carries important business state, transferring it to any number of subscribed consumers. ECST not only allows other systems to reason about state generated in another service, but also to react to changes in that state.

Materialization

> State events let you materialize data quickly and easily. Simply consume the event stream, and materialize the data your service needs into its own state store.

Strong decoupling

> The definition and composition of the state event remains entirely within the producer service's bounded context.

There is no coupling on the internal business logic of the system that produces the record.

Infer changes from state

Consumers can infer all field changes by comparing the current state event to the previous state event for a given key. The service can react accordingly whenever any fields change, as shown in Figure 4-2.

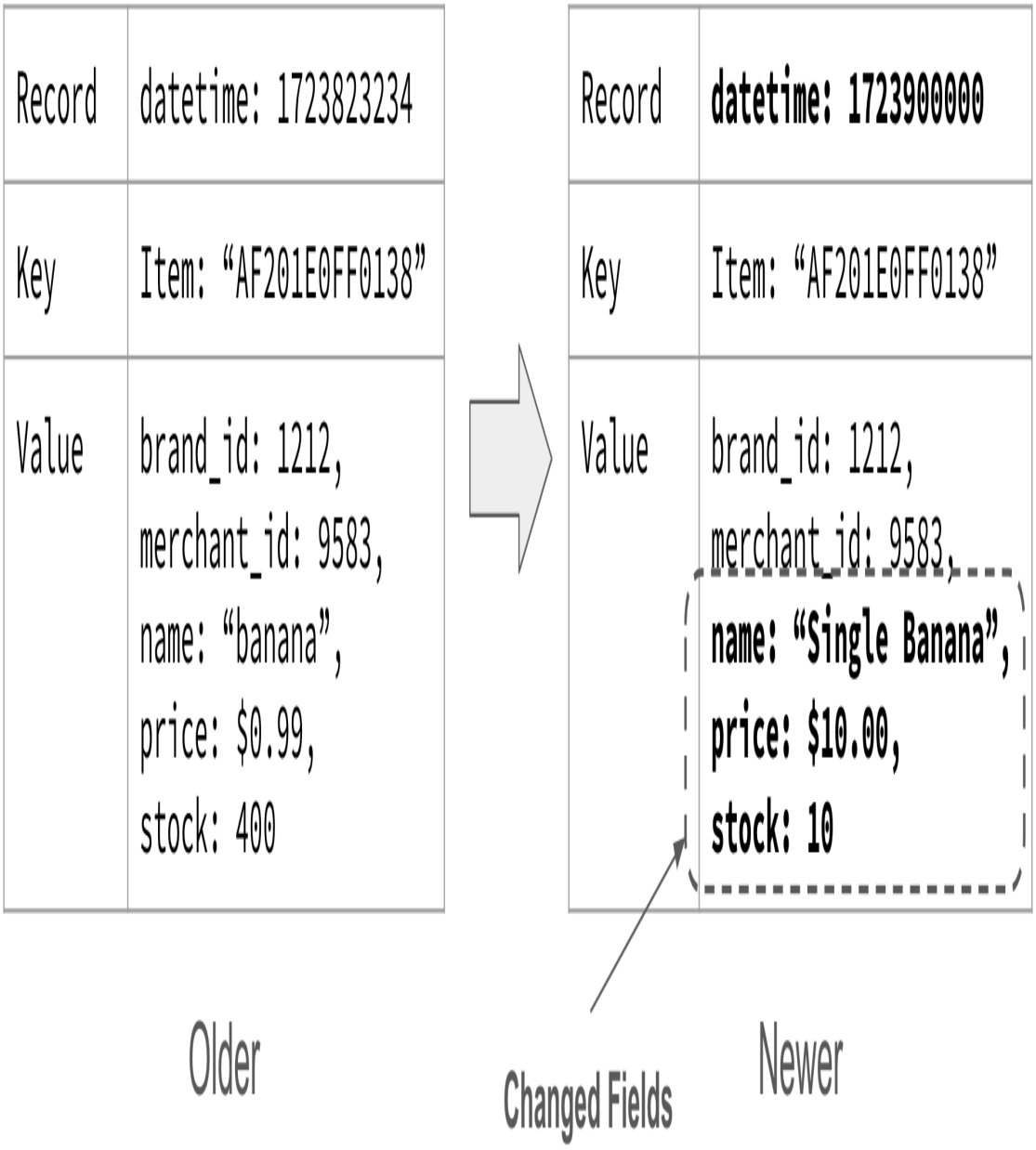


Figure 4-2. Two state events allow you to infer what has changed by comparing the newer event to the older event

In this example, you can see that the newer event has had its name, price, and stock updated. Your microservice can infer the changes that it cares about,

and react accordingly. For example, updating signage, ordering new inventory, or prioritizing alternate options in the customer's search results.

State events may contain just the "now" state or they may contain the "before/after" state (a pattern made popular with "Liberating Data Using Change-Data Capture"). Both options have their own advantages and disadvantages, which we'll examine in turn. For starters, let's take a look at how each of these options affects *compaction* of event streams.

There are two main design strategies for defining the structure and contents of state events:

Current state

> Contains the full public state at the moment the event was created.

Before/after state

> Contains both the full public state *before* the event occurred and the full public state *after* the event occurred.

Let's look into each of these in detail to get a better understanding of their trade-offs.

Current State Events

The event contains only the current state of the entity and requires comparison with a previous state event to determine what has changed. For example, an `inventory` event for a given `item_id` will contain only the latest value for the `quantity` in stock at that point in time. This design strategy has several main benefits:

Lean

> The state events consume a minimal amount of space in the event stream. Network traffic is also minimized.

Simple

The event broker stores any previous state events for that entity, such that if you need historical state, you simply rewind and replay your consumer offsets. You can set independent compaction policies for each event stream depending on your consumer's needs for historical data.

Compactable

You can keep the number of events in the stream proportional to the key space of the domain.

It also has a few nuances that are not quite drawbacks, but rather properties to consider:

Agnostic to why the state changed

The downstream consumer is not provided with the reason *why* the data has changed, only with the new public state. The reason for this is simple: it removes the ability of consumers to create a tight coupling on the *internal state transitions* of the source domain. Think about data in a relational database table—we typically do not communicate *why* that data has changed in the data itself, and the same holds true for state events (Note: hybrid events, covered later in this chapter, bend this rule a bit).

Consumers must maintain state to detect transitions

A consumer must maintain its own state to detect specific changes to certain fields, regardless of how simple or complex its business logic is. For example, a customer changing their address to another country may require you to send them new legal documents, which can differ depending on the country they left and the country they moved to. By making it the consumer's responsibility to materialize state for tracking transitions, the onus of computing the changes entirely up to the consumer. They

can infer *any* changes from one state to another, provided that they portion of state that their business logic cares about. This pattern enables very clean separation of responsibilities between producers and consumers and leads to exceptional flexibility for accommodating business logic changes.

Before/After State Events

This strategy relies on providing the state before a transition occurs and the state after it has occurred. Change-data capture (CDC) systems, as covered in "Liberating Data Using Change-Data Capture", regularly make use of the before/after strategy. The following showcases two before/after `user` events with a simple two-field schema:

```
Key: 26
Value: {
  before: { name: "Adam", country: "Madagascar" },
  after: { name: "Adam", country: "Canada" }
}
```

A follow-up before/after state event that shows the deletion of `Key = 26`. Note that old data still remains in the `before` field:

```
Key: 26
Value: {
  before: { name: "Adam", country: "Canada" },
  after: null
}
```

There are some benefits to this design:

Simple state transitions in a single event

The before/after event showcases every field that has changed within a single transaction, in addition to all of the fields that have not changed. The reason for the change, however, is not included.

Consumers can detect simple changes without maintaining state

> Some consumers can forgo maintaining state if they are only interested in detecting a simple state transition. For example, if we want to send documents to a user who moves from Madagascar to Canada, then our consumer can simply check to see if the `before` and `after` fields of the event match their criteria. However, this doesn't work if Adam moves from Madagascar to Ethiopia, and then soon thereafter moves to Canada, causing two events to occur. The consumer business logic would not be able to trigger on this sequence of events since it doesn't maintain any state. In practice, the theoretical stateless consumer is seldom realized, since the vast majority of services of any reasonable complexity need to maintain state.

There are also a few drawbacks to this design:

Compaction is difficult

> Deleting an event using the before/after logic results in the `after` field being set to null—but the entire value itself is not null. By default, event brokers like Apache Kafka will not recognize this as a tombstone and thus will not delete it. While it may be technically possible to rewrite the compaction logic, it usually isn't feasible, especially if you are relying heavily on SaaS solutions.

In the end you'll have to ensure that you write a secondary tombstone event *after* your initial "before/after" event. Then, you'll rely on the event broker eventually compacting the data away. This double-event strategy is often used by change-data capture systems, such as Debezium (a change-data capture service that will be explored in more detail in "Liberating Data Using Change-Data Capture").

Figure 4-3 illustrates the two events that you would see written to the event stream. The first contains the before/after data (with the after being “null”), while the second event is a tombstone that enables compaction and deletes the old data.

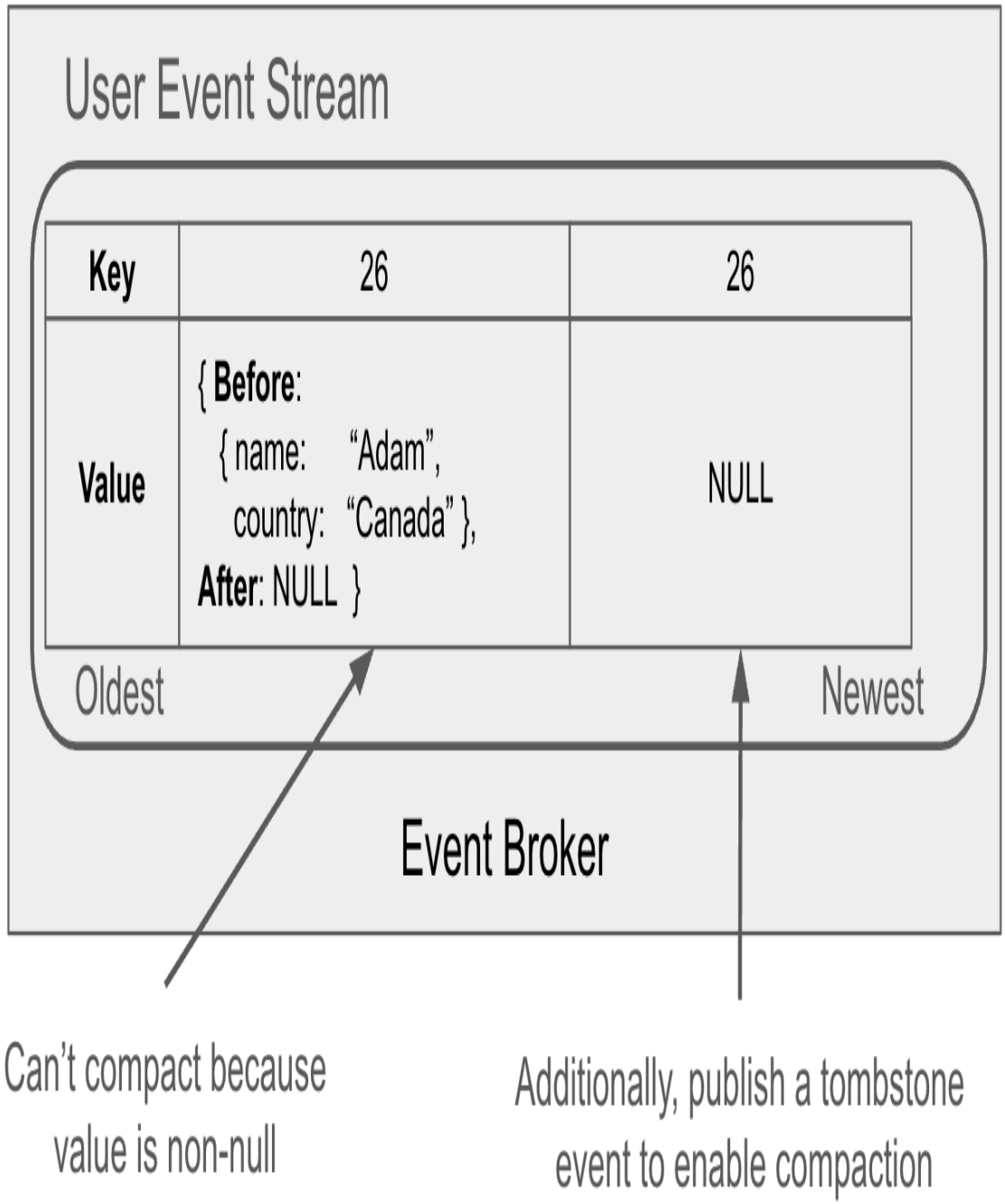


Figure 4-3. Publishing a tombstone after the before/after deletion to enable event stream compaction

Risk of leftover information

Previous data may be accidentally maintained indefinitely in the `before` field unless you issue a series of deletions. Even in the case that the produer publishes tombstone events, there remains a variable delay between the publishing of the tombstone and the asynchronous compaction execution of the event broker.

Doubled data storage and network usage

Before/after events double (on average) the amount of data going over the wire and stored on disk. Consumers, producers, and the event broker each bear part of this load. In some cases this may be trivial. Seldom-updated events or those with low volume are probably nothing to worry about, but extremely high volume event streams can quickly add up the costs. This can be particularly expensive depending on the cross-regional data transfer fees associated with high-availability producer, consumer, and event broker deployments.

Current state events tend to be a far better option than the before/after model for event-streams. Consumers will still need to maintain state for the records they care about for their business processes, but disk space is relatively cheap, and they need only keep the data that they care about. This also simplifies operations for the event broker when compared to before/after, with lower cross-region traffic costs, less broker disk usage, and less broker network usage replication overhead. Further, the risk of leaking data from improper compaction deletion is eliminated.

In the next section, we'll take a look at delta events, where an event is modeled after the change and not the state itself.

Delta Events

The delta event represents a change that has occurred within a specific domain, represented as the edge of a transition in Figure 4-1. Delta events contain *only* the information about the state change, not the past or current state. Delta events are usually phrased as verbs in the past tense, indicating that something has occurred. For example:

- `itemAddedToCart`
- `itemRemovedFromCart`
- `orderPaid`
- `orderShipped`
- `orderReturned`
- `userMoved`
- `userDeleted`

You may find that you're more familiar with these types of events than you are with the state types used for ECST. Delta events have historically been fairly common, particularly in the context of the Lambda architecture (see "The Lambda Architecture"). Delta events are also commonly used *inside* a domain for *event sourcing*, a subject we'll now take a look at in more detail.

Delta Events for Event Sourcing

Event sourcing is an architectural pattern based on recording *what happened* within a domain as a sequence of immutable append-only events. These events are *aggregated* to build up the current state by applying them in the order that they occurred, using domain specific logic, one after another.

This architecture is often promoted as an alternative to the traditional create, read, update, delete (CRUD) model commonly found in relational-database type frameworks. In the CRUD model, the fully mutable state of the entity is directly modified such that only the final state is retained. Though the databases underpinning CRUD can generate an audit log of the changes that

occurred, this log is used primarily for auditing purposes and not for driving business logic.

There are some limitations to the CRUD model that may make event sourcing an attractive alternative. For one, operations must be processed directly against the data store as they are invoked. Under heavy use, this can significantly slow down operations and result in timeouts and failures. Second, high concurrency operations on the same entities can result in data conflicts and failed transactions, further increasing load on the system.

But the CRUD model also contains several distinct advantages. Though it depends largely on the database, most CRUD implementations offer strong read-after-write consistency. It's also fairly intuitive and simple to use, with lots of tools and frameworks supporting it. For many software developers, this is the first model of maintaining state that they encounter. Figure 4-4 shows a series of CRUD events (one create, two updates) applying changes to the refrigerator state. The state is completely mutable, and only the updated state is retained after a create or update command is applied.

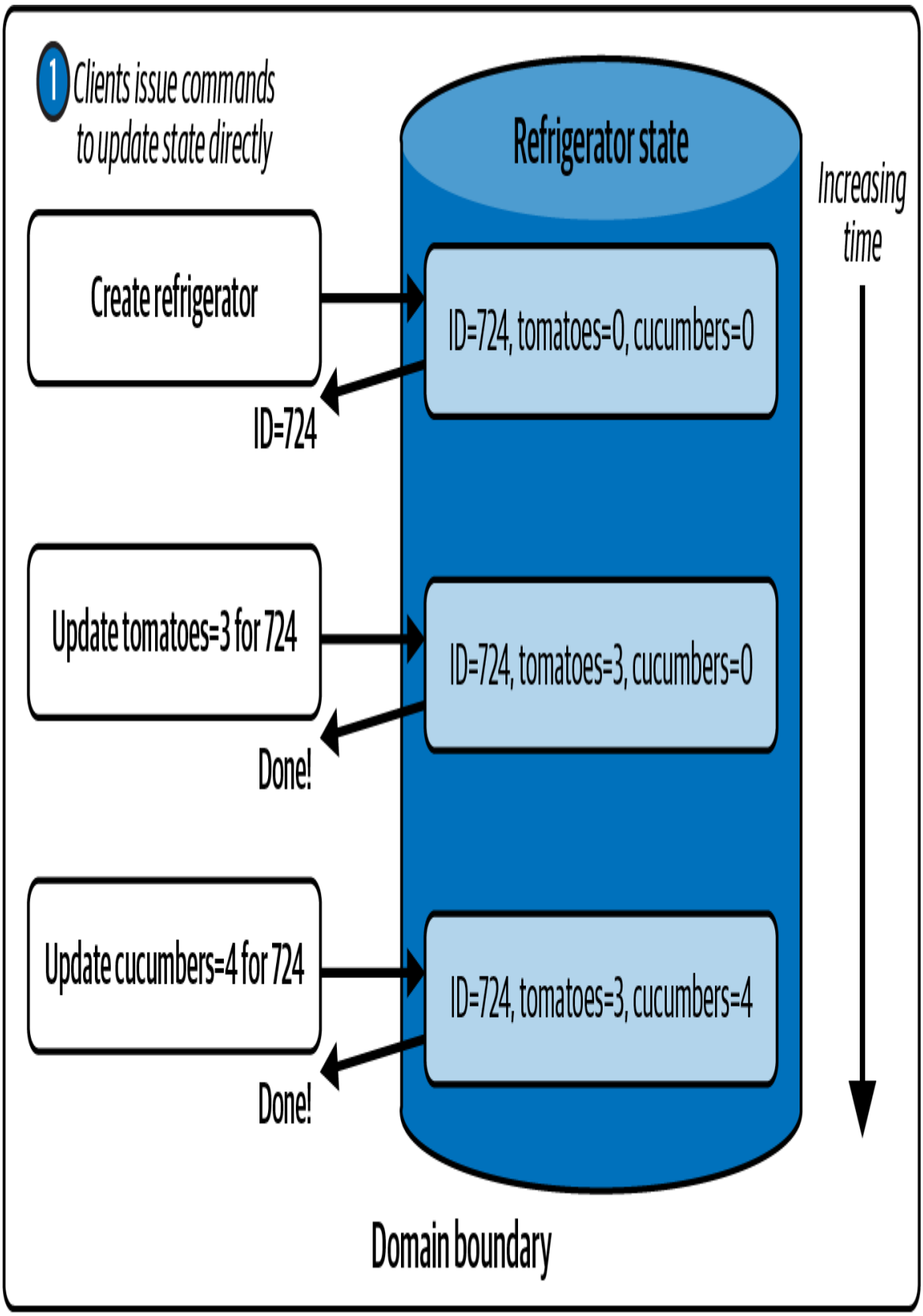

1
Clients issue commands to update state directly
Refrigerator state
Increasing time
Create refrigerator
ID=724, tomatoes=0, cucumbers=0
ID=724
Update tomatoes=3 for 724
ID=724, tomatoes=3, cucumbers=0
Done!
Update cucumbers=4 for 724
ID=724, tomatoes=3, cucumbers=4
Done!
Domain boundary

Figure 4-4. Using CRUD commands to update the contents of the refrigerator, reflected in the database

Under the event sourcing architecture, these create, update, and destroy operations are instead modeled as events that are written to a durable append-only log that retains them indefinitely. It is not uncommon to use a single local database table to act as the append-only log. The changes are then read back from the log and applied in the order they occurred.

While it is also possible to use an event broker like Apache Kafka for event-sourcing, you must ensure you are willing to accept several tradeoffs. For one, the data will travel over a network both to and from the event broker, introducing network latency. Secondly, it is possible that the event broker becomes unavailable, say due to a network failure, which can cause unacceptable delays in processing.

In either case, in event sourcing, the current state is generated by consuming events in the order they are written in the log and applying them one at a time to create the final state.

Figure 4-5 shows the same refrigerator example, with the CRUD operations instead modeled as domain events. And although these sample events are CRUD-like, the domain owner has free reign over designing the deltas to suit their own business use cases. For instance, they could extend the set of events they're creating to also incorporate deltas such as:

- `turn_lights_on`/`turn_lights_off`
- `turn_cooling_on`/`turn_cooling_off`
- `open_door`/`close_door`

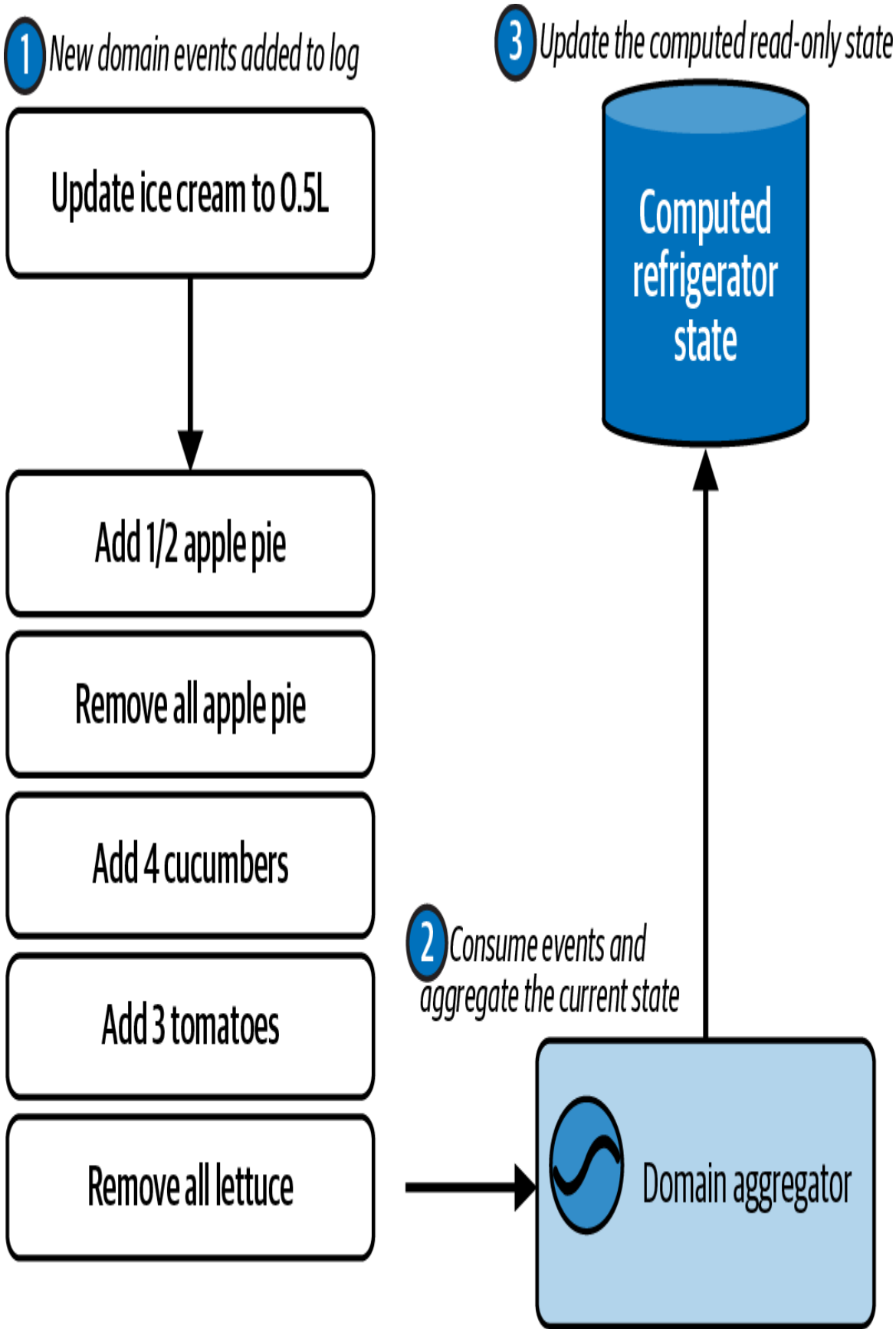

1 New domain events added to log
Update ice cream to 0.5L
Add 1/2 apple pie
Remove all apple pie
Add 4 cucumbers
Add 3 tomatoes
Remove all lettuce
2 Consume events and aggregate the current state
Domain aggregator
3 Update the computed read-only state
Computed refrigerator state

Figure 4-5. Building up the contents of a refrigerator using event sourcing

The domain aggregator (2) is separate from the process that writes the new domain events into the log (1), and allows the write and aggregation processes to be scaled independently. A domain can also contain multiple domain aggregators and may aggregate the same log to two different internal state stores depending on the domain needs.

One of the main drawbacks of event sourcing is that it is an eventually consistent system, which may be a significant obstacle for some use cases. There will always be some delay between writing the event to the log and seeing the materialized result in the state. And because multiple concurrent clients can each write events about the same entity, it becomes difficult to attribute any specific modification in final state to the delta your client just appended. This can make it unsuitable in applications that require strong consistency.

Event sourcing is a reasonable alternative to the CRUD model for building up internal state. The problem with event sourcing comes when it is *misused as a means for interdomain communication*, exposing the internal domain deltas to the outside world for others to couple on (and misinterpret). These domain-specific events and their relationship to the aggregate and to each other can change over time. Just as we do not allow services outside of our domain to couple directly on our data model, we also must not allow services to couple on our private domain events data model.

This isn't to say you cannot expose any events outside of the domain boundary, but *any* event that you expose becomes part of the public data contract. You'll need to ensure that its semantic meaning doesn't drift over time and that the data doesn't evolve in a breaking way. A failure to maintain the boundaries of "events in here" and "events out there" can lead to very tangled coupling, excessive difficulty in refactoring, and subtle errors due to misinterpretation of events by outside consumers.

The Problems with Delta Events

The next few sections illustrate the problems with using delta events for microservices. Keep in mind as you read through this section that delta events are not bad *per se*, but that they are often misused. This section will illustrate the limitations of delta events, while also highlighting where they are useful.

There is an infinite amount of delta event types

First and foremost, there is an infinite amount of delta events that can occur in any domain. This alone should stop most folks from trying to create event-streams with the delta model, but unfortunately it does not. But surely, can it really be the case that there is an *infinite* number of delta events?

In reality, the actual set of delta events necessary for your domain is undoubtedly finite. The real problem is that every consumer of a delta event needs to know precisely how to load it into their own version of state. For many events, this leaves it open to interpretation.

Let's take a look at an example. Figure 4-6 shows a simple set of ecommerce events for constructing the contents of a shopping cart.



Figure 4-6. Shopping cart delta events, used to construct the current state of the shopping cart

`ItemAdded` and `ItemRemoved` are fairly simple: items can be added, or they can be removed. The consumer will need to interpret and apply each of these events, in the correct order, to build up its aggregate. Suppose, though, that a new feature in the domain allows users to update the quantity of items they have in their cart: where previously the domain owner may have issued a remove event first, then an add event with the new quantity, now they may instead simply issue an update.

Figure 4-7 shows this new `UpdatedItemQuantity` event stream published to the world. Now if a consumer needs a model of the shopping cart, they must also account for these updated events in their aggregation code. As the scope of the domain changes, so do the meaning of the events and their relationship to the aggregate.



Figure 4-7. New updated event changes the way the shopping cart delta events are interpreted

One of the common reasons that people (incorrectly) choose to use delta events for cross-domain communication is that they don't believe that other consumers should be required to maintain state to trigger on specific changes. The near-infinite range of possible deltas makes this untenable, but it's a trap that many don't recognize until they're firmly in its grasp.

A simple expansion of the shopping cart domain to incorporate features such as coupons, shipping estimates, and subscriptions increases the amount of information that a consumer must account for, as shown in Figure 4-8.



Figure 4-8. The delta events defining the shopping cart sprawl as new business functionality is added

Exposing this expanded shopping cart domain to consumers requires that the consumers can identify, use, and build a correct aggregate out of these events. But where do the consumers get the information they need to correctly interpret and aggregate this data? From the source producer system. Which leads us to the next major problem of using delta events.

The logic to interpret delta events must be replicated to each consumer

How can a consumer know they're correctly interpreting the delta events? And how does the consumer stay up to date when new domain events are introduced? The key is to make it possible for a consumer to correctly operate without having to continually update their logic to account for new and varied delta events.

In the state model, a consumer only needs to materialize the state events to know they're getting the complete public domain. They may not know *why* the transition occurred (I'll touch on this a bit more later in the chapter), but they can be assured that the entire public domain is there, and that as a consumer, they don't need to worry about correctly building an aggregation.

Figure 4-9 shows two consumers, each of which has replicated the logic from the producer for building up the aggregate state. Consumers are responsible for identifying, understanding, and correctly applying the add, remove, and update domain events to generate the appropriate final state of the aggregate. The complexity of the domain is paramount; very simple domains may be able to account for this, but any domain of meaningful complexity will find this solution untenable.

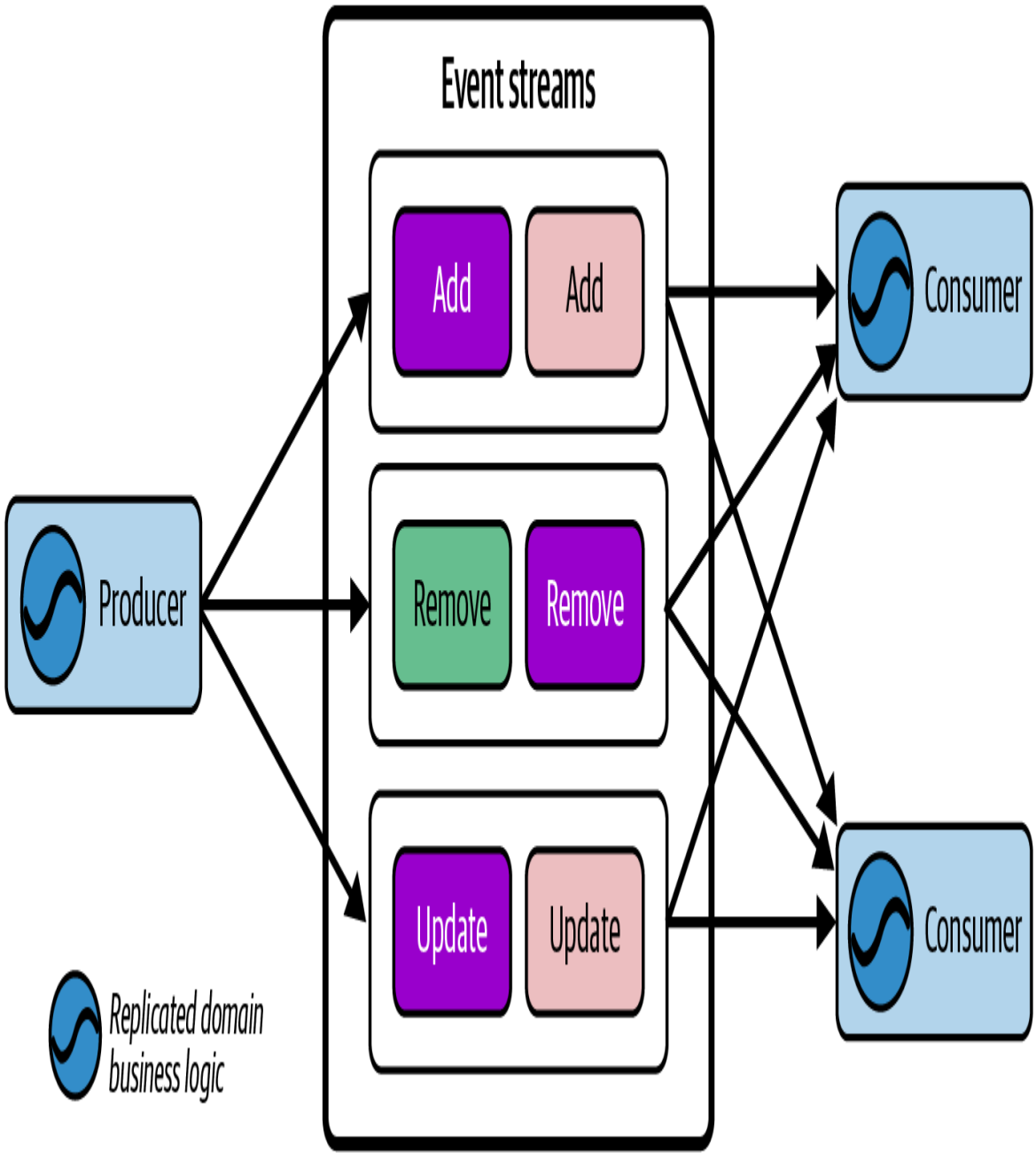


Figure 4-9. The logic to interpret delta events to build state is copied into multiple locations

Intermittent issues can cause further complexities—an event stream hosted on a lagging broker may experience delays in providing some events, resulting in the consumer receiving them out of order from events in other streams. Deltas applied in the wrong order often yield incorrect state transitions and may trigger incorrect business actions.

Additionally, each consumer may implement its own aggregation logic slightly differently—often because a consumer fails to update the aggregation logic as the domain evolves. One consumer may wait up to 30 seconds for late-arriving events, while another consumer may not wait at all and simply discard any late arrivals, resulting in similar yet different aggregates.

Any changes to how the producer aggregates its internal domain, including new events or changed delta semantics, must be propagated to the consumer logic—if you have worked on distributed services (or microservices) before, you may be shuddering at this idea. Using delta events to communicate between domains tightly couples the producer, the event definitions, and the consumers together, and trying to manage this is an exercise in futility.

Delta events map poorly to event streams

In the problems discussed so far, we've operated under the assumption that any new delta events will be immediately identifiable and understandable to consumers, though they may not yet understand how to apply those events to the domain. The reality is far messier. Delta event consumers must be notified when new deltas are created so that they can update their code to integrate the event into their data model.

Coordination can be quite difficult, particularly when there are many different consumers. Herein lies the main problem of this subsection: how do consumers know about the new domain events that they must consider in their model?

One common suggestion that unfortunately misses the point is to simply "put it all in the same event stream so that the consumers have access to it and can choose if they need to use it." Although existing consumers will end up receiving these new event types, this proposal does nothing to solve the code changes and integrations for consumers to use that data.

Additionally, it is far more likely to cause the consumer to throw an exception or skip processing the data. Worse yet, your consumer may experience silent processing errors in its business logic, leading to

cryptically wrong results. This also violates the convention of using a single evolvable schema per event stream, which is a common practice for many of the frameworks and technologies that process event streams.

The critical issue here is that new event definitions require working with those that aggregate the events into a model. If you put the new delta events into new individual streams, you make discovery easier and follow the one-schema-per-stream convention, but your consumers will still need to be manually notified that this new stream exists! In either case, a code update is required to make any sense of this data, while a failure to incorporate it runs the risk of an incorrect aggregate.

Now contrast it with the state model, where the state domain can change as needed and the composition of the state event is encapsulated entirely within the producer service. Any modifications made to the state event occur in one place and one place only, and is reflected in the updated data model published to the event stream. Yes, you may need to handle a schema evolution of the state model event, but only in circumstances that cause a breaking change to the entity data model.

Inversion of ownership: Consumers push their business logic into the producer

The fourth problem with deltas revolves around the ownership and location of business logic. For example, a consumer may need to know when a package has been shipped so that it can send out an email to the intended recipient notifying them that it's on its way. The business logic for determining that the package has shipped must necessarily live in the producer, as in Figure 4-10.

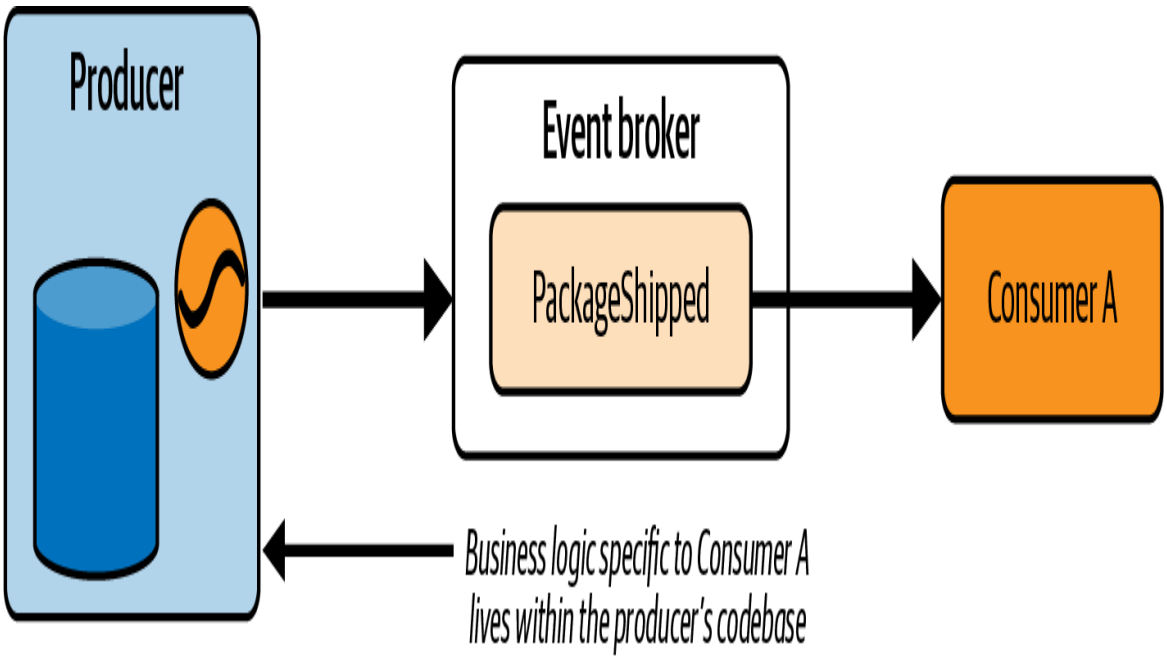


Figure 4-10. Consumer business requirements are pushed into the business logic of the producer; in this case, Consumer A only wants to know when a package is shipped, but not when the package has any other status

However, this quickly becomes untenable with the growing scope of business use cases. Each new business requirement that relies on state transition will similarly need to place its business logic within the producer service (see Figure 4-11) to generate events whenever that "edge" happens. This is prohibitively difficult to scale and manage, let alone track ownership and dependencies.

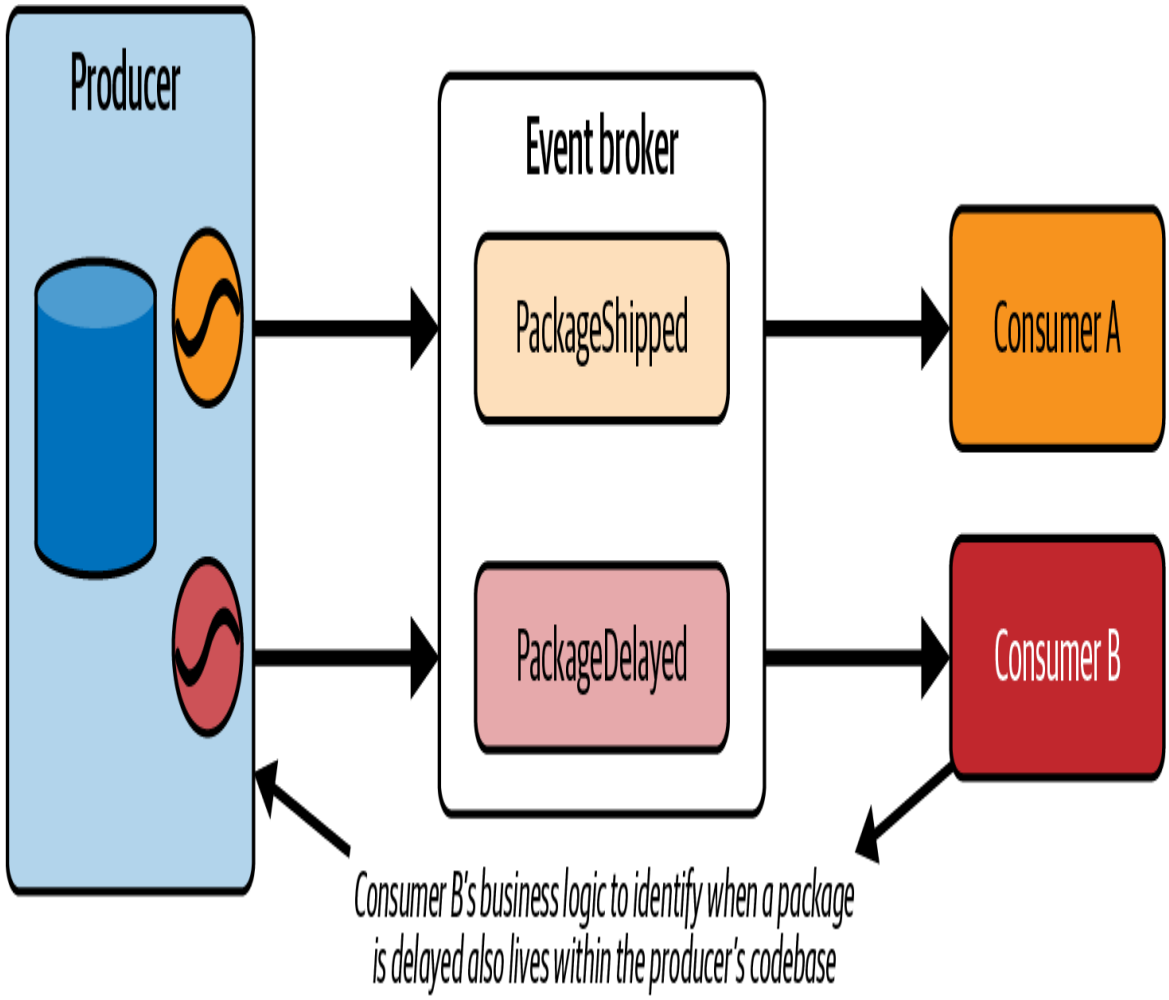


Figure 4-11. The scope of consumer requirements can grow quite large, as there are many possible deltas for most domains of any complexity

The entire purpose of delta events is to avoid maintaining state in the consumer service, but they require that the producer be fully able and willing to fulfill business logic solely for the consumer. For example, consider these reasonably plausible use cases:

- I want to track returns where a user had previously called in to complain: a `userReturnedItemAfterTelephoneComplaint` event.
- I want to know if the user has seen at least three ads for the item and then subsequently purchased it: a `userSawAtLeastThreeAdsThenPurchasedIt` event.

These sample events may seem a bit over the top, but the reality is that these are the sorts of conditions that businesses *do* care about. In each case, the consumer should maintain its own state and build up its own computations of these occurrences but instead avoids it by pushing the responsibility of detecting the edge back to the producer. The resultant is a very tight coupling between the producer and the consumer, and leads to spreading the complexity across multiple code bases.

A final factor is that a single system is seldom able to provide all of the information necessary for these highly specialized events. Consider the example of Figure 4-12. In this example, the consumer needs to act when state from the `advertising` service and the `payments` service (both within their own domains) meet a certain criterion: the user must have been shown an advertisement three times and then eventually have purchased that item.

Even if we convinced the advertising team to produce `userSawAdvertisementThreeTimes` and `userReturnedItemAfterTelephoneComplaint` events, the consumer would still need to store it in its own state store and await the matching purchase from the `payments` service. Even the most complex and convoluted event definition cannot account for handling data that resides entirely in another domain. The consumer must still be able to maintain state, despite our best efforts to avoid it.

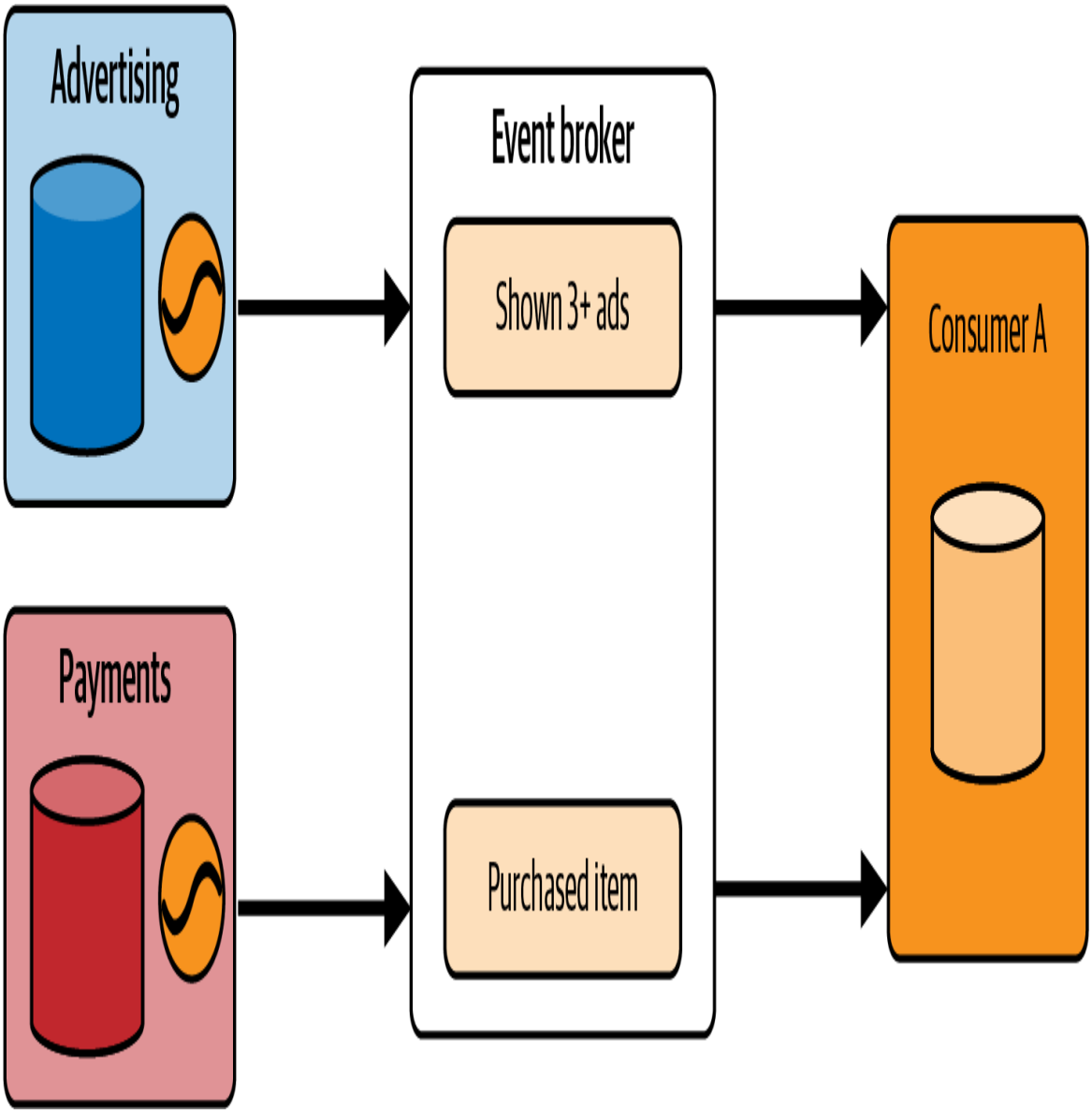


Figure 4-12. Consumer-specific delta event triggering logic is pushed upstream to both the advertising and payments service

And what if our consumer wants to change its business logic from three ads to four? A whole new event definition needs to be negotiated and put in the producer's boundary, which should give you an idea of how poorly this idea fares in practice. It is far more reasonable that the producer output a set of *general purpose state* and let the consumer figure out what it wants to do with those data sets.

Inability to maintain historical data

The fifth and final point against delta events is based on the difficulty of maintaining *usable historical data*. Old state events can simply be compacted, but delta events cannot. It becomes substantially more difficult to manage the ever-increasing log of events as a source of historical information.

Each delta event is essential for aggregating the final state. And there may not only be a single event stream to deal with, but multiple delta streams relating to different deltas within the domain. Figure 4-13 shows an example of three simple shopping cart delta events that have grown very large over the past 10 years—so large that a new consumer might take, say, three weeks of nonstop processing to make it through the volume of data, just to catch up to the current state.

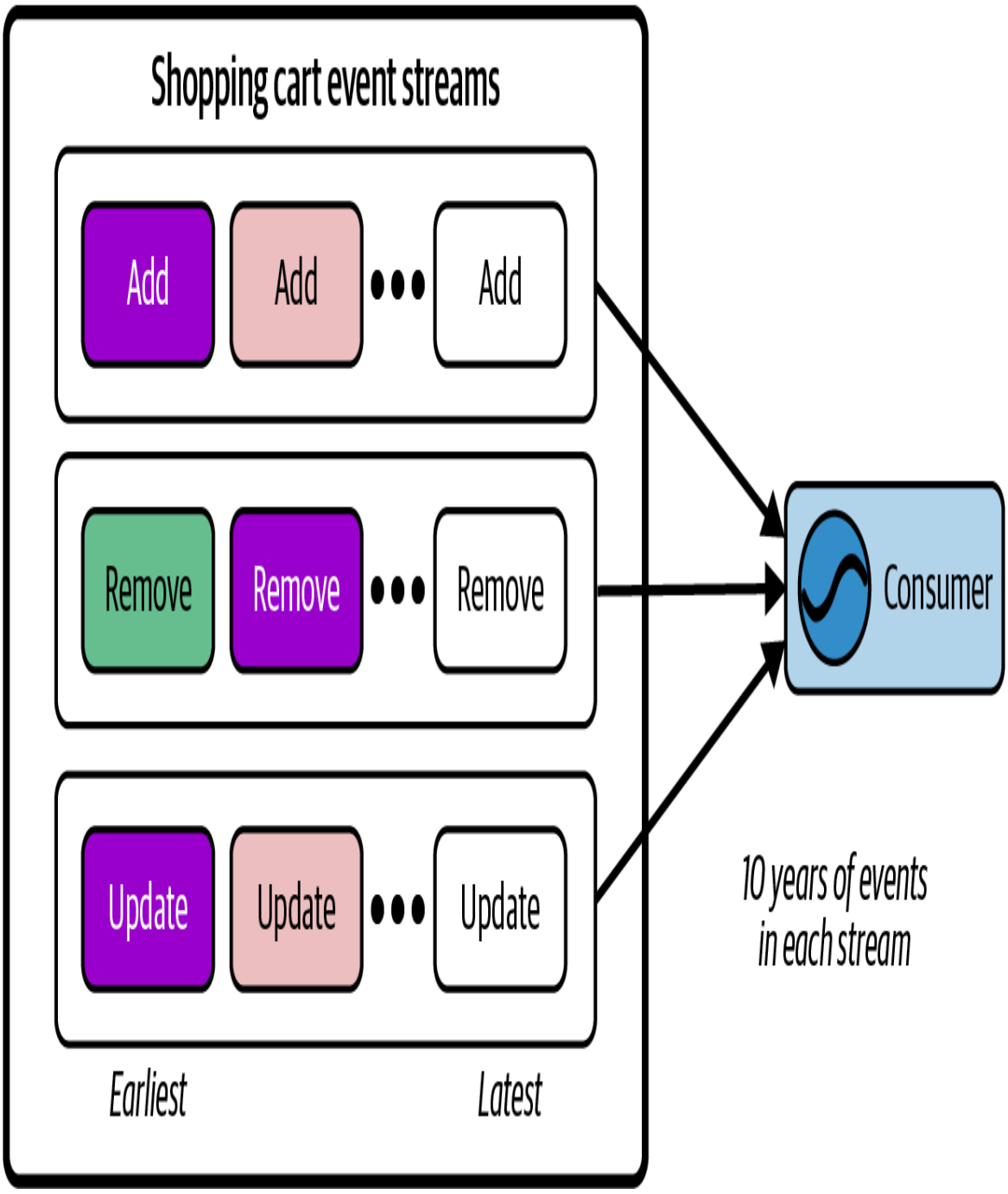


Figure 4-13. There are simply too many delta events in this stream for a new consumer to reasonably consume

While purging old data is certainly one solution, another solution that I have seen attempted is to offload older events into a large side state store, which can be sideloaded into a new consumer. The idea here is that the consumer can load all of these events in parallel, booting up far more quickly. The

problem is that the order in which these events are applied can matter, and just moving the events to a nonstreaming system only to stream them back into new consumers is a bit nonsensical. So the next solution is to build a *snapshot* of the state at that point in time based on all of the delta events. This is shown in Figure 4-14.

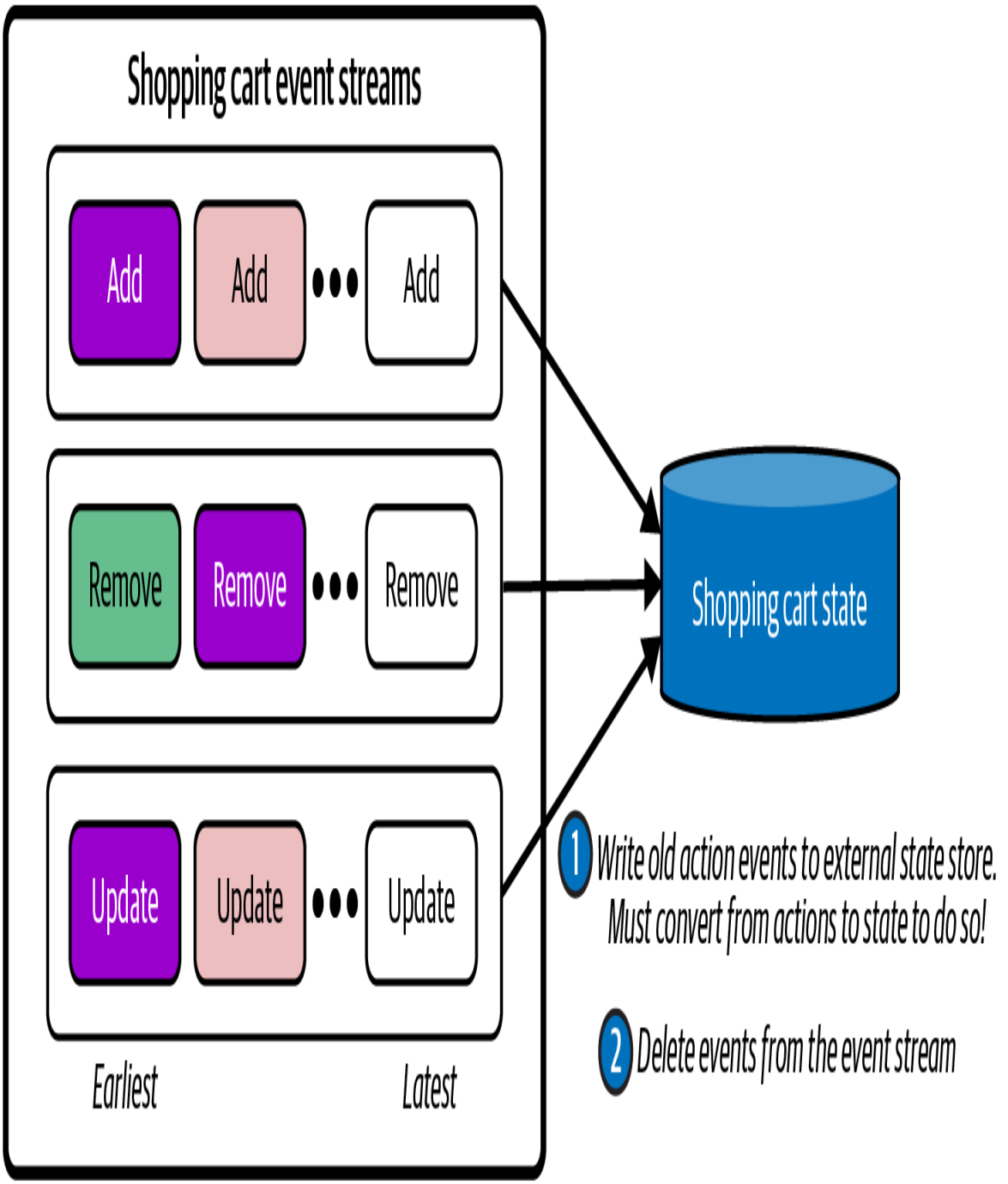


Figure 4-14. Loading the old events into a bootstrapping side store requires aggregating into a state model

There is a bit of irony here. In the attempt to avoid creating a publicly usable definition of state, we find ourselves doing exactly this out of necessity to store the data in a side store. New consumers can certainly boot up far more

quickly using it, but now they have to both read from the snapshot state store and then switch over perfectly to the event stream.

Figure 4-15 shows that we have now come full circle, back to the very Lambda architectures that we had been trying to avoid, along with all its operational complexity and inherent problems.

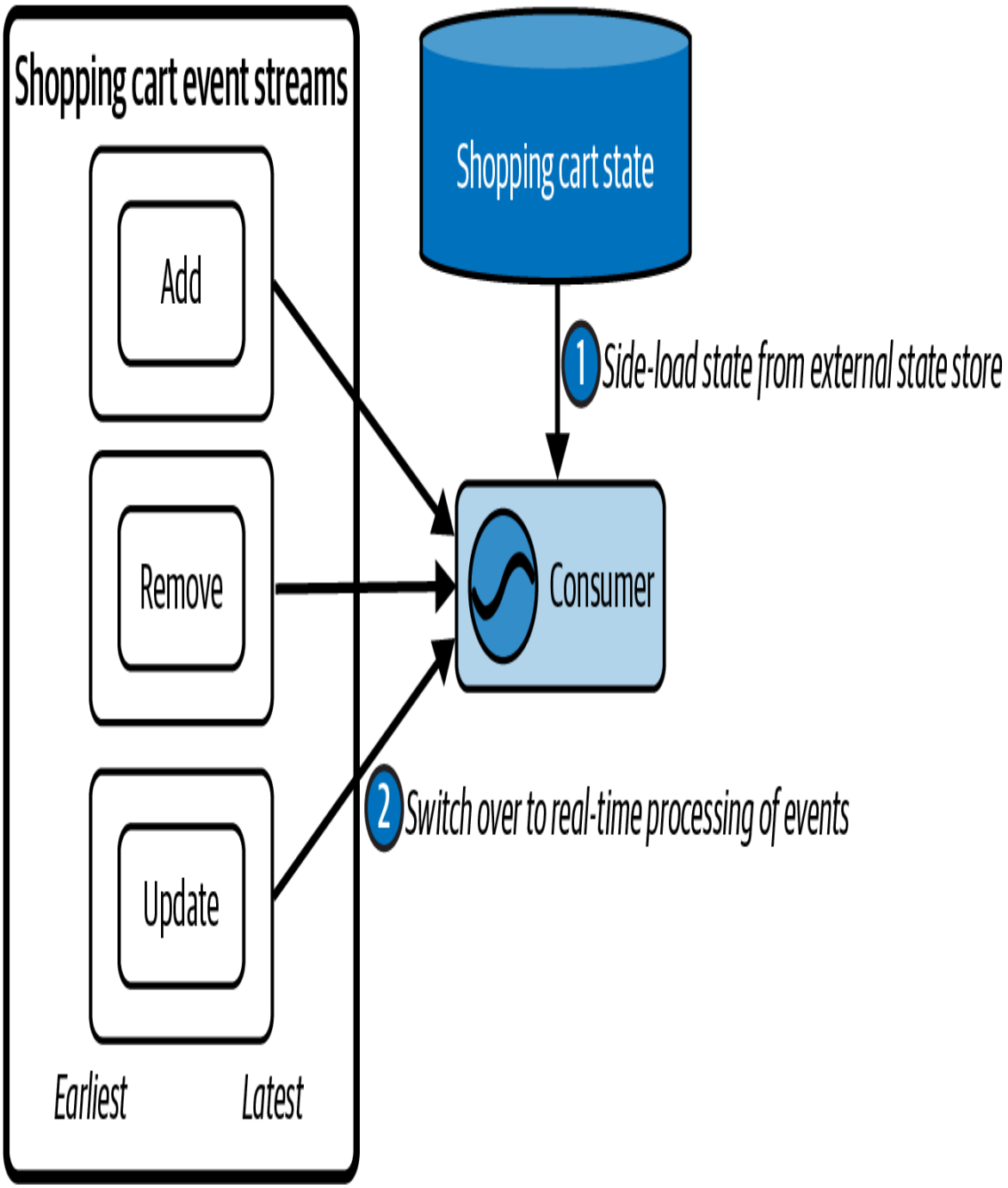


Figure 4-15. This brings us back around to the Lambda architecture involving both delta events, aggregated state, and the need to handle both batch and streaming

Do not let this section on delta problems sour you to their usage. They do have their place in event-driven microservices, as do state events. It's just a matter of knowing when and where to use them, and being aware of the pitfalls and hazards you may encounter along the way.

Where do I use Delta and State Events?

Delta events are best used within the internal boundary of a private domain, where the tight coupling of the event definitions and the logic required to interpret and apply them can be applied consistently. Delta events work fine within a single application for building up state via event sourcing, and can also be very successful between tightly coupled applications.

The following are unfortunately common yet insufficient arguments for using delta events for interdomain communication:

> *Maintaining duplicate state is wasteful, it's going to take up too much disk.*
>
> *The consumers will know what the event means. How could they possibly misinterpret it?*
>
> *C'mon, I really only care about this one transition, it's not a big deal if I couple on it. Just publish a custom event for me.*
>
> —That person who doesn't want to use state events

Using same delta events across domain boundaries is perilous, as illustrated by the five major problems covered in this section. Successfully building up state from delta events across domain boundaries is generally far more challenging than simply relying on state events for ECST. While it is attractive in the sense of reducing state storage requirements, the reality is that disk is extremely cheap in modern event-driven architectures. In contrast, a software engineer's time is far more expensive, and will easily dwarf the costs of state storage in break-fix work if delta usage is not carefully controlled.

Figure 4-16 provides an example for good practices when using state and delta events. In the first (1) bounded context, you can see that there are both delta and state event streams. They're tightly coupled, but they're owned by the same team, and they've decided that delta events work very well within their business use case. However, they do not emit the deltas outside for others to use, but rather produce a state stream.

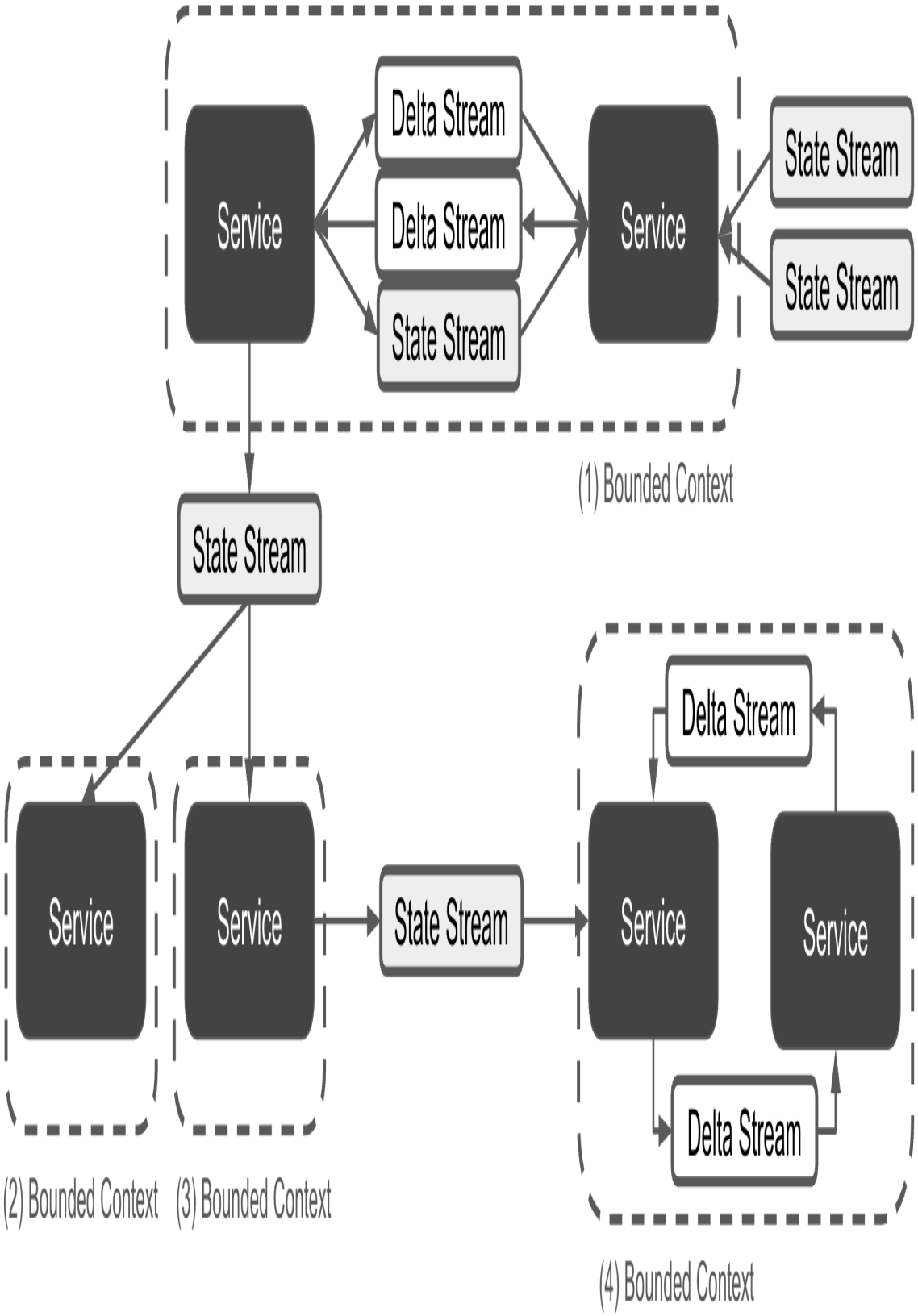

Service
Delta Stream
Delta Stream
State Stream
Service
State Stream
State Stream
(1) Bounded Context
State Stream
Service
Service
State Stream
Delta Stream
Service
Service
Delta Stream
(2) Bounded Context
(3) Bounded Context
(4) Bounded Context

Figure 4-16. State and delta event type usage across an organization

The second (2) and third (3) bounded contexts each consume from the state stream produced by bounded context (1). Both are fully independent of each other, and consume and use the state events as their business logic sees fit. They may also consume from other streams, but these have been omitted for diagram brevity.

Finally, the fourth (4) bounded context consumes from the third (3) bounded context's state stream. It also uses deltas internally to communicate between its two services. Perhaps these delta streams are using multiple event types per stream, and perhaps they're also using side state stores to provide snapshots of state. It doesn't really matter to the outside world, however, as these are concerns solely for the developers of the applications within bounded context four (4).

Having covered both deltas and state, it's now time to complicate things just a bit more with hybrid events.

Hybrid Events: State with a Bit of Delta

Hybrid events are a mixture of state and delta. It's best to think of these as state events that may contain a bit of information about *why* or *how* something happened. Let's look at an example for clarity.

> **CAUTION**
>
> Exposing "how" or "why" something has occurred with a hybrid event may lead to tight consumer coupling on the producer system's business logic. Proceed with caution.

Consider the following scenario. A company provides an online service that requires a user to sign up before using it. There are several ways that a user can sign up:

- Via the main sign up button on the home page

- Via an email advertising link
- Using a third-party account (a Google account, for example)
- The account was manually created for them by an administrator

The consumer wants to know *how* the user signed up. For operational use cases, we want to know which onboarding workflow to serve them when they next log in. For analytical purposes, we may want to know which of our methods of sign-up are the most common so we can allocate our development resources.

One way to model this sign-up is with a `user` state event, with a single enumeration indicating the sign-up mechanism (after all, you can only sign up once!). An example of the record would look like this:

```
Key: "USERID-9283716596927463"
Value: {
  name: "Randolph L. Bandito",
  signup_time: "2022-02-22T22:22:22Z",
  birthday: "2000-01-01T00:00:00Z",
  //An enum of (MAIN, VIA_AD_EMAIL, THIRD_PARTY, or ADMIN)
  method_of_signup: "VIA_AD_EMAIL"
}
```

To create the hybrid event, we incorporated what would otherwise be delta events into a single state event. Instead of `signed_up_via_email`, `signed_up_via_homepage`, `signed_up_via_third_party`, and `signed_up_via_admin` events, we flattened them down into a single enum and appended them to the `user` entity. The domain of values in the user state event needs to account for each of the possible enum settings: for example, we may also want to include information about which third-party sign-in provider was used or which email campaign got the user to sign up.

And herein lies the main issue with hybrid events. The precise mechanism of *how* something came to be in a domain is by and large a private detail, but by exposing this information we also expose the internal business logic process for coupling on by downstream consumers.

The main risk to the consumer of this information is that *how* a user signs up will change over time. This can be both a semantic change in meaning (what exactly is the "main" way to sign up now versus 5 years ago and 5 years in the future?), as well as the expansion or contraction of values in the enum. These semantics are usually only the concern of the source domain, but by exposing these delta-centric seams, they become a concern of the consumer.

There is also the chance (or likelihood) that the producer must update the hybrid event to account for a new means of sign-up: via the company's newly released mobile application (add `VIA_MOBILE_APP` to the `method_of_signup` enum). Consumers of this event must be kept informed of impending changes to this event and must confirm that they can handle processing of this new `method_of_signup` before the event definition is updated. If not, the consumers run the risk of encountering fatal errors during processing, because their business logic won't account for the new type. This is just another aspect of the same issue we saw in "The logic to interpret delta events must be replicated to each consumer".

However, in this example, the risk to the consumer is low, but not zero, for the following reasons:

- How a consumer signed up is immutable. The real risk lies in the meaning of `method_of_signup` drifting over time. The owner of the event can prevent this by providing very clear documentation of the enumeration's meaning (e.g., in the event schema itself) and adhering closely to its own definitions.
- The logic that populates `method_of_signup` is fairly simple overall, and so is much less likely to drift over time. Registering via an email link is a binary delta—you either registered via the email link or you didn't. In contrast, an enum based on the `userReturnedItemAfterTelephoneComplaint` delta event from earlier in the chapter has many more sequential dependencies and ways of misinterpreting it, and is far more likely to drift in meaning over time.

A hybrid event is a trade-off. The risk you incur in using a hybrid event is proportional to the complexity of the delta you are trying to track and the likelihood that it will change over time (intentionally or not). I advise that you try to further decouple your producer and consumer systems to avoid communicating the details of *why* or *how* data has changed. If you choose to include a delta-type field in your event, be aware that it becomes part of your data contract, and carefully consider the coupling it introduces with the source system.

Measurement Events

Measurement events are commonly found in many domains and consist of a complete record of an *occurrence* at a point in time. There are common examples of this in our everyday world: website analytics, perhaps most familiarly embodied by Google Analytics, is one. The user behavior tracking that occurs on every single website, social media experience, and mobile application is another. Every time you click a button, view an ad, or linger on a social media post, it is recorded as a measurement event.

What does a measurement event look like? Here's an example of a user behavior event recording the event of a user seeing an advertisement on a webpage:

```
Key: "USERID-8271949472726174"
Value: {
  utc_timestamp: "2022-01-22T15:39:19Z",
  ad_id: 1739487875123,
  page_id: 364198769786,
  url: https://www.somewebsite.com/welcome.html
}
```

A measurement is a snapshot of state at a specific point in time. However, measurements have a few characteristics that differentiate them from the state events we discussed earlier.

Measurement Events Enable Aggregations

Measurements are often used to create aggregations around a particular key. For example, the `userViewedAd` measurement could be used to compute a multitude of data sets, answering questions like "What is the most popular page_id?", "When do users see the most ads?", and "How many ads does each user see, on average, in a session?"

Measurement Event Sources May Be Lossy

It is not uncommon to lose measurements somewhere between their creation and ingestion into the event stream. For example, ad-blockers are very good at blocking web analytical events, such that your reports and dashboards are unlikely to be completely accurate. They are, however, often *good enough* for many analytical purposes.

Measurement Events Can Power Time-Sensitive Applications

Consider a factory that measures temperature, humidity, and other air quality metrics on its assembly line. One *analytical* use case for these measurements may be to track and identify long-term trends of the factory environment. But an *operational* use case may be to react quickly in the case of divergent sensor values, altering the assembly line throughput or shutting it down altogether if the environmental conditions fail to meet specifications.

In the case of network connectivity issues, it may be the case that the sensors are waiting to publish data that is now 30 to 60 seconds old, while new data piles up behind it. Depending on the *purpose* of the measurement stream and its pre-negotiated service-level objectives, it may choose to discard the old events and simply publish the latest. It really depends heavily on whether this data is being used for real-time purposes or whether it's being used to build a comprehensive historical picture that is tolerant of outages and delays, as is the case in web analytics.

COLLECTING AND USING MEASUREMENTS IN PRACTICE

Early in my career, I worked at RIM, now BlackBerry, collecting measurement data from internal developer BlackBerry devices. Basically, whenever a "bad thing" happened on a device, we would generate a dump of measurements, package it up, and send it to our backend servers for further processing. "Bad things" included dropped calls, dropped text messages, BlackBerry Messenger failing to send messages, cellular modem chip resets, along with custom triggers generated by key business applications. The purpose was to collect all of these measurements for both automated generation of problem reports and to aid developers in debugging.

There are a couple of key things about collecting measurements that I learned then and have carried with me ever since. For one, it was extremely important to have a well-defined schema for the payloads under your control, as it made automated post-processing so much easier (null pointers anyone?) and reduced time spent adding special logic to handle malformed data. Secondly, measurement completeness was more important than real-time performance. It wasn't uncommon for us to receive measurement events that were hours, days, or even weeks old—this could be due to test devices that may have been temporarily disconnected, such as an executive flying from North America to Asia, or even just one of our developers getting stuck in a tunnel on their commute to work. But the SLOs that we had issued to our dependent consumers reflected this, and we only started getting hounded for data if we missed our daily report.

Notification Events

There's one last event type to discuss before we wrap up the chapter. A notification contains a minimal set of information that *something has happened* and a link or URI to the resource containing more information.

Mobile phones are probably the most familiar source of notifications—you have a new message, someone liked your post, or you have enough hearts to resume your free-to-play game—tap here to go to it.

An example of a simple behind-the-scenes notification you may receive on your cell phone could look something like the following. Your instant message application sends out a `NEW_MESSAGE` notification, including a status (for icon display), the name of the application, and a click-through URI to the application itself:

```
Value: {
  status: "NEW_MESSAGE",
  source: "messaging_app",
  application_uri: "/user/chat/192873163812392"
}
```

Notification events are often misused as a means of trying to communicate state without sending state itself. Instead, a pointer to the state is sent in the notification, with the expectation that the recipient will log into the source server and obtain the data. The following shows just such an example, where the notification includes that the status has changed, and there is an access URI to find the complete current state:

```
Key: 12309131238218
Value: {
  status: "PARTIAL_RETURN",
  utc_timestamp: "2021-21-13T13:11:42Z",
  access_uri: "serverURI:8080/orders/values/12309131238218"
}
```

At first glance, this seems to be a neat and trim solution: it allows the consumer to simply query for the full public state upon receiving the event without copying or exposing that data elsewhere. One of the major issues is that the event doesn't actually provide a record of the state *at that point in time*—unless the data contained at `access_uri` is completely immutable (it usually isn't). Since this anti-pattern is usually built on top of a mutable state store, by the time you receive the `PARTIAL_RETURN` notification, the

associated state at `access_uri` may have already been updated again to a new state.

This race condition makes notifications an unreliable mechanism for communicating state. For example, a sale with status updates of `SOLD -> PARTIAL_RETURN -> FULL_RETURN` will emit three distinct events, one for each state. A consumer lagging behind on its processing may not be able to access the `PARTIAL_RETURN` state before it finalizes to `FULL_RETURN` and thus completely miss that full state transition. To make matters worse, a new consumer processing the backlog will not see any of the previous state—only whatever is stored in the `access_uri` at the current wall-clock time.

A final blow to this design is that it adds far more complexity. Not only must the domain owner of the notification publish events, but it must also serve synchronous requests pertaining to that state. This includes managing access control, authorization, and performance scaling for both the event-stream producer and the synchronous query API.

Instead, it is far better just to produce the necessary state of the event as an immutable record of that point in time. It takes very little effort and greatly simplifies data communication between domains.

Event Design for Data Privacy and the Right to Be Forgotten

Private data, such as personally identifiable information (PII) and financial information, tend to have high security requirements. These requirements are often codified in law. For example, General Data Protection Regulation (GDPR) applies to any organization that processes data of people located within Europe. It requires the secure handling, storage, and deletion of customer data. ttps://oreil.ly/d3620[Article 17] requires providing individuals the right to have their personal data deleted upon request, without a significant delay.

However, event streams are immutable. You can't go in and alter the data once its been written. While [[Link to Come]] covers some techniques for

handling bad data that you can apply to eliminating PII from your streams, your best bet is to encrypt the sensitive data first.

Field-level encryption ("Field-Level Encryption") is one encryption option, while another is to encrypt the entire event. In both cases, you'll probably use a key-management service (KMS) to hold on to the keys and provide a secure API for requesting access. It's common to use a unique key *per user*, *per account*, or *per other important entity*. Why? When it comes time to delete the data of the specific, say, user, you can instead just delete the key in the KMS to make the entire body of encrypted data inaccessible. This is known as crypto-shredding.

It's important to note that crypto-shredding simply makes the data cryptographically unavailable. It is *usually* sufficient for meeting data privacy requirements, but you'll need to check with your own company's lawyers to ensure you stay on the right side of the law.

Consumers of the encrypted events can contact the KMS and request access to the decryption keys. Provided they have the correct permissions, the KMS passes them back the decryption keys to allow them to decrypt the data themselves.

> **WARNING**
>
> Ensure that your consumers maintain the same level of encyption as the source stream when storing event data to its private data store. Microservice owners are responsible for securing their own data, which means its best to avoid storing sensitive data unless its absolutely necessary for your service.

There are several complications that make event encryption and crypto-shredding an important consideration:

Large amounts of data

> Large amounts of data may be stored in backups, tape drives, cold cloud storage, and other expensive and slow-to-access mediums. It can be very expensive and extremely time-

consuming to read in historical data, selectively delete records, and then write it back to storage. Crypto-shredding enables you to avoid having to search through every single piece of old data in your organization.

Partially encrypted data is still useful

Deleting just a user's PII is often sufficient for meeting the GDPR Article 17 requirements, while the remaining data may still be useful for your consumers. Field-level encryption lets you select which data you want to encyrpt, instead of encrypting the whole event.

Data across multiple systems

Deleting the decryption keys simultaneously halts data access across all consumer services. You don't need to worry about when the data is deleted, especially for systems that are slow to delete their data.

Further defense in depth

Crypto-shredding provides an additional layer of security for preventing data security incidents. Leaking encrypted data is far less damaging than leaking unencrypted data, and helps reduce both the risk and the impact of a data security breach.

Crypto-shredding doesn't protect you from consumers who negligently store decrypted data or the decryption keys. Ensure that consumers have clear and simple info security policies to follow, time-limit decryption key retention, and never write decrypted data to disk.

Data encryption is an important component of designing events, particularly as it can be difficult to add it in later. It will also most likely result in a breaking change for your consumers, as they will need to implement

decryption operations, KMS connectivity, and validate their data retention policies against the data privacy needs.

Summary

We covered a lot of ground in this chapter, so let's take a moment to recap before moving on.

Events can primarily be defined as state or delta. State events enable event-carried state transfer and are your best option for communicating data between domains. State events rely on event broker features, such as indefinite retention, durable state, and compaction to help us manage the volume of events. The state design allows us to leverage the event broker as the primary source of data, enables the use of the Kappa architecture, and deftly avoids the pitfalls associated with its predecessor, the Lambda architecture.

Delta events are a common way of thinking about event-driven architectures, but they are insufficient for cross-domain communication. Deltas belong firmly in the camp of the event sourcing and tightly-coupled inter-application communication. They can be invaluable for communication within a singular bounded context. Misuse of delta events occurs when coupling by external parties is allowed. This results in the exposure of internal business logic, processes, and events that should remain private. Simply put, do not use delta events for cross-domain coupling.

Measurement events record occurrences, such as those from human users, distributed systems, and Internet of Things (IoT) devices. Measurement events have their roots in the data analytics domain and consist of a snapshot of the localized state at a precise moment in time. These events are frequently used to compose detailed aggregates or to react to rapid measurement changes.

Both hybrid and notification events should be used with caution, if at all. Hybrid events are primarily a state event but can expose information pertaining to *why* something happened, akin to a delta event. This forms a

seam that introduces tight coupling, particularly when the *why* changes with time, and can lead to tight coupling and dependencies spread across multiple services.

While this chapter has discussed how to design events, the next chapter shows how to generate events from existing data sources. The vast majority of event-driven microservice architectures emerge as an expansion to existing systems and services, and getting their business data into event streams is of the utmost importance. The state event type will feature heavily in the next chapter.

Chapter 5. Integrating Event-Driven Architectures with Existing Systems

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 6th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

Transitioning an organization to an event-driven architecture requires the integration of existing systems. Your organization may have one or more monolithic relational database applications. Point-to-point connections between various implementations are likely to exist. Perhaps there are already event-like mechanisms for transferring bulk data between systems, such as regular syncing of database dumps via an intermediary file store location. This chapter covers how you can start making important business data available through event streams instead.

In any business domain, there are entities and events that are commonly required across multiple subdomains. For example, an ecommerce retailer will need to supply product information, prices, stock, and images to various bounded contexts. Perhaps payments are collected by one system but need to be validated in another, with analytics on purchase patterns performed in a

third system. Making this data available in a central location as the new single source of truth allows each system to consume it as it becomes available. Migrating to event-driven microservices requires making the necessary business domain data available in the event broker, consumable as event streams. Doing so is a process known as *data liberation*, and involves *sourcing* the data from the existing systems and state stores that contain it.

Data produced to an event stream can be accessed by any system, event-driven or otherwise. While event-driven applications can use streaming frameworks and native consumers to read the events, legacy applications may not be able to access them as easily due to a variety of factors, such as technology and performance limitations. In this case, you may need to *sink* the events from an event stream into an existing state store.

There are a number of patterns and frameworks for sourcing and sinking event data. For each technique, this chapter covers why it's necessary, how to do it, and the tradeoffs associated with different approaches. Then, we'll review how data liberation and sinking fit in to the organization as a whole, the impacts they have, and ways to structure your efforts for success.

What Is Data Liberation?

Data liberation is the identification and publication of cross-domain data sets to their corresponding event streams and is part of a *migration strategy* for event-driven architectures. Cross-domain data sets include any data stored in one data store that is required by other systems. Point-to-point dependencies between existing services and data stores often highlight the cross-domain data that should be liberated, as shown in Figure 5-1, where three dependent services are querying the legacy system directly.

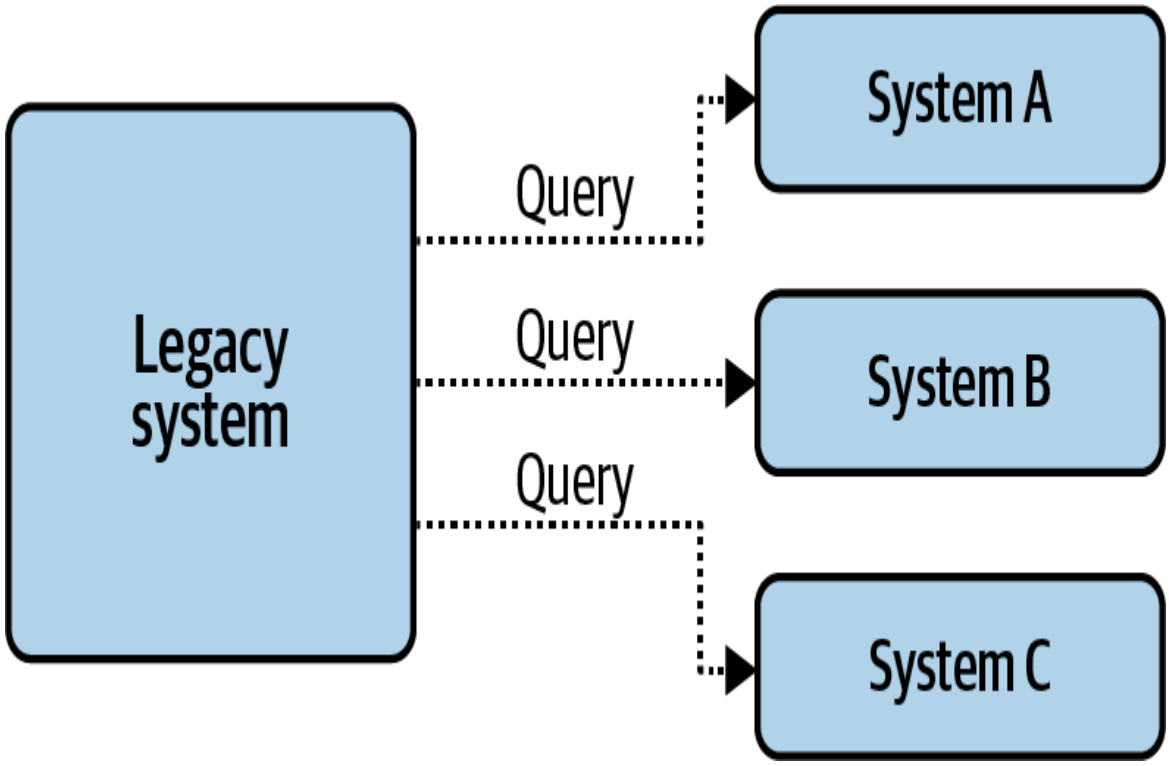


Figure 5-1. Point-to-point dependencies, accessing data directly from the underlying service

Data liberation enforces two primary features of event-driven architecture: the single source of truth and the elimination of direct coupling between systems. Systems no longer need to couple directly to the underlying data stores or application APIs, but instead can couple solely on the event streams. The post-liberation workflow is shown in Figure 5-2.

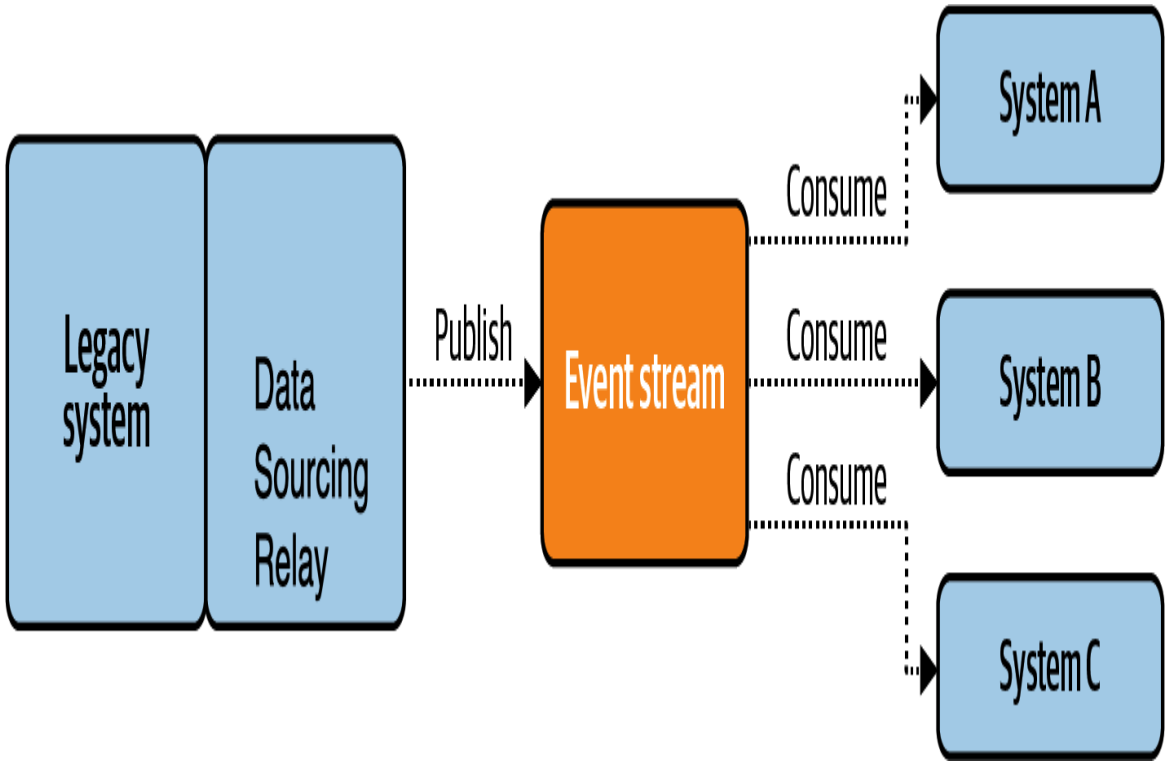


Figure 5-2. Post-data-liberation workflow

The Dual Write Anti-Pattern

The dual write anti-pattern is when a service writes data to two data stores independently of one another. Figure 5-3 illustrates a service writing a block of data to both a state store and an event stream in an event broker. The service cannot use a simple transaction to commit the data as both services are fully independent of one another. Committing data to one service may succeed with the commit to the other subsequently failing.

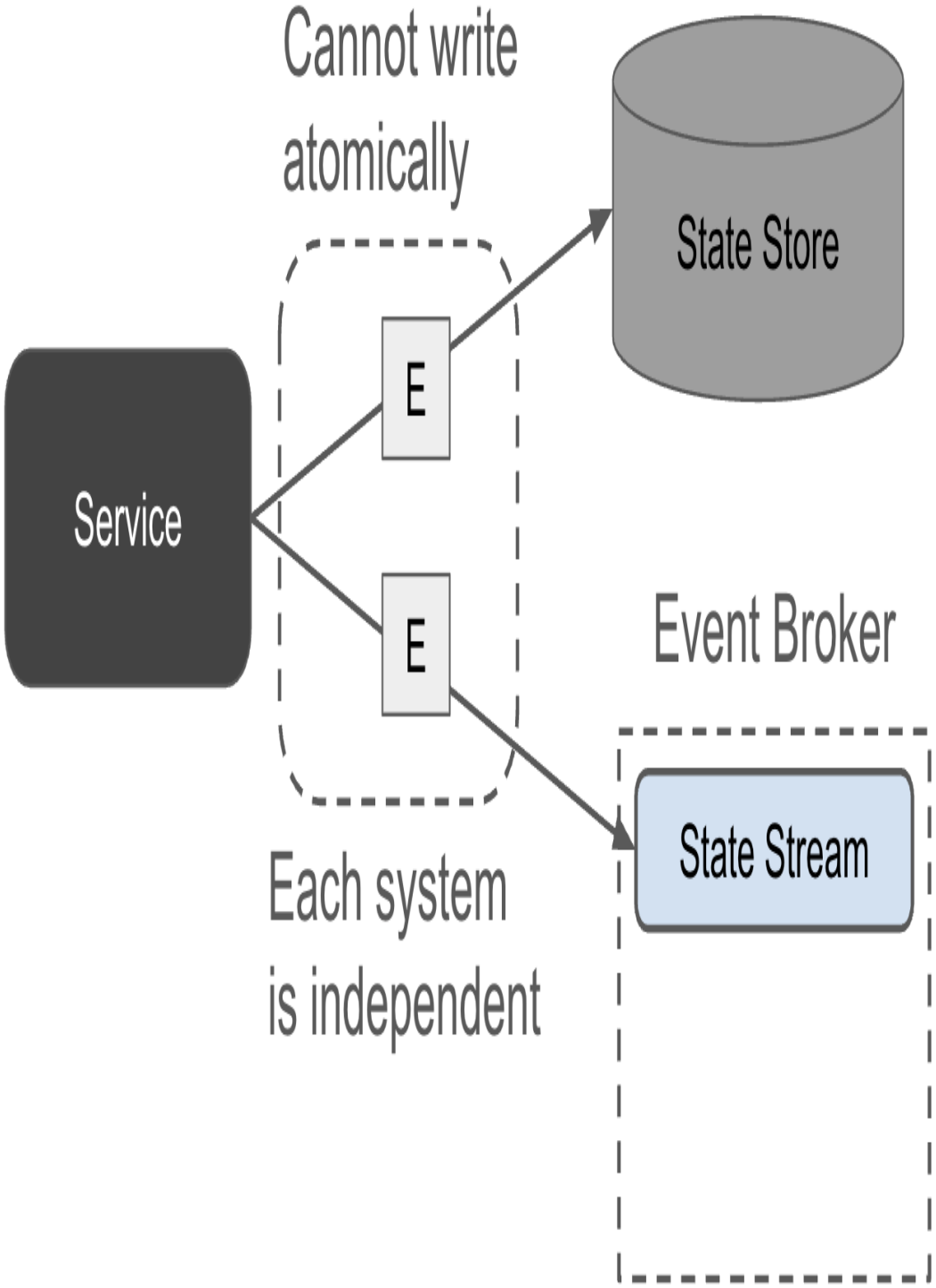

Cannot write
atomically
State Store
E
Service
E
Event Broker
State Stream
Each system
is independent

Figure 5-3. A service cannot write to a datastore and an event stream in a single atomic transaction

A developers that discovers they cannot write to both a data store and an event stream atomically may just shrug their shoulders and do it anyway, without the transaction. Their service may simply retry on a failure to commit, but there are other reasons why a commit may fail: invalid schema, service permanently moved, or a major prolonged outage due to hardware or configuration issues. Eventually the producer service will have to give up, at which point the data in the two services has permanently diverged.

Services that support two-phase commits (2PC) can indeed participate in a distributed transaction with multiple parties. But the reality is that many data stores do not support 2PC, nor do many event brokers. Apache Kafka has a project for 2PC, but it remains in progess at the time of publication of this book.

The good news is that there are several strategies that you use for liberating data that do not involve 2PC, covered in the next section.

Data Liberation Patterns

There are three main data liberation patterns that you can use to extract data from the underlying data store. Since liberated data is meant to form the new single source of truth, it follows that it must contain the entire set of data from the data store. Additionally, this data must be kept up to date with new insertions, updates, and deletes.

Query-based

> You extract data by querying the underlying state store. This can be performed on any data store.

Log-based

> You extract data by following the append-only log for changes to the underlying data structures. This option is

available only for select data stores that maintain a log of the modifications made to the data.

Outbox Table-based

In this pattern, you first push data to a table used as an output queue. Another thread or separate process queries the table, emits the data to the relevant event stream, and then deletes the associated entries. This method requires that the data store support both transactions and an output queue mechanism, usually a standalone table configured for use as a queue.

While each pattern is unique, there is one commonality among the three. Each should produce its events in sorted timestamp order, using the source record's most recent `updated_at` time in its output event record header. This will generate an event stream timestamped according to the event's occurrence, not the time that the producer published the event. This is particularly important for data liberation, as it accurately represents when events actually happened in the workflow. Timestamp-based interleaving of events is discussed further in Chapter 8.

Data Liberation Frameworks

One method of liberating data involves the usage of a dedicated, centralized framework to extract data into event streams. Examples of centralized frameworks for capturing event streams include Kafka Connect (exclusively for the Kafka platform), Debezium.io, and Apache NiFi. They'll connect to a whole suite of different data stores, applications, and SaaS endpoints with off-the shelf connection options. And some connector frameworks will also let you write your own connectors for less common use cases.

Each framework allows you to execute a query against the underlying data set with the results piped through to your output event streams. Each option is also scalable, such that you can add further instances to increase the capacity

for executing change-data capture (CDC) jobs (covered in the next section). They support various levels of integration with the schema registry offered by Confluent (Apache Kafka), but customization can certainly be performed to support other schema registries. See [[Link to Come]] for more information.

Let's take a look at our first mechanism for getting data into streams: change-data capture.

Liberating Data Using Change-Data Capture

One of the chief methods for liberating data relies on the data store's own underlying immutable log (e.g. binary log for MySQL, *write-ahead* logs for PostgreSQL). This immutable append-only data structure preserves the data store's data integrity. An inserted record is first written into the durable log before being applied to the underlying data model. If the database were to fail mid-write, then the durable log is replayed to the disk, to ensure that no information is lost and the data model remains consistent.

Change-data capture is a process that reads (or *tails*) these logs, converts the individual data store changes into events, and writes them to the event stream. These changes include the creation, deletion, and updating of individual records, as well as the creation, deletion, and altering of the individual data sets and their schemas. While many databases provide some form of read-only access to the durable logs, others, like MongoDB, provide CDC events directly instead of tailing the log.

The data store's log seldom contains the entire history of data. To turn the entire data store dataset into an event stream, you'll need to create a snapshot of the existing state from the data set itself. I'll cover that more in the next section on snapshotting.

Not all data stores implement an immutable logging of changes, and of those that do, not all of them have off-the-shelf connectors available for extracting the data. This approach is mostly applicable to select relational databases, such as MySQL and PostgreSQL, though any data store with a set of

comprehensive changelogs is a suitable candidate. Many other modern data stores expose event APIs that act as a proxy for a physical write-ahead log. For example, MongoDB provides a Change Streams interface, whereas Couchbase provides replication access via its internal replication protocol.

Figure 5-4 shows a MySQL database emitting its binary changelog. A Kafka Connect service, running a Debezium connector, is consuming the raw binary log. Debezium parses the data and converts it into discrete events. Next, an event router emits each event to a specific event stream in Kafka, depending on the source table of that event. Downstream consumers are now able to access the database content by consuming the relevant event streams from Kafka.

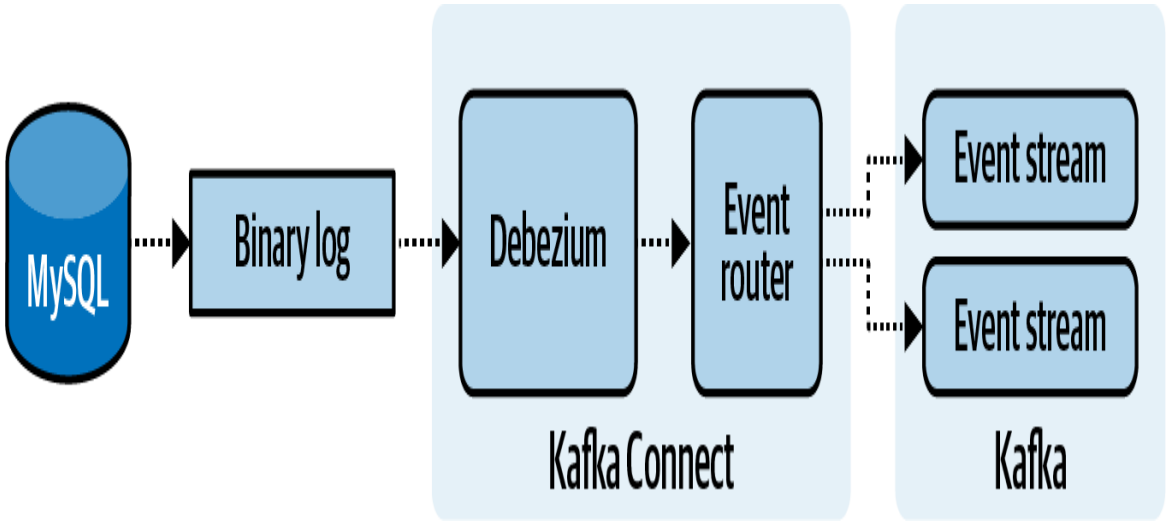


Figure 5-4. The end-to-end workflow of a Debezium capturing data from a MySQL database's binary log, and writing it to event streams in Kafka

CDC gives you *every* change made in the database. You will not miss any transitions, and your derived event stream will contain every update. CDC frameworks provide configuration files that let you specify which fields to include in your event, letting you filter out internal data from the outside world. You can also choose to include metadata like the `serverId`, `transactionId`, and other database or connector-specific content. CDC commonly uses `before` and `after` fields that detail the full state of the row or document before the change and after the change, as covered in "Before/After State Events".

You must checkpoint progress when capturing events from the changelogs, though depending on the tooling you use, this may already be built in. In the event that the change-data capture mechanism fails, the checkpoint is used to restore the last stored changelog index. This approach can only provide *at-least-once* production of records, which tends to be suitable for the entity-based nature of data liberation. The production of an additional record is inconsequential since updating entity data is idempotent.

While there are several options available for sourcing data from changelogs, Debezium remains the most popular and the defacto leader in the field. It supports many of the most popular data stores supports the most common ones, and can produce records to both Apache Kafka and Apache Pulsar.

Having covered the basics of CDC, let's now turn our attention to how you can get the initial data set into the event stream.

Snapshotting the Initial Data Set State

The data store log is unlikely to contain all changes since the beginning of time, as it's primarily a mechanism to ensure data store consistency. The data store continually merges the log data down into the underlying data store model, only keeping a short window of data (e.g. several GBs).

Snapshotting is the process of loading all the *current* data into the event stream, directly from the data store (and not the log). It can be a very resource-intensive activity, as you must query, return, copy, convert to events, and write to event streams every entity of data from the source data set. Only once you've completed your snapshot can you move to live capturing of records.

The snapshot usually involves a large, performance-impacting query on the table and is commonly referred to as *bootstrapping*. You must ensure that there is overlap between the records in the bootstrapped query results and the records in the log, so that you do not miss any record when you swich over to live CDC.

There are further complications beyond the size of the query. A data store typically serves live business use cases and cannot be interrupted or degraded without some sort of consequence. However, and without getting too into the weeds, snapshotting typically requires an unchanging set of data to ensure consistency, and to accomplish that, it usually locks the table that it is querying. This means that your important production table may be locked for several hours while the snapshot completes. I bet your database administrator will be thrilled. And even if it's brief, it may still be unacceptable. So what else can you do?

One option is to snapshot from a read-only replica. You take the snapshot against the replica, leaving the production data store alone. Once completed, you can unlock the table, which will then be populated by the replicated updates from the production data store. At this point, you can swap over to change-date capture, or a periodic query as covered in the next section.

Some CDC frameworks, like Debezium.io, provide a mechanism to snapshot specific data stores without locking the table. Normal operational reads and writes can continue uninterrupted with the resultant snapshot being eventually consistent with the current table's state. A Debezium blog post, explains this innovation. It is based on a paper from Netflix that states:

> *DBLog utilizes a watermark based approach that allows us to interleave transaction log events with rows that we directly select from tables to capture the full state. Our solution allows log events to continue progress without stalling while processing selects. Selects can be triggered at any time on all tables, a specific table, or for specific primary keys of a table. DBLog executes selects in chunks and tracks progress, allowing them to pause and resume. The watermark approach does not use locks and has minimum impact on the source.*
>
> —Andreas Andreakis and Ioannis Papapanagiotou, Netflix

Live table snapshots that don't degrade performance or block production use cases are a massive improvement from early CDC technologies. This let you focus on *using* the data to build event-driven applications, instead of just wrangling the data out of the data store.

The CDC system merges the incremental snapshot query results with the log records. The snapshot query data is written directly to the event stream, except in the case where a *newer* record from the log has already been written for the given key. In this case, the older record is discarded, as it is no longer a valid representation of the latest data. Once completed, the CDC process continues to tail the data store's log indefinitely.

Next, a few of the main benefits and drawbacks of using the CDC with data store logs.

Benefits of Change-Data Capture Using Data Store Logs

Benefits of using data store logs include the following:

Delete tracking

> Data store logs contain deletions, so that you can see when a record has been deleted from the data store. These can be converted into tombstone events to enable downstream deletions and stream compaction.

Minimal effect on data store performance

> For data stores predicated on logs, change-data capture can be performed without any impact to the data store's performance. For those that use change tables, such as in SQL Server, the impact is related to the volume of data.

Low-latency updates

> Updates propagate as soon as the event is written to the data store log, resulting in event streams with relatively low latency.

Nonblocking snapshots

> CDC can create snapshots without interfering with the normal operations of the source data store.

Drawbacks of Change-Data Capture Using Data Store Logs

Downsides to using data store logs include the following:

Exposure of internal data models

> The internal data model is completely exposed in the changelogs. Isolation of the underlying data model must be carefully and selectively managed.

Denormalization outside of the data store

> Changelogs contain only the event data. Some CDC mechanisms can extract from materialized views, but for many others, denormalization must occur somewhere downstream. This typically leads to the creation of highly normalized event streams, requiring downstream microservices to handle foreign-key joins and denormalization, which is covered in more detail in Chapter 6.

Brittle dependency between data set schema and output event schema

> Valid data model changes in the source data store, such as altering a data set or redefining a field type, may cause breaking changes to downstream consumers.

Liberating Data with a Polling Query

With query-based data liberation you query the data store, get the results, convert them into events, and write them to the event stream. You can write your own code to do this for you, or you can rely on the likes of Kafka Connect or Debezium, as mentioned previously. Given the wealth of connectors available today, I recommend that you start with a purpose-built framework instead of trying to rebuild your own.

> **NOTE**
>
> Query-based data liberation doesn't use the underlying data store log. Both snapshots and incremental iterations are consumerd entirely via the data store query API.

There are two main stages to the periodic query pattern: the initial snapshot, and the incremental phase.

The snapshot process is identical to that found in "Snapshotting the Initial Data Set State", though there is no data store log to swap over to once completed. The iterative phase starts where the snapshot left off.

Query-based polling requires identifying which records have changed since the last polling iteration. For example, an `updated_at` or `modified_at` timestamp is a common choice of field to evaluate in your query, as illustrated by the Kafka Connect JDBC connector. For each iteration of your polling loop, it uses the highest `updated_at` timestamp from the last query's results, then inputs it as the starting point for the next polling loop. The returned rows are converted into events and the connector updates its stored `updated_at` timestamp for the next loop. You may also choose to use an auto-incrementing ID if it is available in your table.

The event schemas are automatically inferred from the query results - at least for the off-the-shelf frameworks. If you're rolling your own polling query, then you'll need to generate a schema that suits your needs. That being said, if the upstream data store model changes, then you run the risk that your event schema will also change.

Query-based polling, particularly when rolling your own, relies heavily on locking tables to get a consistent state. It is overall a dated process for bootstrapping, and while it's still technically *valid*, it's far more common to use CDC over iterative queries. Since the first edition of this book CDC has substantially improved, with query-based polling relegated mostly to data stores and APIs that provide no underlying data store log access.

Incremental Updating

The first step of any incremental update is to ensure that the necessary timestamp or auto-incrementing ID is available in the records of your data set. There must be a field that the query can use to filter out records it has already processed from those it has yet to process. Data sets that lack these fields will need to have them added, and the data store will need to be configured to populate the necessary `updated_at` timestamp or the auto-incrementing ID field. If the fields cannot be added to the data set, then incremental updates will not be possible with a query-based pattern.

The second step is to determine the frequency of polling and the latency of the updates. Higher-frequency updates provide lower latency for data updates downstream, though this comes at the expense of a larger total load on the data store. It's also important to consider whether the interval between requests is sufficient to finish loading all of the data. Beginning a new query while the old one is still loading can lead to race conditions, where older data overwrites newer data in the output event streams.

Once the incremental update field has been selected and the frequency of updates determined, the final step is to perform a single bulk load before enabling incremental updates. This bulk load must query and produce all of the existing data in the data set prior to further incremental updates.

Benefits of Query-Based Updating

Advantages of query-based updating include the following:

Customizability

> Any data store can be queried, and the entire range of client options for querying is available.

Independent polling periods

> Specific queries can be executed more frequently to meet tighter SLAs (service-level agreements), while other more expensive queries can be executed less frequently to save resources.

Isolation of internal data models

Relational databases can provide isolation from the internal data model by using views or materialized views of the underlying data. This technique can be used to hide domain model information that should not be exposed outside of the data store.

> **CAUTION**
>
> Remember that the liberated data will be the single source of truth. Consider whether any concealed or omitted data should instead be liberated, or if the source data model needs to be refactored. This often occurs during data liberation from legacy systems, where business data and entity data have become intertwined over time.

Drawbacks of Query-Based Updating

Downsides to query-based updating include the following:

Required `updated-at` *timestamp*

The underlying table or namespace of events to query must have a column containing their `updated-at` timestamp. This is essential for tracking the last update time of the data and for making incremental updates.

Only detects soft deletes

Outright deleting a record from the data store will not result in any events showing up in the query. Thus, you must use a soft delete, where the records are marked as deleted by a specific `is_deleted` column.

Brittle dependency between data set schema and output event schema

Data set schema changes may occur that are incompatible with downstream event format schema rules. Breakages are increasingly likely if the liberation mechanism is separate

from the code base of the data store application, which is usually the case for query-based systems.

May miss intermittent data values

Data is synced only at polling intervals, and so a series of individual changes to the same record may only show up as a single event.

Production resource consumption

Queries use the underlying system resources to execute, which can cause unacceptable delays on a production system. This issue can be mitigated by the use of a read-only replica, but additional financial costs and system complexity will apply.

Query performance varies due to data size

The quantity of data queried and returned varies depending on changes made to the underlying data. In the worst-case scenario, the entire body of data is changed each time. This can result in race conditions when a query is not finished before the next one starts.

Transactional data inconsistencies

Consider a transaction that writes to two tables, with one table referencing the other by primary or foreign key. Periodically querying the source tables means that you may generate records for one stream that reference a value that in the other stream, though it hasn't yet been captured from its table. This can lead to intermittent errors when your consumers need to reconcile the data.

Liberating Data Using Transactional Outbox Tables

A transactional outbox table is a dedicated database table that acts as a buffer for data to be written to the event stream. When you update your *internal domain model*, you select only the data that you want to expose to the outside world and write it to the outbox. Then, a separate asynchronous process, such as a dedicated CDC connector, consumes the data from the outbox and writes it to the event stream. Figure 5-5 shows the end-to-end workflow.

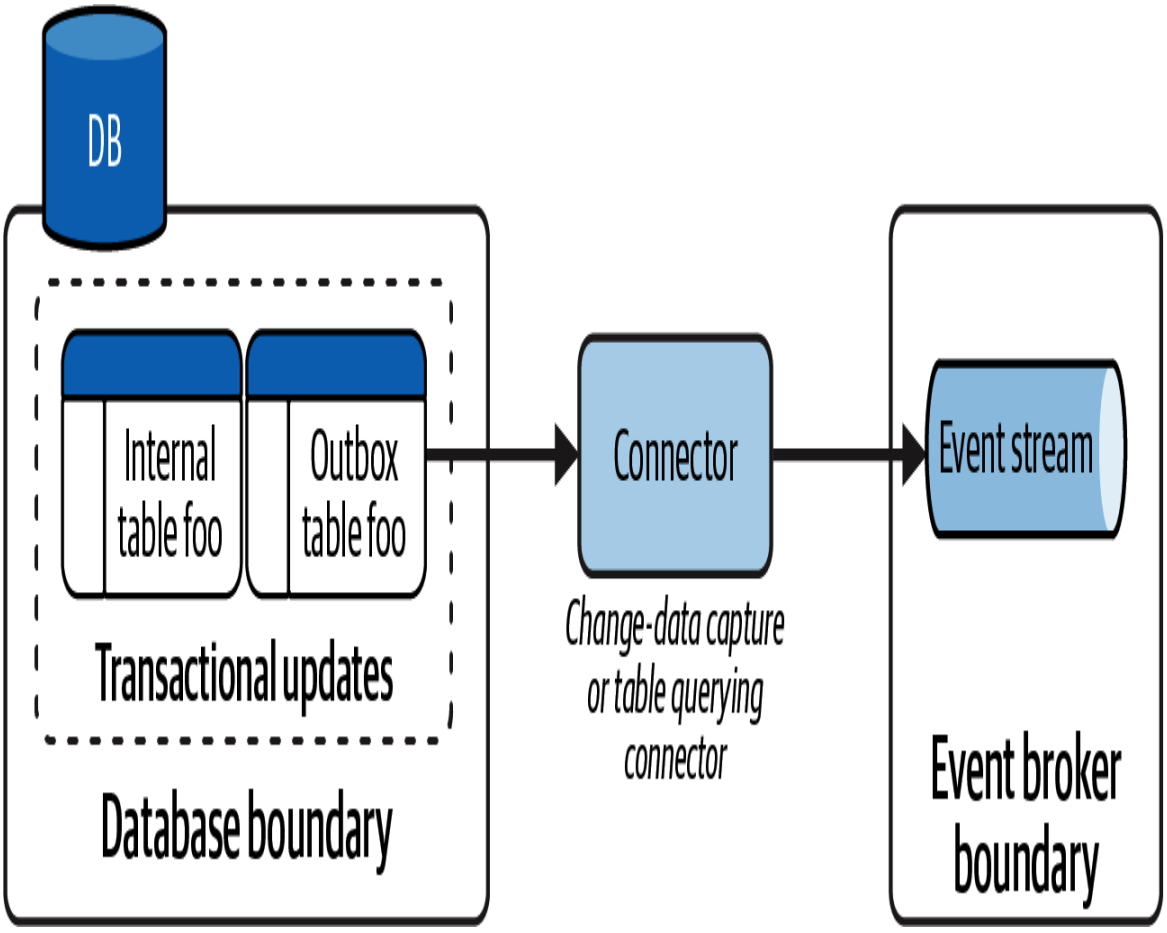


Figure 5-5. The end-to-end workflow of an outbox table and connector solution

The outbox table pattern leverages the transactional capabilities and durability of the data store to act as a *write-ahead log*. Updates to the internal table and the outbox are bundled into a *single transaction*, such that the updates occur only if the entire transaction succeeds. A failure to do so

may eventually result in divergence with the event stream as the single source of truth, which can be difficult to detect and repair.

Once the connector commits the records to the output event stream, you can delete the records from the outbox. In the case of any failure, be it the data store, the consumer/producer, or the event broker itself, outbox records will still be retained without risk of loss. This pattern provides at-least-once delivery.

Example 5-1 illustrates (in Python) an atomic update of an `EcomItem` in a MySQL database. The internal model update is executed prior to the transactional outbox update, though both are wrapped within a single transaction for consistency.

Example 5-1. Atomic update of the internal model and the transactional outbox

```
try:
    conn = mysql.connector.connect(host='localhost',
                                   database='python_db',
                                   user='abellemare',
                                   password='definitelynotpassword')
    conn.autocommit = False
    cursor = conn.cursor()

    # (1) Perform the internal domain model update
    internal_model_update = """
        Update EcomItem
        set price = 1299.99
        where id = 4291"""
    cursor.execute(internal_model_update)

    # (2) Select the subdomain of the internal model
    # to write to the transactional outbox
    internal_sub_model_query = """
        Select name, price
        from EcomItem
        where id = 4291"""
    cursor.execute(internal_sub_model_query)
    name_and_price = cursor.fetchone()

    if (name_and_price == None):
      raise Exception("Unexpected missing record. Can't get name_and_price")

    # (3) Insert the selected data into the outbox
```

```
    outbox_insert = """INSERT INTO EcomItem_Outbox (id, name, price)
                       VALUES (4291, %s, %s)"""

   # Pass the name and price in to replace the query wildcards
    cursor.execute(outbox_insert, name_and_price)

    # Commit the internal and outbox updates atomically
    conn.commit()

except mysql.connector.Error as error:
    # reverting changes because of exception
    conn.rollback()

finally:
    # Close the database connection
    if conn.is_connected():
        cursor.close()
        conn.close()
```

This example code first (1) updates the `EcomItem price` to `$1299.99`. Next (2), it selects the `name` and `price` from the table that was just updated. Finally, it composes the event and writes it into the `EcomItem_Outbox` format (3). Table 5-1 shows what this data looks like, which is really just the same as any other relational database table.

Table 5-1. `EcomItem_Outbox` table definition

id: INT NOT NULL	name: VARCHAR(255)	price: DECIMAL(13,2) NOT NULL	Datetime: DEF/ CURRENT_TIM
4291	“Fancy Laptop”	1299.99	2022-06-22 11:3

There are two things to note about this SQL table definition. One, is is using `NOT NULL` for each of the mandatory fields expected in our event. Inserting a new event into the outbox will fail unless all constraints are met. Two, the `Datetime` field uses the default `CURRENT_TIMESTAMP` if it is not provided by the application’s code. You may use this timestamp as part of the schema for

your event, or you may use it to populate the event's metadata to indicate when the event was created.

> **TIP**
>
> Ensure that the transactional outbox table schema and the event schema are compatible. The fields of one schema should map 1:1 with the other. Use pre-deployment scripts to validate that the outbox table schema and the event schema match to avoid easily preventable runtime errors.

The transactional outbox table is a relatively invasive approach to change-data capture as you must modify either the data store or the application layer, requiring involvement by the data store and/or application owner.

> **BUILT-IN CHANGE-DATA TABLES**
>
> Some databases, such as SQL Server, do not provide change-data capture logs, but instead provide change-data tables. These tables are often used to audit the operations of the database and come as a built-in option. External services, such as the aforementioned Kafka Connect and Debezium, can connect to databases that use a CDC table instead of a CDC log and use the query-based pattern to extract events and produce them to event streams.

The records in outbox tables require a strict ordering identifier to ensure that intermediate states are recorded and emitted in the same order they occurred. Alternatively, you may choose instead to simply upsert by primary key, overwriting previous entries and forgoing intermediate state. In this case, overwritten records will not be emitted into the event stream.

Performance Considerations

The inclusion of outbox tables introduces additional load on the data store and its request-handling applications. For small data stores with minimal load, the overhead may go completely unnoticed. Alternately, it may be quite

expensive with very large data stores, particularly those with significant load and many tables under capture. The cost of this approach should be evaluated on a case-by-case basis and balanced against the costs of a reactive strategy such as parsing the change-data capture logs.

Isolating Internal Data Models

An outbox table need not map 1:1 with the internal domain. In fact, one of the major benefits of the outbox is that the data store client can isolate the internal data model from downstream consumers. The internal data model may use several highly normalized tables that are optimized for relational operations but are largely unsuitable for consumption by downstream consumers. Even simple domains may comprise multiple tables, which if exposed as independent streams would require reconstruction for usage by downstream consumers. It can quickly becomes extremely expensive in terms of operational overhead, as multiple downstream teams will have to reconstruct the domain model and deal with handling relational data in event streams.

> **WARNING**
>
> Exposing the internal data model to downstream consumers is an anti-pattern. Downstream consumers should only access data formatted with public-facing data contracts as described in Chapter 3.

The data store client can instead denormalize data upon insertion time such that the outbox mirrors the intended public data contract, though this does come at the expense of additional performance and storage space. Another option is to maintain the 1:1 mapping of changes to output event streams and denormalize the streams with a downstream event processor dedicated to just this task. Chapter 6 covers these options in more detail, comparing and contrasting possible options.

The extent to which the internal data models are isolated from external consumers tends to become a point of contention in organizations moving

toward event-driven microservices. Some teams may be unable or unwilling to maintain data model isolation, most commonly citing a lack of resources. However, it's a cost that is paid either upstream at the producer, or paid downstream for each and every consumer of the data. Isolating the internal data model is essential for ensuring decoupling and independence of services and to ensure that systems need only change due to new business requirements. The next chapter covers data model isolation in more detail.

Ensuring Schema Compatibility

Schema serialization (and therefore, validation) can also be built into the transactional outbox workflow. Schema validation can be performed either before or after the event is written to the outbox table. Success means the event can be proceed in the workflow, whereas a failure may require manual intervention to determine the root cause and avoid data loss.

Serializing prior to committing the transaction to the outbox table provides the strongest guarantee of data consistency. A serialization failure will cause the transaction to fail and roll back any changes made to the internal tables, ensuring that the outbox table and internal tables stay in sync. This process is shown in Figure 5-6. A successful validation will see the event serialized and ready for event stream publishing. The main advantage of this approach is that data inconsistencies between the internal state and the output event stream are significantly reduced. The event stream data is treated as a first-class citizen, and publishing correct data is considered just as important as maintaining consistent internal state.

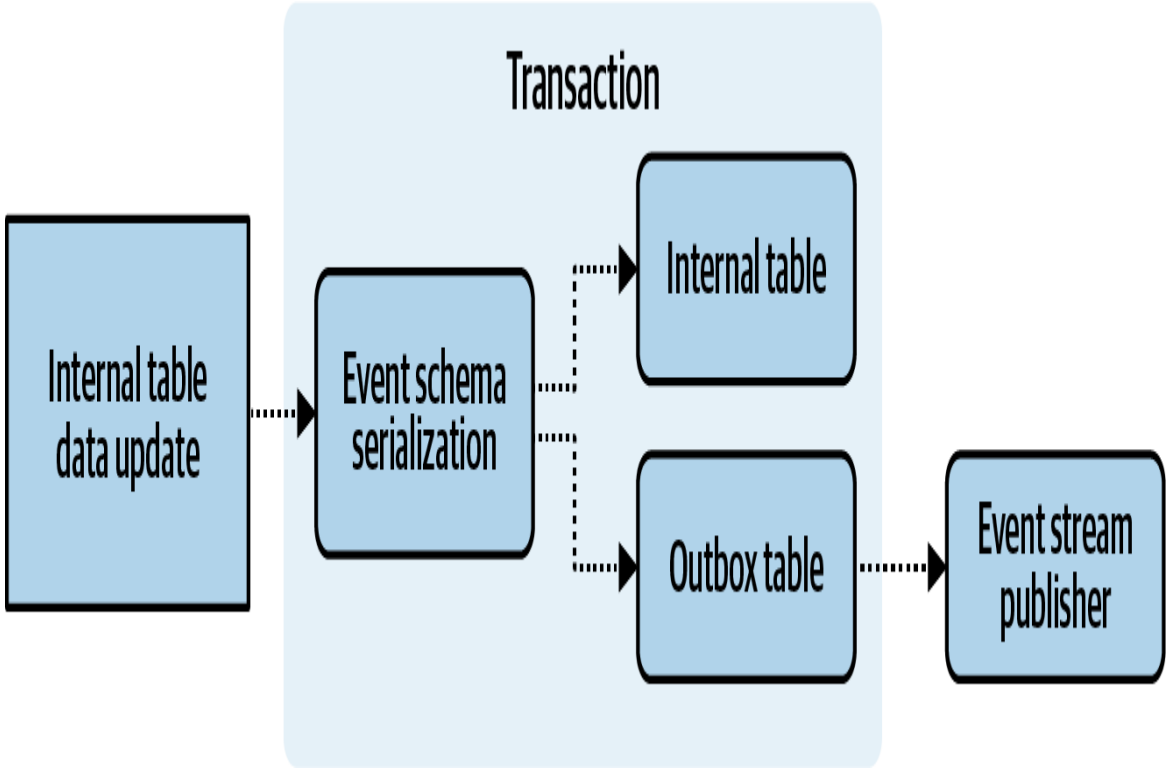


Figure 5-6. Serializing change-data before writing to outbox table

Serializing before writing to the outbox also provides you with the option of using a single outbox for all transactions. The format is simple, as the content is predominantly serialized data with the target output event stream mapping. This is shown in Figure 5-7.

Outbox table

id (autoincrement)	created_at	serialized_key	serialized_value	output_stream
8273	2020-07-07T07:43:10	A0 FB 24	0112 C5 BB D4	Accounts
8274	2020-07-07T07:43:10	DE A8 EF	25 6B EA F9 76	Users

Figure 5-7. A single output table with events already validated and serialized (note the output_stream entry for routing purposes)

One (often major) drawback of serializing before publishing is that performance may suffer due to the serialization overhead. This may be inconsequential for light loads but could have more significant implications for heavier loads. You will need to ensure your performance needs remain met.

Alternately, serialization can be performed after the event has been written to the outbox table, as is shown in Figure 5-6.

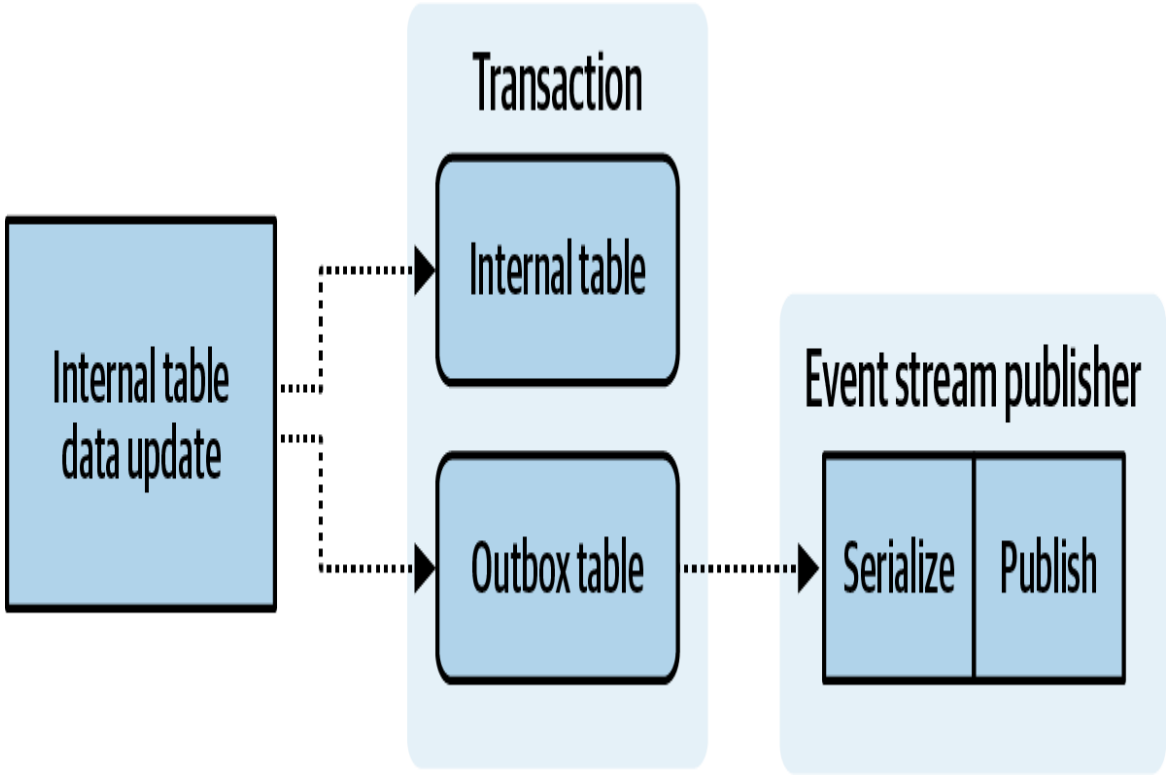


Figure 5-8. Serializing change-data after writing to outbox table, as part of the publishing process

With this strategy you typically have independent outboxes, one for each domain model, mapped to the public schema of the corresponding output event stream. The publisher process reads the unserialized event from the outbox and attempts to serialize it with the associated schema prior to producing it to the output event stream. Figure 5-9 shows an example of multiple outboxes, one for a User entity and one for an Account entity.

User table outbox

id (autoincrement)	created_at	name	address	country
23	2020-05-20T16:30:00	Justin T.	123 Road Lane	Canada
24	2020-05-20T16:30:01	Robert O.	60 3rd Street	USA

Account table outbox

id (autoincrement)	created_at	account_id	user_id	type
45	2020-06-02T09:10:10	623	2729	Cash
46	2020-06-03T11:53:01	638	4291	Credit

Figure 5-9. Multiple outbox tables (note that the data is not serialized, which means that it may not be compatible with the schema of the output event stream)

A failure to serialize indicates that the data of the event does not comply with its defined schema and so cannot be published. This is where the serialization-after-write option becomes more difficult to maintain, as an already completed transaction will have pushed incompatible data into the outbox table, and there is no guarantee that the transaction can be reversed.

In reality, you will typically end up with a large number of unserializable events in your outbox. Human intervention will most likely be required to try to salvage some of the data, but resolving the issue will be time-consuming and difficult and may even require downtime to prevent additional issues. This is compounded by the fact that *some* events may indeed be compatible

and have already been published, leading to possible incorrect ordering of events in output streams.

> **TIP**
>
> Before-the-fact serialization provides a stronger guarantee against incompatible data than after-the-fact and prevents propagation of events that violate their data contract. The tradeoff is that this implementation will also prevent the business process from completing should serialization fail, as the transaction must be rolled back.

Validating and serializing before writing ensures that the data is being treated as a first-class citizen and offers a guarantee that events in the output event stream are eventually consistent with the data inside the source data store, while also preserving the isolation of the source's internal data model. This is the strongest guarantee that a change-data capture solution can offer.

Benefits of event-production with outbox tables

There transactional outbox advantages include the following:

Multilanguage support

This approach is supported by any client or framework that exposes transactional capabilities.

Exactly-once outbox semantics

Transactions ensure that both the internal model and the outbox data are created atomically. You will not miss any changes in your database provided you wrap the updates in a transaction.

Early schema enforcement

The outbox table can provide a strongly defined schema that represents the event stream's schema. Alternatively, valid schemas at runtime by serializing the data written into the outbox.

Internal data model isolation

Data store application developers can select which fields to write to the outbox table, keeping internal fields isolated.

Denormalization

The service can denormalize the data before writing it to the outbox table.

Drawbacks of event production with outbox tables

Producing events via outbox tables has several disadvantages as well:

Database must support transactions

Your database must support transactions. If it does not, you'll have to choose a different data access pattern.

Application code changes

The application code must be changed to enable this pattern, which requires development and testing resources from the application maintainers.

Business process performance impact

The performance impact to the business workflow may be nontrivial, particularly when validating schemas via serialization. Failed transactions can also prevent business operations from proceeding.

Data store performance impact

The performance impact to the data store may be nontrivial, especially when a significant quantity of records are being written, read, and deleted from the outbox.

> **NOTE**
>
> Performance impacts must be balanced against other costs. For instance, some organizations simply emit events by parsing change-data capture logs and leave it up to downstream teams to clean up the events after the fact. This incurs its own set of expenses in the form of computing costs for processing and standardizing the events, as well as human-labor costs in the form of resolving incompatible schemas and attending to the effects of strong coupling to internal data models. Costs saved at the producer side are often dwarfed by the expenses incurred at the consumer side for dealing with these issues.

Capturing Change-Data Using Triggers

Trigger support predates many of the auditing, binlog, and write-ahead log patterns examined in the previous sections. Many older relational databases use triggers as a means of generating audit tables. As their name implies, triggers are set up to occur automatically on a particular condition. If it fails, the command that caused the trigger to execute also fails, ensuring update atomicity.

You can capture row-level changes to an audit table by using an `AFTER` trigger. For example, after any `INSERT`, `UPDATE`, or `DELETE` command, the trigger will write a corresponding row to the change-data table. This ensures that changes made to a specific table are tracked accordingly.

Consider the example shown in Figure 5-10. User data is being upserted to a user table, with a trigger capturing the events as they occur. Note that the trigger is also capturing the time at which the insertion occurred as well as an auto-incrementing sequence ID for the event publisher process to use.

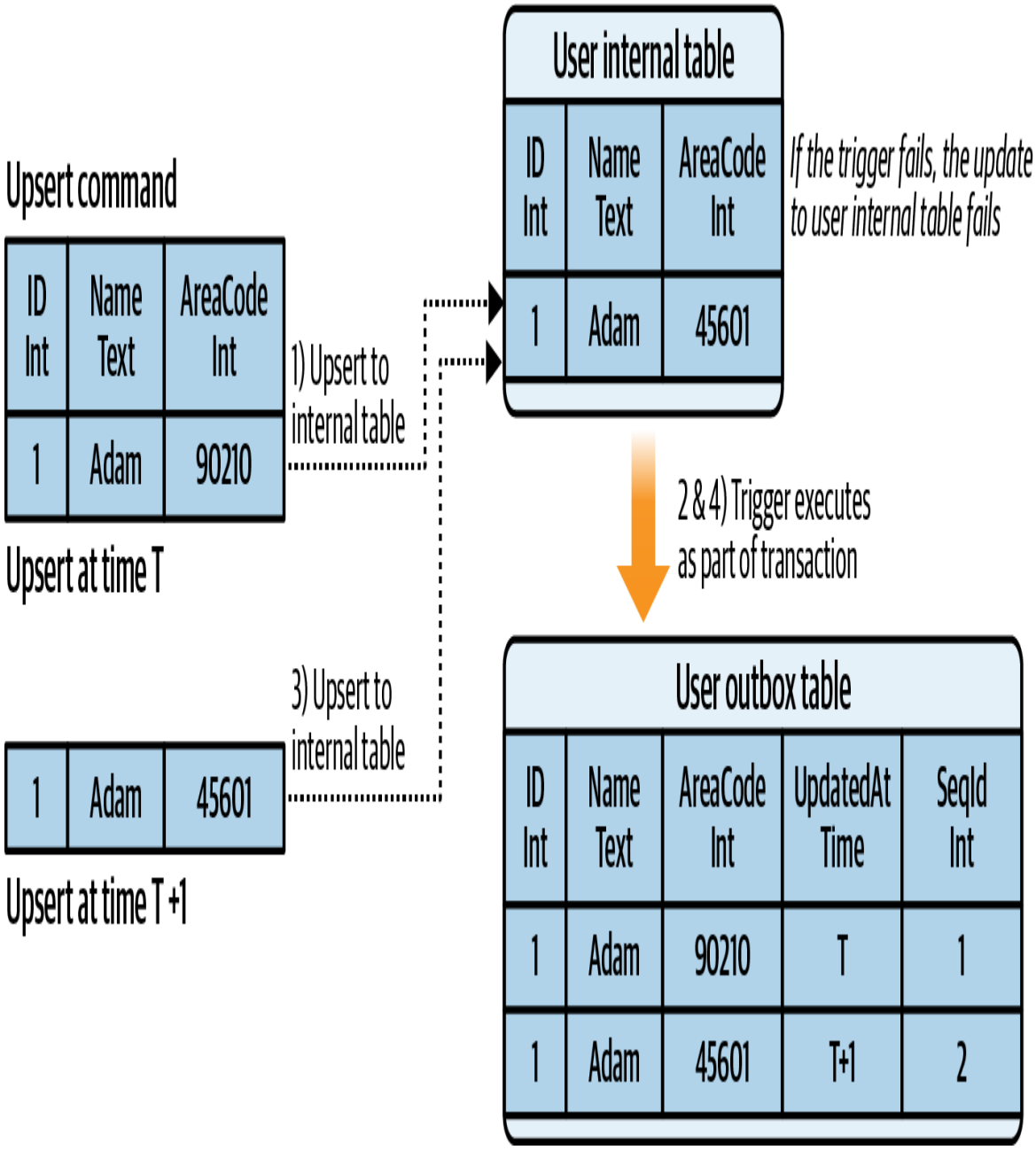


Figure 5-10. Using a trigger to capture changes to a user table

You generally cannot validate the change-data with the event schema during the execution of a trigger, though it is not impossible. One main issue is that it may simply not be supported, as triggers execute within the database itself, and many are limited to the forms of language they can support. While PostgreSQL supports C, Python, and Perl, which may be used to write user-defined functions to perform schema validation, many other databases do not provide multilanguage support. Finally, even if a trigger is supported, it may

simply be too expensive. Each trigger fires independently and requires a nontrivial amount of overhead to store the necessary data, schemas, and validation logic, and for many system loads the cost is too high.

Figure 5-11 shows a continuation of the previous example. After-the-fact validation and serialization is performed on the change-data, with successfully validated data produced to the output event stream. Unsuccessful data would need to be error-handled according to business requirements, but would likely require human intervention.

The change-data capture table schema is the bridge between the internal table schema and the output event stream schema. Compatibility among all three is essential for ensuring that data can be produced to the output event stream. Because output schema validation is typically not performed during trigger execution, it is best to keep the change-data table in sync with the format of the output event schema.

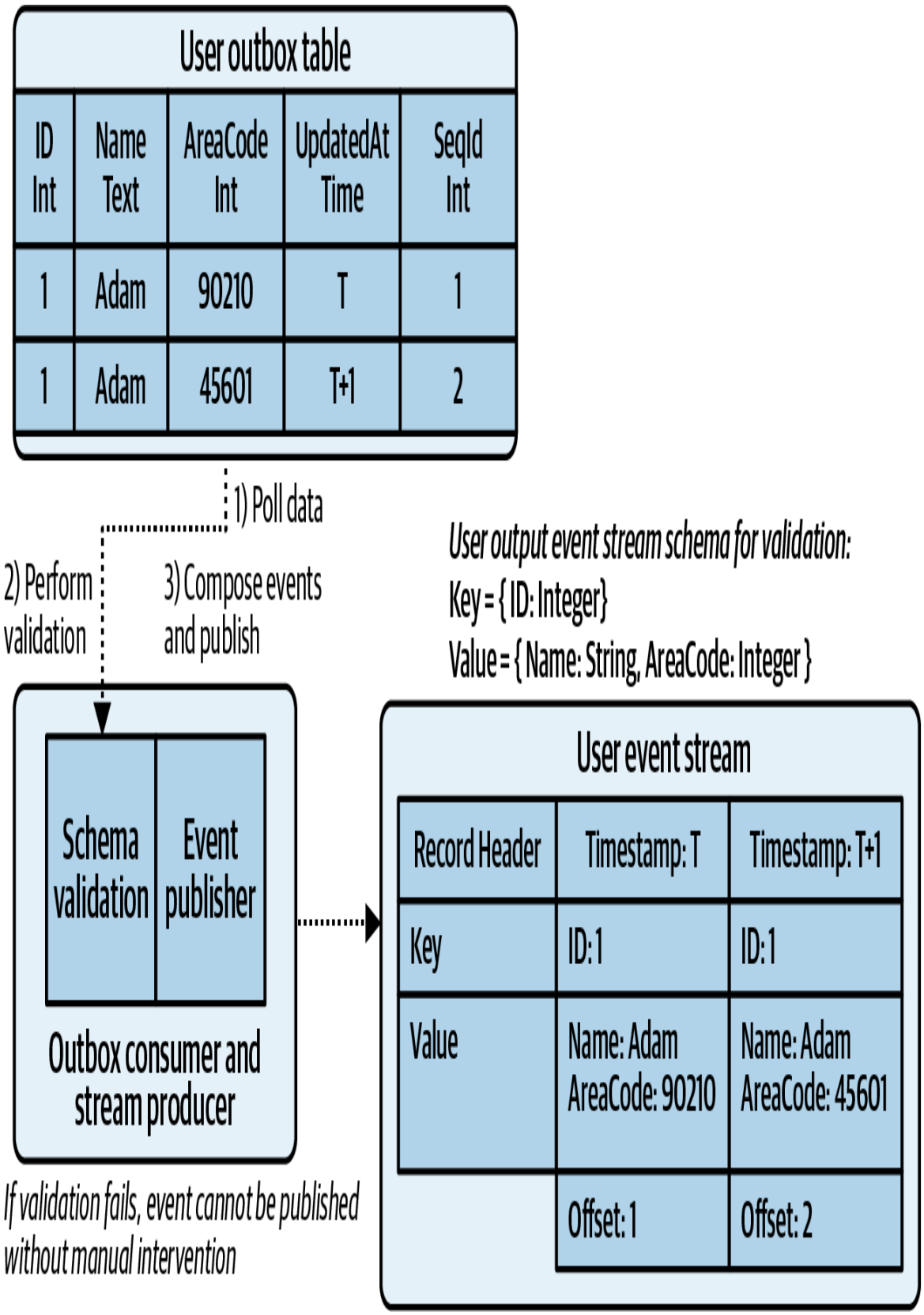


Figure 5-11. After-the-fact validation and production to the output event stream

> **TIP**
>
> Compare the format of the output event schema with the change-data table during testing. This can expose incompatibilities before production deployment.

That being said, triggers can work great in many legacy systems. Legacy systems tend to use, by definition, old technology; triggers have existed for a very long time and may very well be able to provide the necessary change-data capture mechanism. The access and load patterns tend to be well defined and stable, such that the impact of adding triggering can be accurately estimated. Finally, although schema validation is unlikely to occur during the triggering process itself, it may be equally unlikely that the schemas themselves are going to change, simply due to the legacy nature of the system. After-the-fact validation is only an issue if schemas are expected to change frequently.

> **WARNING**
>
> Try to avoid the use of triggers if you can instead use more modern functionality for generating or accessing change-data, such as a designated CDC system. You should not underestimate the overhead performance and management required for a trigger-based solution, particularly when many dozens or hundreds of tables and data models are involved.

Benefits of using triggers

Benefits of using triggers include the following:

Supported by most databases

Triggers exist for most relational databases.

Low overhead for small data sets

Maintenance and configuration is fairly easy for a small data sets with infrequent updates.

Customizable logic

Trigger code can be customized to expose only a subset of specific fields. This can provide some isolation into what data is exposed to downstream consumers.

Drawbacks of using triggers

Some cons of using triggers are:

Performance overhead

Triggers execute inline with actions on the database tables and can consume non-trivial processing resources. Depending on the performance requirements and SLAs of your services, this approach may cause an unacceptable load.

Change management complexity

Changes to application code and to data set definitions may require corresponding trigger modifications. Necessary modifications to underlying triggers may be overlooked by the system maintainers, leading to data liberation results that are inconsistent with the internal data sets. Comprehensive testing should be performed to ensure the trigger workflows operate as per expectations.

Poor scaling

The quantity of triggers required scales linearly with the number of data sets to be captured. This excludes any additional triggers that may already exist in the business logic, such as those used for enforcing dependencies between tables.

After-the-fact schema enforcement

Schema enforcement for the output event occurs only after the record has been published to the outbox table. This can lead to unpublishable events in the outbox table.

> **TIP**
>
> Some databases allow for triggers to be executed with languages that can validate compatibility with output event schemas during the trigger's execution (e.g., Python for PostgreSQL). This can increase complexity and processing costs, but significantly reduces the risk of downstream schema incompatibilities.

Making Data Definition Changes to Data Sets Under Capture

Integrating data definition changes can be difficult in a data liberation framework. Data migrations are a common operation for many relational database applications and need to be supported by capture. Data definition changes for a relational database can include adding, deleting, and renaming columns; changing the column type; and adding or removing defaults. While all of these operations are valid data set changes, they can create issues for the production of data to liberated event streams.

> **NOTE**
>
> *Data definition* is the formal description of the data set. For example, a table in a relational database is defined using a *data definition language* (DDL). The resultant table, columns, names, types, and indices are all part of its data definition.

The most important component to successfully navigating data definition changes in your source data sets is to get deeply integrated with their change management process. If it's your responsibility to ensure that data entering an event stream remains compatible with its established schema, then you should do everything you can to stay abreast of any incoming changes.

Specific actions you can take include:

Observe Data Store Pull Requests

> Register for notifications to change requests for any code that touches the data stores under capture. You will see any proposed changes and can identify if they will impact your code.

Build Integration Tests

> Add integration tests to the codebase that pulls down the event stream schema and compares it to the schema of the data store under capture. This should notify the developer making the changes during development time, giving you both more time to find a common solution.

Establish Deployment Validations

> Add the intergration tests to the deployment pipeline to ensure that no code is deployed without successfully validating (or overriding) the schema validation step.

The reality is that you're bridging a schema from one independent system (the data store) to another independent system (the event broker). This step will always be a bit precarious, particularly because team boundaries tend to align on this very same system gap.

TIP

A big benefit of the outbox table pattern is that the code that populates the outbox resides inside the same code base as the main application. Any breaking changes to the code that populates the outbox will result in compilation and testing errors, letting you take action before breaking anything in production.

Compromises for Data Liberation

The liberated event stream must eventually, and accurately, reflect the source data set. Event-driven architectures rely on the liberated event streams for event-carried state transfer (“State Events and Event-Carried State Transfer”), such that they can materialize the event data back into a copy of the original table.

However, legacy systems do not rebuild their data sets from these liberated event streams. They typically have their own backup and restore mechanisms, such as snapshots, checkpoints, and their own write-ahead logs. But regardless of their restoration mechanisms, there remains a risk that the restored state doesn’t mirror a restoration created by materializing the event stream.

Converting the producer to a *read your own write* pattern is one option for restoring state. This involves rewriting your application to first write to the event stream, and then read back the data that it just wrote to build its internal state. This is known as the *read your own writes* pattern, as is illustrated in Figure 5-12.

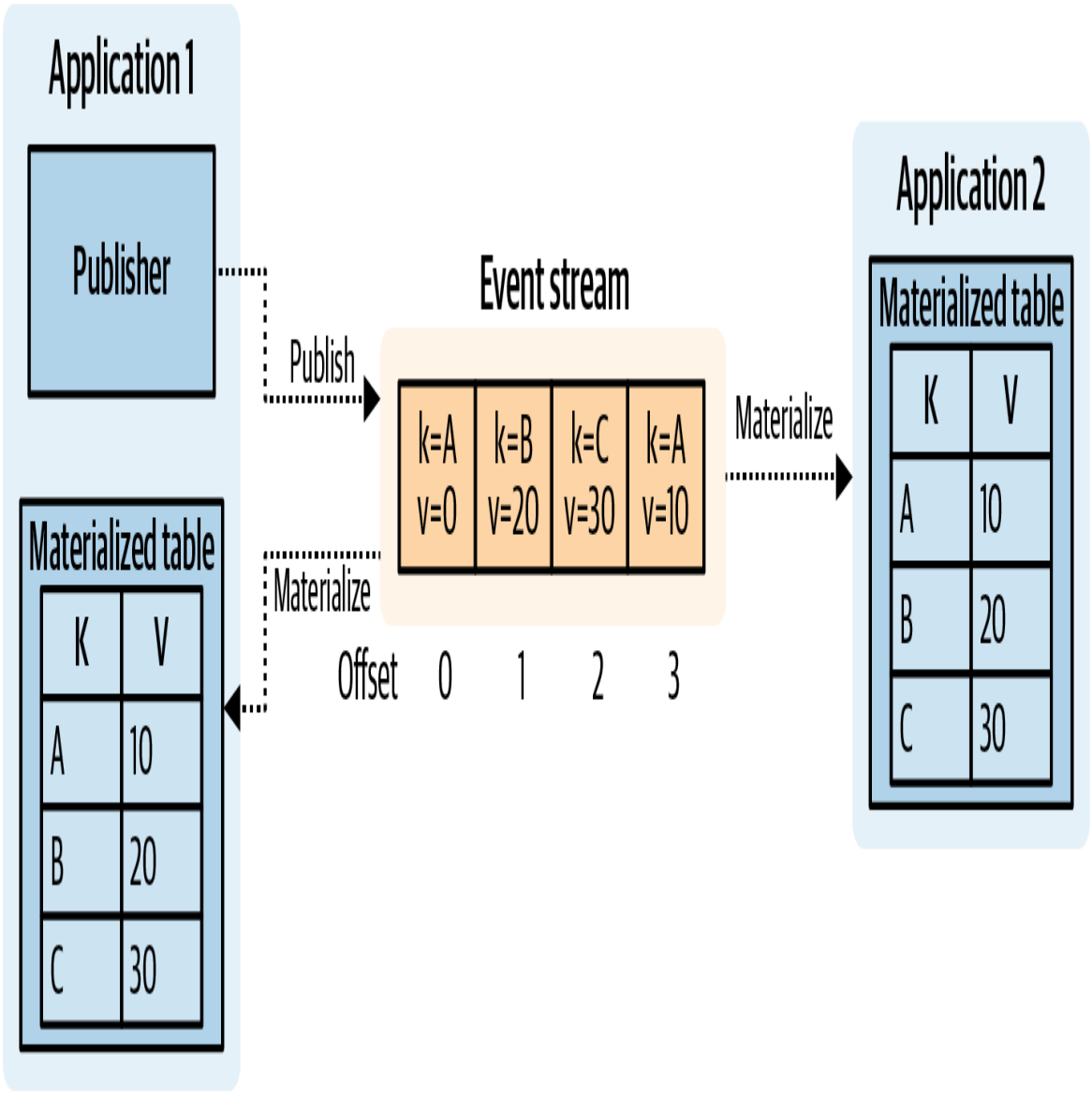


Figure 5-12. Publish to stream before materializing

The main advantages of this pattern include:

- **Single Source of Truth:**: The producer service gets a full copy of the same data as every other service subscribing to the event stream. Materializations are fully convergent with one another.
- **Failure Recovery from Stream:** Any producer failures are remedied simply by reading back the event stream to get a consistent view of where it left off. If data is missing, the stream can be rewound and replayble.

The main downsides include:

- **Increased latency:** The data is written to the stream first and then read back. The network round trips will increase latency, and may not be suitable for services that require very low latency.
- **Not suitable for all data store use cases:** Complex multi-step transactions that require IDs and values from previous steps are largely unsuitable for this pattern. Writing to the event stream first means that your data store operations are largely limited to basic upserts.
- **Requires refactoring:** Legacy systems require refactoring to use the read your own write pattern, and it may prove to be too expensive to commit to outside of very simple use cases.

Now while it would be ideal if we could use a single event stream for building and restoring state for both the producer and the consumer, in practice you'll often have to look for other options. Legacy systems are often unable to undertake such a major change, and the development cycles may simply not be available.

There is an opportunity for compromise here. You can rely on data liberation patterns to extract the data out of the data store and create the necessary event streams. This is a form of unidirectional event-driven architecture, as the legacy system will *not* be reading back from the liberated event stream, as shown in Figure 5-12.

Instead, the fundamental goal is to keep the internal data set synchronized with the external event stream through strictly controlled publishing of event data. The event stream will be eventually consistent with the internal data set of the legacy application, as shown in Figure 5-13.

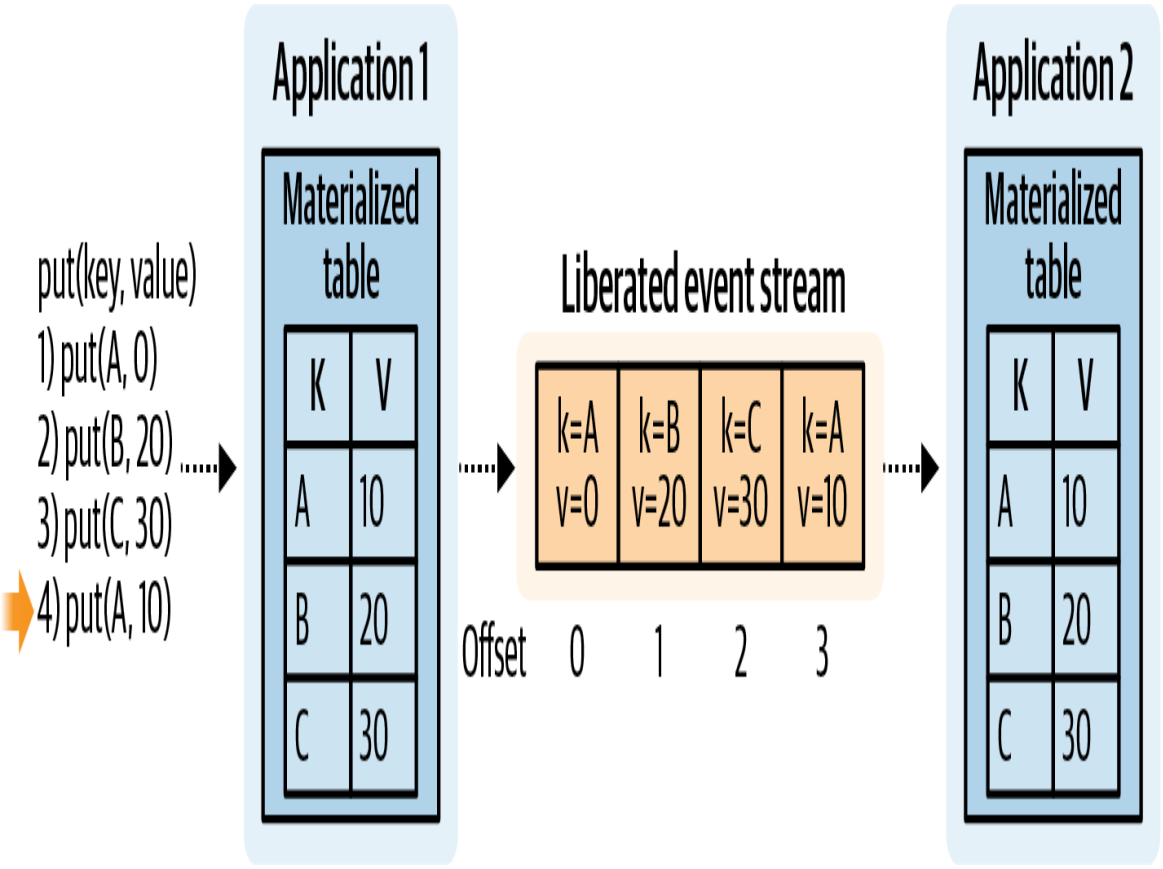


Figure 5-13. Liberating and materializing state between two services

Sinking Event Data to Data Stores

Sinking data from event streams consists of consuming event data and inserting it into a data store. This is facilitated either by the centralized framework or by a standalone microservice. Any type of event data, be it entity, keyed events, or unkeyed events, can be sunk to a data store.

Event sinking is particularly useful for integrating non-event-driven applications with event streams. The sink process reads the event streams from the event broker and inserts the data into the specified data store. It keeps track of its own consumption offsets and writes event data as it arrives at the input, acting completely independently of the non-event-driven application.

A typical use of event sinking is replacing direct point-to-point couplings between legacy systems. Once the data of the source system is liberated into

event streams, it can be sunk to the destination system with few other changes. The sink process operates both externally and invisibly to the destination system.

Teams that need to perform batch-based big-data analysis are also customers for data in event streams. They usually access the event streams by sinking data to a distributed file system, which enables large parallel batch-based processing. The Hadoop Distributed File System (HDFS) is just one such example of an early distributed file system, that led to the cloud-native successors we see today: Amazon S3, Azure Blob Storage, and Google Cloud Storage. Table formats, particularly those backed by Apache Iceberg and [Delta Lake]*https://delta.io/* are also a common destination for event stream data.

Using a common platform like Kafka Connect allows you to specify sinks with simple configurations and run them on the shared infrastructure. Standalone microservice sinks provide an alternative solution. Developers can create and run them on the microservice platform and manage them independently.

The Impacts of Sinking and Sourcing on a Business

A centralized framework allows for lower-overhead processes for liberating data. This framework may be operated at scale by a single team, which in turn supports the data liberation needs of other teams across the organization. Teams looking to integrate then need only concern themselves with the connector configuration and design, not with any operational duties. This approach works best in larger organizations where data is stored in multiple data stores across multiple teams, as it allows for a quick start to data liberation without each team needing to construct its own solution.

There are two main traps that you can fall into when using a centralized framework. First, the data sourcing/sinking responsibilities are now shared between teams. The team operating the centralized framework is responsible

for the stability, scaling, and health of both the framework and each connector instance. Meanwhile, the team operating the system under capture is independent and may make decisions that alter the performance and stability of the connector, such as adding and removing fields, or changing logic that affects the volume of data being transmitted through the connector. This introduces a direct dependency between these two teams. These changes can break the connectors, but may be detected only by the connector management team, leading to linearly scaling, cross-team dependencies. This can become a difficult-to-manage burden as the number of changes grows.

The second issue is a bit more pervasive, especially in an organization where event-driven principles are only partially adopted. Systems can become too reliant upon frameworks and connectors to do their event-driven work for them. Once data has been liberated from the internal state stores and published to event streams, the organization may become complacent about moving onward into native event-driven microservices. Teams can become overly reliant upon the connector framework for sourcing and sinking data, and choose not to refactor their applications or develop new event-driven services. In this scenario they instead prefer to just requisition new sources and sinks as necessary, leaving their entire underlying application completely ignorant to events.

> **WARNING**
>
> CDC tools are *not* the final destination in moving to an event-driven architecture, but instead are primarily meant to help bootstrap the process. The real value of the event broker as the data communication layer is in providing a robust, reliable, and truthful source of event data decoupled from the implementation layers, and the broker is only as good as the quality and reliability of its data.

Both of these issues can be mitigated through a proper understanding of the role of the change-data capture framework. Perhaps counterintuitively, it's important to minimize the usage of the CDC framework and have teams implement their own change-data capture (such as the outbox pattern) despite the additional up-front work this may require. Teams become solely

responsible for publishing and their system's events, eliminating cross-team dependencies and brittle connector-based CDC. This minimizes the work that the CDC framework team needs to do and allows them to focus on supporting products that truly need it.

Reducing the reliance on the CDC framework also propagates an "event-first" mind-set. Instead of thinking of event streams as a way to shuffle data between monoliths, you view each system as a direct publisher and consumer of events, joining in on the event-driven ecosystem. By becoming an active participant in the EDM ecosystem, you begin to think about *when* and *how* the system needs to produce events, about the data *out there* instead of just the data *in here*. This is an important part of the cultural shift toward successful implementation of EDM.

For products with limited resources and those under maintenance-only operation, a centralized source and sink connector system can be a significant boon. For other products, especially those that are more complex, have significant event stream requirements, and are under active development, ongoing maintenance and support of connectors is unsustainable. In these circumstances it is best to schedule time to refactor the codebase as necessary to allow the application to become a truly native event-driven application.

Finally, carefully consider the tradeoffs of each of the CDC strategies. This often becomes an area of discussion and contention within an organization, as teams try to figure out their new responsibilities and boundaries in regard to producing their events as the single source of truth. Moving to an event-driven architecture requires investment into the data communication layer, and the usefulness of this layer can only ever be as good as the quality of data within it. Everyone within the organization must shift their thinking to consider the impacts of their liberated data on the rest of the organization and come up with clear service-level agreements as to the schemas, data models, ordering, latency, and correctness for the events they are producing.

Summary

Data liberation is an important step toward providing a mature and accessible data communication layer. Legacy systems frequently contain the bulk of the core business domain models, stored within some form of centralized implementation communication structure. This data needs to be liberated from these legacy systems to enable other areas of the organization to compose new, decoupled products and services.

There are a number of frameworks, tools, and strategies available to extract and transform data from their implementation data stores. Each has its own benefits, drawbacks, and tradeoffs. Your use cases will influence which options you select, or you may find that you must create your own mechanisms and processes.

The goal of data liberation is to provide a clean and consistent single source of truth for data important to the organization. Access to data is decoupled from the production and storage of it, eliminating the need for implementation communication structures to serve double duty. This simple act reduces the boundaries for accessing important domain data from the numerous implementations of legacy systems and directly promotes the development of new products and services.

There is a full spectrum of data liberation strategies. On one end you will find careful integration with the source system, where events are emitted to the event broker as they are written to the implementation data store. Some systems may even be able to produce to the event stream first before consuming it back for their own needs, further reinforcing the event stream as the single source of truth. The producer is cognizant of its role as a good data-producing citizen and puts protections in place to prevent unintentional breaking changes. Producers seek to work with the consumers to ensure a high-quality, well-defined data stream, minimize disruptive changes, and ensure changes to the system are compatible with the schemas of the events they are producing.

On the other end of the spectrum, you'll find the highly reactive strategies. The owners of the source data in the implementation have little to no visibility into the production of data into the event broker. They rely

completely on frameworks to either pull the data directly from their internal data sets or parse the change-data capture logs. Broken schemas that disrupt downstream consumers are common, as is exposure of internal data models from the source implementation. This model is unsustainable in the long run, as it neglects the responsibility of the owner of the data to ensure clean, consistent production of domain events.

The culture of the organization dictates how successful data liberation initiatives will be in moving toward an event-driven architecture. Data owners must take seriously the need to produce clean and reliable event streams, and understand that data capture mechanisms are insufficient as a final destination for liberating event data.

Chapter 6. Denormalization and Eventification

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 7th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

NOTE

Portions of this chapter previously appeared in *Building an Event-Driven Data Mesh*.

Events are often sourced from normalized data models, particularly if you're using change-data capture (see Chapter 5) to load in data from relational tables. Relational databases resolve relationships at query time, and are built on the premise that they have access to all necessary data at query time.

In constrast, event-driven microservices rely on data provided by events, and tend to work best when the necessary data is all within the event itself. While they can resolve joins between streams and materialized tables, these operations tend to be slower and more expensive than in a dedicated relational database. Furthermore, the languages and frameworks that support

streaming joins are few and far between, and are covered in more detail in both Chapter 11, Chapter 10, and [[Link to Come]].

Eventification is a implementation pattern for denormalizing and remodeling highly relational streams into forms that are more suitable for event-driven consumers. The end goal is to reduce the amount of repetitive, error-prone, and expensive denormalizations that each consumer must do, when sourcing data from relational systems. It is more challenging to deal with denormalization in a set of event streams than it is a set of tables, so you must carefully consider what *should* and what *should not* go into a denormalized event stream.

Let's look at an example.

Consider the following relational ecommerce, merchant, and inventory relational data model, as represented in a relational database. The `EcomItem` shown in Table 6-1 has a foreign-key relationship (`merchant_id`) to the `Merchant` table in Table 6-2. `EcomItem` also has a primary-key relationship (`inventory_id`) to the `Inventory` table in Table 6-3, which contains the quantity of items currently in stock.

Table 6-1. `EcomItem` table definition

id	name	price	merchant_id	inve
4291	"Mirage Block Set"	1299.99	4	4429

Table 6-2. `Merchant` table definition

id	name	premium_partner	updated_at
4	"Devin's Trading Cards"	true	2019-08-12 19:00:37

Table 6-3. `Inventory` table definition

id	quantity_in_stock	updated_at
44291	3	2021-01-09 07:33:13

Though this model is simplified, you'll commonly find similar models in many online ecommerce marketplace platforms—think of businesses like Amazon, eBay, Alibaba, Etsy, and Shopify, to name a few.

`Merchant.premium_parter` specifies if the merchant has signed on and paid for the premium business experience: better advertising deals, preferred advertisement placement, and preferred search result placement. Additional operational features include the ability to upload videos and additional pictures per `EcomItem`, create item bundles, and offer AI-generated custom deals to find satisfactory price points for cost-savvy customers. Most of these services require the `EcomItem` information to serve to the end customer—but they each also require the merchant information for branding and display purposes as well as determining if the merchant requires `premium_partner` treatment.

Due to the highly relational nature of these streams, each consumer service will need to do the same foreign-key joins of `EcomItem` to `Merchant` to get the data into a proper format for its business use cases. Figure 6-1 shows a small subset of consumers that each need to consume and join each event stream.

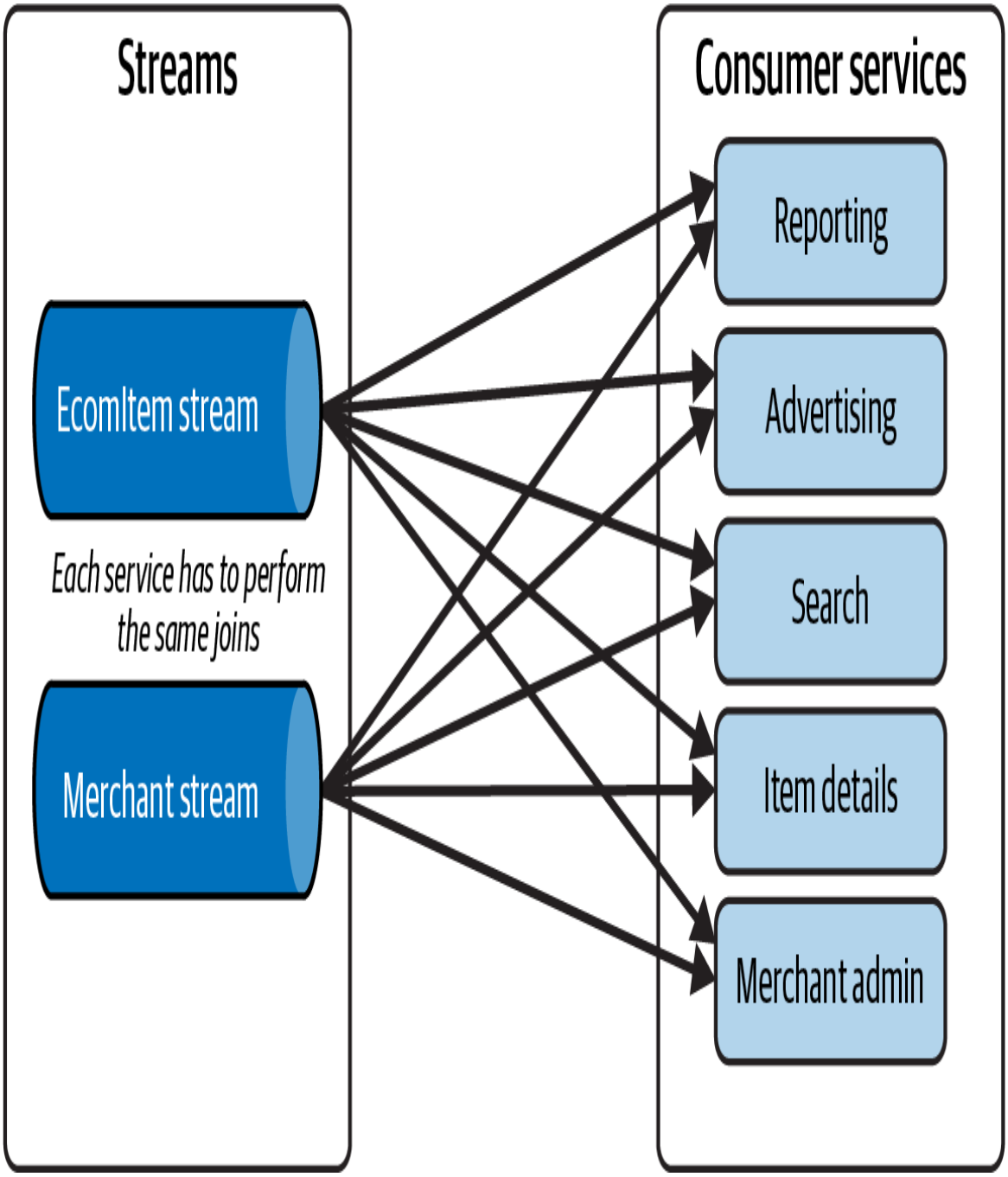


Figure 6-1. Each consumer has to join the same data to determine premium partner status

There are some significant downsides to this arrangement:

Repeated processing

Each consumer needs to execute these joins on its own. Each client will have duplicate code and use a similar amount of resources for the joins. For small data sets, it may be

inconsequential, but for processing data at scale, including dealing with partitioned event streams, it can become quite expensive.

Significant client code constraints

Each consumer client must resolve the joins on its own. The majority of scalable streaming join solutions, such as Apache Kafka Streams, Apache Flink, and Apache Spark, are Java virtual machine (JVM)-based, but some also support Python and SQL. However, there is no first-class support for other popular programming languages such as Go, JavaScript, Ruby, Rust, etc. You may also choose to resolve joins by using adding a dedicated relational database to your microservice, but it may be entirely unsuitable for your microservice's other needs.

`EcomItem` contains a reference to the `Merchant`, which contains the necessary `premium_partner` status field. Denormalizing that information into an `Enriched_EcomItem` will alleviate all downstream consumers from having to do that work themselves. We'll leave `Inventory` outside of the `Enriched_EcomItem` for now, and return to it later in this chapter.

There are several factors to consider when performing eventification:

Consumer requirements and use cases

Ease of use is the chief goal of eventification. Your consumer's requirements, if known, are an important consideration for determining what data to put into an event and what data to leave out. Denormalizing simple foreign key relationships is often one of the top requirements, regardless of business logic.

Degree of denormalization

How much you denormalize the data depends on the consumer use cases as well as the size of the data and the frequency of change. This is covered further in more detail in "What Should Go In the Event? And What Should Stay Out?".

Keys for joining on related streams

As discussed in [[Link to Come]], the event keys you select during eventification should enable easy joining on other event streams. Ensure your event is keyed in a way that makes it compatible for joining to related streams that are not required by most consumers. For example, both `Inventory` and `Enriched_EcomItem` are joinable via their primary key ids.

Structuring the external data model

Eventification provides you with the opportunity to convert and standardize data as well as conceal portions of the internal source data model. This ensures your consumers only couple on a public data contract, and not whatever internal data sources they can get their hands on.

There are two main areas where you can implement the eventification pattern: when writing to the outbox table and when using a dedicated service to join normalized streams. Let's look at each of these in turn.

Eventification at the Transactional Outbox

One option is to select internal model data and denormalize it prior to writing it to the transactional outbox ("Liberating Data Using Transactional Outbox Tables").

Example 6-1 shows a code sample for remodeling relational data into a more suitable denormalized event format. It is based off the code example from

Example 5-1, and shows denormalization of `EcomItem` and `Merchant` within a singular transaction.

Example 6-1. Eventification and denormalization prior to writing event data to the outbox

```
try:
    # Stubbing out previous example code to save some space
    conn = ...
    cursor = conn.cursor()
    internal_model_update = ...
    cursor.execute(internal_model_update)
    # Just committed the internal model updates

    # Next, select fields from the internal model and
    # denormalize by joining against the Merchant table
    internal_model_query = """
     select e.name, e.price, m.name as merchant_name, m.premium_partner
     from EcomItem as e, Merchant as m,
     join on e.merchant_id = Merchant.id
     where e.id = 4291"""

    cursor.execute(internal_sub_model_query)
    result = cursor.fetchone()

    # Create the insert statement for the outbox table
    outbox_insert = """
      INSERT INTO
        Enriched_EcomItem_Outbox (id, name, price, merchant_name, premium_partner)
      VALUES (4291, %s, %s, %s, %s)"""

    cursor.execute(outbox_insert, result)
    # Commit the internal and outbox updates atomically
    conn.commit()

except mysql.connector.Error as error:
    conn.rollback()
finally:
    if conn.is_connected():
        cursor.close()
        conn.close()
```

This code creates an entry for `Enriched_EcomItem_Outbox` in response to the application creating or updating the data for `EcomItem.id=4291`. The internal relational model remains encapsulated within the database, while the outbox provides a data model more suitable for event-driven consumers.

But what about when the `Merchant` data changes? Consider a single merchant that decides to pay for premium partner status. Any previously created `Enriched_EcomItem` events must *also* be updated to reflect the current premium status. A failure to do so means that the event stream is permanently inaccurate and no longer reflects reality, which will likely cause problems in how downstream consumers react to and process that merchant's data.

Simply put, if you're going to denormalize data, you're going to need to update the event whenever *any* field changes. This remains true regardless of *how* you go about denormalizing data, be it from the inside with an outbox table or with a dedicated eventification service, as we shall see in the next section.

A transactional outbox table works really well as a low-overhead way of isolating the internal and external data models. If your source data resides inside a relational database, it's very easy to rely on the database engine to quickly and efficiently denormalize data into an event-friendly format instead of leaving it up to each consumer to handle. By making event streams easy to use, you improve the data product user experience for your consumers, helping them get on with the business of using the data instead of just struggling with it.

However, it's not always possible to build an outbox table into a database, let alone denormalize your data within the transaction. Aside from databases that simply do not support transactions, legacy systems with no active development as well as those with strict performance requirements may not be suitable candidates. Whenever a transactional outbox isn't feasible, it's best to look at eventification in a dedicated service residing outside of the source domain. Let's take a look at that now.

Eventification in a Dedicated Service

Eventification *outside* of the source database requires a purpose-built microservice or stream SQL application. For example, Figure 6-2 shows a

high-level overview of a dedicated eventification service joining `EcomItem` and `Merchant` data together to form a single enriched stream for downstream use.

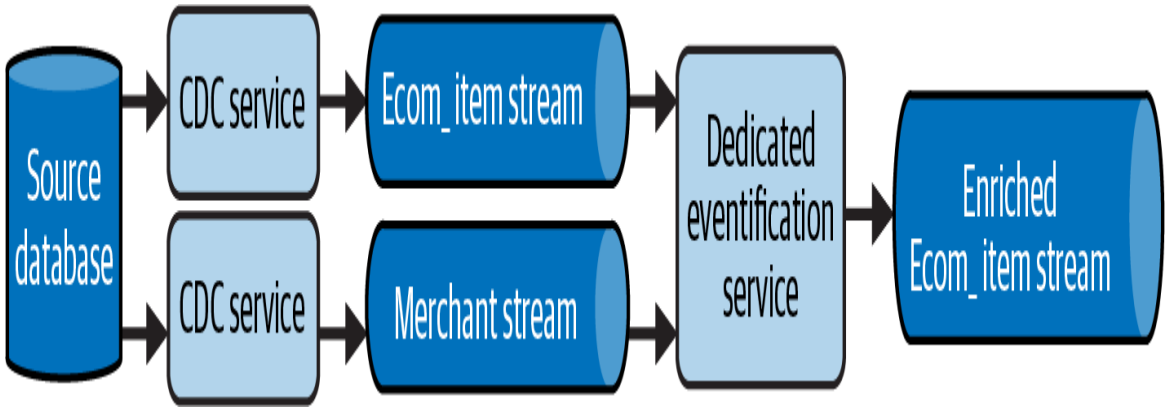


Figure 6-2. Eventification within a dedicated service using CDC event streams

Event-driven processing technologies like Kafka Streams, Flink, and Spark provide high-level frameworks that make it easy to join your data together. These will be discussed in more detail in Chapter 11 and Chapter 10, but for now just keep it simple and look at what a streaming SQL join may look like.

Using the same `Merchant` and `EcomItem` models as before, you could create a simple SQL streaming query that joins the two into a single state-based event. The code would look something like Example 6-2, ignoring the overhead of declaring the `EcomItem` and `Merchant` tables.

Example 6-2. A Flink SQL version of an eventification service for EcomItem and Merchant

```
SELECT *
FROM EcomItem
INNER JOIN Merchant
ON EcomItem.merchantId = Merchant.id
```

SQL eventification services make it easy to resolve table-table streaming joins and denormalization of entities. SQL is very common, easy to learn, and easy to read. And since a key aspect of eventification is simple denormalization, it becomes very easy to see if someone has loaded some business logic into the query. Remember - the main purpose of eventification

is just to make it easier on consumers to use the data - we're *not* executing any complex business logic, filters, or other application-specific logic.

There are some significant advantages to using an external eventification microservice, including:

- Reduced demand on the database's resources
- Independently scalable from the database itself
- Simplification of the source domain's application logic

However, there are also some challenges to handle:

- Synchronizing changes to the database tables, the connectors, and the eventification microservice
- Finding a robust stream joiner framework; there are only a few options, and they're primarily JVM or SQL-based.
- Arguing over ownership of the eventification microservice (hint: the best team to own it is the team that owns the source data)

Regardless of whether you choose to denormalize your events inside the source database, outside the database using a purpose-built microservice, or at the consumer itself, it's critical to consider the impact on your consumers. Focus on making the event data as easy to understand and use as possible, rely on well-defined schemas, and always work towards reducing the chances that they might misunderstand or misinterpret the data.

What Should Go In the Event? And What Should Stay Out?

There is a fine balancing act for determining what data to include and what data to exclude in an event. There are several factors that influence this decision. Consumer needs, the update frequency, the data size, and the resultant total load are all considerations.

Let's consider an extension of our ecommerce example. `EcomItem.inventory_id` specifies a relationship with the `Inventory` data (Table 6-3), indicating how many of the item are in stock.

Every time the inventory changes, say due to a sale, a return, or a received shipment, the inventory domain can emit a new state event with the updated inventory. For products that change extremely frequently (think a Black Friday "door crasher" sale), you could end up with a veritable barrage of inventory events.

> **NOTE**
>
> The `Inventory` producer can choose to emit inventory updates periodically, perhaps every 5 or 30 seconds. This can reduce the frequency of updates, at the expense of increasing the latency between updates.

Joining the `EcomItem` with `Inventory` (`EcomItem.inventory_id = Inventory.id`) will trigger a corresponding join on the materialized `EcomItem` table on every `Inventory` update. This can be quite a lot of events, and consumers that don't care about the inventory quantity will have a lot of inconsequential updates to work through. This costs money and time for consumer and producer alike, meaning that the first factor we need to consider is **frequency**.

Similarly, consider `Reviews` that contains the top 100 reviews for a given `EcomItem`. While this data may not change frequently, joining it with `EcomItem` means that every single time we update either `Reviews` or `EcomItem`, we also re-emit the entire payload containing the top 100 reviews. Just like with frequent updates, many consumers may not care at all about the contents but still need to contend with the very large data size coming through the stream. This is the second factor to watch out for: **size**.

Finally, you may discover that you have both data with a very high rate of change from one stream and data with a very large payload size from another stream. Joining the two means that you'll now have *a lot* of data with a high

rate of change. This can cause a compounding high load on your event broker, your eventification process, and your consumers. High **frequency** data multiplied by large **size** equals a very expensive denormalization proposition.

As a general rule, avoid joining in data that *changes frequently* or is *large in size*. You may be better off leaving those components out of the enriched event and letting your consumers choose to integrate it into their own service.

Figure 6-3 provides a visualization of the final state of the `Enriched_EcomItem` eventification service. The most commonly used data is joined into the enriched result. `Reviews` are omitted from enrichment due to the large size of the data, while `Inventory` is similarly omitted due to its high frequency updates.

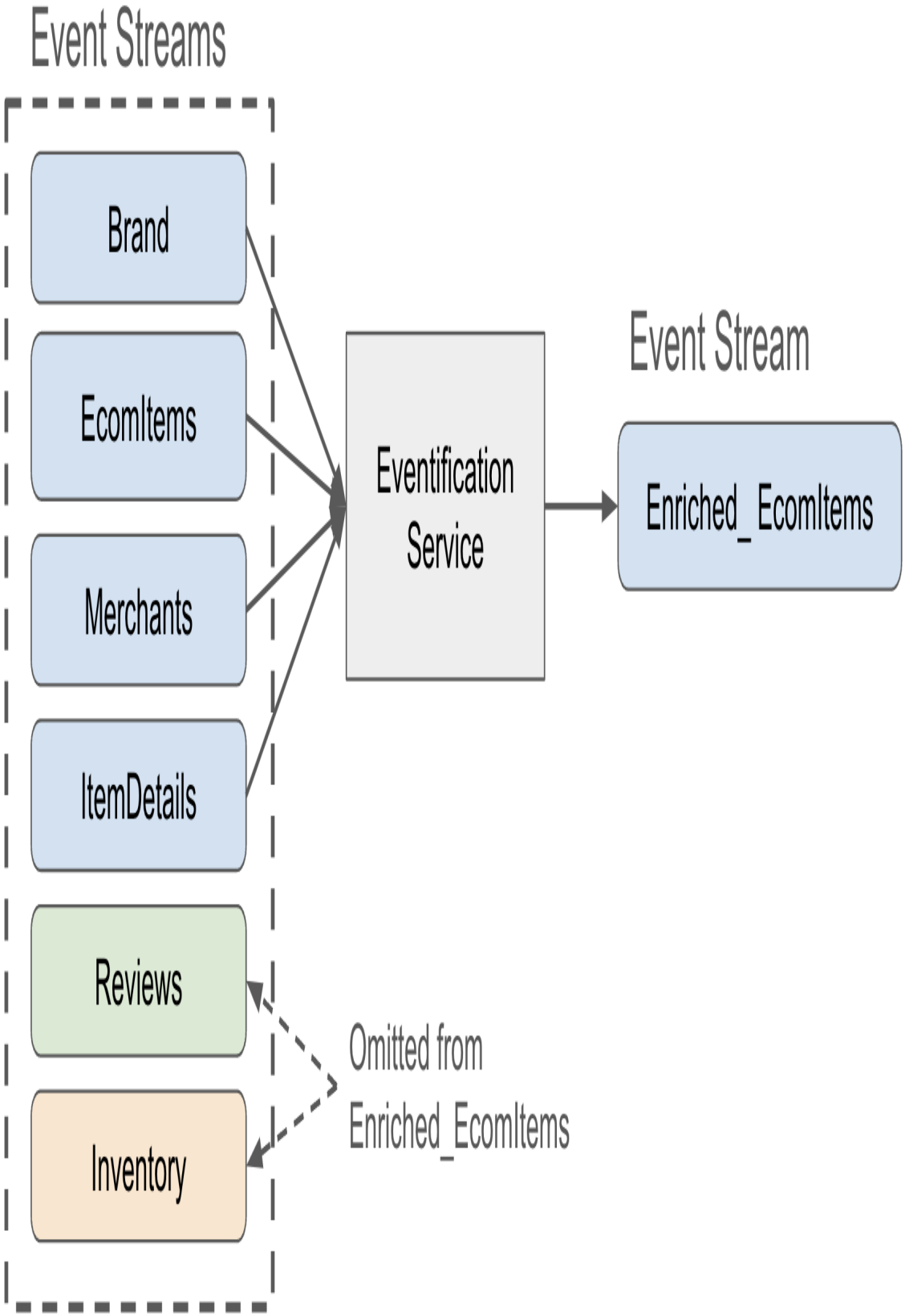

Event Streams
Brand
EcomItems
Merchants
ItemDetails
Reviews
Inventory
Eventification Service
Event Stream
Enriched_EcomItems
Omitted from Enriched_EcomItems

Figure 6-3. Eventification of EcomItem with commonly used data joined into the enriched result

At the end of the day, there is no magic answer or definitive cutoff for what should and shouldn't go in an event. The heuristics presented in this section should give you a better understanding of what to look for and how to think about the impact of joining in data. There remains a few more things to discuss before closing out this chapter, so let's move on into slowly-changing dimensions.

Slowly Changing Dimensions

Continuing with the ecommerce example, you may notice that there are many `EcomItems` for a given `Merchant`. If we want to denormalize the `EcomItem.merchantId` data, we'll have to join against `Merchant`. Think about what you may know about your favorite merchants. Do they change their names often? Change their locations? Not really. We can surmise that `Merchant` data may not change for a very long time, though it certainly *could* be changed at any moment.

This type of data is known as a *slowly changing dimension* (SCD), which is usually static but can change unpredictably. SCDs have long been the provenance of the data analytics space, but the terminology is useful for describing data models and how they relate to event streams.

Relational databases typically handle this form of data by normalizing it into its own table. If the `Merchant` changes their name, you simply update the `Merchant.name` in the appropriate table.

Event streams, unfortunately, are more akin to the document model. *If* you decide to emit the denormalized `Merchant` data into your event stream, you're going to need to consider what happens if that SCD changes. If that `Merchant` is selling millions of `EcomItems`, and you denormalized that data together, then you're going to need to update all of those events.

Although there are (at least) 8 distinct SCD subtypes, the two most relevant for event stream consideration are `Type 1` and `Type 2`. Let's take a closer

look at both of these to see how they relate to data denormalization with event streams.

Type 1: Overwrite with the new value

With `Type 1` modeling, only the most recent value is retained. Say we have a merchant *without* premium status, as per the account shown in Example 6-3. If this merchant decides to pay for premium status, we'd need to produce a new entity event to overwrite the previous one.

Example 6-3. Type 1: The first state event of Devin's Trading Cards

```
{
    "name": "Devin's Trading Cards",
    "premium_partner": false,
    "updated_at": "2011-08-10 13:22:09"
}
```

Example 6-4 shows the results of the updated merchant entity event with `premium_partner=true`.

Example 6-4. Type 1: Updated premium status (and timestamp) for Devin's Trading Cards

```
{
    "name": "Devin's Trading Cards",
    "premium_partner": true,
    "updated_at": "2019-08-12 19:00:27"
}
```

The previous record is no longer required, and the event broker can compact away the old event at a later date. While you may choose to retain the data for a longer period of time, `Type 1` dimensional modeling doesn't account for a history of changes — this is where `Type 2` comes in.

Type 2: Append the new value

In `Type 2` dimension modeling, the changed value of is appended to the state and emitted as a new event. The consumer then has access to both the current and previous versions of the field, and can use it for determining what the entity looked like at a specific point in time. A microservice can react to the

`Merchant.premium_partner` status set to `true`, while the same data can be used to power analytics that focus on premium status over time.

Successfully implementing a `Type 2` SCD requires the producer to keep track of what it has already written to the stream, so it can accurately produce an updated event. Example 6-5 illustrates the updated `Merchant` data event detailing the timelines of changes to the data.

Example 6-5. Type 2: Updated `Merchant` data is appended to the event, complete with version IDs for previous updates

```
[
    {
        "name": "Devin's Trading Cards",
        "premium_partner": false,
        "updated_at": "2011-08-10 13:22:09",
        "version": "0"
    },
    {
        "name": "Devin's Trading Cards",
        "premium_partner": true,
        "updated_at": "2019-08-12 19:00:27",
        "version": "1"
    }
]
```

The default selection for a state event is usually `Type 1`. Your application publishes only the latest data, and leaves it up to the event stream to provide the history of previous values. Your consumer model needs to *opt in* to maintaining the previous values within its own domain data store.

`Type 2` models *force* the consumer to consider the impact of the dimension changes. The full history of previous values require careful consideration when joining together, for you will get different results depending on *when* you want to represent the resultant join. If the consumer wants only the latest value, it still needs to explicitly select out of the history. While `Type 2` models are more verbose and require a bit more overhead for consumers to use, they reduce the chance of a consumer overlooking the history of changes.

Summary

Converting your data into forms that your consumers can easily use is important to building a functional event-driven architecture. Many source data structures are not well-suited for direct conversion into events, and lead to downstream problems if used directly.

Relational data models are one of the most common sources of event data, particularly when bootstrapping via change-data capture. The streams reflect the internal models of the source database, leading to tight coupling, repetitive and expensive joins, and overly complex microservices.

You can denormalize and isolate events via eventification, most commonly as part of the transactional outbox pattern or as a standalone application. In either case, it is essential that you limit eventification to simplifying the data formats through joins and standardizations. Do not apply custom consumer business logic, that belongs downstream in the individual consumers.

Event size and update frequency are two of the most important factors to consider when building enriched events. Be careful of adding in information that is not commonly used, that is very large, or that will cause a very high rate of updates.

Chapter 7. Stateful Event-Driven Microservices

> **A NOTE FOR EARLY RELEASE READERS**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 8th chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

Event-driven microservices (stateful or not) typically follow the same three basic steps.

1. React to the event, e.g. from an input stream, a remote request from a client, a sensor reading, or a row insertion to a database.
2. Process that event
3. Produce any resultant output events

There's a lot of detail that's missing from this simple list, but it shows the heart of how the vast majority of event-driven microservices operate. This chapter will dig deeper into the role that *state* plays, particularly as most microservices must maintain some degree of state to actually do anything useful.

This three-step list is important in evaluating state for event-driven microservices, particularly as the services scale, fail, and recover. How do

we ensure that the correct partitions are assigned to correct consumer instances? And how do we handle *multiple* streams, each with their own partitioning and partitioners? This chapter will provide you the answers.

State stores play an important role in the vast majority of event driven microservices, and go hand-in-hand with stream partition assignments. They are quite frankly, are essential for performing any business functionality beyond just toy-level complexity. Each microservice is responsible for maintaining its own state, including creating it, updating it, reading it, and destroying it.

Event Stream Partitions and Consumer Assignments

"The Basics of Event-Driven Microservices" introduced the concept of consumer groups. Each microservice has a consumer group that it uses to track its offsets and stream consumption. In addition, the consumer microservice must also designate consumption of the input event stream to specific microservice instances. This is called *partition assignment*.

Some event brokers, such as Apache Kafka, delegate partition assignment to the first online client for each consumer group. As consumer group leader, this instance is responsible for performing the partition assignment duties, ensuring that input event-stream partitions are correctly assigned whenever new instances join that consumer group.

Other event brokers, such as Apache Pulsar, maintain a centralized ownership of partition assignment within the broker. In this case, the partition assignment and rebalancing are done by the broker, but the mechanism of identification via consumer group remains the same. Partitions are assigned, and work can begin from the last known offsets of consumed events.

Work is typically momentarily suspended while partitions are reassigned to avoid assignment race conditions. This ensures that any revoked partitions are no longer being processed by another instance before assignment to the new instance, eliminating any potential duplicate output.

The leader (or broker) will issue a *rebalance* when a new instance joins the consumer group, thereby rebalancing the partitions and redistributing the workload. Similarly, when a consumer leaves a consumer group and fails to rejoin within a certain period of time (e.g. 1 minute), the leader (or broker) will issue a rebalance to redistribute the orphaned partitions to another consumer instance.

Partition assignments are very closely related to state. Any corresponding state built from previously assigned partitions also needs to be reassigned during a rebalance. There are a few ways to handle this in practice, either by relying on the internal state stores of stream processing frameworks (see more in Chapter 10, Chapter 11) or by using external state stores that are independent of repartitioning. This chapter will examine internal and external state stores more in the next section.

For now, let's shift to looking at a specific stream-related problem. Let's say you're building a stateful microservice and you you have the data you need in an event stream. However, it's *not* partitioned according to the key your service needs. Does that mean you have to recreate the entire stream from the source? Not necessarily. This is where *repartitioning* comes in.

Repartitioning Event Streams

Event streams are partitioned according to the event key and the producer's event partitioner logic. Most commonly, the event partitioner is a deterministic hash function, meaning that the same key will always be assigned to the same partition for data locality's sake.

Repartitioning is the act of producing a *new event stream* derived from the original stream, with one or more of the following properties:

Different partition count

> With either a higher partition to increase processing parallelization, or a specific partition count to copartition with another stream (discussed more in the next section).

Different event key

Change the event key, often by rekeying on a data contained within the event's value.

Different event partitioner

Change the logic that selects the partition to write the event to.

Consider an example. Suppose there is a stream of user data coming in from a web-facing endpoint. The user's interaction are converted into unkeyed events, with the payload of the events containing both a user-id (`id`) and other arbitrary event data (`x`).

Say that the consuming microservice requires that all data belonging to a particular user `id` is contained within the *same partition*, regardless of how the source event stream is partitioned. In this case, the service assigns the key to the `id` and writes it to a new event stream with a partition count of 3, as shown in Figure 7-1.

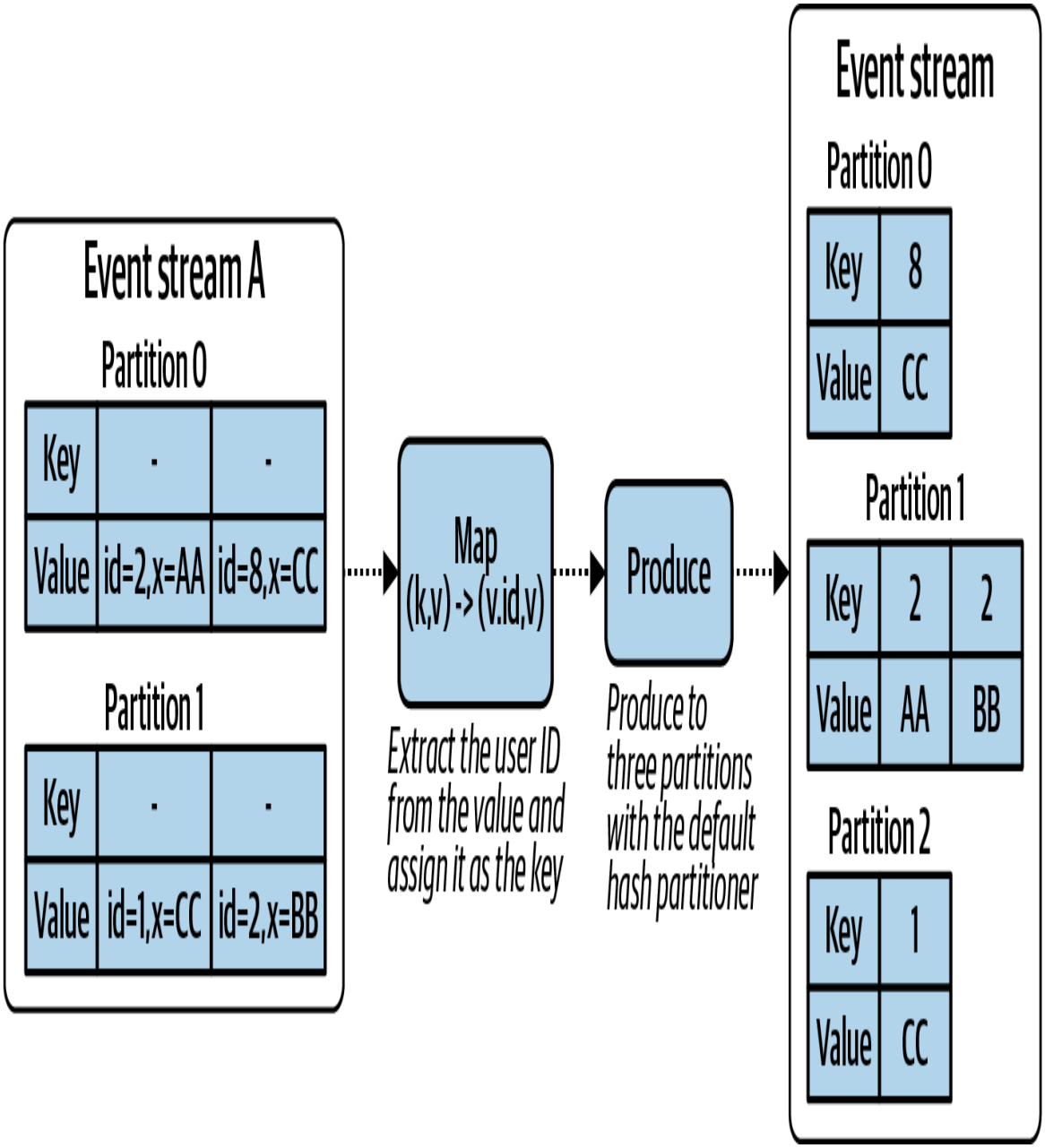


Figure 7-1. Repartitioning an event stream

There are a few of things to observe from this example.

- Events can be introduced out-of-order. You may notice that user `id=2` has events both in both partition 0 and partition 1 in the original stream. The order that they end up in the output stream is not necessarily deterministic, and you may need to account for out-of-order events. This is covered more in "Out-of-Order and Late-Arriving Events"

> **TIP**
>
> If the order of data in an event stream is extremely important, then it's best to generate it *in order* right from the start. The event stream in this example could simply be keyed by `id` from the start, and written in the actual order of occurrence.

- The throughput of the microservice may be limited by how fast it can repartition the data. For example, if you decided to repartition from 2 partitions to 1000 partitions, it is unlikely that the service could benefit from the additional partitions. It would remain bottlenecked on the much lower 2 partition input.

Repartitioning streams lets you change the key, the partition assignor, and the partition count. And this, in turn, lets you copartition your stream with another stream.

Copartitioning Event Streams

Copartitioning is the repartition of an event stream into a new one with the *same partition count* and the *same partition assignor* as another stream. Copartitioning ensures data locality as several stateful operations, such as joins and multi-stream aggregations, require that all events of a given key go to the same processing instance.

Consider an example based off of Figure 7-1. Say that you now need to join the repartitioned user event stream with a user entity stream, keyed on that same user `id`. The joining of these streams is shown in Figure 7-2.

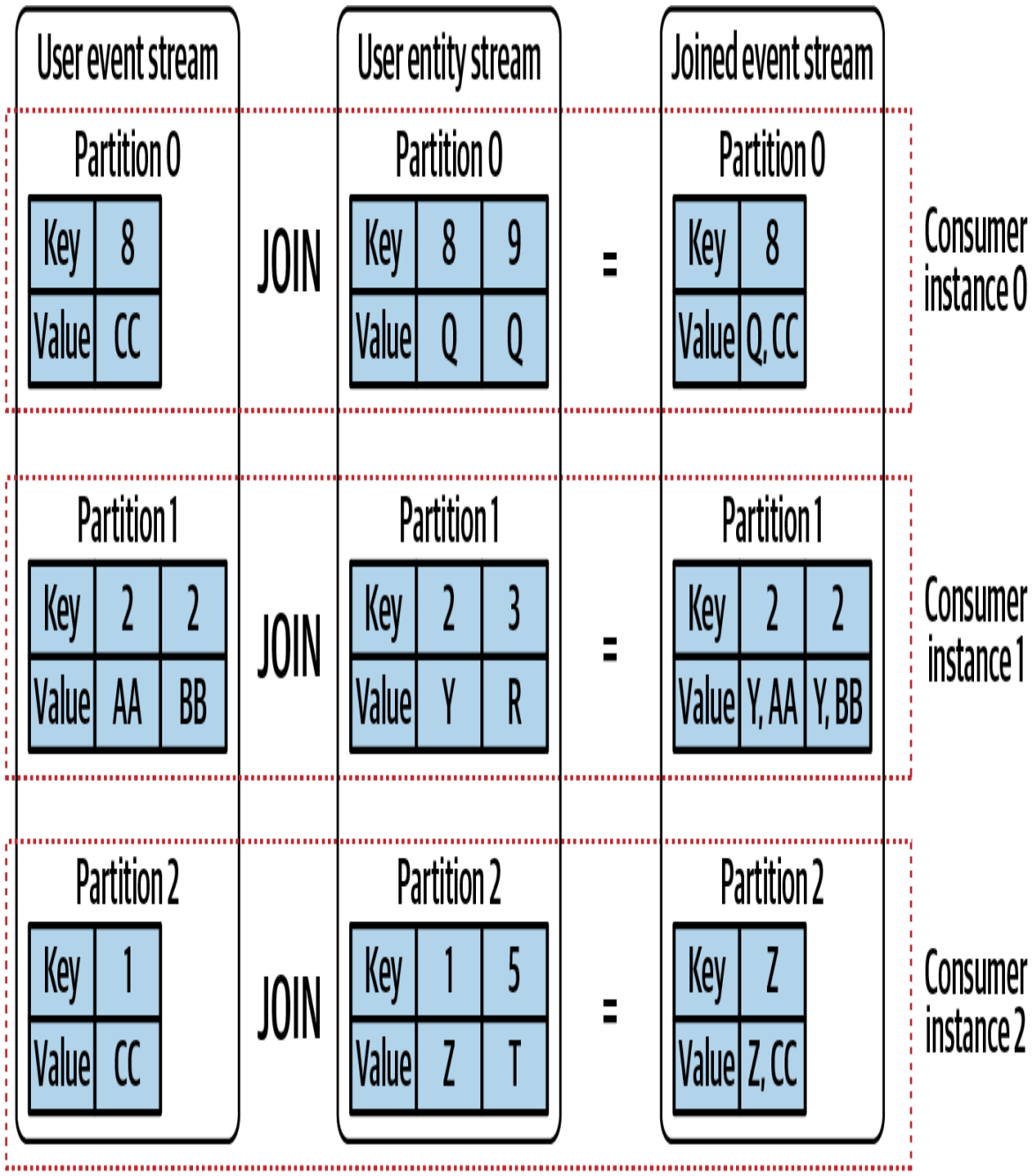


Figure 7-2. Copartitioned user event and user entity streams

Both streams have the same partition count and have had their events partitioned using the same partition assignor. Note that the key distribution of each partition matches the distribution of the other stream and that each join is performed by its own consumer instance. The next section covers how partitions are assigned to a microservice instance to leverage copartitioned streams, as was done in this join example.

Assigning Partitions within a Consumer Group

Partition assignment is quite trivial when there are no relationships between the streams - a simple round-robin assignment is more than enough.

Full-featured frameworks, like those in Chapter 10 and Chapter 11, provide under-the-hood hands-off partition assignments. The frameworks can infer from the application code which streams need to be copartitioned (or repartitioned), and will ensure that the partition assignor acts accordingly.

Basic consumer clients typically do *not* provide automatic copartitioning compatible assignments. You'll typically have to manage these manually at a more granular level, which can be challenging when you have multiple sets of copartitioned streams in more complicated microservices.

As a brief example, consider a service that consumes from three streams, two of which are copartitioned (say for a join operation).

Figure 7-3 shows two consumer instances, each with its own set of assigned partitions. C0 has two sets of copartitioned partitions compared to one for C1, since assignment both began and ended on C0.

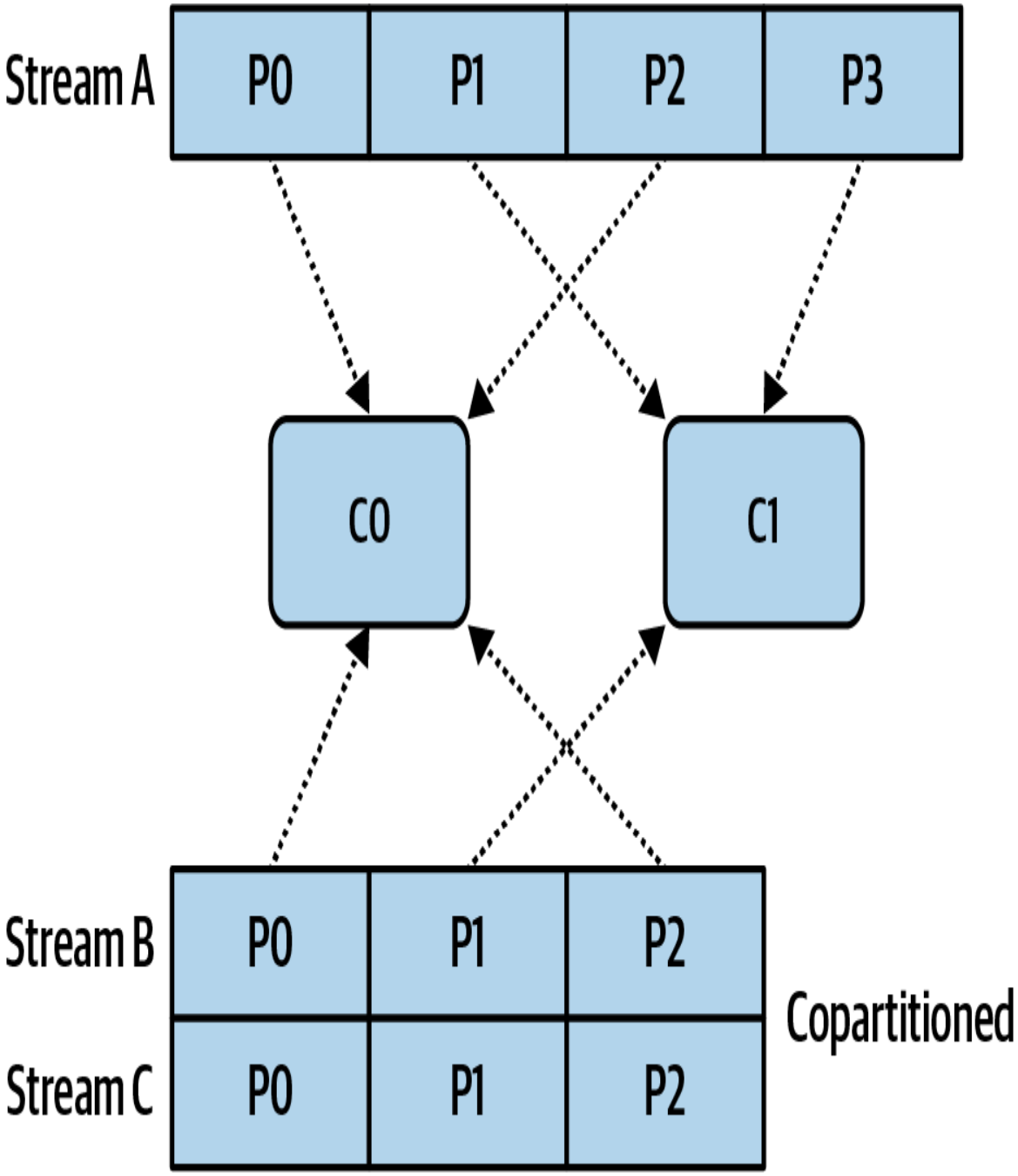


Figure 7-3. A round-robin partition assignment for two consumer instances and three streams

Additional consumer instances added to the consumer group cause a rebalance. Figure 7-4 shows the effects of adding two more consumer instances.

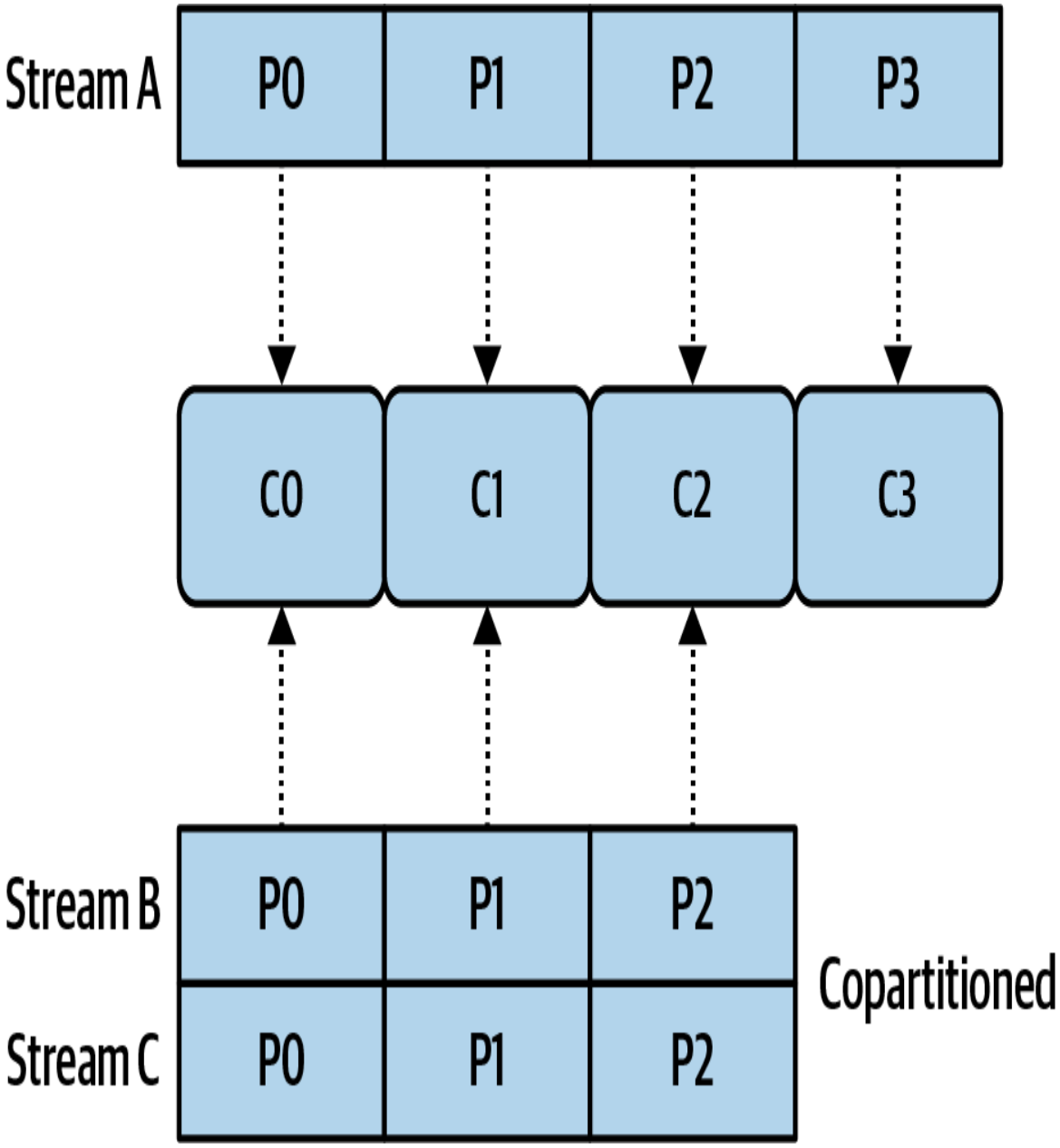


Figure 7-4. Round-robin partition assignments for four consumer instances and three streams

C2 is now assigned the copartitioned P2s, as well as stream A's P2. C3, on the other hand, only has partition P3 from stream A because there are no additional partitions to assign. Adding any further instances will not result in any additional parallelization.

Example 7-1.

The exact mechanisms of consumer group membership, partition assignments, and rebalancing all rely heavily on your event broker and the processing

framework choices. Check the documentation to ensure you have a proper understanding and test it to make sure it behaves as expected before moving to production.

While assigning the right partitions to the right instances is a big part of stateful event-driven microservices, it's time to take a look at adding the state stores themselves.

Selecting a State Store for your Microservice

Let's start with an example to anchor the exploration of state. Consider Figure 7-5, a service built to package up customer `Orders` based on the `Inventory` available.

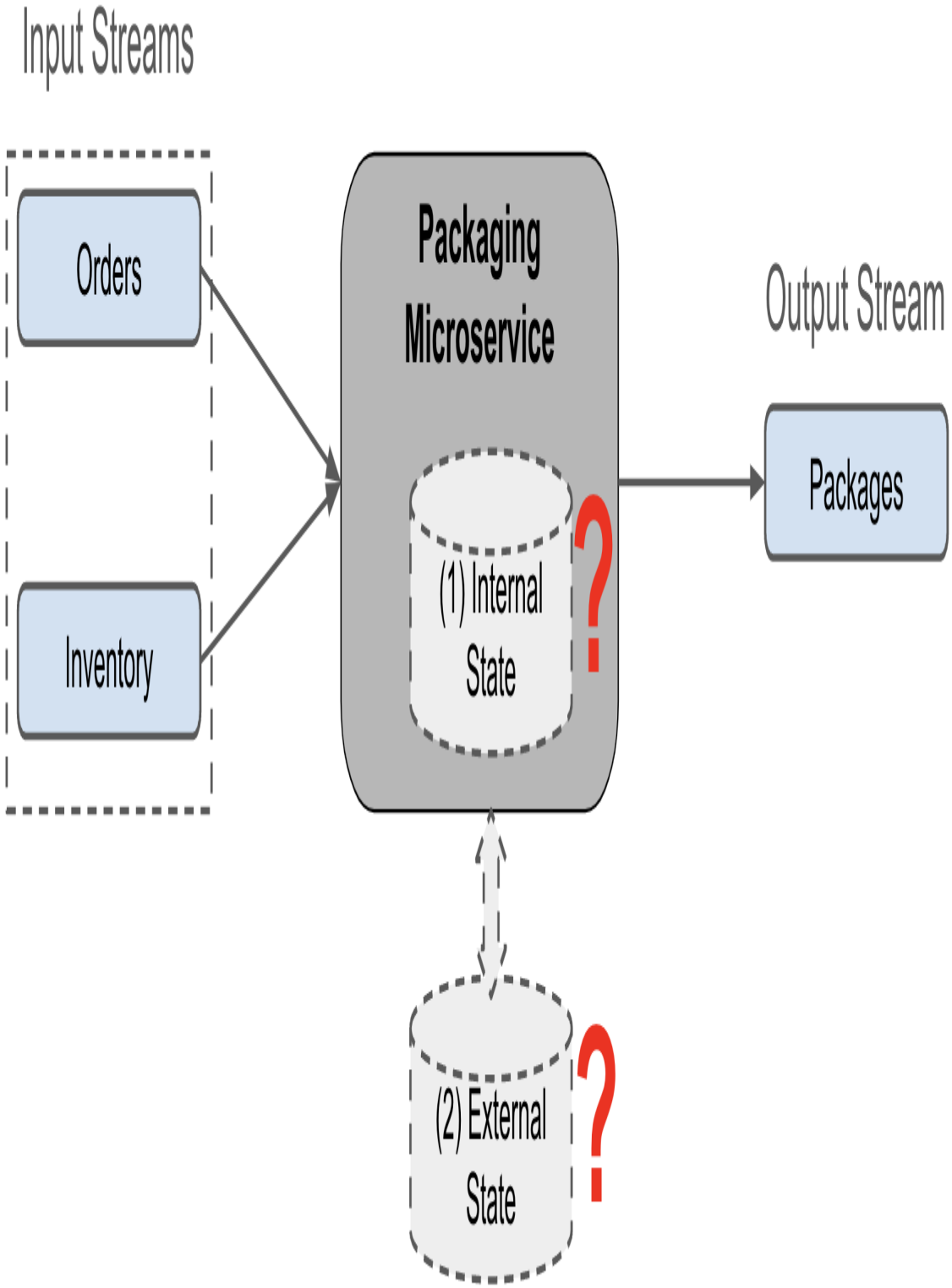


Figure 7-5. A shipment packaging service loading state to either an internal or external state store

This service will need to materialize both Orders and Inventory events, figure out what *can* be shipped right away and what *can not*. Secondly, the service is also responsible for ensuring that the products are put into the box, that the shipping label is affixed, and that the box is then made ready for dispatch to the customer.

For the sake of simplicity and to focus on the role of state in microservices, we're going to pretend that at worst, an order is only delayed awaiting new inventory. Additionally, we'll assume that there are no flaws in the packing of the package, particularly due to the imperfect mapping of the digital world to the real physical world. More on that later though. (TODO - When?)

Internal state stores are located in the same container or VM as the microservice, and are typically allocated in memory or on volatile disk. They remain online when the service is online. If the microservice is halted or restarted, then so is the state store.

External state stores exists outside of the processor's container or VM. It is largely independent of the microservice's life cycle, remaining online and available even if the microservice itself is restarted or shut down.

The first step in building this or any stateful microservice is to figure out what kind of state store (or stores) your service needs. In this case, is the service best suited to an external state store, or an internal one? The next sections will help you answer that question for yourself.

Materializing State to an Internal State Store

Internal state stores are located in the same container or VM as the microservice, and are typically allocated in memory or on volatile disk. They act primarily as high-performance and low-latency caches, since communication between the service and state store do not need to go over the network. The internal state store's life cycle is tied to the microservice. If the microservice is halted or restarted, then so is the state store.

Each microservice instance materializes the events from its assigned partitions, keeping each partition's data logically separate within the store.

These logically separate materialized partitions permit a microservice instance to simply drop the state for a revoked partition after a consumer group rebalance. This avoids resource leaks and multiple sources of truth by ensuring that materialized state exists only on the instance that owns the partition. New partition assignments can be rebuilt by consuming the input events from the event stream or from the changelog.

High-performance key/value stores, such as RocksDB, are typically used to implement internal state stores. They are optimized to be highly efficient with local solid-state drives (SSDs), enabling performant operations on data sets that exceed main memory allocation. While key/value stores tend to be the most common implementation for internal state stores, any form of data store can be used. A relational or document data store implementation would not be unheard of, but again, it would need to be instantiated and contained within each individual microservice instance.

Internal state stores are typically volatile, and restarting the microservice will cause you to lose your state. Upon reloading, the state store will need to reload its state from a secondary location. State store snapshots stored to an external location is one option for restoration, and we'll take a closer look at that later in this chapter when we get to external data stores. The other form of restoration relies on the stream itself - in this case, a *changelog*. Let's take a look at that now.

Using a Changelog Event Stream as State Recovery

A *changelog* is a record of all changes made to the data of the state store. It is the stream in the table-stream duality, with the table of state transformed into a stream of individual events. As a permanent copy of the state maintained *outside* of the microservice instance, the changelog can be used to rebuild state as shown in Figure 7-6, and serves as a checkpoint.

TIP

Changelogs optimize the task of rebuilding failed services because they store the results of previous processing, allowing a recovering processor to avoid reprocessing all input events.

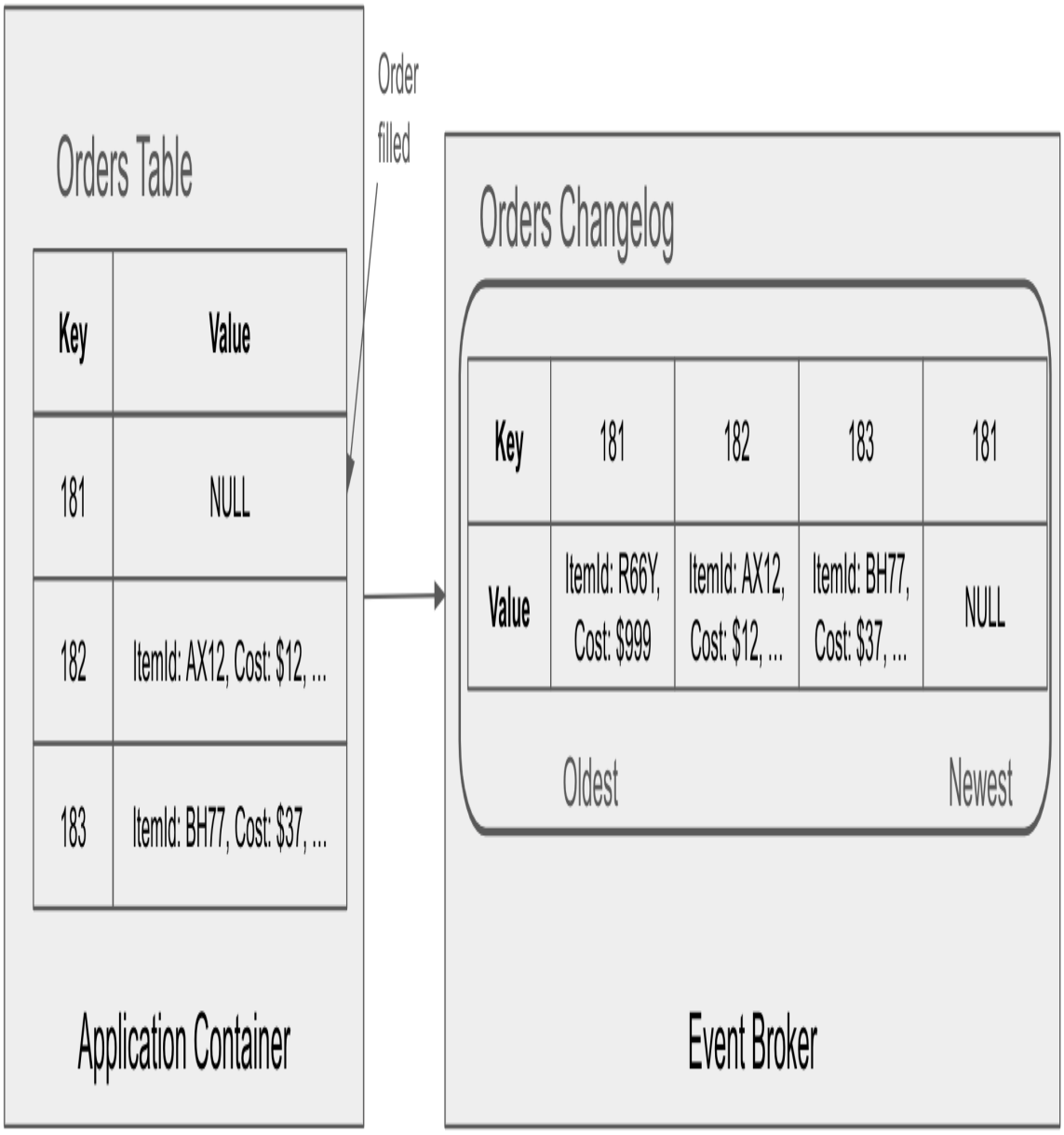


Figure 7-6. A state store with changelogging enabled

In this example, the Orders table provides a materialized view of the orders that are ready to be packaged. Once the order is packaged, the record can be deleted from the table. A tombstone record is issued into the Orders changelog, such that the earlier data associated with the `key 181` can be removed from the stream.

Changelog streams are stored in the event broker just like any other stream, though its write and read permissions should remain strictly limited to the associated application. No other services should access the changelog's data. Changelogs are commonly compacted, particularly since their primary purpose is to restore state, and not track a history of changes as is more common in entity streams.

During restoration, the newly created application instance loads the data from the associated changelog partitions, as shown in Figure 7-7.

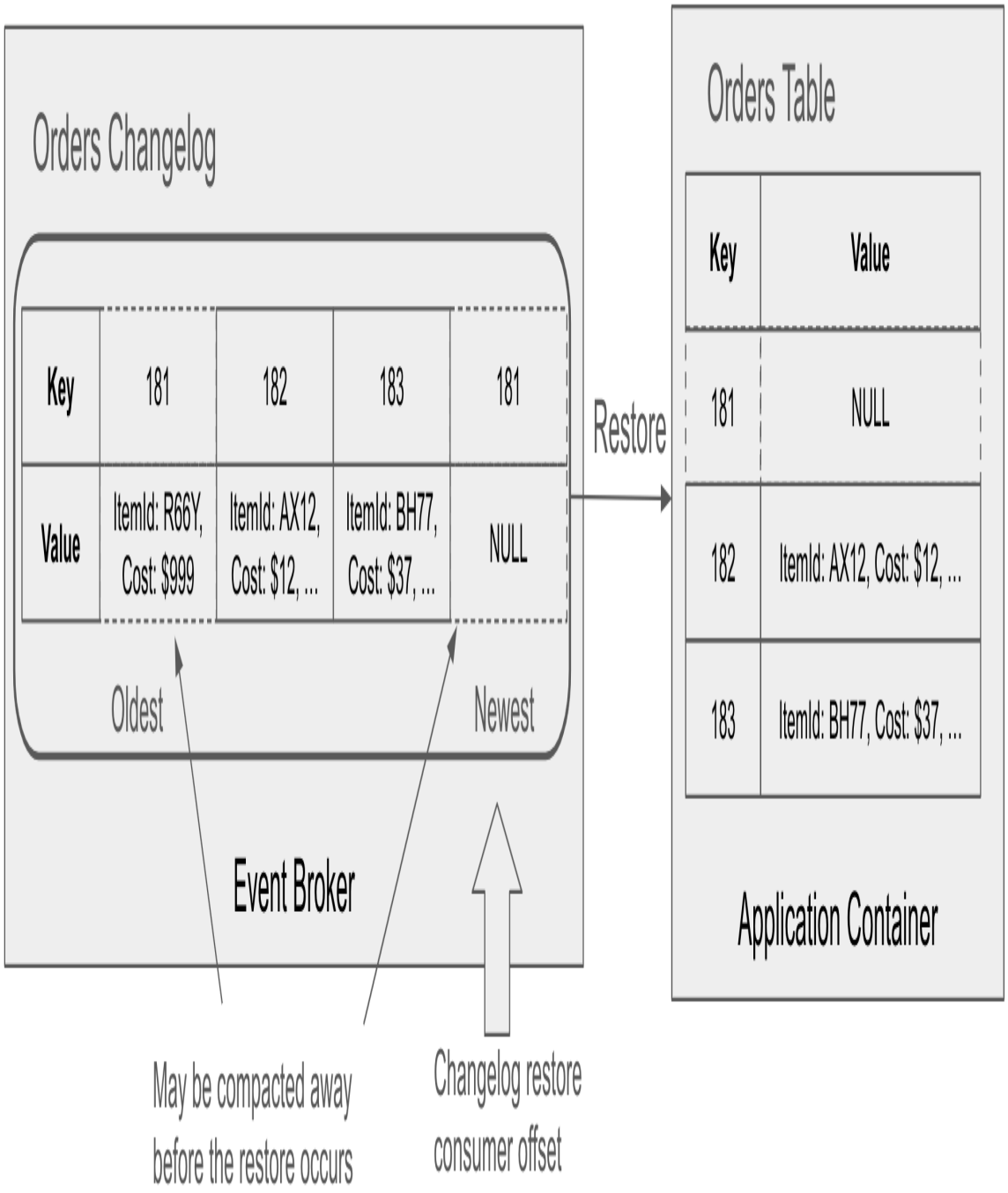


Figure 7-7. Restoring the application's state store from a single changelog partition

The application can restore its state as quickly as the events can be read and inserted into their state store. In this example, note that `key 181` may or may not have been asynchronously compacted away. In the case that it has been

compacted, then it simply won't be loaded into the table. If it has not, then it will be loaded and subsequently deleted. Note that no business logic is triggered during state restoration.

Changelogs are either provided as a built-in feature, such as in the Kafka Streams client, or independently implemented by the application developer. Basic producer/consumer clients tend not to provide any changelogging or stateful support.

Materializing Global State

A *global state store* is a special form of the internal state store. Instead of materializing only the partitions assigned to it, a global state store materializes the data of *all* partitions for a given event stream to each microservice instance. Global state is also known as broadcast state, and in either case, each microservice instance generates a complete materialization of the entire input topic, as shown in Figure 7-8.

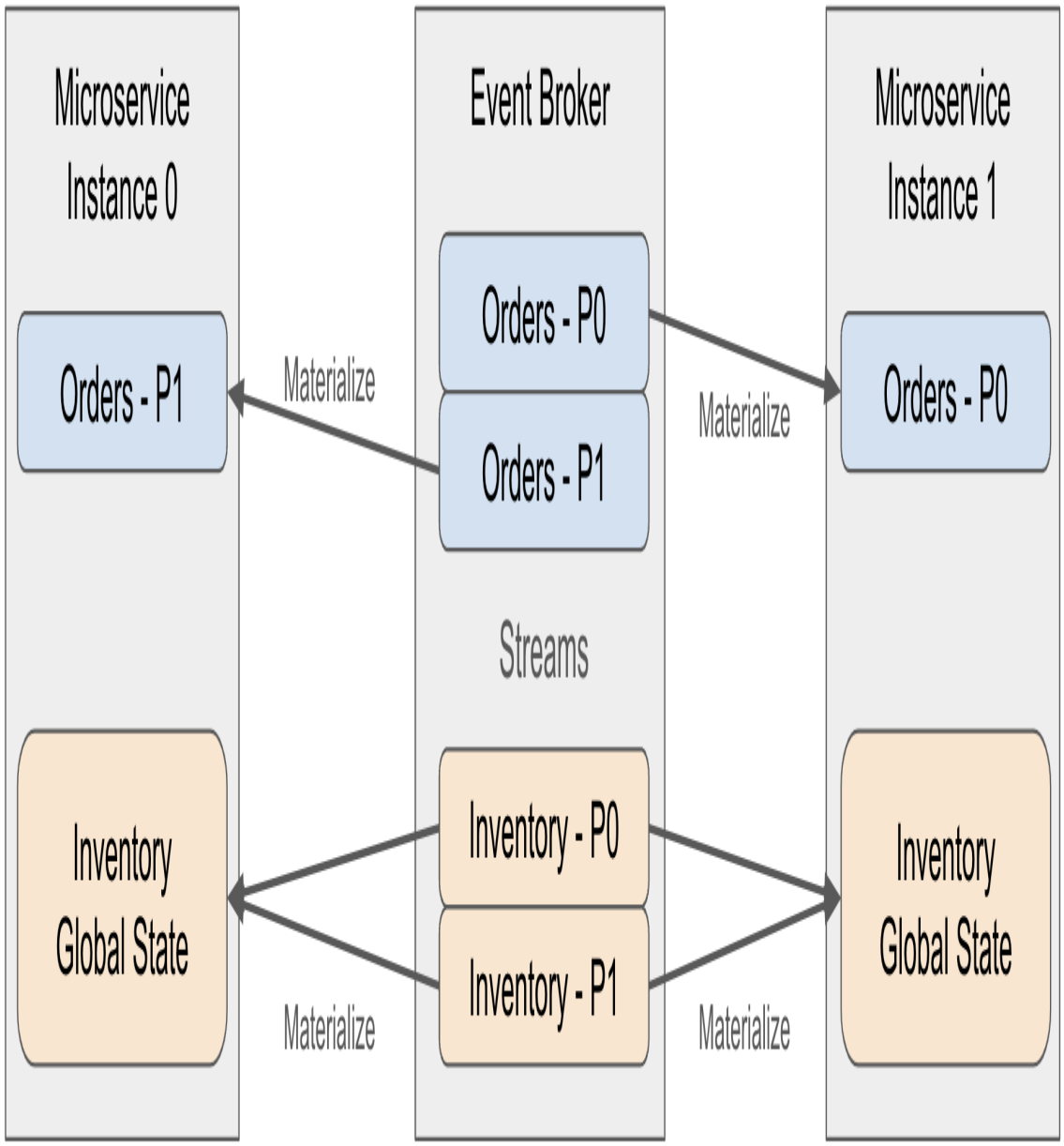


Figure 7-8. Global materialized state and nonglobal materialized state

In this example, we're decided to load the entire state of Inventory into each microservice instance. Each instance can then check available inventory before asking for an Order to be packaged, so that it doesn't waste the time of the warehouse personnel. Global state stores are useful when each microservice instance requires a full data set, *and* when that data set is small, commonly used, and seldom-changing.

Global state stores are typically not used to drive event-driven logic, but rather as the means for providing data lookups.

Advantages of Internal State Stores

Scalability requirements are offloaded from the developer

A major benefit of using internal state stores with changelog recovery is that scalability requirements are fully offloaded to the event broker and compute resource clusters. Each application can be scaled simply by increasing and decreasing the instance count, allowing the application development team to focus strictly on writing application logic. Meanwhile, the microservices capability teams provides the scaling mechanisms common to all event-driven microservices.

High-performance disk-based options

Maintaining all state within main memory is not always possible in an event-driven microservice, especially if costs are to be kept low. Physically attached local disk can be quite performant for the majority of modern microservice use cases. Local disk implementations tend to favor high random-access patterns, generally supported by SSDs. For instance, the latency for a random-access read from an SSD using RocksDB is approximately 65 microseconds, which means a single thread will have a sequential access ceiling of approximately 15.4k requests/second. In-memory performance is significantly faster, serving millions of random-access requests per second as the norm. The local-disk and local-memory approach allows for extremely high throughput and significantly reduces the data-access bottleneck.

Flexibility to use network-attached disk

Microservices may also use network-attached disk instead of local disk, which significantly increases the read/write latency. Since events typically must be processed one at a time to maintain temporal and offset order, the single processing thread will spend a lot of time awaiting read/write

responses, resulting in significantly lower throughput per processor. This is generally fine for any stateful service that doesn't need high-performance processing, but can be problematic if event volumes are high.

Accessing "local" data stored on network-attached disk has a much higher latency than accessing physically local data stored in the system's memory or attached disk. While RocksDB paired with a local SSD has an estimated throughput of 15.4k request/second, introducing a network latency of only 1 mS round-trip time to an identical access pattern reduces the throughput cap to just 939 requests/second. While you might be able to do some work to parallelize access and reduce this gap, remember that events must be processed in the offset sequence in which they are consumed and that parallelization is not possible in many cases.

One major benefit of network-attached disk is that the state can be maintained in the volume and migrated to new processing hardware as needed. When the processing node is brought back up, the network disk can be reattached and processing can resume where it left off, instead of being rebuilt from the changelog stream. This greatly reduces downtime since the state is no longer completely ephemeral as with a local disk, and also increases the flexibility of microservices to migrate across compute resources, such as when you are using inexpensive on-demand nodes.

Disadvantages of Internal State Stores

Limited to using runtime-defined disk

Internal state stores are limited to using only disk that is defined and attached to the node at the service's runtime. Changing either the size or quantity of attached volumes typically requires halting the service, adjusting the volumes, then restarting the service. In addition, many compute resource management solutions allow only for volume size to be *increased*, as decreasing a volume's size means that data would need to be deleted.

Wasted disk space

Data patterns that are cyclical in nature, such as the traffic generated to a shopping website at 3 p.m. versus 3 a.m., can require cyclical storage volume. That is, these patterns may require a large maximum disk for peak traffic but only a small amount otherwise. Reserving full disk for the entire time can waste both space and money when compared to using external services that charge you only per byte of data stored.

Scaling and Recovery of Internal State

Scaling processing up to multiple instances and recovering a failed instance are identical processes from a state recovery perspective. The new or recovered instance needs to materialize any state defined by its topology before it can begin processing new events. The quickest way to do so is to reload the changelog topic for each stateful store materialized in the application.

Using hot replicas

While it is most common to materialize only a single partition per microservice instance, you can also materialize a standby "hot replica" on another instance. Apache Kafka has this functionality built into its Streams framework via a simple configuration setting. It provides highly available state stores and enables the microservice to tolerate instance failures with zero downtime.

To simplify this example, let's assume that each order is only for one product and that both inventory and product streams are keyed on `productId`. In other words, they're *copartitioned*, and packaging up an Order from P0 (Partition-0) only requires data from Inventory P0. Figure 7-9 shows what a standard three-instance deployment would look like, with each copartitioned data set located in a single instance.

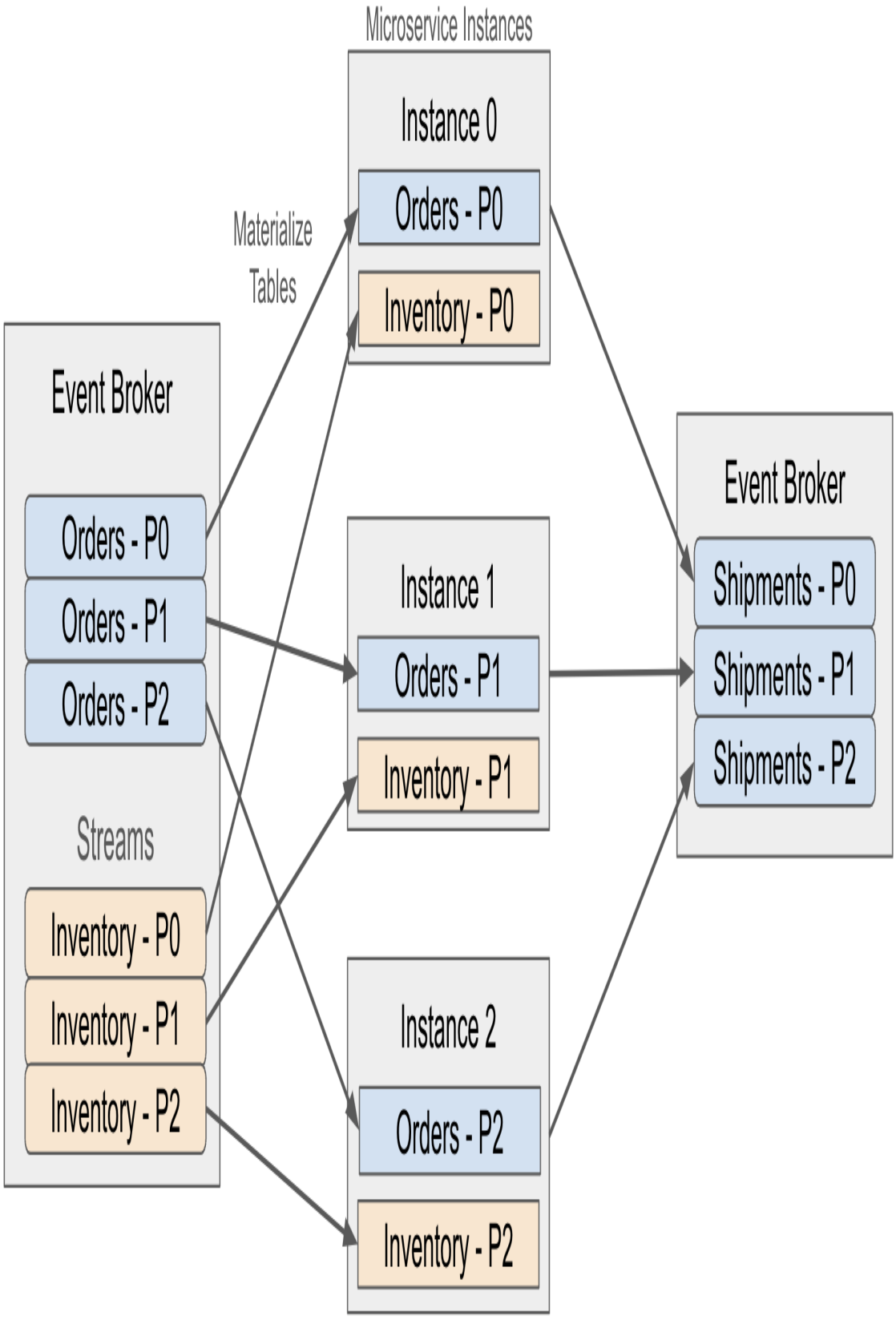

Microservice Instances
Instance 0
Orders - P0
Inventory - P0
Materialize
Tables
Event Broker
Orders - P0
Orders - P1
Orders - P2
Streams
Inventory - P0
Inventory - P1
Inventory - P2
Instance 1
Orders - P1
Inventory - P1
Instance 2
Orders - P2
Inventory - P2
Event Broker
Shipments - P0
Shipments - P1
Shipments - P2

Figure 7-9. Joining Orders with Inventory, with one partition per instance

The processing load is split up between the three microservice instances, and we're able to compute the Shipments accordingly. However, in the event that an instance fails, we'd have to stop processing the assigned partitions, shift them to a new consumer instance, reload the state from the changelog, and then resume processing.

Figure 7-10 shows a three-instance microservice deployment with an internal state store replication factor of 2. Each of the Orders and Inventory partitions are materialized twice, once as the primary and once as a replica. Each instance must manage its replicas just as it would its primaries, except that the replicas do not drive any logic.

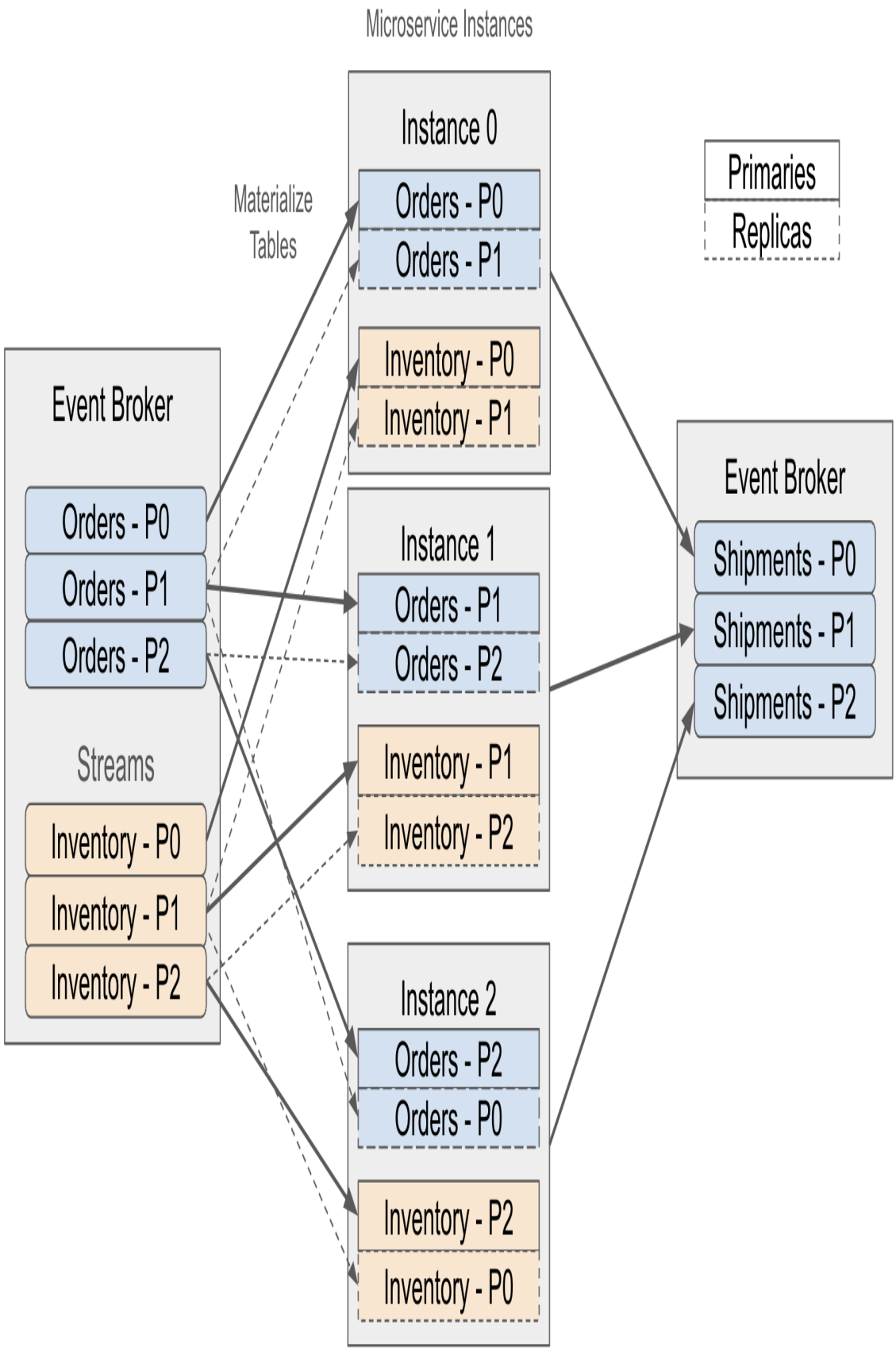

Microservice Instances
Instance 0
Orders - P0
Orders - P1
Inventory - P0
Inventory - P1
Materialize
Tables
Primaries
Replicas
Event Broker
Orders - P0
Orders - P1
Orders - P2
Streams
Inventory - P0
Inventory - P1
Inventory - P2
Instance 1
Orders - P1
Orders - P2
Inventory - P1
Inventory - P2
Event Broker
Shipments - P0
Shipments - P1
Shipments - P2
Instance 2
Orders - P2
Orders - P0
Inventory - P2
Inventory - P0

Figure 7-10. Joining Orders with Inventory, with one hot replica of each partition co-located in another microservice instance

When an instance is terminated, the consumer group leader eventually rebalances the partition assignments. The partition assignor determines the location of the hot replica (it previously assigned all partitions and knows all partition-to-instance mappings) and reassigns the partitions accordingly.

In Figure 7-11, instance 1 has terminated, and the remaining microservice instances rebalance their partition assignments. Instance 0 takes over the workload that instance 1 used to do, as it contains the hot P1 replicas.

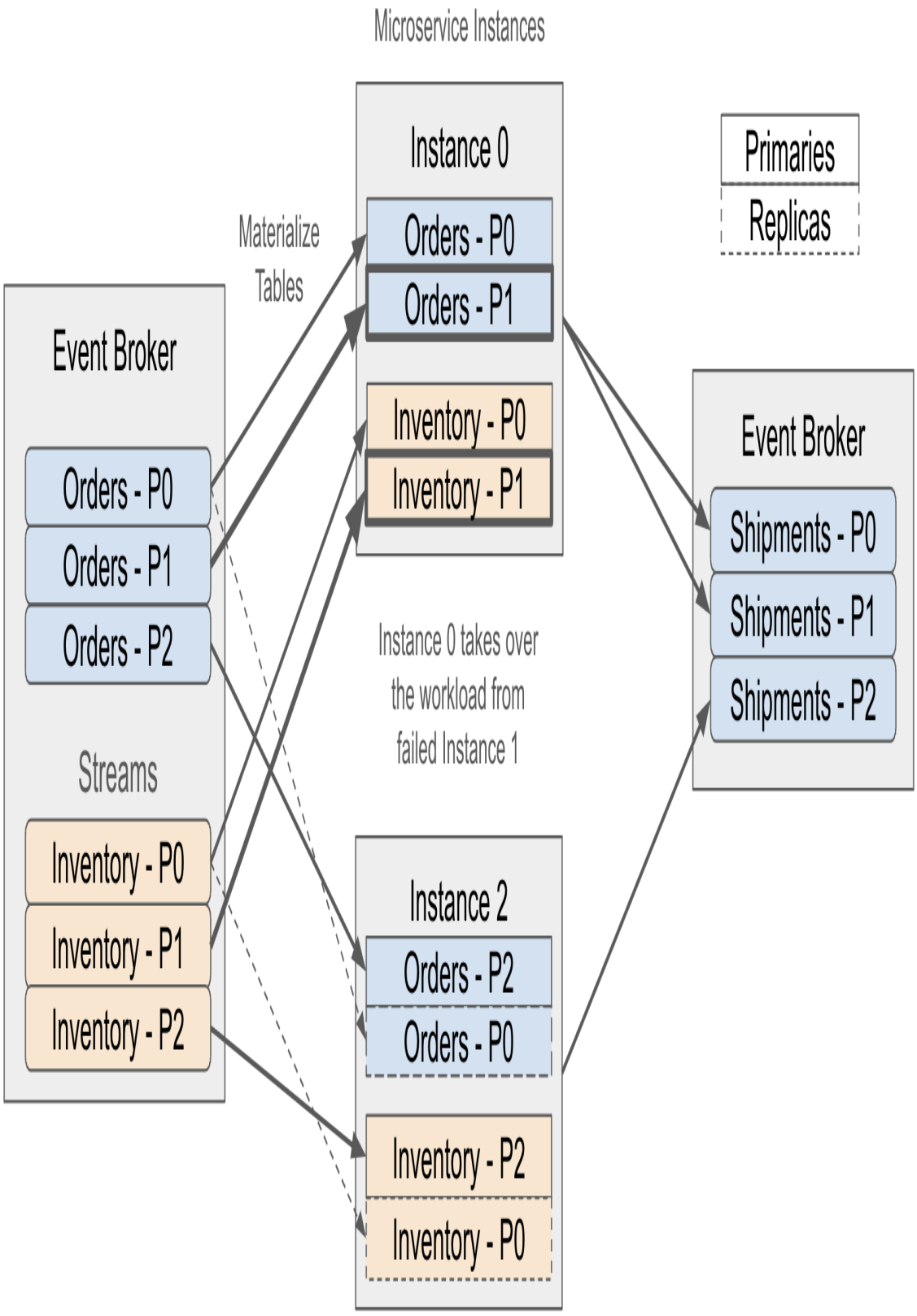

Microservice Instances
Instance 0
Primaries
Replicas
Materialize
Tables
Orders - P0
Orders - P1
Event Broker
Orders - P0
Orders - P1
Orders - P2
Inventory - P0
Inventory - P1
Event Broker
Shipments - P0
Shipments - P1
Shipments - P2
Instance 0 takes over
the workload from
failed Instance 1
Streams
Inventory - P0
Inventory - P1
Inventory - P2
Instance 2
Orders - P2
Orders - P0
Inventory - P2
Inventory - P0

Figure 7-11. Partition assignment rebalancing due to instance 1 termination

Once processing has resumed, new hot replicas must be built from the changelog to maintain the minimum replica count. In this case, you may see Instance 2 building a hot replica for P1, while Instance 1 builds a hot replica for P2. However, if the original Instance 1 comes back online, then they'll redistribute their workloads back to the original configuration once it has rebuilt its internal state.

> **NOTE**
>
> This section addresses hot failovers in general. For specifics, read your prcoessing framework's documentation to get a better understanding of the hot partition failover and failback mechanisms.

Hot-replicas trade off additional disk usage and replica maintenance processing in exchange for reducing downtime when an instance fails. Low-latency use cases will find hot replicas to be very useful, whereas workloads that can tolerate a higher latency and periodic outages may find them unnecessary.

Restoring and scaling from changelogs

When a newly created microservice instance joins the consumer group, any stateful partitions that it is assigned can be reloaded simply by consuming from its changelog. During this time the instance should not be processing new events, as doing so could produce nondeterministic and erroneous results.

Restoring and scaling from input event streams

If no changelog is maintained, the microservice instance can rebuild its state stores from the input streams. It must consume all of its input events from the very beginning of its assigned event stream partitions. Each event must be consumed and processed in strict incrementing order, its state updated, and any subsequent output events produced.

> **WARNING**
>
> Consider the impact of events produced during a full reprocessing. Downstream consumers may need to process these idempotently or eliminate them as duplicates. You may also choose to implement a mechanism that halts output events until a given progress condition is met.

Restoring from the input event streams instead of a changelog can take much longer to rebuild state. It's best to employ this strategy only for simple topologies where duplicate output is not a concern, the entity event streams are well-compacted, and input event stream retention is short.

Materializing State to an External State Store

External state stores exist outside the microservice's containers or virtual machines. The external state store either uses its own dedicated hardware, runs inside its own container or VM, or more commonly nowadays, is provided by a fully managed cloud service.

You can implement an external data store using your preferred technology, but you should select it based on the needs of the microservice's problem space. Some common examples of external store services include relational databases; document databases; graph databases; geospatial search systems; and distributed, highly available key/value stores.

Keep in mind that while a specific microservice's external state store may use a common data storage platform, the data set itself must remain logically isolated from all other microservice implementations. Sharing materialized state between microservices is an (unfortunately somewhat common) anti-pattern for implementers of external data stores who seek to use a common materialized data set to serve multiple business needs. This can lead to tight coupling between otherwise completely unrelated products or features and should be avoided.

> **WARNING**
>
> Do not share direct state access with other microservices! Instead, all microservices must materialize their own copy of state. This eliminates direct couplings and isolates microservices against unintentional changes, but at the expense of extra processing and data storage resources.

Having already examined the advantages of internal state, let's now look at what external state has to offer.

Advantages of External State

Full data locality

Unlike internal state stores, external state stores can provide access to all materialized data for each microservice instance. A single materialized data set eliminates the need for partition locality when you are performing lookups, relational queries on foreign keys, and geospatial queries between a large number of elements. However, each microservice instance remains responsible for materializing and processing its own assigned partitions.

> **TIP**
>
> Use state stores with strong read-after-write guarantees to eliminate inconsistent results when using multiple instances.

Maintains State During Microservice Outages

The data stored in the external state store is independent of the lifecycle of the microservice instances. Failures, rolling restarts, and temporary halts of the microserivce don't require you to rebuild state when starting back up, reducing your downtime.

Technology Options

External data stores can leverage technology that the organization is already familiar with, reducing the time and effort it takes to deliver a microservice to production. Basic consumer/producer patterns are especially good candidates for using external data stores, as covered in Chapter 9. Function-as-a-Service solutions are also excellent external data store candidates, as covered in [[Link to Come]].

Drawbacks of External State

Management of multiple technologies

External state stores are managed and scaled independently of the microservice business logic solution. One of the risks of an external data store is that the microservice owner is now on the hook for ensuring that it is maintained and scaled appropriately. Each team must implement proper resource allocation, scaling policies, and system monitoring to ensure that their data service is suitable and resilient for the microservice's load. Managed data services provided by the organization's capabilities team or by a third-party cloud platform vendor can help distribute some of this responsibility.

> **NOTE**
>
> Each team must fully manage the external state stores for its microservices. Do not delegate responsibility of external state store management to its own team, as this introduces a technical cross-team dependency. Compose a list of acceptable external data services with guides on how to properly manage and scale them. This will alleviate each team from having to independently discover and implement its own management solutions.

Performance loss due to network latency

Accessing data stored in an external state store has a much higher latency than accessing data stored locally in memory or on disk. In "Advantages of Internal State Stores", you saw that using a network-attached disk introduces a slight network delay and can significantly reduce throughput and performance.

While caching and parallelization may reduce the impact of the network latency, the tradeoff is often added complexity and an increased cost for additional memory and CPU. Not all microservice patterns support caching and parallelization efforts either, with many requiring the processing thread to simply block and wait for a reply from the external data store.

Financial cost of external state store services

Direct financial costs tend to be higher with external data stores than with similarly sized internal data stores. Hosted external state store solutions often charge by the number of transactions, the size of the data payload, and the retention period for the data. They may also require over-provisioning to handle bursty and inconsistent loads. On-demand pricing models with flexible performance characteristics may help reduce costs, but you must be sure they still meet your performance needs.

Full data locality

Though also listed as a benefit, full data locality can present some challenges. The data available in the external state store originates from multiple processors and multiple partitions, each of which is processing at its own rate. It can be difficult to reason about (and debug) the contributions of any particular processing instance to the collective shared state.

You'll need to be careful about race conditions and nondeterministic behavior, as each microservice instance operates on its own independent stream time. The stream time guarantees of a single microservice instance do not extend to all of them.

For example, one instance may attempt to join an event on a foreign key that has not yet been populated by a separate instance. Reprocessing the same data at a later time may execute the join. Because the stream-processing of each instance is completely separate from the others, any results obtained through this approach are likely to be nondeterministic and nonreproducible.

Scaling and Recovery with External State Stores

Scaling and recovery of microservices using an external state store is pretty easy. You simply add a new instance with the necessary credentials to access the state store. In contrast, scaling and recovery of the underlying state store are dependent completely upon the selected technology. It may be simple with a cloud service, it may be difficult if you're doing it in-house.

To reiterate an earlier point, having a list of acceptable external data services with guides on how to properly manage, scale, back up, and restore them is essential for providing developers a sustainable way forward. Unfortunately, the number of state store technologies is prohibitively large and effectively impossible to discuss in this book. Instead, I'll simply generalize the strategies of rebuilding state into three main techniques: via source streams, via changelogs, and via snapshots.

Recovery using the source streams::

Rewinding the consumer offsets to rebuild from the start of the stream is one option for rebuilding state. This method incurs the longest downtime of all options, but is easily reproducible and relies only on the persistent storage of the event broker to maintain the source data. Keep in mind that this option is effectively a full application reset and will also result in the reproduction of any output events (unless intentionally discarded by the microservice).

Recovery Using changelogs

External state stores typically do not rely on using broker-stored changelogs to record and restore state, though there is no rule preventing this. Just like when rebuilding from source streams, you must create a fresh copy of the state store. If rebuilding from changelogs, the microservice consumer instances must ensure they rebuild the entire state as stored in the changelog before resuming processing.

> **WARNING**
>
> Rebuilding external state stores from source event streams or changelogs can be prohibitively time-consuming due to network latency overhead. Make sure you can still meet the microservice SLAs in such a scenario.

Recovery Using Snapshots or Checkpoints

It is far more common for external state stores to provide their own backup and restoration process. Snapshots (or checkpoints) consist of the data store offloading an image of the state at a certain point in time to yet-another external storage location (for e.g., Amazon S3).

The challenge with snapshots is ensuring that, upon restoration, your microservice's consumer offsets line up with the restored state. Too late and you'll miss events, suffering data loss. Too early and you'll process duplicate results, which may or may not cause an issue, depending on your service and downstream dependencies.

If your microservice is idempotent and duplicate data is not a problem, you can set your consumer offsets to a time period before the snapshot was taken. This will ensure that no data is missed, providing "at-least-once" event processing guarantees.

If the stored state is not idempotent and any duplicate events are not acceptable, then you should *store your consumer's partition offsets alongside the data within the data store*. This is best done in an atomic transaction, as it will ensure that your state and consumer offsets are perfectly in sync. Then, when your service restores its state from the snapshot, it can set its consumer group offsets to the snapshot values. This will enable your service to have "effectively once" (sometimes called "exactly once") event processing guarantees. This is covered in more detail in "Maintaining consistent state via state store transactions".

Rebuilding the External State Store

New business requirements sometimes require a change to the state store models of a service. You may need to add new information to existing events, perform some extra join steps with another materialized table, or otherwise store newly derived business data. In the vast majority of cases, it is far better to rebuild the external state store than to try to migrate your data.

> **WARNING**
>
> Data migrations are very risky, and should be avoided whenever possible. The migrated data may be inconsistent with what is actually written in the internal stream the original source of truth from the input stream, resulting in inconsistent output. It can be very difficult to track these errors down, and any time or money saved via a data migration is often accounted for 10-fold in trying to fix the unintended consequences.

Rebuilding the microservice's state stores by is your best and safest option for accounting for changes in the data model.

First, update your code to account for the changed data models. You'll want to make sure that everything works in testing before you switch over to production. Secondly, *create a new instance of the external state store*, and *do not delete the original yet!* You may need it to fall back on if things go poorly. Third, rewind the updated microservices' consumer offsets to the very beginning of the stream. Fourth, start it up and let it repopulate the data store.

Note that just like internal state store rebuilding, you may need to implement a "mute" to prevent your service from outputting events during the rebuild. Alternately, you may choose not to, and simply output the new results as your business logic executes on the historical data.

Finally, once you have verified that everything is operational with the newly migrated service, you can delete the old state store and any changelogs or snapshots.

> **TIP**
>
> If your application is critically reliant upon a source of input data, you must ensure that it remains readily available for reprocessing. It effectively serves as your snapshot for restoring state, and needs to be protected accordingly.

As an example, consider a scenario where a company starts to offer international shipping. Previously, they may only have shipped nationally and not have bothered to collect the country information of their customer. Now, they collect the country information during signup, as their packaging service may need that information to best select a shipper for their goods.

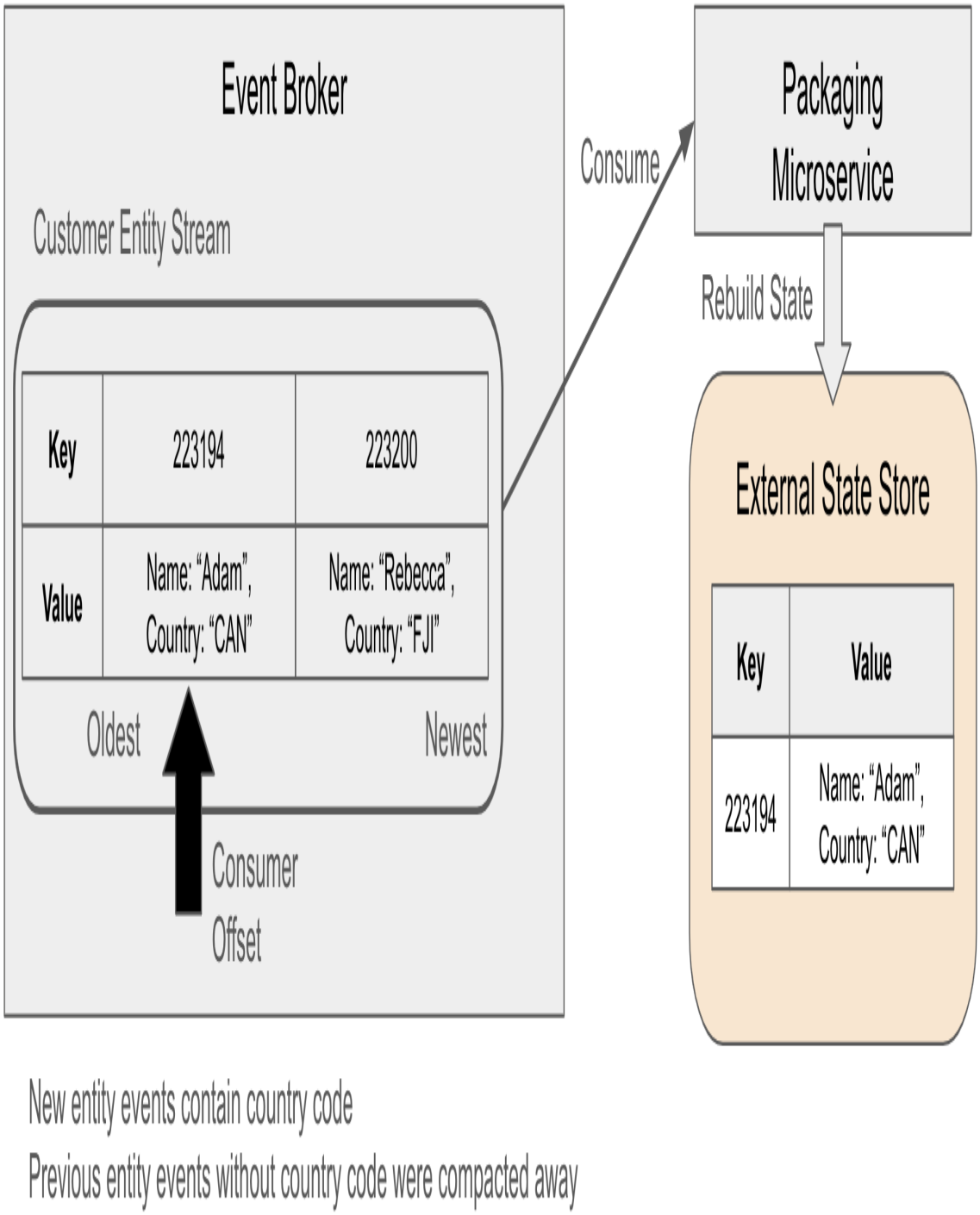


Figure 7-12. The packaging microservice rebuilding its external state store, to make business decisions off the newly added `Country` field

The microserivce that populates the Customer stream will have evolved its schema to include `Country` as a field type. But the packaging service will need to rebuild its state to include the Customer's country as part of its state store. Thus, you can rewind your service to the start of the Customer stream and rebuild the external state store.

Building State via Event-Sourcing

Event-sourcing is an architectural pattern based on recording what happened as a sequence of events to an immutable append-only log. You can then *aggregate* the current state by applying the events in exactly the same order that they appear in the log.

Events in the event-sourced log can be delta events, state events, hybrid events, or a mixture of many different types. Event-sourcing events are very tightly coupled to the system that writes and reads them, and they are *not* suitable for consumption by other microservices in other domains across the organization.

This architecture is often promoted as an alternative to the traditional create, read, update, delete (CRUD) model commonly found in relational-database type frameworks. In the CRUD model, a mutable entity is modified such that only the final state is retained. You have no access to the operations that led to the final state, and cannot replay them in the case that there's a problem in the logic that made the computations.

There are some limitations to the CRUD model that may make event sourcing an attractive alternative. For one, the data store must process operations as they are invoked, and under heavy use can result in significantly slower operations and failures. Secondly, high-concurrency operations on the same entities can result in data conflicts and failed transactions, further increasing load on the system. But the CRUD model also contains several distinct advantages.

Though it depends largely on the data store, most CRUD implementations offer strong read-after-write consistency. They're also fairly intuitive and

simple to use, with lots of tools and frameworks supporting their usage. For many software developers, the CRUD model, with its directly mutable entities, is the first one they learn. Figure 4-4 shows a series of CRUD events (one create, two updates) applying changes to the refrigerator state. The data store only retains the updated state after processing any the opeations.

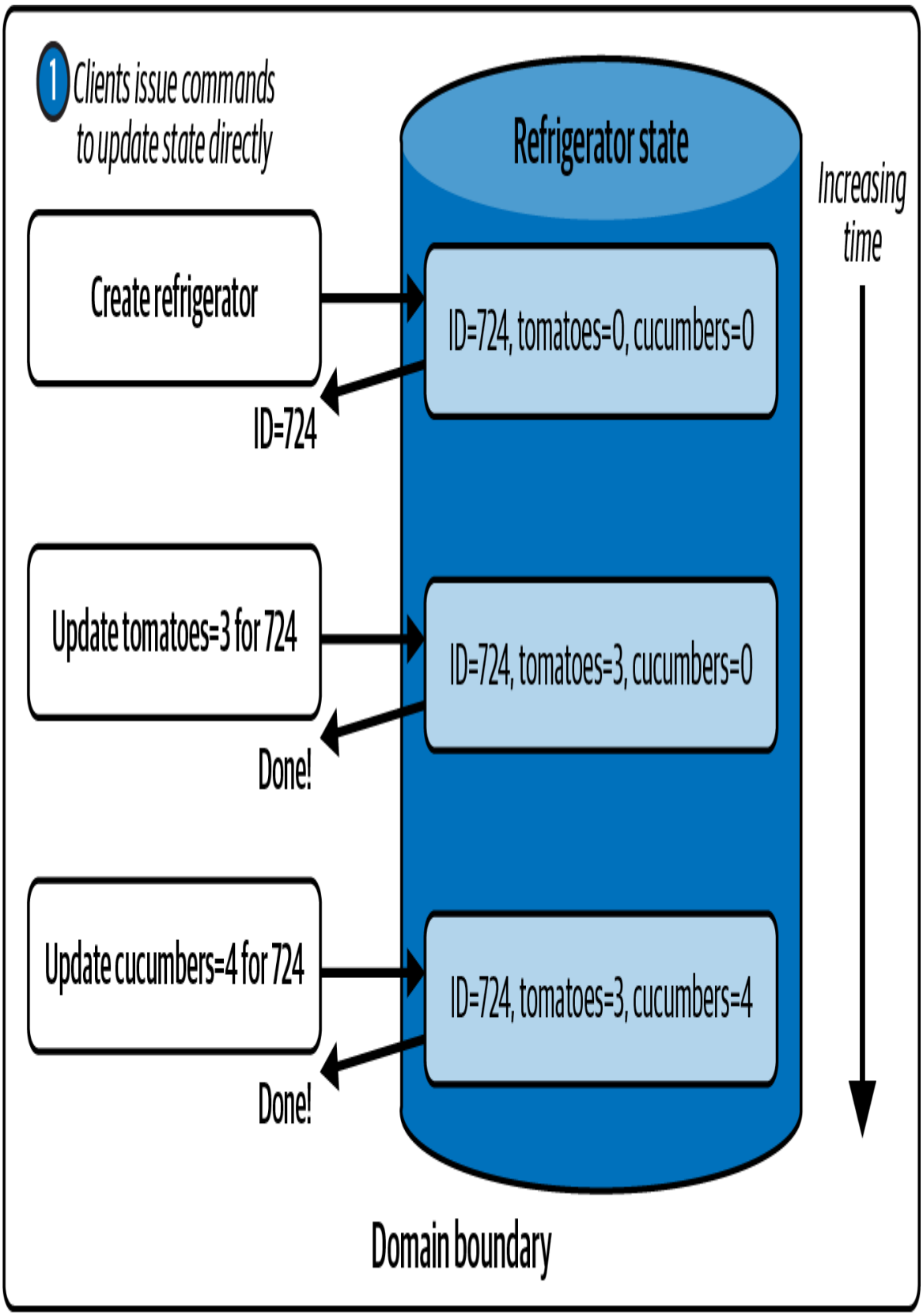

1
Clients issue commands to update state directly
Refrigerator state
Increasing time
Create refrigerator
ID=724, tomatoes=0, cucumbers=0
ID=724
Update tomatoes=3 for 724
ID=724, tomatoes=3, cucumbers=0
Done!
Update cucumbers=4 for 724
ID=724, tomatoes=3, cucumbers=4
Done!
Domain boundary

Figure 7-13. Using CRUD commands to update the contents of the refrigerator, reflected in the database

Under the event sourcing architecture, you instead model the operations as events, then append them to the log. There are many ways to build an event-sourcing append-only log. One option is to use a data store table, and simply append events as they occur. A second option, if you're willing to tolerate the round-trip latency, is to write it to your event broker. A third option is to use a dedicated event-sourcing data base or framework, as is covered later in [[Link to Come]].

Figure 4-5 shows the same refrigerator example, with the CRUD operations instead modeled as domain events. Although these events remain CRUD-like, it's important to note that the service owner can implement them however they like, using event types such as deltas, state, hybrid, measurements, and notifications. They can also add new event types, such as:

- `turn_lights_on/turn_lights_off`
- `turn_cooling_on/turn_cooling_off`
- `open_door/close_door`
- `make_ice`
- `self_check`

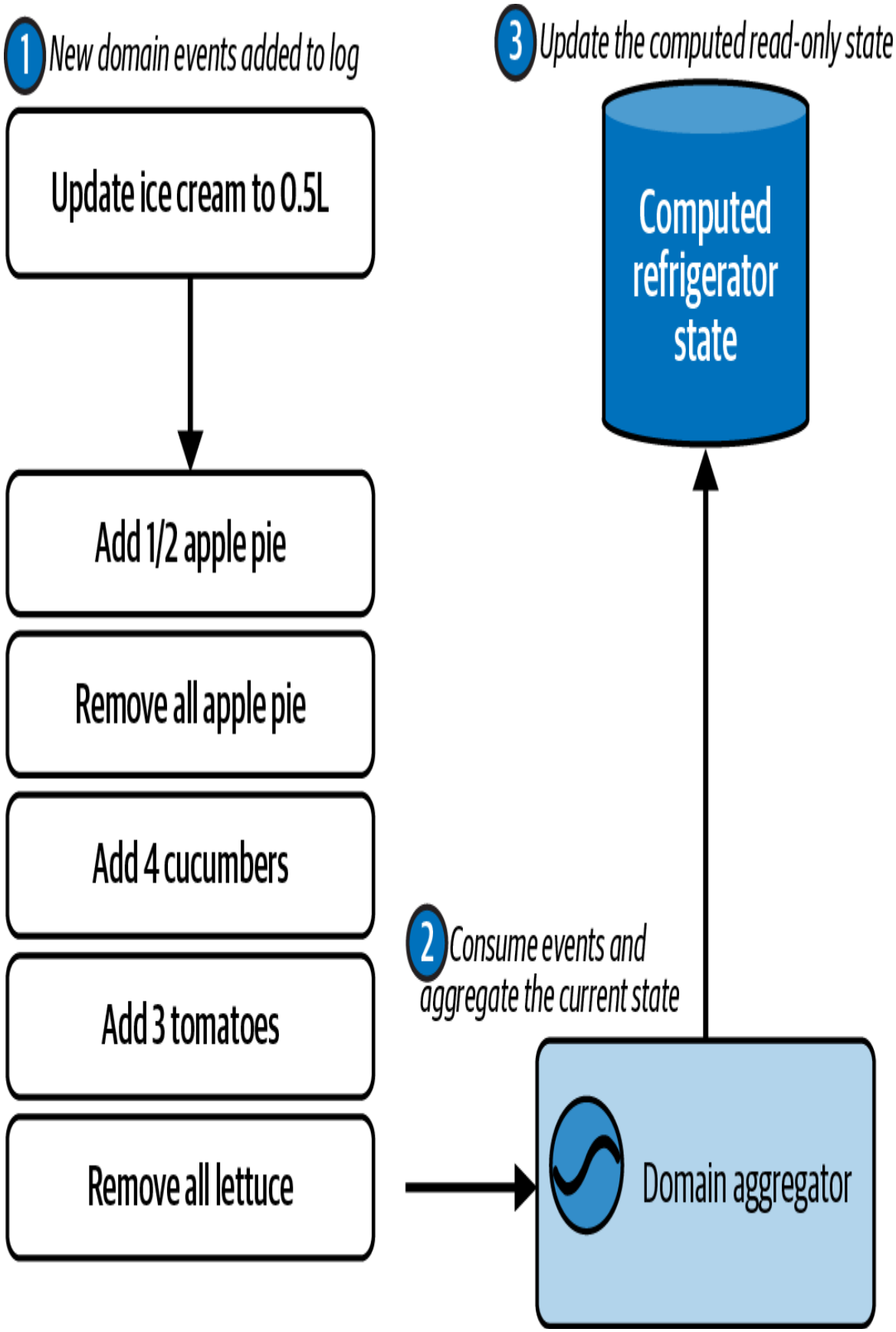

1 New domain events added to log
Update ice cream to 0.5L
Add 1/2 apple pie
Remove all apple pie
Add 4 cucumbers
Add 3 tomatoes
Remove all lettuce
2 Consume events and aggregate the current state
Domain aggregator
3 Update the computed read-only state
Computed refrigerator state

Figure 7-14. Building up the contents of a refrigerator using event sourcing

The domain aggregator (2), a formal term for *the business logic that builds up the state*, is independent from the process that writes the events to the log (1). It builds up the state by applying its business logic to the event stream, and stores the results inside its proprietary state store (3). This state store is read-only to every other component in the microservice, with the exception of the domain aggregator that builds it.

One of the main drawbacks of event sourcing is that it is eventually consistent, which may be problematic for for some use cases. There will always be some delay between writing the event to the log and seeing the materialized result in the state. And because multiple concurrent clients could write events about the same entity, it becomes difficult to attribute any specific modification in final state to the delta your client just appended. This can make it unsuitable in applications that require strong consistency.

So why would you use event sourcing? It's an excellent way to keep a log of all operations relevant to building up state. For example, financial laws may require you to keep a log of all operations performed on every account, including deposits, debits, and transfers. Event sourcing provides both the audit log and the operational log, ensuring that your systems and your logs match. It also provides your application the means to rebuild its state in the case of a failure or a bug in the code.

As an indication to the power and usability of event sourcing, popular databases like MySQL and PostgreSQL use their own proprietary append-only logs to build up their database state. In fact, these are the very same logs that you can attach your connectors to, as covered in "Liberating Data Using Change-Data Capture".

Event sourcing is an alternative to the CRUD model for building up internal state. Ensure that you keep the events private to your microservice, as the writing and reading logic is tightly coupled to the event structure. It is also important that you keep the events private to your application, and don't open them up for wider consumption. External coupling on the event sourcing log is akin to giving direct read access to an underlying data store, and any

changes made to the events could cause failures in the improperly coupled consumers.

[[ch-stateful-effectively once-processing]] === Transactions and Effectively Once Processing

Effectively once processing is the guarantee that any updates made to the single source of truth are consistently applied, regardless of any failure to the producer, the consumer, or the event broker. In particular, it means that you can commit your consumer offsets *and* your output event stream records in a single transaction. Transactional support is required to ensure effectively once processing.

Transactional support is offered by Apache Kafka and Apache Pulsar. Much like a relational database can support multi-table updates in a single transaction, an event broker implementation may also support the atomic writing of multiple events to multiple separate event streams.

Effectively once processing is also sometimes described as *exactly once processing*, though this is not quite accurate. An event-driven microservice may process the same data multiple times, say due to a consumer failure and subsequent recovery, but fail to commit its offsets and increment its stream progress. However, the service still executes the business logic for every event, *including any side effects* such as publishing data to external endpoints or communicating with a third-party service. It doesn't mean that your processing and side effects will only occur once for every event.

Idempotent writes are another commonly supported feature among event broker implementations such as Apache Kafka and Apache Pulsar. They allow for an event to be written once, and only once, to an event stream. They use a combination of unique process IDs and monotonically increasing sequences to fence out records produced more than once. In short, it'll let your service safely retry record production without accidentally adding duplicates to the output event stream.

Not all event brokers may support transactions or idempotent production. In some cases it won't matter, but in other cases you may need to build your own workaround.

Example: effectively once Processing for an Inventory Accounting Service

The inventory accounting service is responsible for issuing an event when the inventory of any given item is low. The microservice must piece together the current inventory available for each product based on a chain of additions and subtractions made over time. Selling items to customers, losing items to damage, and losing items to theft are all events that reduce inventory. Receiving shipments, accepting customer returns, and finding lost inventory in the warehouse increase it. These events are shown in the same event stream for simplicity in this example, as illustrated in Figure 7-15.

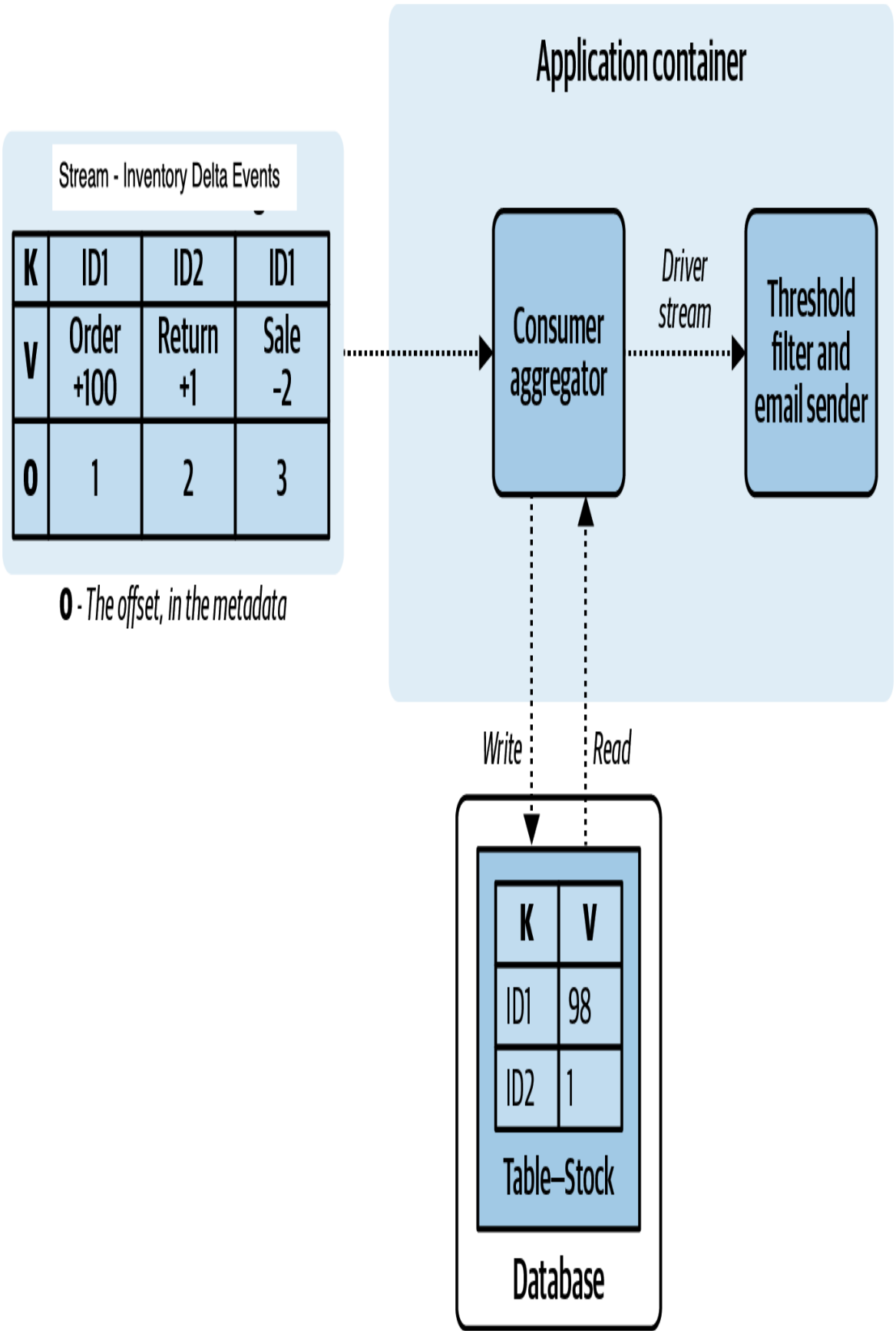

Application container
Stream - Inventory Delta Events
K
ID1
ID2
ID1
V
Order +100
Return +1
Sale -2
O
1
2
3
O - The offset, in the metadata
Consumer aggregator
Driver stream
Threshold filter and email sender
Write
Read
K
V
ID1
98
ID2
1
Table–Stock
Database

Figure 7-15. A simple inventory accounting service

This inventory accounting service is quite simple. It calculates the current running total based off of the events and stores it in its data store. The business logic filters on a threshold value to decide when to issue a purchase event that requests the ordering of more inventory. To get an accurate count, you must ensure that each input event is applied effectively once to the aggregated state. At-least-once and at-most-once processing will give incorrect results.

This service benefits greatly from effectively once semantics as it is entirely event-driven. It doesn't need to issue any commands or requests to external systems, and can ensure that its state and offsets are atomically committed (provided you broker supports transactions - see next section).

Effectively Once Processing with Client-Broker Transactions

Effectively once processing can be facilitated by any event broker that supports transactions. Transactions ensure that output events, changelog events, and consumer offset output events, are committed atomically to the event broker, as shown in Figure 7-16.

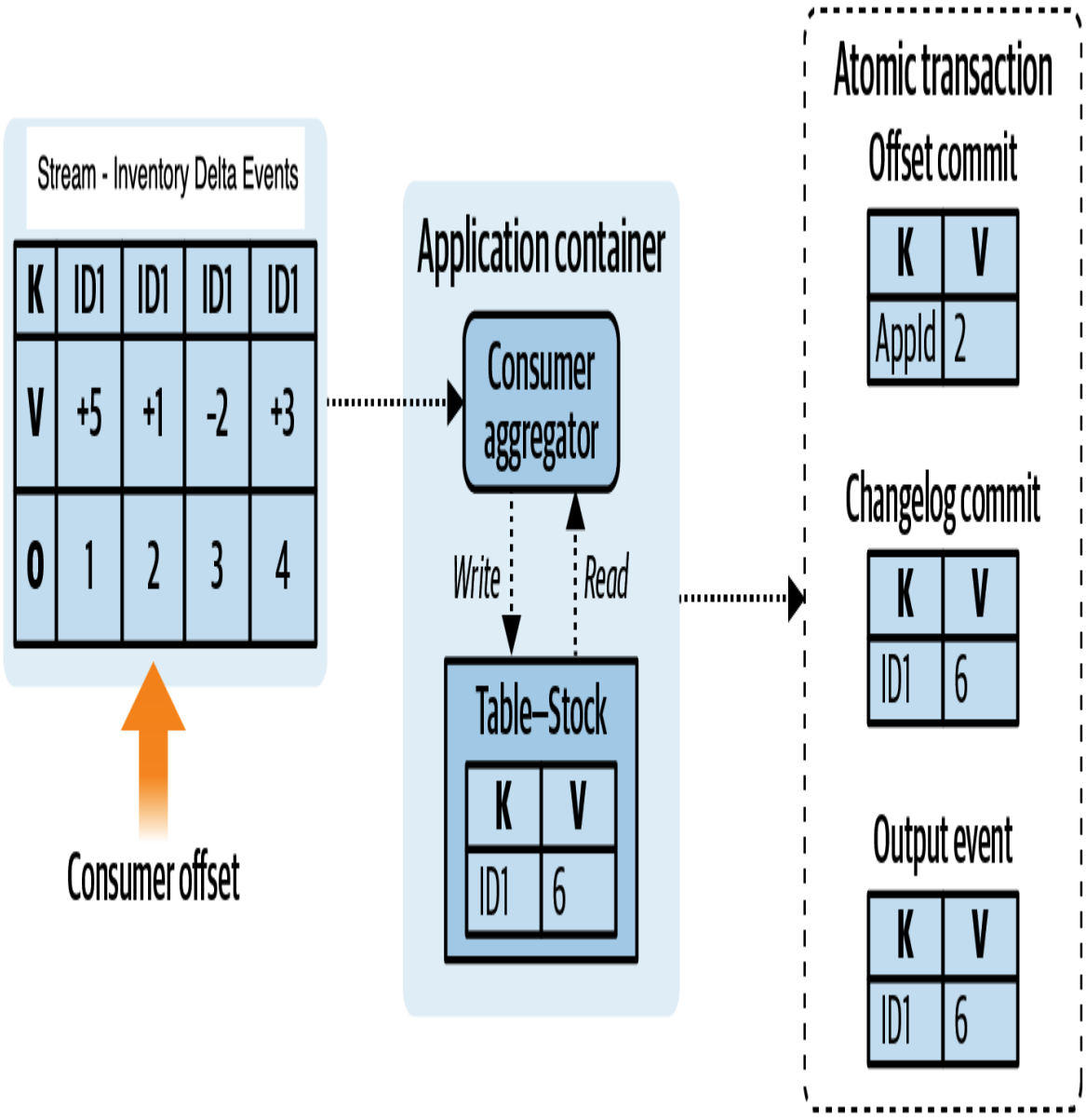


Figure 7-16. Client-broker transactions—committing offsets and changelogs

The atomic transaction between the producer client and the event broker will publish all events to their corresponding event streams. If the failure persists longer than the retry timeouts, as in Figure 7-17, then the broker will abort the transaction, and no events will be committed. In the case of transient errors, the producer can simply retry committing its transaction, as it is an idempotent operation.

Event stream consumers typically abstain from processing events that are in uncommitted transactions. It's a bit academic really, as there are vanishingly

few scenarios where you want the guarantees of a transaction, but don't care if the transaction is actually committed! The consumer, however, must respect offset order, and so it'll wait for the pending transactions to commit or abort.

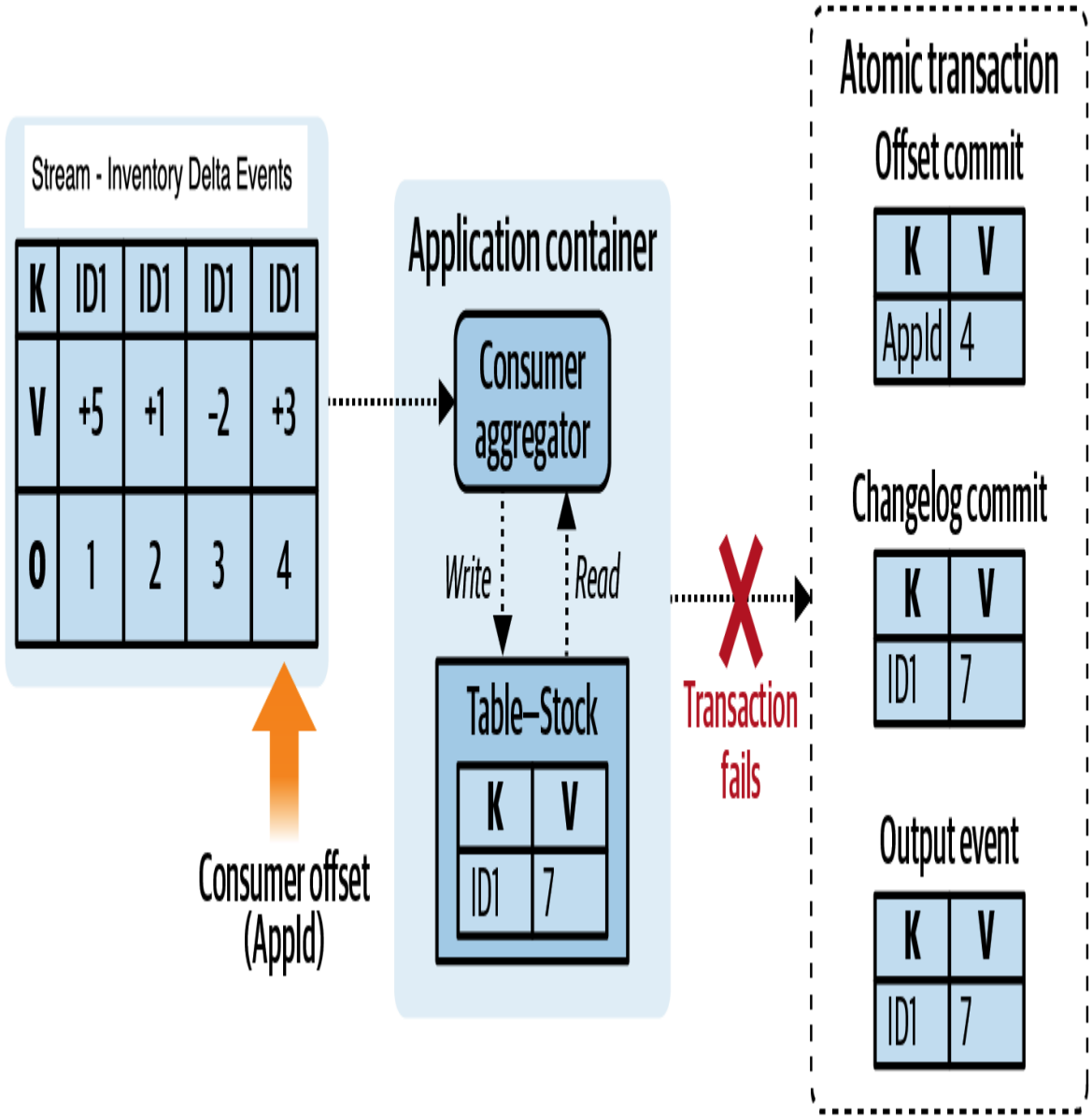


Figure 7-17. Failed commit for a client-broker transaction

In the case that the producer suffers a fatal exception during a transaction, its replacement instance can simply be rebuilt by restoring from the changelogs as shown in Figure 7-18. The consumer group offsets of the input event streams are also reset according to the last known good position stored in the offset event stream.

New transactions can begin once the producer is recovered, and all previous incomplete transactions are failed and cleaned up by the event broker. The transactional mechanisms will vary to some extent depending on the broker implementation, so make sure to familiarize yourself with the one you are using.

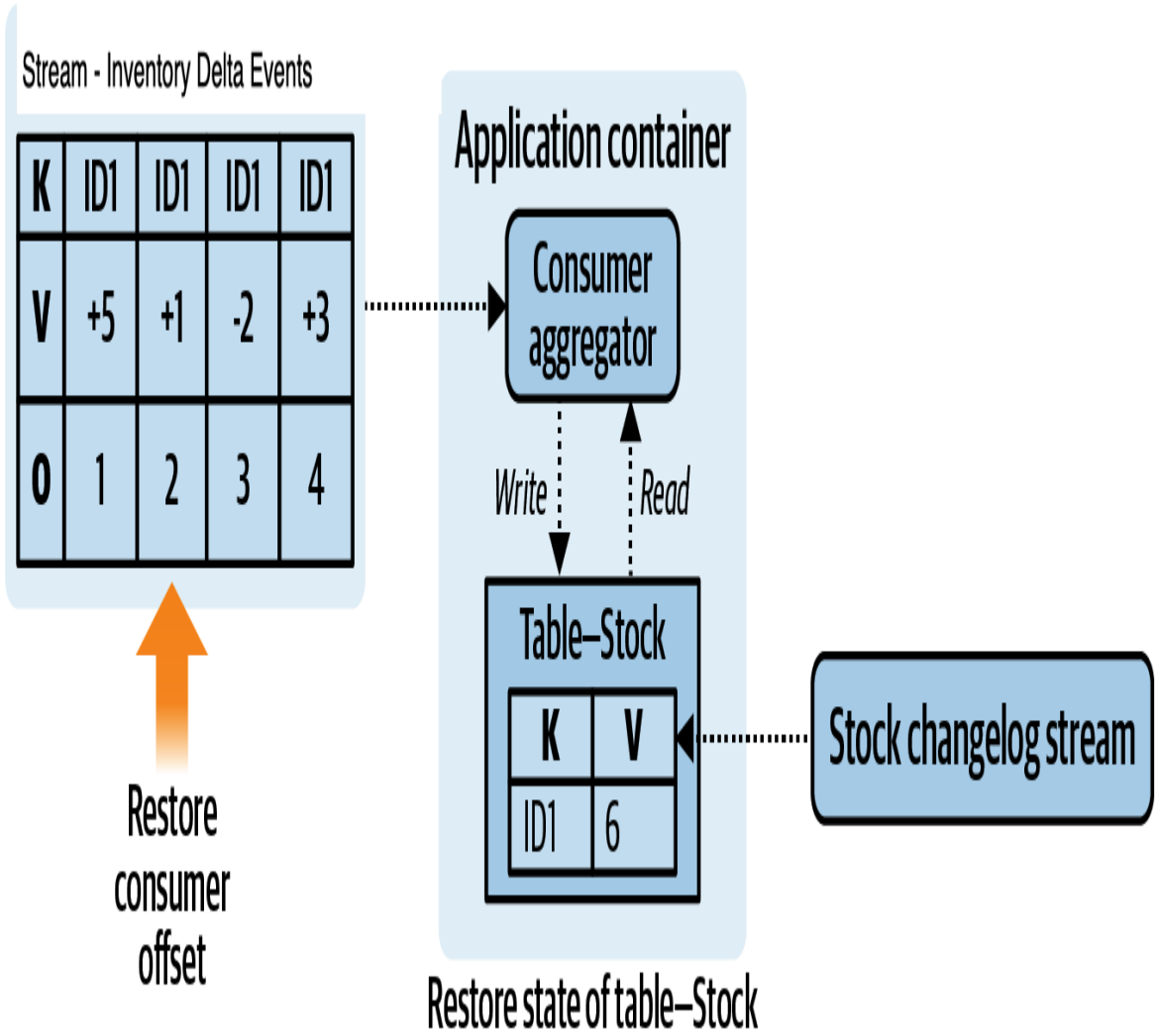


Figure 7-18. Restoring the state from the broker using changelogs and previous offsets

Effectively Once Processing Without Client-Broker Transactions

Effectively once processing of events is also possible for implementations that do not support client-broker transactions, though it requires more work and a careful consideration of duplicate events. First, if upstream services

are not able to provide effectively once event production guarantees, then it is possible that they may produce duplicate records. Any duplicate events created by upstream processes need to be identified and filtered out.

Second, state and offset management need to be updated in a *local transaction* to ensure that the event processing is applied only once to the system state. By following this strategy, clients can be assured that the internal state generated by their processor is consistent with the logical narrative of the input event streams. Let's take a look at these steps in more detail.

> **WARNING**
>
> It is better to use an event broker and client that support idempotent writes than it is to try to solve deduplication after the fact. The former method scales well to all consumer applications, whereas the latter is expensive and difficult to scale as each consumer must implement safeguards.

Generating duplicate events

Duplicate events are generated when a producer successfully writes the events to an event stream, but either fails to receive a write acknowledgment and retries, or it crashes before updating its own consumer offsets. These scenarios are slightly different:

Producer fails to receive acknowledgment from broker and retries

> In this scenario, the producer still has the copies of the events to produce in its memory. These events, if published again, may have the same timestamps (if they use event creation time) and same event data, but will be assigned new offsets. Idempotent production between a client and broker normally filter these out.

Producer crashes immediately after writing, before updating its own consumer offsets

In this case, the producer will have successfully written its events, but will *not* have updated its consumer offsets yet. This means that when the producer comes back up, it will repeat the work that it had previously done, creating logically identical copies of the events but with new timestamps. If processing is deterministic, then the events will have the same data. New offsets will also be assigned, making it difficult to tell that they are in fact duplicates.

> **NOTE**
>
> *Idempotent production* can mitigate failures due to crashes and retries between the client and the broker. It cannot mitigate duplicates introduced through faulty business logic.

Identifying duplicate events

If idempotent production of events is *not* available and there are duplicates (with unique offsets and unique timestamps) in the event stream, then it is up to you to mitigate their impact. First, determine if the duplicates actually cause any problems. In many cases duplicates have a minor, if not negligible, effect and can simply be ignored.

For those scenarios where duplicate events *do* cause problems, you will need to figure out how to identify them. One way to do this is to have the producer generate a unique ID for each event, such that any duplicates will generate the same unique hash.

This hash function is often based on the properties of the internal event data, including the key, values, and the event time of the source events (or the query time of the request-response query that triggered it). This approach tends to work well for events that have a large data domain, but poorly for events that are logically equivalent to one another. Here are a few scenarios where you could feasibly generate a unique ID:

- A bank account transfer, detailing the source, destination, amount, date, and time

- An ecommerce order, detailing each product, the purchaser, date, time, total amount, and payment provider
- Inventory debited for shipment purposes, where each event has an associated `orderId` (uses an existing unique data ID)

One factor these examples have in common is that each ID is composed of elements with a very high cardinality (that is, uniqueness). This significantly reduces the chances of duplicates between the IDs. The deduplication ID (dedupe ID) can either be generated with the event or be generated by the consumer upon consumption. Your best option is to generate the ID with the event, as all consumers can benefit from it and you can freely change the logic that generates it.

> **WARNING**
>
> Guarding against duplicate events produced without a key is extremely challenging, as there is no guarantee of partition locality. Produce events with a key, respect partition locality, and use idempotent writes whenever possible.

Guarding against duplicates

Any effectively once consumer must either identify and discard duplicates, perform idempotent operations, or consume from event streams that have idempotent producers. Idempotent operations are not possible for all business cases, and without idempotent production you must find a way to guard your business logic against duplicate events. This can be an expensive endeavor, as it requires that each consumer maintain a state store of previously processed dedupe IDs. The store can grow very large depending on the volume of events and the offset or time range that the application must guard against.

Perfect deduplication requires that each consumer indefinitely maintain a lookup of each dedupe ID already processed, but time and space requirements can become prohibitively expensive if an attempt is made to guard against too large a range. In practice, deduplication is generally only

performed for a specific rolling time-window or offset-window as a best-effort attempt.

> **TIP**
>
> Keep deduplication stores small by using time-to-live (TTL), a maximum cache size, and periodic deletions. The specific settings needed will vary depending on the sensitivity of your application to duplicates and the impact of duplicates occurring.

Deduplication should be attempted only within a single event stream partition, as deduplication between partitions will be prohibitively expensive. Keyed events have an added benefit over unkeyed events, since they consistently map to the same partition.

Figure 7-19 shows a deduplication store in action. In this figure you can see the workflow that an event goes through before being passed off to the actual business logic. In this example the TTL is arbitrarily set to 8,000 seconds, but in practice would need to be established based on business requirements.

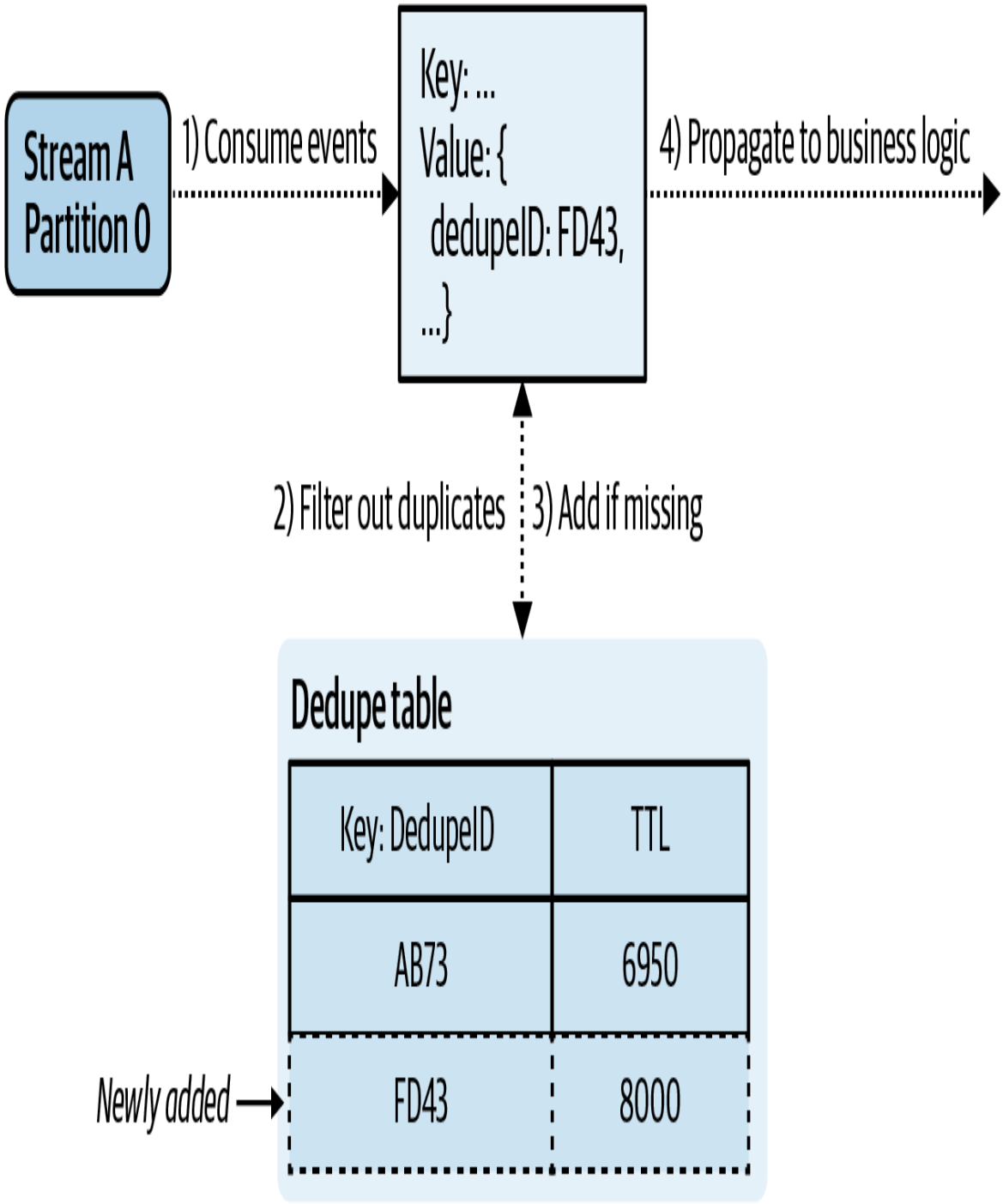


Figure 7-19. Deduplication using persisted state

> **TIP**
>
> A maximum cache size is used in the deduplication store to limit the number of events maintained, particularly during reprocessing.

Note that you are also responsible for maintaining durable backups of the deduplication table, the same as any other materialized table. In the case of a failure, the dedupe table must be rebuilt prior to resuming the processing of new events.

Maintaining consistent state via state store transactions

A microservice can leverage the transactional capabilities of its own state store instead of the event broker to perform effectively once processing. However, this requires moving the consumer group's input offset management from the event broker into the data store. Then, a single transaction can atomically update both the microservice's computed state *and* the consumer group input offsets. Any changes made to the state coincide completely with those made to the consumer offsets, maintaining consistency.

In the case of a service failure, such as a timeout when committing to the data store, then microservice can simply abandon the transaction and revert to the last known good state. All consumption is halted until the data service is back online, at which point consumption is restored from the last known good offset.

By keeping the official record of offsets synchronized with the state in the data store, you have a consistent record of state that the service can recover from. This process is illustrated in Figures 7-20, 7-21, and 7-22.

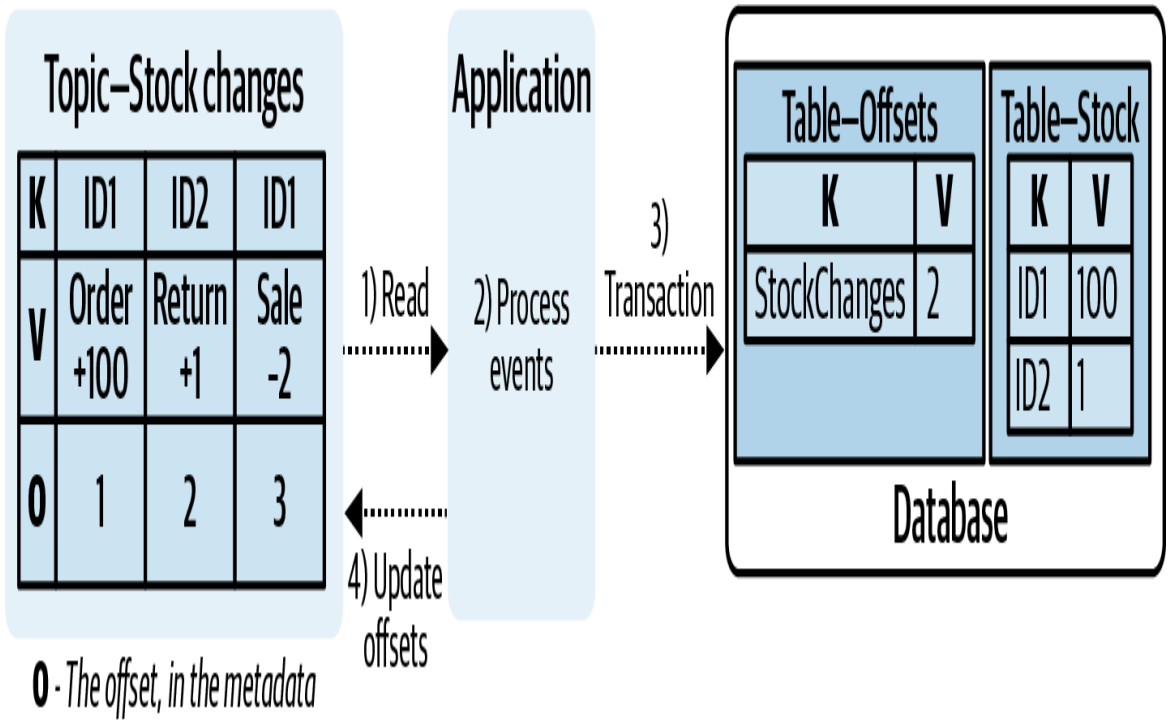


Figure 7-20. Normal transactional processing of events

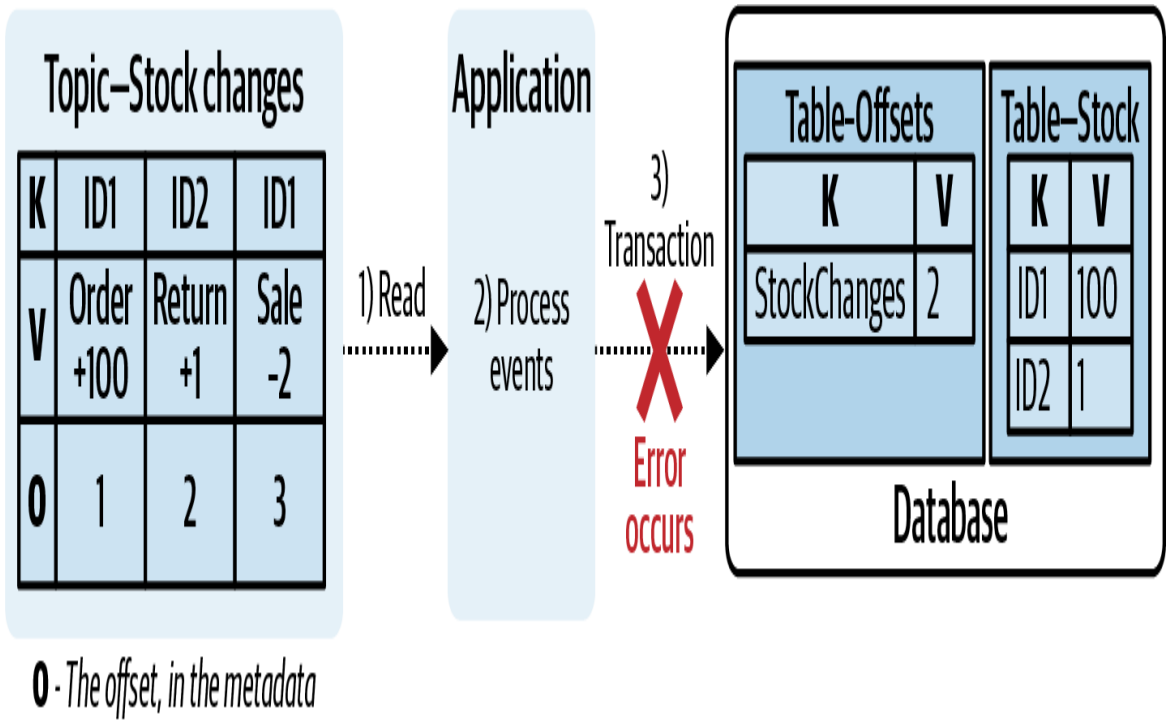


Figure 7-21. Failure occurs in transactional processing

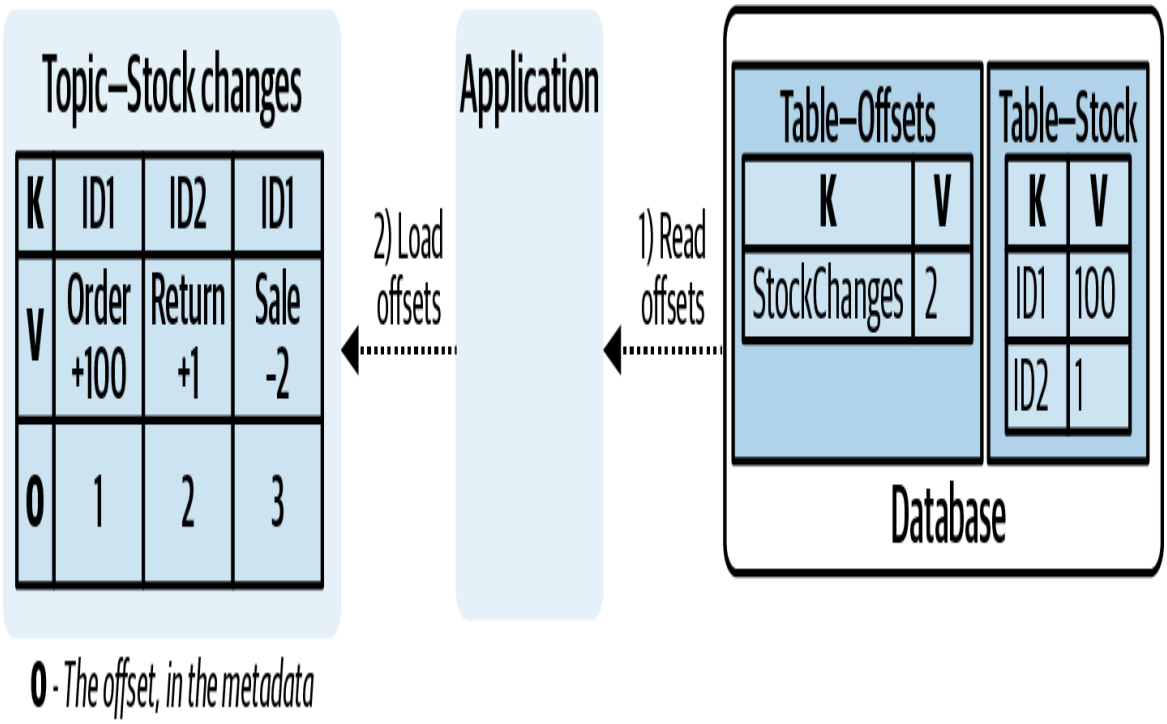


Figure 7-22. Recovery of offsets during state restoration process

Note that this approach gives your processor effectively once *processing*, but not effectively once event *production*. Any events produced by this service are subject to the limitations of at-least-once production. After all, in this scenario, there is no atomic transactions between the producer and the broker.

Summary

This chapter covered at internal and external state stores, how they work, their advantages, their disadvantages, and when to use them. Internal state stores can support high-performance processing, while external state stores can provide a range of flexible options for supporting the business needs of your microservices. Latency, throughput, cost, scaling, and data access patterns are each significant factors for determining whether an internal or external data store is best for your microservice.

Changelogs play an important role in the backup and restoration of microservice state stores, though they remain native only to a narrow selection of event-driven frameworks. Alternately, you could choose to rely

on transaction-supporting databases with regularly scheduled snapshots as your data store of choice.

Event brokers that support transactions can enable extremely powerful effectively once processing, offloading the responsibility of duplication prevention from the consumer, while deduplication efforts can enable effectively once processing in systems without such support.

Chapter 8. Deterministic Stream Processing

> **A NOTE FOR EARLY RELEASE READERS**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 9th chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

Event-driven microservices typically have topologies that are more complex than those introduced in the previous chapter. They consume events from multiple partitioned event streams, maintain state, repartition data, and emit new events to other streams. They are also subject to all the same failure modes and faults as any other application, which can result in out-of-order events, late events, and questions around how events will be processed in actuality.

Here are the three main questions addressed in this chapter:

- How does a microservice choose the order of events to process when consuming from multiple partitions?
- How does a microservice handle out-of-order and late-arriving events?
- How do we ensure that our microservices produce deterministic results when processing streams in near–real time versus when

processing from the beginning of the streams?

We can answer these questions by examining timestamps, event scheduling, watermarks, and stream times, and how they contribute to deterministic processing. Bugs, errors, and changes in business logic will also necessitate reprocessing, making deterministic results important. This chapter also explores how out-of-order and late-arriving events can occur, strategies for handling them, and mitigating their impact on our workflows.

> **NOTE**
>
> This chapter is fairly information-dense despite my best efforts to find a simple and concise way to explain the key concepts. There are a number of sections where I will refer you to further resources to explore on your own, as the details often go beyond the scope of this book.

Determinism with Event-Driven Workflows

An event-driven microservice has two main processing states. It may be processing events at near–real time, which is typical of long-running microservices. Alternately, it may be processing events from the past in an effort to catch up to the present time, which is common for underscaled and new services.

If you were to rewind the consumer group offsets of the input event streams to the beginning of time and start the microservice run again, would it generate the same output as the first time it was run? The overarching goal of deterministic processing is that a microservice should produce the same output whether it is processing in real time or catching up to the present time.

Note that there are workflows that are explicitly nondeterministic, such as those based on the current wall-clock time and those that query external services. External services may provide different results depending on when they are queried, especially if their internal state is updated independently of that from the services issuing the query. In these cases there is no promise of

determinism, so be sure to pay attention to any nondeterministic operations in your workflow.

Fully deterministic processing is the ideal case, where every event arrives on time and there is no latency, no producer or consumer failures, and no intermittent network issues. Since we have no choice but to deal with these scenarios, the reality is that our services can only achieve a best effort at determinism. There are a number of components and processes that work together to facilitate this attempt, and in most cases best-effort determinism will be sufficient for your requirements. There are a few things you need to achieve this: consistent timestamps, well-selected event keys, partition assignment, event scheduling, and strategies to handle late-arriving events.

Timestamps

Events can happen anywhere and at any time and often need to be reconciled with events from other producers. Synchronized and consistent timestamps are a hard requirement for comparing events across distributed systems.

An event stored in an event stream has both an offset and a timestamp. The offset is used by the consumer to determine which events it has already read, while the timestamp, which indicates when that event was created, is used to determine when an event occurred relative to other events and to ensure that events are processed in the correct order.

The following timestamp-related concepts are illustrated in Figure 8-1, which shows their temporal positions in the event-driven workflow:

Event time

> The local timestamp assigned to the event by the producer at the time the event occurred.

Broker ingestion time

> The timestamp assigned to the event by the event broker. You can configure this to be either the event time or the ingestion time, with the former being much more common.

In scenarios where the producer's event time is unreliable, broker-ingestion time can provide a sufficient substitute.

Consumer ingestion time

The time in which the event is ingested by the consumer. This can be set to the event time specified in the broker record, or it can be the wall-clock time.

Processing time

The wall-clock time at which the event has been processed by the consumer.

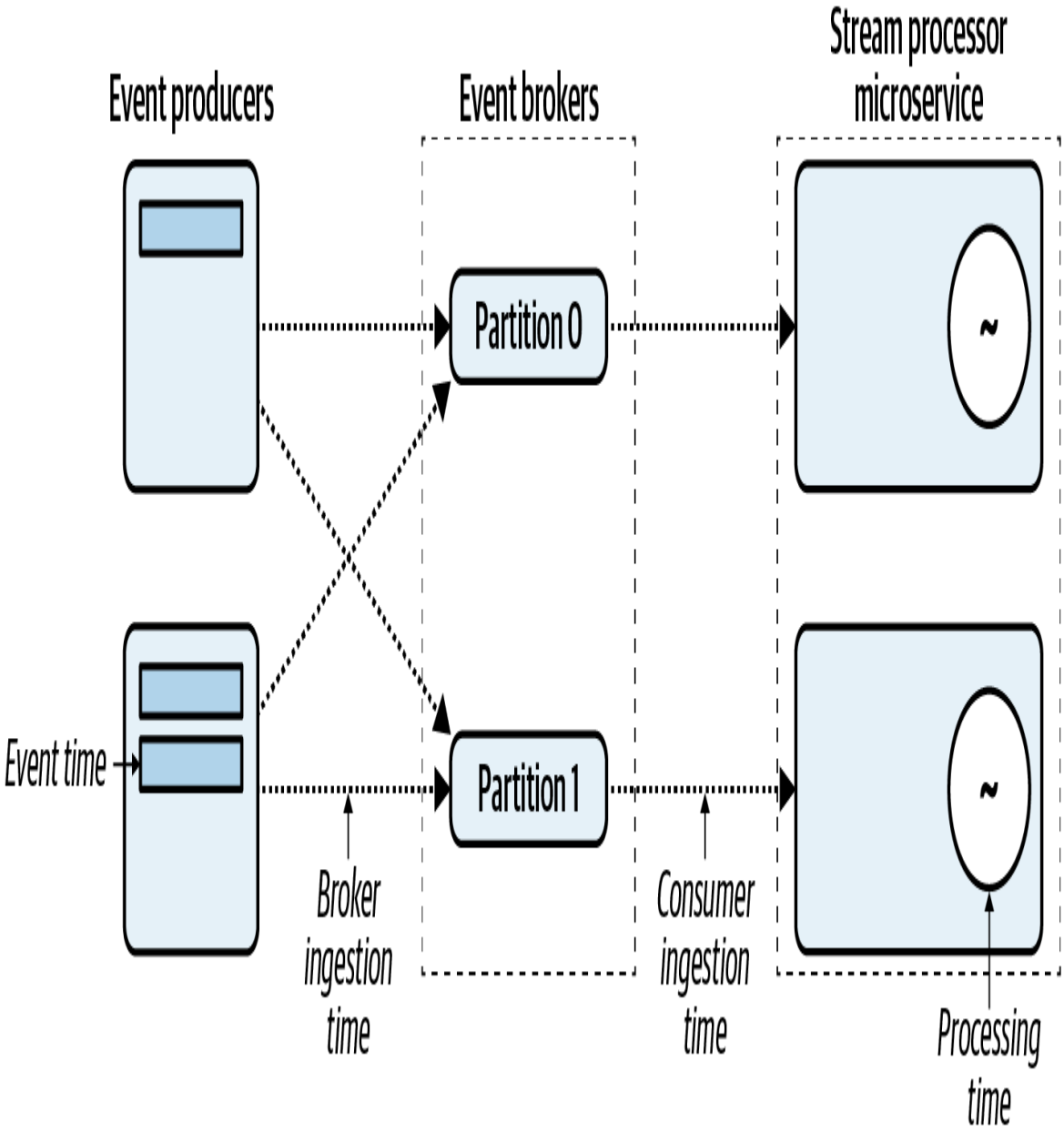


Figure 8-1. Event scheduler ordering the input events by timestamp

You can see that it's possible to propagate the event time through the event broker to the consumer, enabling the consumer logic to make decisions based on *when* an event happened. This will help answer the three questions posed at the start of the chapter. Now that we've mapped out the types of timestamps, let's take a look at how they're generated.

Synchronizing Distributed Timestamps

A fundamental limitation of physics is that two independent systems cannot be guaranteed to have precisely the same system-clock time. Various physical properties limit how precise system clocks can be, such as material tolerances in the underlying clock circuitry, variations in the operating temperature of the chip, and inconsistent network communication delays during synchronization. However, it is possible to establish local system clocks that are *nearly* in sync and end up being good enough for most computing purposes.

Consistent clock times are primarily accomplished by synchronizing with Network Time Protocol (NTP) servers. Cloud service providers such as Amazon and Google offer redundant satellite-connected and atomic clocks in their various regions for instant synchronization.

Synchronization with NTP servers within a local area network can provide very accurate local system clocks, with a drift of only a few mS after 15 minutes. This can be reduced to 1 mS or less with more frequent synchronizations in best-case scenarios according to David Mills, NTP's inventor, though intermittent network issues may prevent this target from being reached in practice. Synchronization across the open internet can result in much larger skews, with accuracy being reduced to ranges of +/− 100mS, and is a factor to be considered if you're trying to resynchronize events from different areas of the globe.

NTP synchronization is also prone to failure, as network outages, misconfiguration, and transient issues may prevent instances from synchronizing. The NTP servers themselves may also otherwise become unreliable or unresponsive. The clock within an instance may be affected by multitenancy issues, just as in VM-based systems sharing the underlying hardware.

For the vast majority of business cases, frequent synchronization to NTP servers can provide sufficient consistency for system event time. Improvements to NTP servers and GPS usage have begun to push NTP synchronization accuracy consistently into the submillisecond range. The creation time and ingestion time values assigned as the timestamps can be

highly consistent, though minor out-of-order issues will still occur. Handling of late events is covered later in this chapter.

Timestamps provide a way to process events distributed across multiple event streams and partitions in a consistent temporal order. Many use cases require you to maintain order between events based on time, and need consistent, reproducible results regardless of when the event stream is processed. Using offsets as a means of comparison works only for events within a single event stream partition, while events quite commonly need to be processed from multiple different event streams.

For example, a bank must ensure that both deposit and withdrawal event streams are processed in the correct temporal order. It keeps a stateful running tally of withdrawals and deposits, applying an overdraft penalty when a client's account balance drops below $0. For this example, the bank has its deposits in one event stream and its withdrawals in another stream, as shown in Figure 8-2.

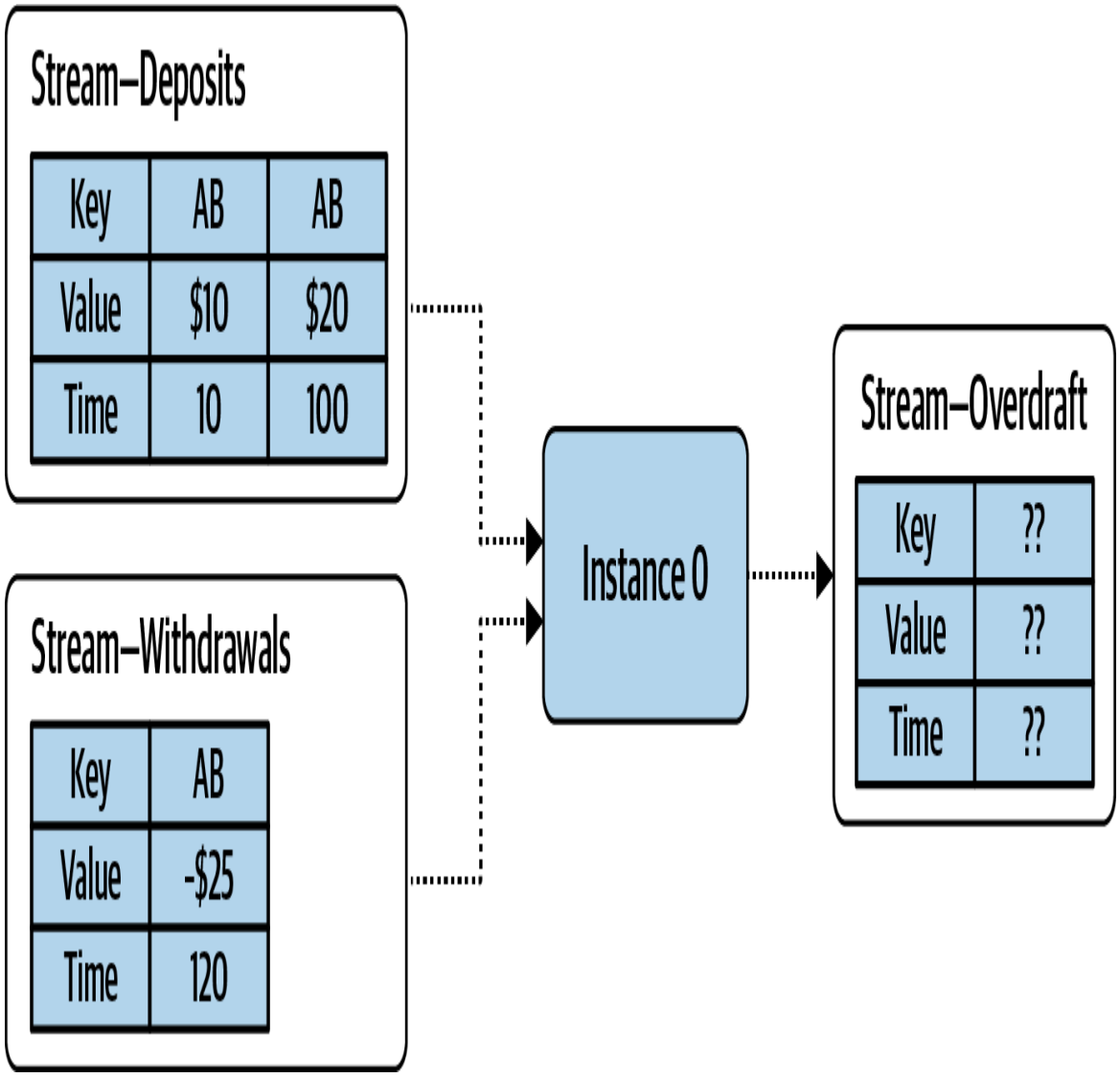


Figure 8-2. In which order should events be processed?

A naive approach to consuming and processing records, perhaps a round-robin processor, might process the $10 deposit first, the $25 withdrawal second (incurring a negative balance and overdraft penalties), and the $20 deposit third. This is incorrect, however, and does not represent the temporal order in which the events occurred. This example makes clear that you must consider the event's timestamp when consuming and processing events. The next section discusses this in greater detail.

Event Scheduling and Deterministic Processing

Deterministic processing requires that events be processed consistently, such that the results can be reproduced at a later date. Event scheduling is the process of selecting the next events to process when consuming from multiple input partitions. For an immutable log-based event stream, records are consumed in an offset-based order. However, as Figure 8-2 demonstrates, the processing order of events must be interleaved based on the *event time* provided in the record, regardless of which input partition it comes from, to ensure correct results.

> **TIP**
>
> The most common event-scheduling implementation selects and dispatches the event with the oldest timestamp from all assigned input partitions to the downstream processing topology.

Event scheduling is a feature of many stream-processing frameworks, but is typically absent from basic consumer implementations. You will need to determine if it is required for your microservice implementation.

> **WARNING**
>
> Your microservice will need event scheduling if the order in which events are consumed and processed matters to the business logic.

Custom Event Schedulers

Some streaming frameworks allow you to implement custom event schedulers. For example, Apache Samza lets you implement a `MessageChooser` class, where you select which event to process based on a number of factors, such as prioritization of certain event streams over others, the wall-clock time, event time, event metadata, and even content within the

event itself. You should take care when implementing your own event scheduler, however, as many custom schedulers are nondeterministic in nature and won't be able to generate reproducible results if reprocessing is required.

Processing Based on Event Time, Processing Time, and Ingestion Time

A time-based order of event processing requires you to select *which* point in time to use as the event's timestamp, as per Figure 8-1. The choice is between the locally assigned event time and broker ingestion time. Both timestamps occur only once each in a produce-consume workflow, whereas the wall-clock and consumer ingestion times vary depending on when the application is executed.

In most scenarios, particularly when all consumers and all producers are healthy and there is no event backlog for any consumer group, all four points in time will be within a few seconds of each other. Contrarily, for a microservice processing historic events, event time and consumer ingestion time will differ significantly.

For the most accurate depiction events in the real world, it is best to use the locally assigned event time *provided you can rely on its accuracy*. If the producer has unreliable timestamps (and you can't fix it), your next best bet is to set the timestamps based on when the events are ingested into the event broker. It is only in rare cases where the event broker and the producer cannot communicate that there may be a substantial delay between the true event time and the one assigned by the broker.

Timestamp Extraction by the Consumer

The consumer must know the timestamp of the record before it can decide how to order it for processing. At consumer ingestion time, a *timestamp extractor* is used to extract the timestamp from the consumed event. This extractor can take information from any part of the event's payload, including the key, value, and metadata.

Each consumed record has a designated event-time timestamp that is set by this extractor. Once this timestamp has been set, it is used by the consumer framework for the duration of its processing.

Request-Response Calls to External Systems

Any non-event-driven requests made to external systems from within an event-driven topology may introduce nondeterministic results. By definition, external systems are managed externally to the microservice, meaning that at any point in time their internal state and their responses to the requesting microservice may differ. Whether this is significant depends entirely on the business requirements of your microservice and is up to you to assess.

Watermarks

Watermarking is used to track the progress of event time through a processing topology and to declare that all data of a given event time (or earlier) has been processed. This is a common technique used by many of the leading stream-processing frameworks, such as Apache Spark, Apache Flink, Apache Samza, and Apache Beam. A whitepaper from Google describes watermarks in greater detail and provides a good starting point for anyone who would like to learn more about it.

A watermark is a declaration to downstream nodes *within the same processing topology* that all events of time *t* and prior have been processed. The node receiving the watermark can then update its own internal event time and propagate its own watermark downstream to its dependent topology nodes. This process is shown in Figure 8-3.

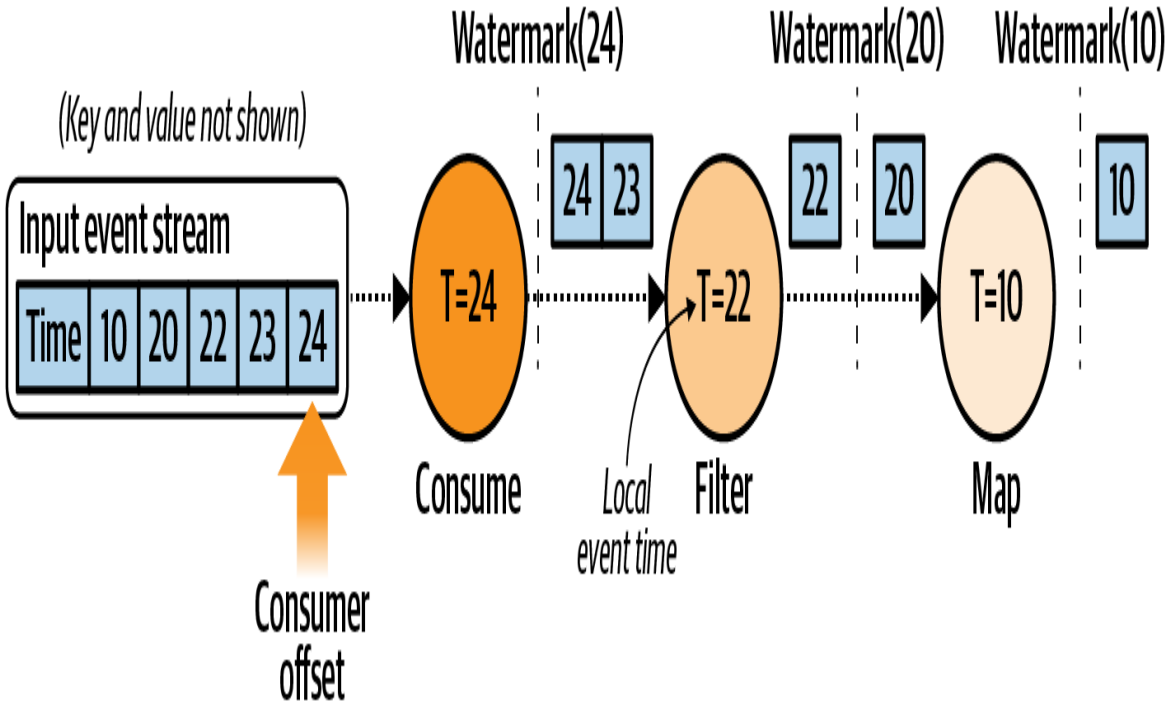


Figure 8-3. Watermark propagation between nodes in a single topology

In this figure, the consumer node has the highest watermark time because it's consuming from the source event stream. New watermarks are generated periodically, such as after a period of wall-clock or event time has elapsed or after some minimum number of events has been processed. These watermarks propagate downstream to the other processing nodes in the topology, which update their own event time accordingly.

> **NOTE**
>
> This chapter only touches on watermarks to give you an understanding of how they're used for deterministic processing. If you would like to dig deeper into watermarks, consider Chapters 2 and 3 of the excellent book *Streaming Systems*, by Tyler Akidau, Slava Chernyak, and Reuven Lax (O'Reilly, 2018).

Watermarks in Parallel Processing

Watermarks are particularly useful for coordinating event time between multiple independent consumer instances. Figure 8-4 shows a simple processing topology of two consumer instances. Each consumer instance

consumes events from its own assigned partition, applies a `groupByKey` function, followed by an `aggregate` function. This requires a *shuffle*, where all events with the same key are sent to a single downstream aggregate instance. In this case, events from instance 0 and instance 1 are sent to each other based on the key to ensure all events of the same key are in the same partition.

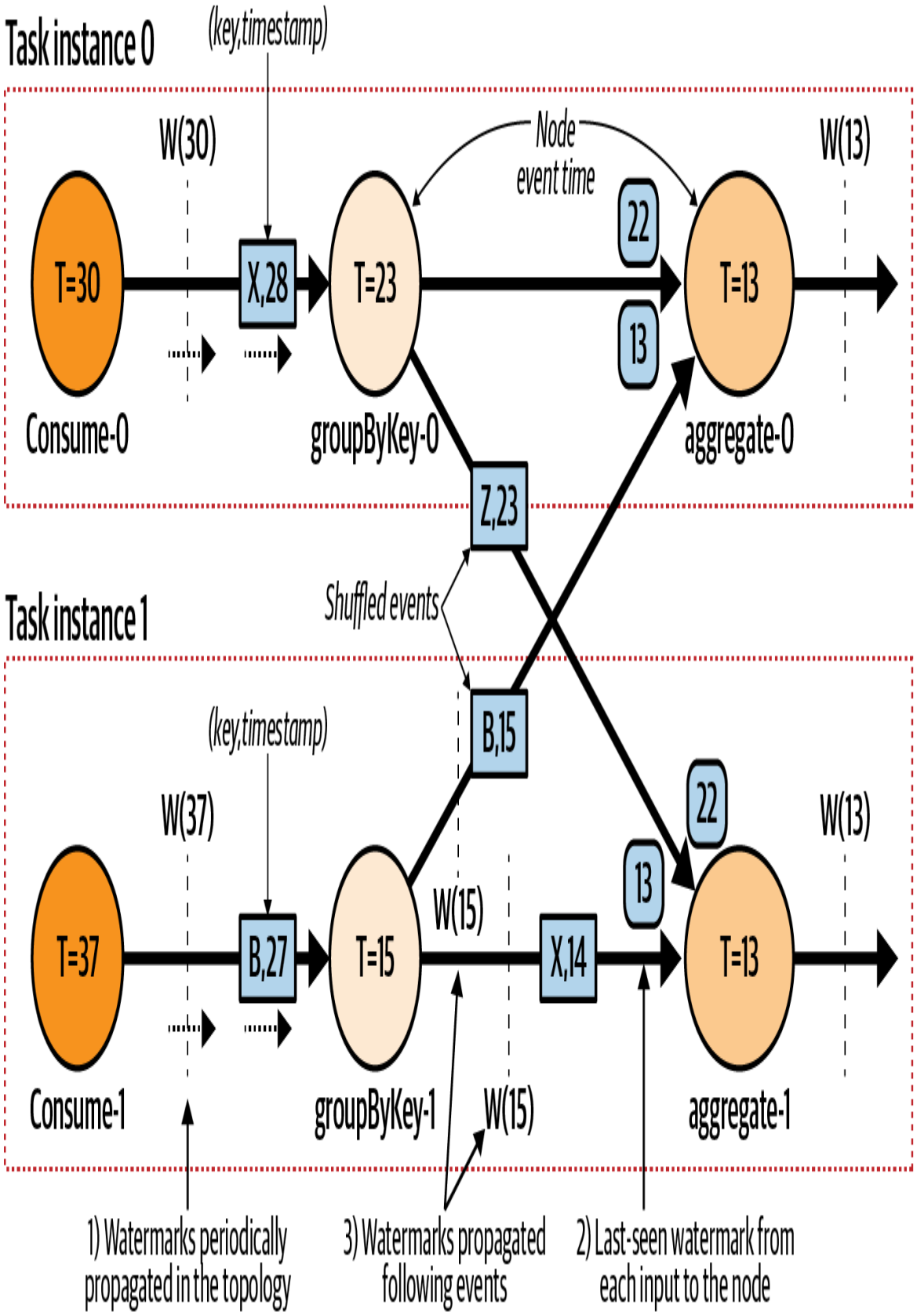


Figure 8-4. Watermark propagation between nodes in a single topology with multiple processors

There is a fair bit to unpack in this diagram, so let's take a look at it from the start.

Watermarks are generated at the source function, where the events are consumed from the event stream partition. The watermarks define the event time at that consumer and are propagated downstream as the event time of the consumer node is incremented (#1 in Figure 8-4).

Downstream nodes update their event time as the watermarks arrive, and in turn generate their own new watermark to propagate downstream to its successors. Nodes with multiple inputs, such as `aggregate`, consume events and watermarks from multiple upstream inputs. The node's event time is the *minimum* of all of its input sources' event times, which the node keeps track of internally (#2 in Figure 8-4).

In the example, both `aggregate` nodes will have their event time updated from 13 to 15 once the watermark from the groupByKey-1 node arrives (#3 in Figure 8-4). Note that the watermark does not affect the event scheduling of the node; it simply notifies the node that it should consider any events with a timestamp earlier than the watermark to be considered late. Handling late events is covered later in this chapter.

Spark, Flink, and Beam, among other heavyweight processing frameworks, require a dedicated cluster of processing resources to perform stream processing at scale. This is particularly relevant because this cluster also provides the means for cross-task communications and centralized coordination of each processing task. Repartitioning events, such as with the `groupByKey` + `aggregate` operation in this example, use cluster-internal communications and *not* event streams in the event broker.

Stream Time

A second option for maintaining time in a stream processor, known simply as *stream time*, is the approach favored by Apache Kafka Streams. A consumer application reading from one or more event streams maintains a stream time for its topology, which is the highest timestamp of processed events. The

consumer instance consumes and buffers events from each event stream partition assigned to it, applies the event-scheduling algorithm to select the next event to process, and then updates the stream time if it is larger than the previous stream time. Stream time will never be decreased.

Figure 8-5 shows an example of stream time. The consumer node maintains a single stream time based on the highest event-time value it has received. The stream time is currently set to 20 since that was the event time of the most recently processed event. The next event to be processed is the smallest value of the two input buffers—in this case, it's the event with event time 30. The event is dispatched down to the processing topology, and the stream time will be updated to 30.

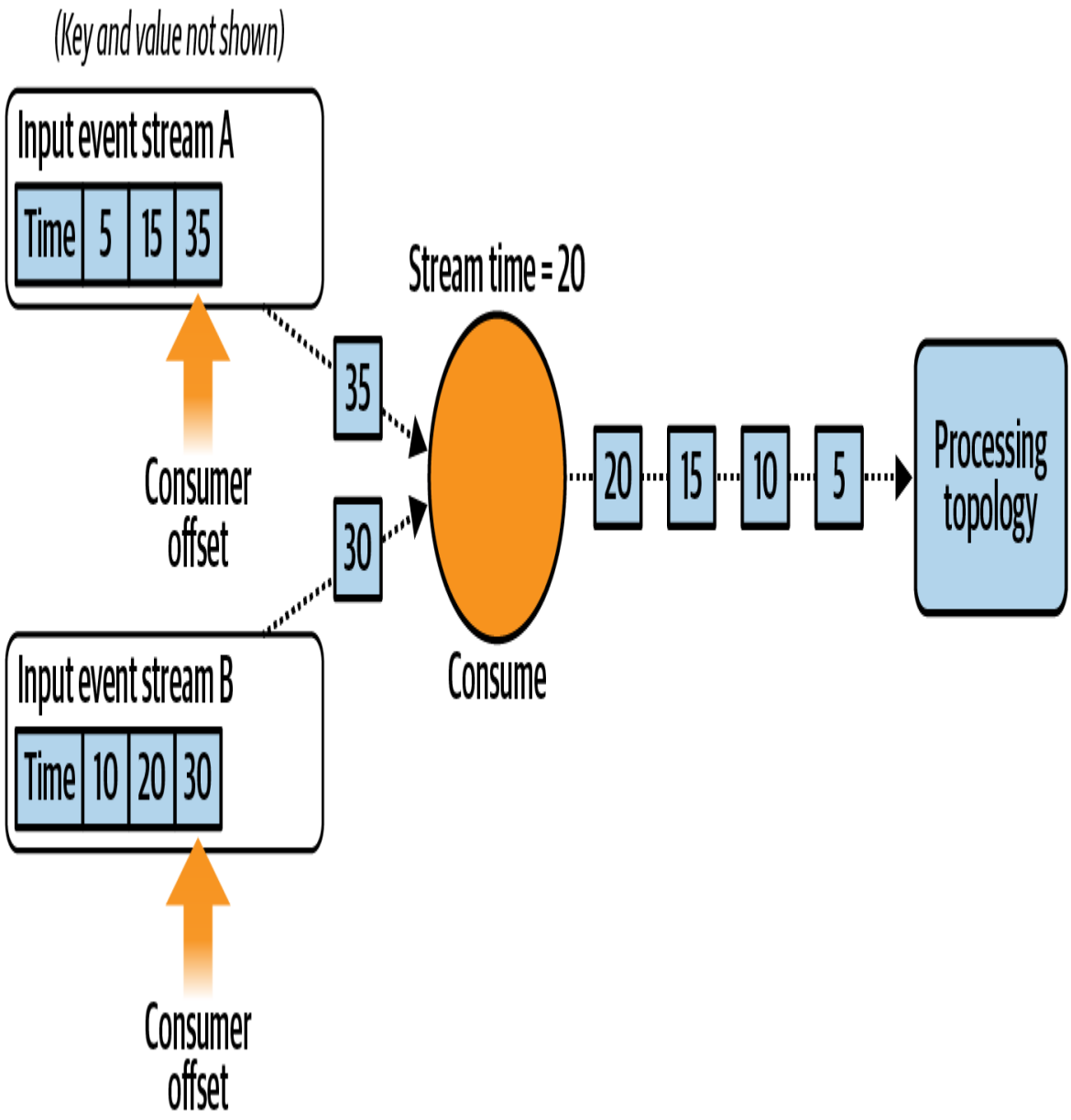


Figure 8-5. Stream time when consuming from multiple input streams

Stream time is maintained by processing each event completely through the topology before processing the next one. In cases where a topology contains a repartition stream, each topology is split into two, and each subtopology maintains its own distinct stream time. Events are processed in a depth-first manner, such that only one event is being processed in a subtopology at any given time. This is different than the watermark-based approach where events can be buffered at the inputs of each processing node, with each node's event time independently updated.

Stream Time in Parallel Processing

Consider again the same two-instance consumer example from Figure 8-4, but this time with the stream time approach championed by Kafka Streams (see Figure 8-6). A notable difference is that the Kafka Streams approach sends the repartitioned events *back* to the event broker using what's known as an *internal event stream*. This stream is then reconsumed by the instances, with all repartitioned data colocated by key within single partitions. This is functionally the same as the shuffle mechanism within the heavyweight cluster, but does not require a dedicated cluster (note: Kafka Streams is very microservice friendly).

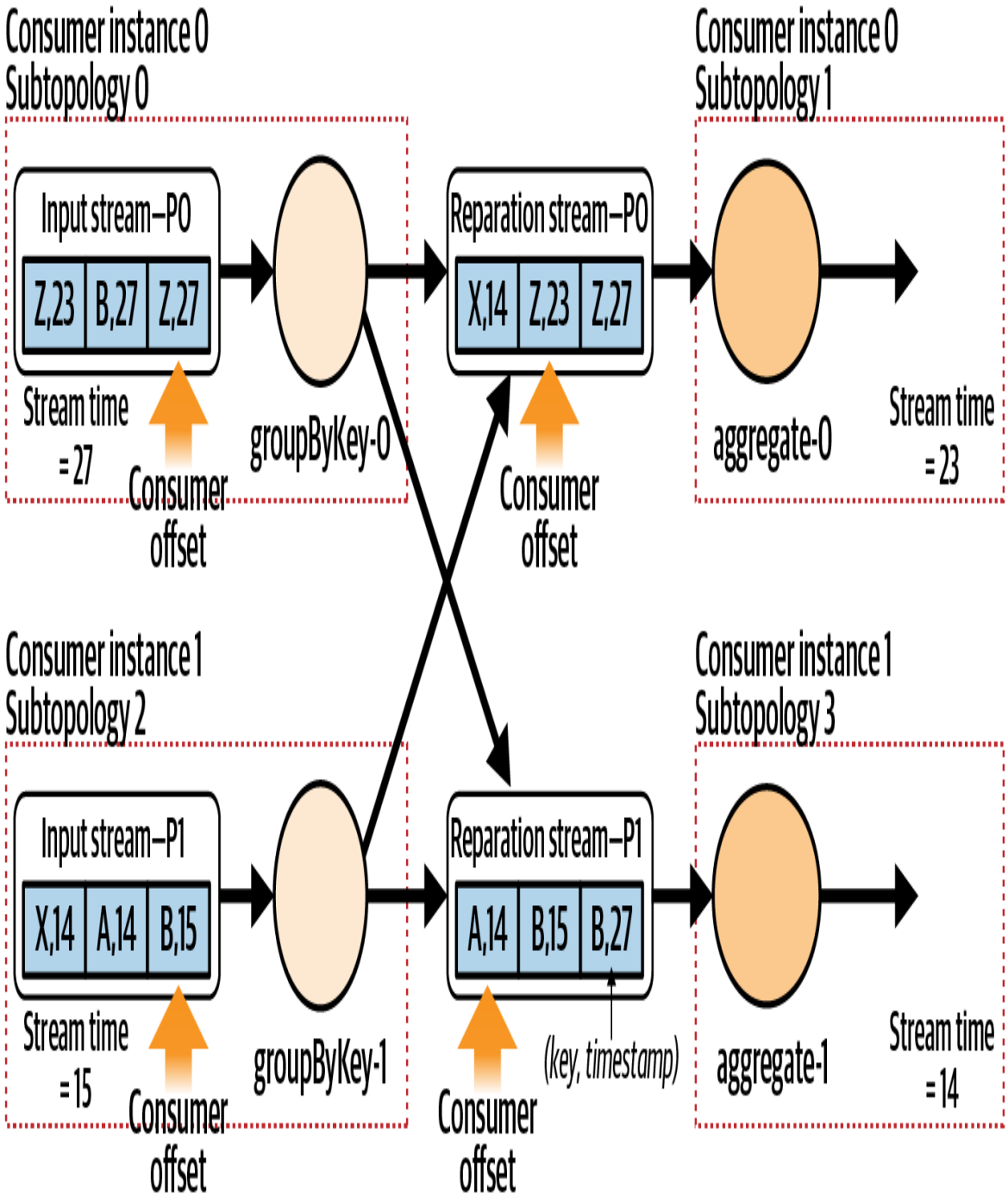


Figure 8-6. Shuffling events via a repartition event stream

In this example, events from the input stream are repartitioned according to their key and are written into the repartition event stream. The events keyed on `A` and `B` end up in `P1`, while the events keyed on `X` and `Z` end up in `P0`. Also note that the event time has been maintained for each event, and not overwritten by the current wall-clock time. Recall that the repartition should

be treated only as a logical shuffling of existing event data. Rewriting the event time of an event would completely destroy the original temporal ordering.

Notice the subtopologies shown in the figure. Because of the repartition event stream, the processing topology is effectively cut in half, meaning that work on each subtopology can be done in parallel. Subtopologies `1` and `3` consume from the repartition stream and group events together, while subtopologies `0` and `2` produce the repartitioned events. Each subtopology maintains its own stream time, since both are consuming from independent event streams.

> **TIP**
>
> Watermarking strategies can also use repartition event streams. Apache Samza offers a standalone mode that is similar to Kafka Streams, but uses watermarking instead of stream time.

Out-of-Order and Late-Arriving Events

In an ideal world, all events are produced without issue and available to the consumer with zero latency. Unfortunately for all of us living in the real world, this is never the case, so we must plan to accommodate out-of-order events. An event is said to be out of order if its timestamp isn't equal to or greater than the events ahead of it in the event stream. In Figure 8-7, event `F` is out of order because its timestamp is lower than `G`'s, just as event `H` is out of order as its timestamp is lower than `I`'s.

Bounded data sets, such as historical data processed in batch, are typically fairly resilient to out-of-order data. The entire batch can be thought of as one large window, and an event arriving out of order by many minutes or even hours is not really relevant provided that the processing for that batch has not yet started. In this way, a bounded data set processed in batch can produce results with high determinism. This comes at the expense of high latency, especially for the traditional sorts of nightly batch big-data processing jobs

where the results are available only after the 24-hour period, plus batch processing time.

For unbounded data sets, such as those in ever-updating event streams, the developer must consider the requirements of latency and determinism when designing the microservice. This extends beyond the technological requirements into the business requirements, so any event-driven microservice developer must ask, “Does my microservice handle out-of-order and late-arriving events according to business requirements?” Out-of-order events require the business to make specific decisions about how to handle them, and to determine whether latency or determinism takes priority.

Consider the previous example of the bank account. A deposit followed by an immediate withdrawal must be processed in the correct order lest an overdraft charge be incorrectly applied, regardless of the ordering of events or how late they may be. To mitigate this, the application logic may need to maintain state to handle out-of-order data for a time period specified by the business, such as a one-hour grace window.

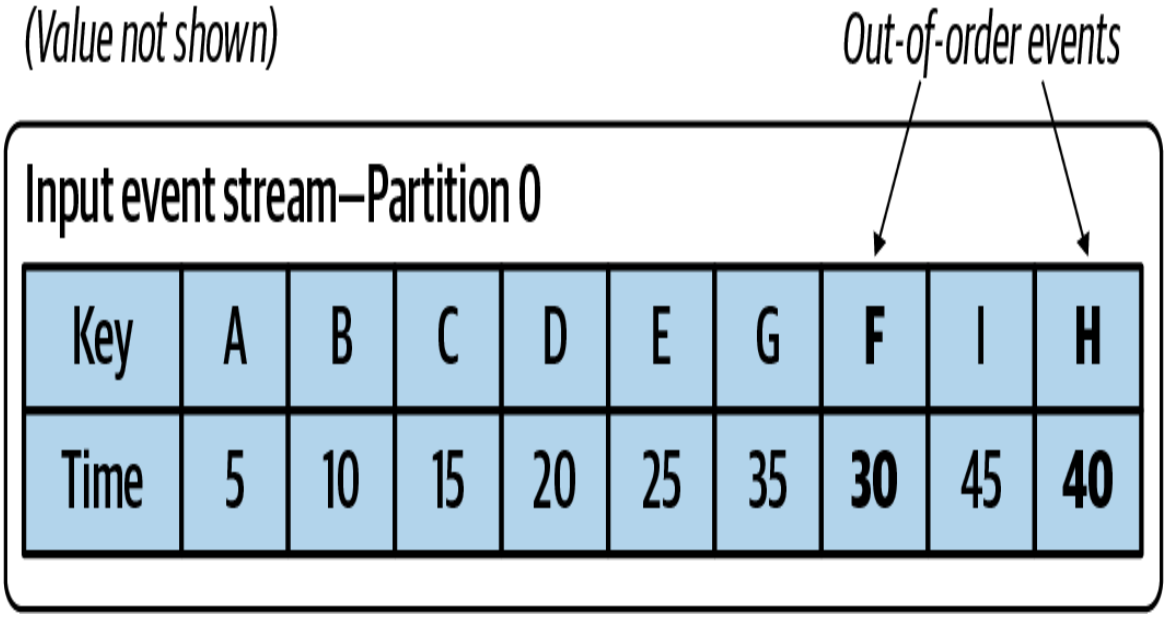


Figure 8-7. Out-of-order events in an event stream partition

> **TIP**
>
> Events from a single partition should always be processed according to their offset order, regardless of their timestamp. This can lead to out-of-order events.

An event can be considered *late* only when viewed from the perspective of the consuming microservice. One microservice may consider *any* out-of-order events as late, whereas another may be fairly tolerant and require many hours of wall-clock or event time to pass before considering an event to be late.

Late Events with Watermarks and Stream Time

Consider two events, one with time t, the other with time t'. Event t' has an *earlier* timestamp than event t.

Watermarks

> The event t' is considered late when it arrives *after* the watermark $W(t)$. It is up to the specific node how to handle this event.

Stream time

> The event t' is considered late when it arrives *after* the stream time has been incremented past t'. It is up to each operator in the subtopology how to handle this event.

> **TIP**
>
> An event is late only when it has missed a deadline specific to the consumer.

Causes and Impacts of Out-of-Order Events

There are several ways that out-of-order events can occur.

Sourcing from out-of-order data

The most obvious, of course, is when events are sourced from out-of-order data. This can occur when data is consumed from a stream that is already out of order or when events are being sourced from an external system with existing out-of-order timestamps.

Multiple producers to multiple partitions

Multiple producers writing to multiple output partitions can introduce out-of-order events. Repartitioning an existing event stream is one way in which this can happen. Figure 8-8 shows the repartitioning of two partitions by two consumer instances. In this scenario the source events indicate which product the user has interacted with. For instance, Harry has interacted with products ID12 and ID77. Say that a data analyst needs to rekey these events on the user ID, such that they can perform session-based analysis of the user's engagements. The resultant output streams may end up with some out-of-order events.

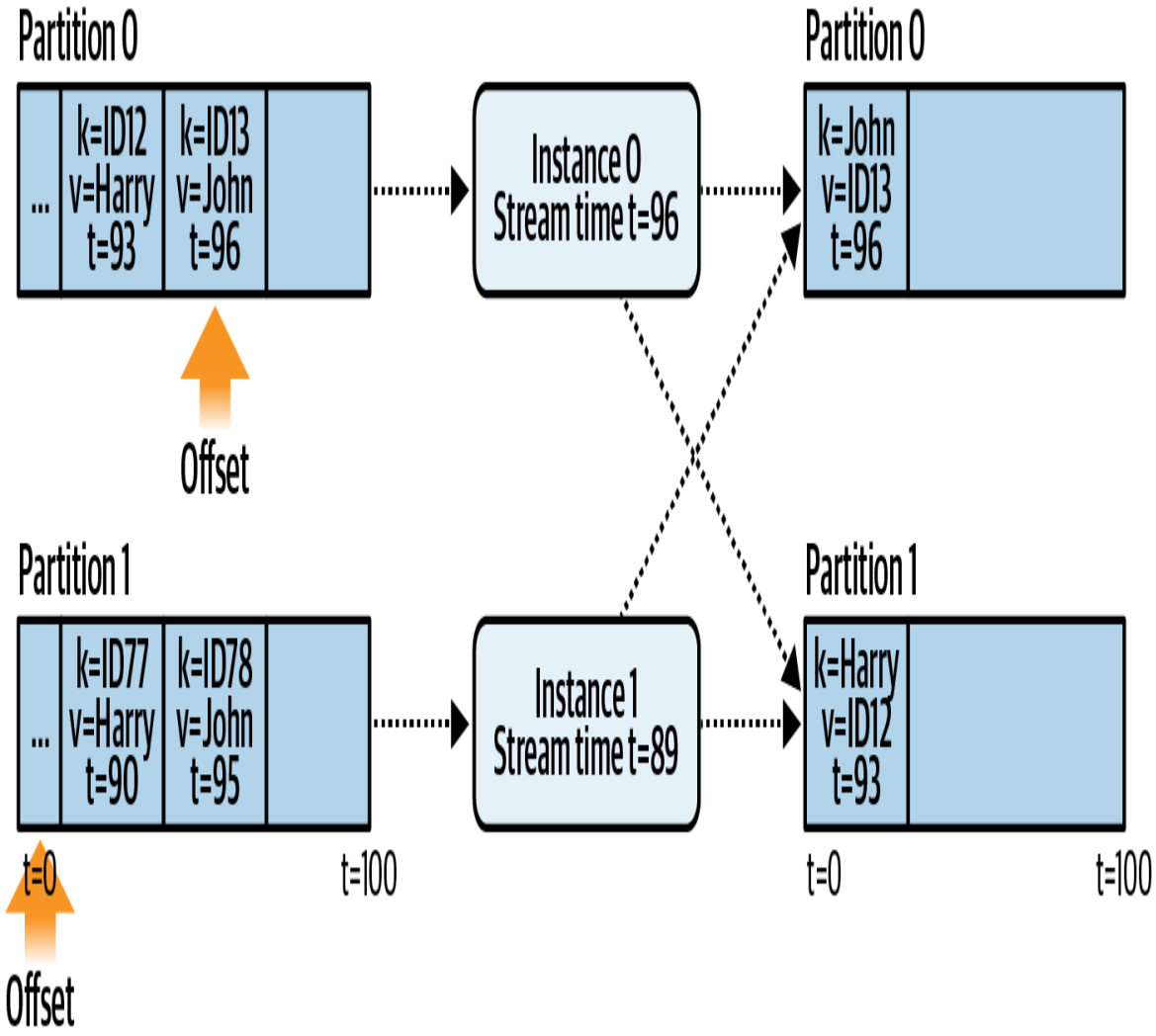


Figure 8-8. Shuffling events via a repartition event stream

Note that each instance maintains its own internal stream time and that there is no synchronization between the two instances. This can cause a time skew that produces out-of-order events, as shown in Figure 8-9.

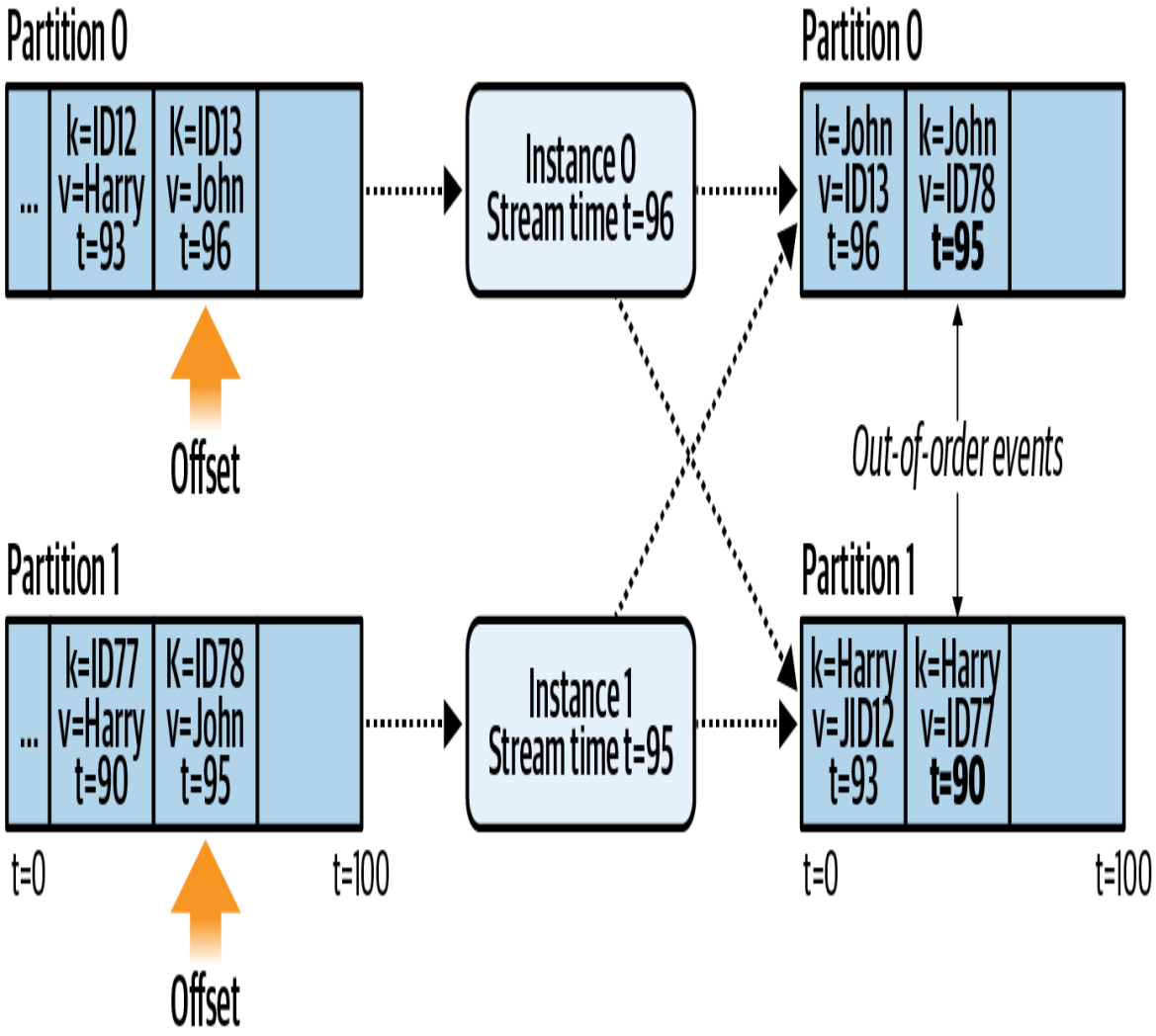


Figure 8-9. Shuffling events via a repartition event stream

Instance 0 was only slightly ahead of instance 1 in stream time, but because of their independent stream times, the events of time $t = 90$ and $t = 95$ are considered out of order in the repartitioned event stream. This issue is exacerbated by unbalanced partition sizes, unequal processing rates, and large backlogs of events. The impact here is that the previously *in-order* event data is now *out of order*, and thus as a consumer you cannot depend on having consistently incrementing time in each of your event streams.

> **TIP**
>
> A single-threaded producer will not create out-of-order events in normal operation unless it is sourcing its data from an out-of-order source.

Since the stream time is incremented whenever an event with a higher timestamp is detected, it is possible to end up in a scenario where a large number of events are considered late due to reordering. This may have an effect on processing depending on how the consumers choose to handle out-of-order events.

Time-Sensitive Functions and Windowing

Late events are predominantly the concern of time-based business logic, such as aggregating events in a particular time period or triggering an event after a certain period of time has passed. A late event is one that arrives after the business logic has already finished processing for that particular period of time. Windowing functions are an excellent example of time-based business logic.

Windowing means grouping events together by time. This is particularly useful for events with the same key, where you want to see *what happened* with events of that key in that period of time. There are three main types of event windows, but again, be sure to check your stream-processing framework for more information.

> **TIP**
>
> Windowing can be done using either event time or processing time, though event-time windowing typically has more business applications.

Tumbling windows

A tumbling window is a window of a fixed size. Previous and subsequent windows do not overlap. Figure 8-10 shows three tumbling windows, each

aligned on *t*, *t* + 1, and so on. This sort of windowing can help answer questions such as "When is the peak hour for product usage?"

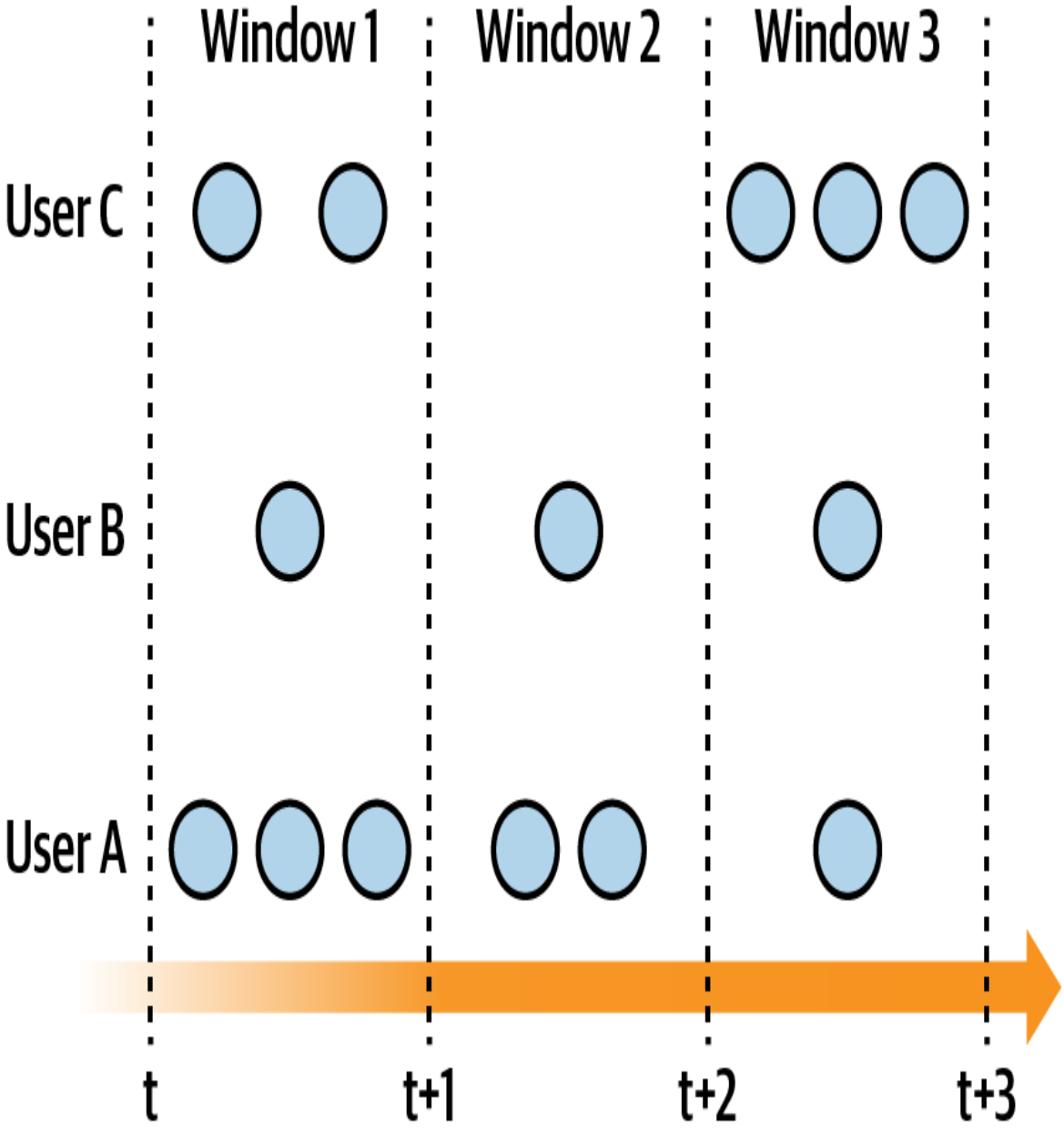


Figure 8-10. Tumbling windows

Sliding windows

A sliding window has a fixed window size and incremental step known as the *window slide*. It must reflect only the aggregation of events currently in the window. A sliding window can help answer questions such as "How

many users clicked on my product in the past hour?" Figure 8-11 shows an example of the sliding window, including the size of the window and the amount that it slides forward.

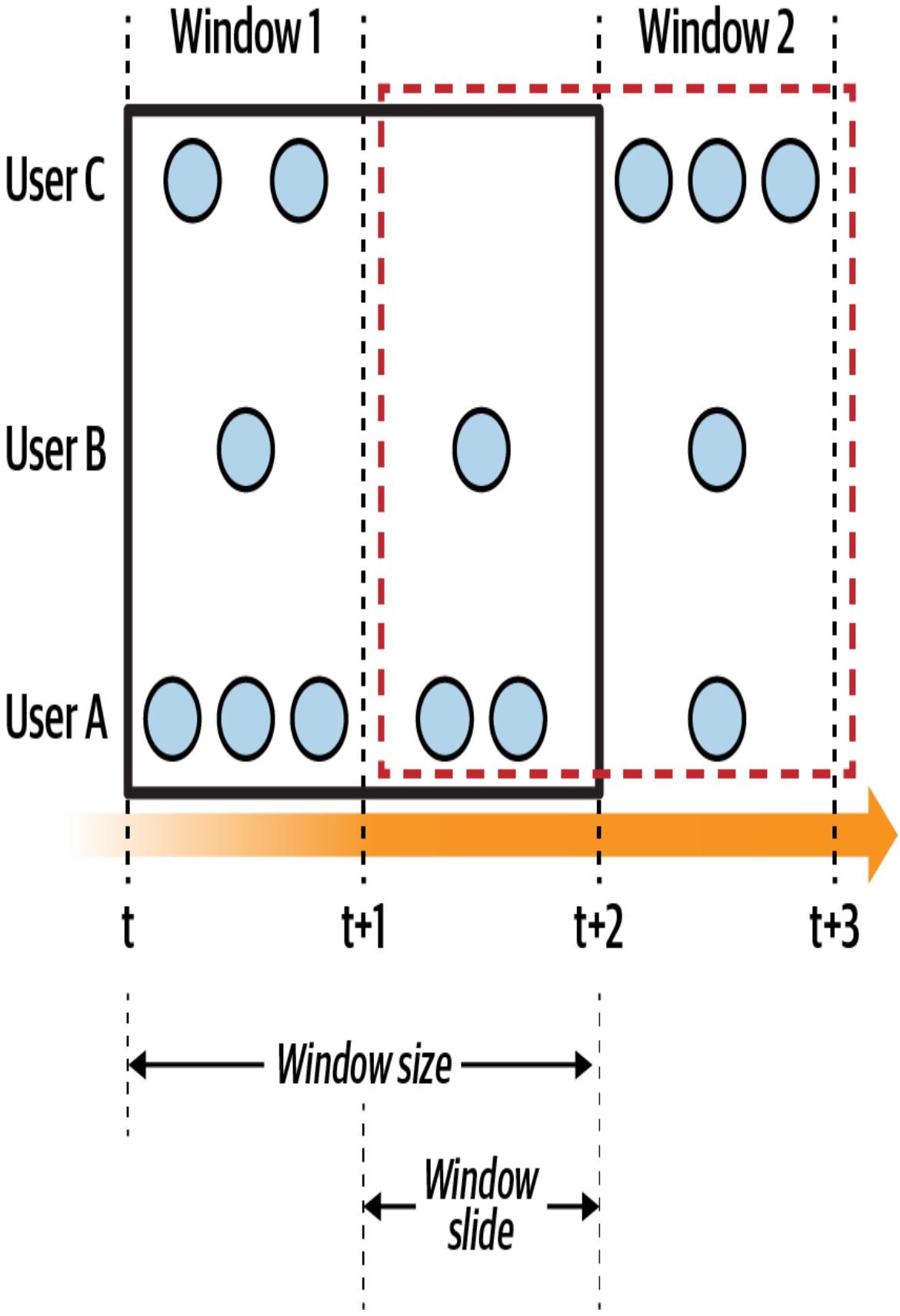

Window 1
Window 2
User C
User B
User A
t
t+1
t+2
t+3
Window size
Window slide

Figure 8-11. Sliding windows

Session windows

A session window is a dynamically sized window. It is terminated based on a timeout due to inactivity, with a new session started for any activity happening after the timeout. Figure 8-12 shows an example of session windows, with a session gap due to inactivity for user C. This sort of window can help answer questions such as "What does a user look at in a given browsing session?"

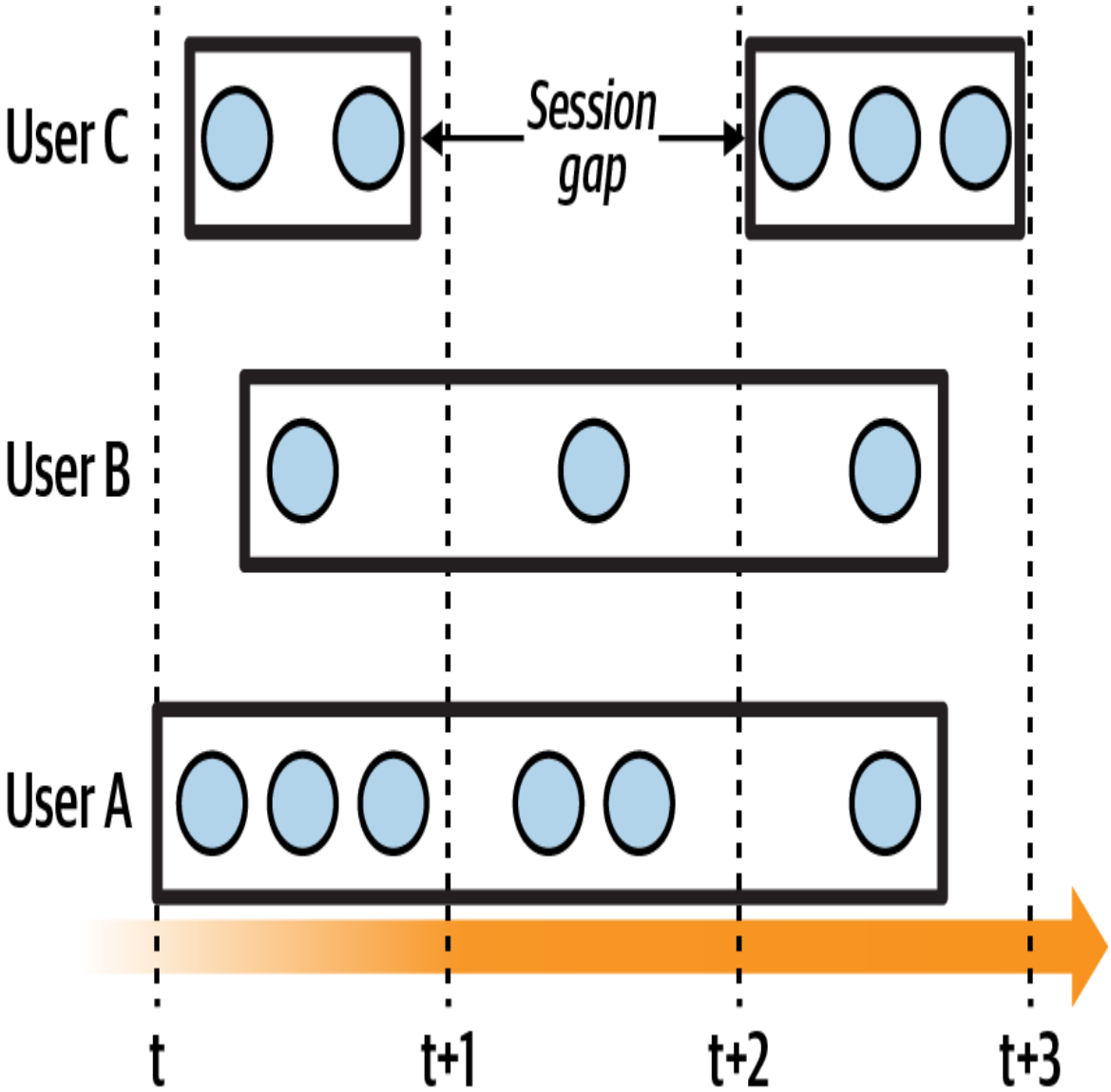


Figure 8-12. Session windows

Each of these window functions must deal with out-of-order events. You must decide how long to wait for any out-of-order events before deeming them too late for consideration. One of the fundamental issues in stream processing is that you can never be sure that you have received all of the events. Waiting for out-of-order events can work, but eventually your service will need to give up, as it cannot wait indefinitely. Other factors to consider include how much state to store, the likelihood of late events, and the business impact of not using the late events.

Handling Late Events

The strategy for handling out-of-order and late-arriving events should be determined at a *business level* prior to developing an engineering solution, as strategies will vary depending on the importance of the data. Critical events such as financial transactions and system failures may be required to be handled regardless of their position in the stream. Alternately, measurement-style events, such as temperature or force metrics, may simply be discarded as no longer relevant.

Business requirements also dictate how much latency is acceptable, as waiting for events to arrive may increase determinism but at the expense of higher latency. This can negatively affect the performance characteristics of time-sensitive applications or those with tight service-level agreements. Thankfully, microservices provide the flexibility necessary to tailor determinism, latency, and out-of-order event handling characterstics on a per-service basis.

When your stream processor windowing operation sees an event with an event time greater than the window cutoff, it starts a new window and closes off the old window, emitting it to the downstream output. Since the input event streams can be temporally out of order, what happens to the next event that should have gone into the initial window? You have a few options:

Discard the event

Simply drop the event. The window is closed, and any time-based aggregations are already complete.

Keep window open and delay output

Delay output of the window results until a fixed amount of time has passed. This provides the consumers with a higher degree of completeness at the expense of increased latency.

Grace period

Output the windowed result as soon as the window is deemed complete. The windowed result is not closed, however, but kept around and available for the predetermined grace period. Any late arriving events that belong to that window are merged in, and the updated output will be written to the event stream.

Regardless of how long a microservice waits, eventually events will simply be too late and will need to be discarded. There is no cut-and-dry technical rule for how your microservice should handle late events; just ensure that your business requirements are sufficiently met. If the protocol for handling late events is not specified in the microservice's business requirements, then the business must work to resolve that.

Here are a few questions to help you determine good guidelines for handling late events:

- How likely are late events to occur?
- How long does your service need to guard against late events?
- What are the business impacts of dropping late events?
- What are the business benefits of waiting a long time to capture late events?
- How much disk or memory does it take to maintain state?

- Do the expenses incurred in waiting for late events outweigh the benefits?

Reprocessing Versus Processing in Near-Real Time

Immutable event streams allow consumers to rewind their offsets and replay consumption from an arbitrary offset. This is known as *reprocessing*, and it's something that every event-driven microservice designer needs to consider. Reprocessing is typically performed only on event-driven microservices that use event time for processing events, and not those that rely on wall-clock time aggregations and windowing.

New consumer services are effectively reprocessing some, all, or none of the historical data. Processing historical data is pretty straightforward for a new service: you simply point the consumer offsets to the start of the input streams and let it process at its own rate. The service materializes state, computes results, and writes resultant output events as necessary.

Event scheduling is an important part of correctly reprocessing an event stream. It ensures that microservices process events in the same order that they did during near–real time. Handling out-of-order events is also an important part of this process, as repartitioning an event stream through the event broker (instead of using a heavyweight framework like Spark, Flink, or Beam) can cause out-of-order events.

Here are the steps to follow when you want to reprocess your event streams:

1. *Determine the starting point*. As a best practice, all stateful consumers should reprocess events from the very beginning of *each* event stream that they are subscribed to. This applies to entity event streams in particular, as they contain important facts about the entities under question. Note that if your service only ever cares about, say, the last 30 days of data, then you may rightfully choose to only reprocess data that is younger than that.

2. *Determine which consumer offsets to reset*. Any streams that contain events used in stateful processing should be reset to the very beginning, as it is difficult to ensure that you will end up with a correct state if you start in the wrong location (consider what would happen if you reprocessed someone's bank balance and accidentally omitted previous paychecks).

3. *Consider the volume of data*. Some microservices may process huge quantities of events. Consider how long it may take to reprocess the events, and any bottlenecks that may exist. Quotas (see [[Link to Come]]) may be needed to ensure that your microservice doesn't overwhelm the event broker with I/O.

4. *Consider the time to reprocess*. It is possible that reprocessing may take many hours to do, so it's worth calculating how much downtime you may need. Ensure that your downstream consumers are also okay with possibly stale data while your service reprocesses. Scaling the number of consumer instances to maximum parallelism can significantly reduce this downtime and can be scaled down once reprocessing has completed.

5. *Consider the impact*. Some microservices perform actions that you may not want to occur when reprocessing. For instance, a service that emails users when their packages have shipped should *not* re-email users when reprocessing events, as it would be a terrible user experience and completely nonsensical from a business perspective. Carefully consider the impact of reprocessing on the business logic of your system, as well as potential issues that may arise for downstream consumers.

Intermittent Failures and Late Events

An event may be late during near-real-time processing (watermark or stream time is incremented) but may be available as expected within the event stream during event-stream reprocessing. This issue can be hard to detect,

but it really illustrates the connected nature of event-driven microservices and how upstream problems can affect downstream consumers. Let's take a quick look at how this can occur.

Producer/Event Broker Connectivity Issues

In this scenario, records are *created* in timestamp order but can't be *published* until a later time (see Figure 8-13). During normal operation, producers send their events as they occur, and consumers consume them in near-real-time. This scenario is tricky to identify when it's happening and can go unnoticed even in retrospect.

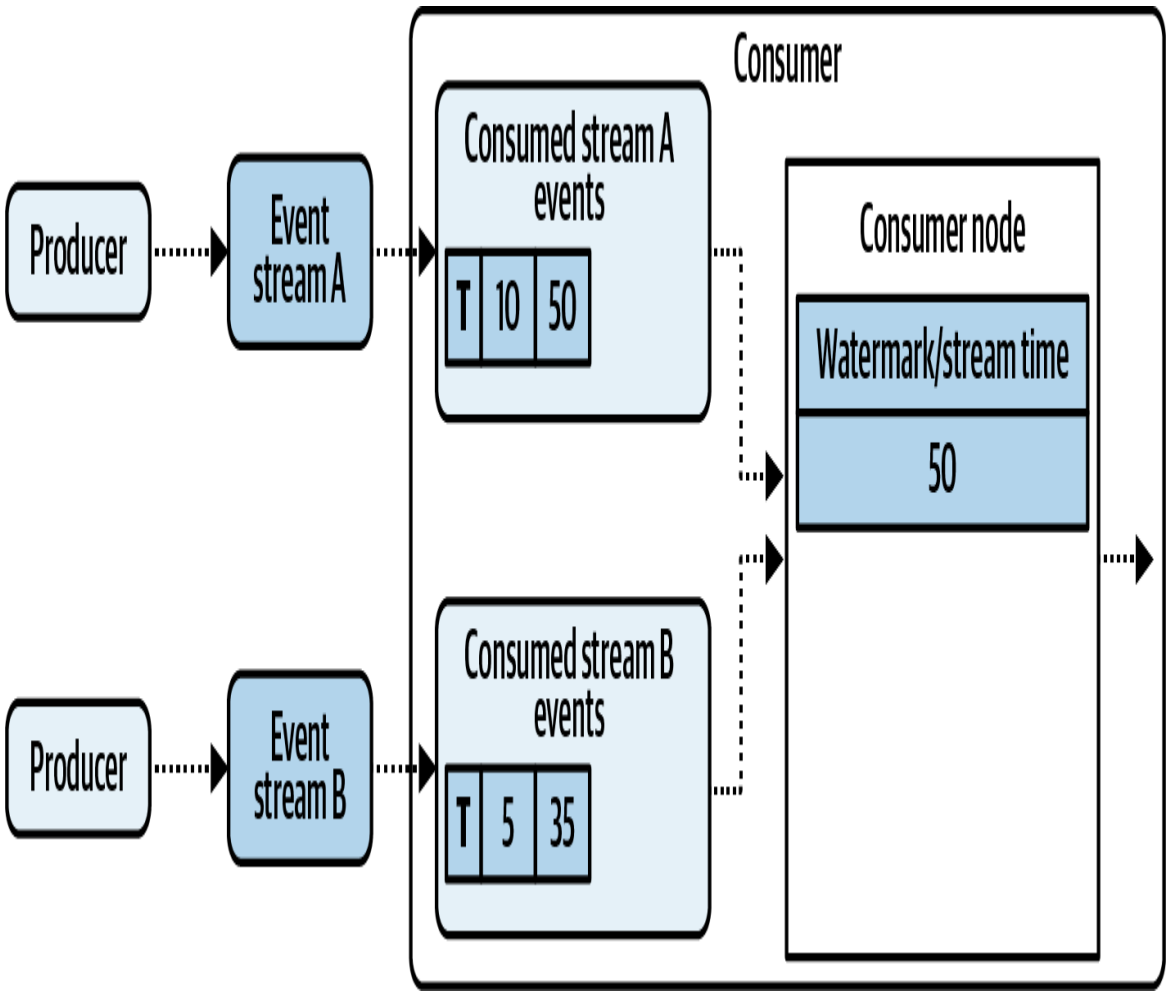


Figure 8-13. Normal operation prior to producer/broker connection outage

Say a producer has a number of records ready to send, but it is unable to connect to the event broker. The records are timestamped with the *local* time that the event occurred. The producer will retry a number of times and either eventually succeed or give up and fail (ideally a noisy failure so that the faulty connection can be identified and rectified). This scenario is shown in Figure 8-14. Events from stream A are still consumed, with the watermark/stream time incremented accordingly. However, upon consuming from stream B, the consumer ends up with no new events, so it can simply assume that no new data is available.

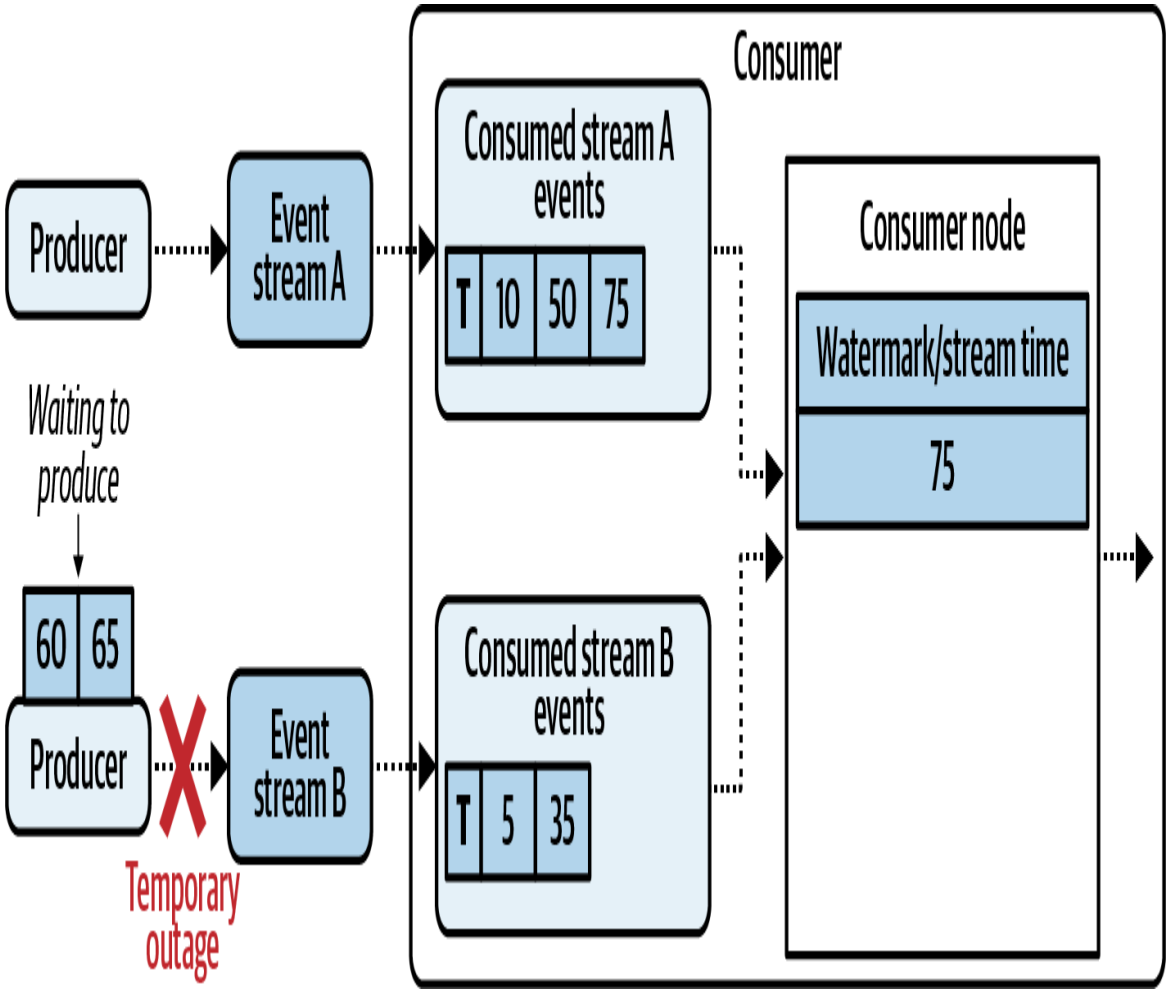


Figure 8-14. Temporary producer/broker connection outage

Eventually the producer will be able to write records to the event stream. These events are published in the correct event-time order that they actually occurred, but because of the wall-clock delay, near-real-time consumers will

have marked them as late and treat them as such. This is shown in Figure 8-15.

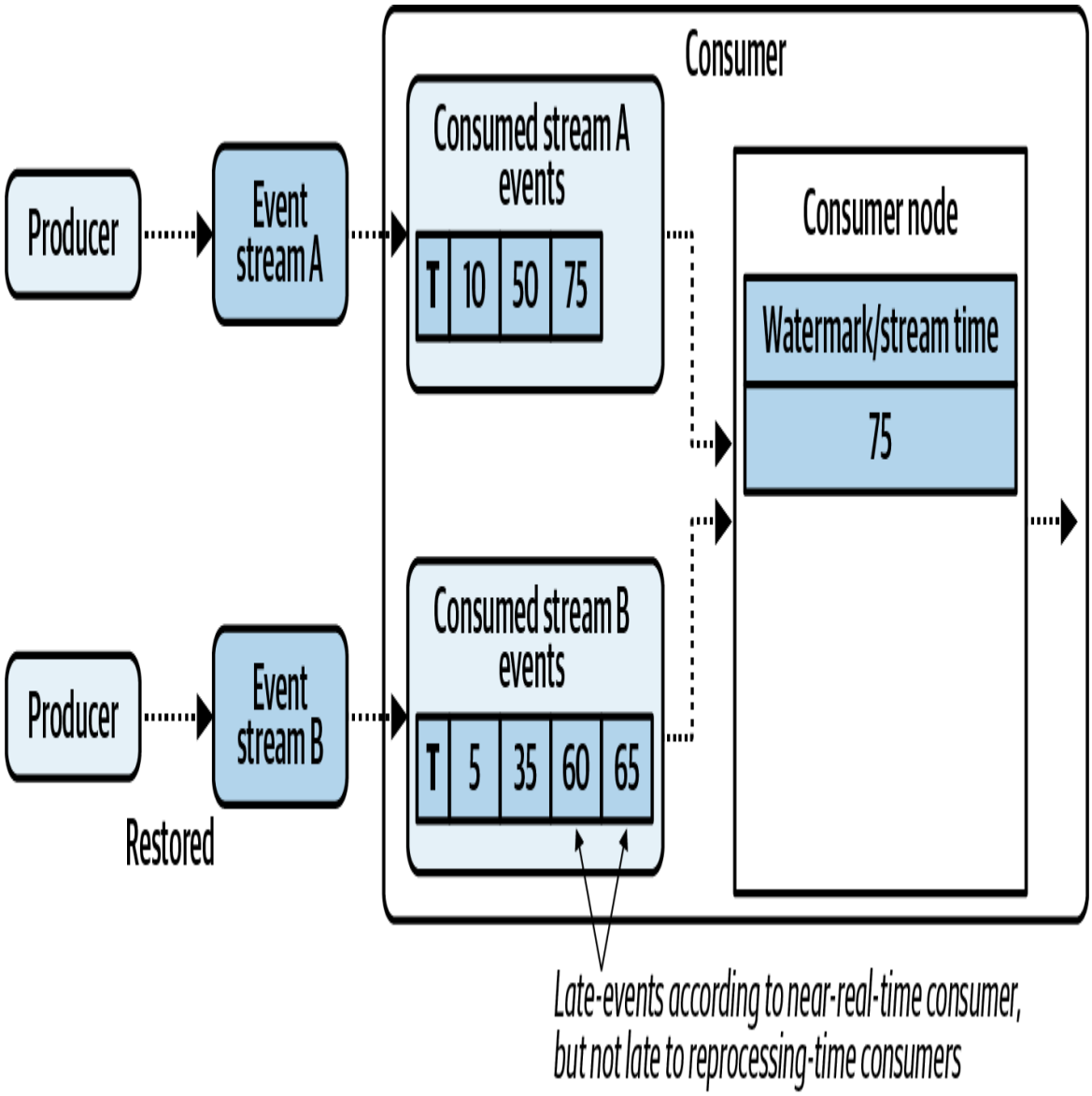


Figure 8-15. The producer is able to reconnect and publish its temporarily delayed events, while the consumer has already incremented its event time

One way to mitigate this is to wait a predetermined amount of time before processing events, though this approach does incur a latency cost and will only be useful when production delays are shorter than the wait time. Another option is to use robust late-event-handling logic in your code such that your business logic is not impacted by this scenario.

Summary and Further Reading

This chapter looked at determinism and how best to approach it with unbounded streams. It also examined how to select the next events to process between multiple partitions to ensure best-effort determinism when processing in both near-real-time and when reprocessing.

The very nature of an unbounded stream of events combined with intermittent failures means that full determinism can never be completely achieved. Reasonable, best-effort solutions that work most of the time provide the best tradeoff between latency and correctness.

If you would like to read more about watermarks, check out Tyler Akidau's excellent articles, "The World Beyond Batch Streaming 101" and 102. Additional considerations and insights into distributed system time can be found in Mikito Takada's online book *Distributed Systems for Fun and Profit*.

In the next chapter, we'll take a look at orchestration and choreography, and how both of those workflow types play essential roles in a mciroservice architecture.

Part III. Event-Driven Microservices Frameworks

There are many different technologies, languages, databases, and frameworks you can use to build microservices. This set of chapters covers the more common options available, and will provide you with enough information to let you make your own decisions when building your own microservices.

Chapter 9. Basic Producer and Consumer Microservices

> **A NOTE FOR EARLY RELEASE READERS**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 11th chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

Basic producer and consumer (BPC) microservices ingest events from one or more event streams, apply any necessary transformations or business logic, and emit any necessary events to output event streams. Synchronous request-response I/O may also be a part of this workflow, but that topic is covered in more detail in [[Link to Come]]. This chapter focuses strictly on event-driven components.

BPC microservices are characterized by the use of basic consumer and producer clients. Basic consumer clients do not include any event scheduling, watermarks, built-in materialization mechanisms, changelogs, or horizontal scaling of processing instances with local state stores. These capabilities typically belong only to more full-featured frameworks, which Chapters 10 and 11 will discuss in more depth. While it is certainly possible for you to develop your own libraries to provide these features, doing this is quite beyond the scope of this chapter. Thus, you must carefully consider whether the BPC pattern will work for your business requirements.

Producer and consumer clients are readily available in most commonly used languages, lowering the cognitive overhead in getting started with event-driven microservices. The entire workflow of the bounded context is contained within the code of the single microservice application, keeping the responsibilities of the microservice localized and easy to understand. The workflow can also easily be wrapped into one or more containers (depending on the complexity of the implementation), which can then be deployed and executed with the microservice's container management solution.

Where Do BPCs Work Well?

BPC microservices can fulfill a wide range of business requirements despite lacking most of the full-featured framework components. Simple patterns such as stateless transformations are easily implemented, as are stateful patterns where deterministic event scheduling is not required.

External state stores are more commonly used than internal state stores in BPC implementations, as scaling local state between multiple instances and recovering from instance failures is difficult without a full-featured streaming framework. External state stores can provide multiple microservice instances with uniform access as well as data backup and recovery mechanisms.

Let's look at a few use cases in which basic BPC implementations work particularly well.

Integration with Existing Systems Using the Sidecar Pattern

The basic producer/consumer client is an optimal way to integrate event-driven services into legacy codebases. Legacy systems can easily leverage the basic producer to write their own events to the stream using at-least-once semantics. Additionally, they can also consume and process events, inserting them into their own databases or running their own reactive code.

In some scenarios it's not possible to safely modify the legacy codebase to produce and consume data from event streams. The *sidecar pattern* is particularly applicable to this scenario, as it enables some event-driven functionality without affecting the source codebase.

Consider an ecommerce store frontend that displays all the inventory and product data it has available. Previously, the frontend service would source all of its data by synchronizing with a read-only subordinate data store using a scheduled batch job, as in Figure 9-1.

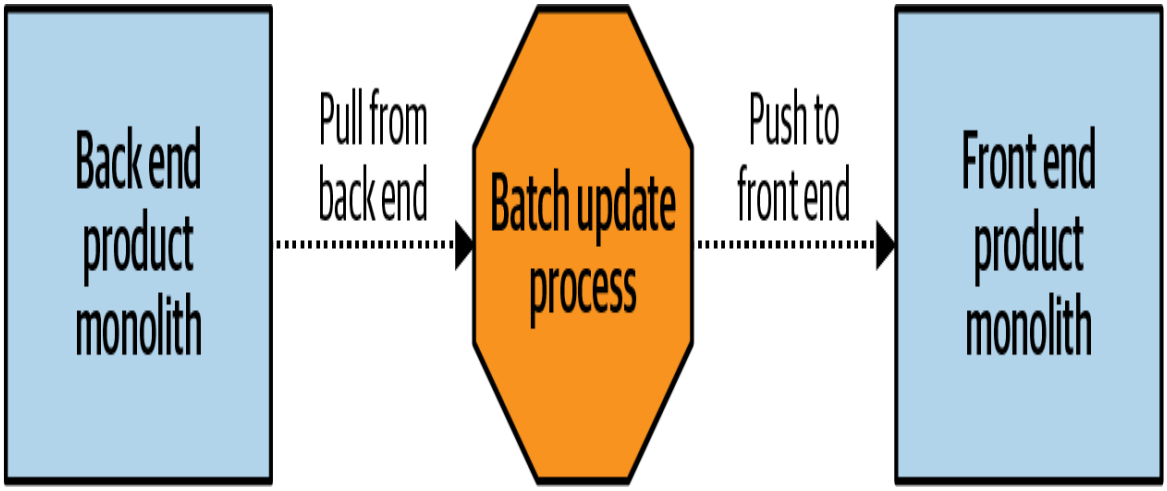


Figure 9-1. Scheduled batch between monoliths

In the event-driven architecture world there are two event streams, one with `Product Info` and one with `Product Inventoy` levels. Each stream is a mirror of the table that contains the source data in the database. Consumers that need product inventory data can consume from one, while consumers that need product information can consume from the other. In this case, the consumer needs data from both streams.

A sidecar implementation has one service that sinks the data from both streams into the data store, using a BPC to consume and upsert the data into the associated data set, as in Figure 9-2. In return, the frontend system gains access to a near-real time data feed of product updates, without having to change any of the system code.

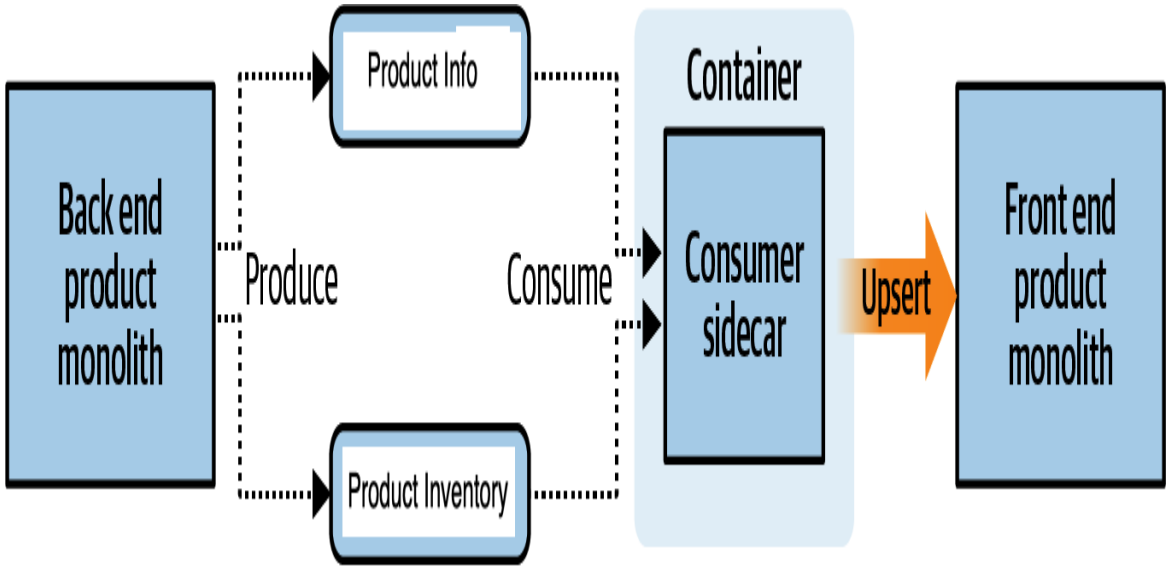


Figure 9-2. Using the sidecar to upsert data into the frontend data store

The sidecar resides inside its own container but *is part of the single deployable* of the frontend service. For example, you may choose to deploy it as a Kuberenetes sidecar, or even within the same container as the microservice application itself.

The sidecar pattern is pretty useful, as it allows you to add new functionality to a system without requiring significant changes to the legacy codebase. It is most commonly used to add event-driven processing capabilities to applications that would otherwise remain left out.

The main drawback of the sidecar pattern is the increase in complexity and coordination. Testing the application becomes more complicated, as you must also include the stream-processing framework as part of your testing considerations. Handling the bounded context of the microservice and stream-processing component is also more difficult, as you must ensure you can deploy and rollback the components together.

Stateful Business Logic That Isn't Reliant Upon Event Order

Many business processes do not have any requirements regarding the order in which events arrive and are processed, but they do require that *all* necessary

events *eventually* arrive. This is known as a *gating pattern* and is one in which a BPC can work well.

For example, say that you work for a book publisher, and there are three things that must be done before a book can be sent to the printer. The order that these tasks are completed is not important, but it is important that each one is completed prior to releasing the book to the printers:

Contents

> The contents of the book must have been written.

Cover art

> The cover art for the book must have been created.

Pricing

> The prices must be set according to regions and formats.

Each of these event streams acts as a driver of logic. When a new event comes in on any of these streams, it is first materialized in its proper table and subsequently used to look up every other table to see if the other events are present. Figure 9-3 illustrates an example.

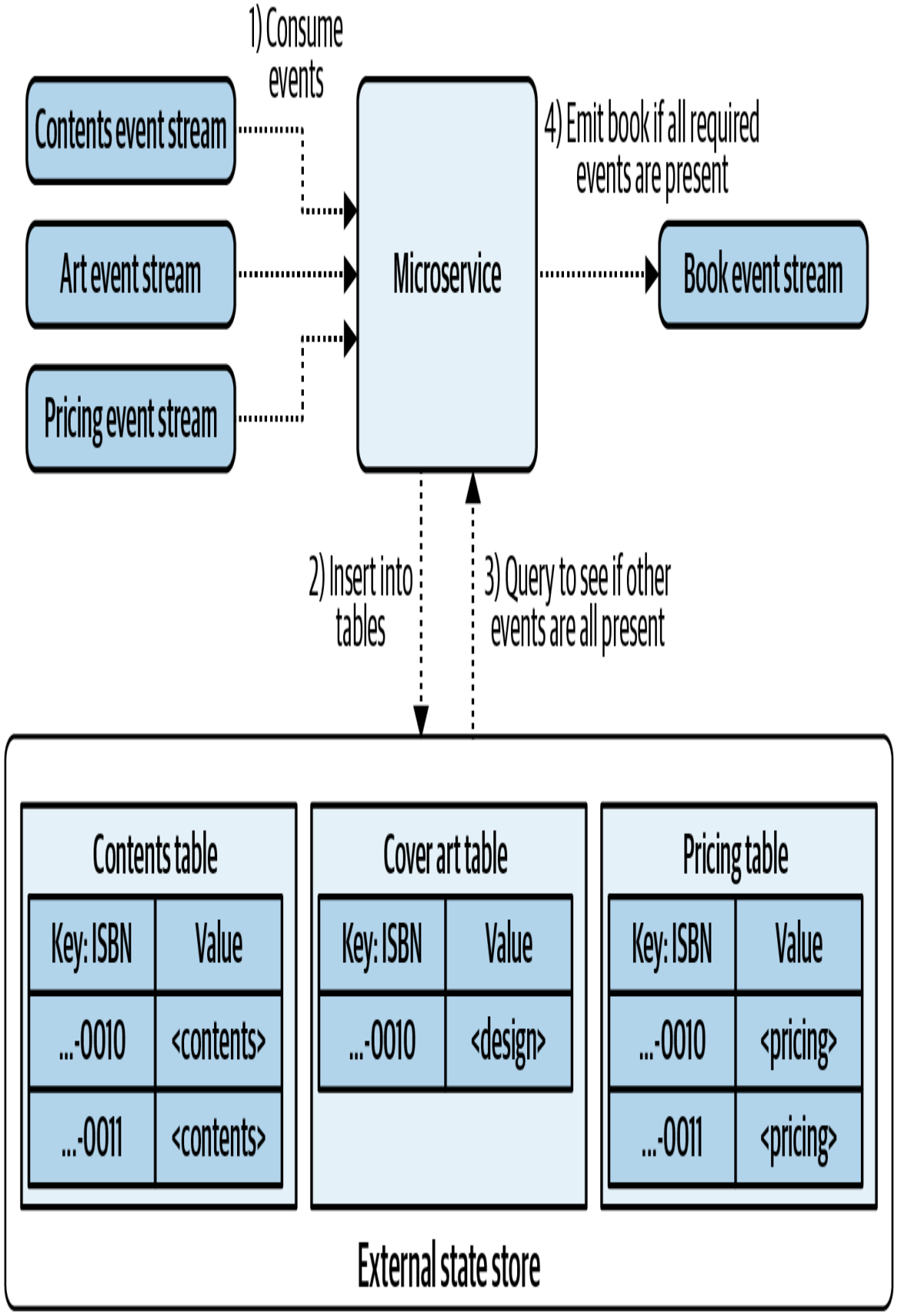

1) Consume events
Contents event stream
Art event stream
Pricing event stream
Microservice
4) Emit book if all required events are present
Book event stream
2) Insert into tables
3) Query to see if other events are all present
Contents table
Key: ISBN
Value
...-0010
<contents>
...-0011
<contents>
Cover art table
Key: ISBN
Value
...-0010
<design>
Pricing table
Key: ISBN
Value
...-0010
<pricing>
...-0011
<pricing>
External state store

Figure 9-3. Gating the readiness of a book

In this example, the book ending with `ISBN 0010` will already have been published to the output book event stream. Meanwhile, the book ending with `ISBN 0011` is currently waiting for cover art to be available and has not been published to the output stream.

> **TIP**
>
> Explicit approval from a human being may also be required in the gating pattern. This is covered in more detail in [[Link to Come]].

When the Data Store Does Much of the Work

A BPC is also suitable when the underlying data store performs most of the business logic, such as a geospatial data store; document data store; and machine learning, AI, and neural network applications. An ecommerce company may ingest new products scraped from websites and classify them using a BPC microservice, utilizing a batch-trained machine learning categorizer service in the backend. All of the complex business logic is performed offline in batch, while the classification is performed in realtime via lookups a per-event basis.

Alternately, user behavior events collected from user devices, websites, and applications, may be correlated with a geospatial data store to determine the nearest retailers from which to show advertisements.

In both scenarios the complexity of processing the event is offloaded almost entirely to the underlying data store. The producer and consumer components simply provide a mechanism to consume events, process them, and emit the results downstream.

Independent Scaling of the Processing and Data Store

The processing needs and the data storage needs of a microservice are not always linearly related. For instance, the volume of events that a

microservice must process may vary with time. One common load pattern, which is incorporated into the following example, mirrors the sleep/wake cycle of a local population, with intensive activity during the day and very low activity during the night.

Consider a scenario where user behavior events are aggregated into 24-hour sessions. The session data is used to determine the most recently popular products for a given user, and in turn are used to drive advertisements for more related products. Upon completing a 24-hour aggregation session, the service emits the results downstream in its own event and flushes the contents from its data store.

An option for per-user aggregation maintained in a external key-value data store is shown in Example 9-1.

Example 9-1. Data storage with a composite primary key with a list of `productIds` as the value

Key	Value
`userId, timestamp`	`List(productId)`

The processing needs of the service change with the sleep/wake cycles of the people using the product. At nighttime, when most users are asleep, the service requires very little processing power perform aggregations compared to what's required during the day. In the interest of saving money on processing power, the service is scaled down at night.

The partition assignor can reassign the input partitions to a single processor instance, as it can handle the reduced scale of user events. Note that despite the volume of events being low, the domain of potential users remains constant. Since anyone may be logged in at any time, the service requires full access to each and every user aggregations. In other words, scaling down the processing power has no impact on the state's upper size boundary that it must maintain.

During the day, additional processing instances can be brought online to handle the increased event load. The query rate of the data store will also increase in this particular scenario, but caching, partitioning, and batching can help keep the load lighter than the linear increase in processing requirements.

> **TIP**
>
> Cloud service providers typically offer high performance key-value storage that accommodates independent read/write scaling, while simultaneously keeping costs proportional to your read/write access patterns.

Summary

The BPC pattern is simple yet powerful. It forms the foundation of many stateless and stateful event-driven microservice patterns. You can easily implement stateless streaming and simple stateful applications using the BPC pattern.

The BPC pattern is also flexible. It pairs well with implementations where the data storage layer does most of the business work. You can use it as an interfacing layer between event streams and legacy systems, as well as leverage external stream-processing systems to augment its capabilities.

A major shortcoming is that BPCs lack the more advanced capabilities required for handling relationships between streams. For example, if you want to join data from event streams together, you must load it all into a database and join it there. Additionally, BPCs also lack out-of-the-box mechanisms such as state materialization, event scheduling, and timestamp-based decision-making. The next two chapters cover how heavyweight and lightweight frameworks provide all of these features, and more.

Chapter 10. Heavyweight Framework Microservices

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 12th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

This chapter and the next cover the full-featured frameworks most commonly used in event-driven processing. Frequently referred to as *streaming frameworks*, they provide mechanisms and APIs for consuming, processing, and producing event streams. These frameworks can be roughly divided into heavyweight frameworks, which are covered in this chapter, and lightweight frameworks, which are covered in the next.

Streaming frameworks provide full-featured support for primary key joins, foreign key joins, aggregations, and time-based operations. They're also capable of handling very large data sets, far beyond what a single instance could typically handle. Basic producer/consumer frameworks, as well as FaaS frameworks, typically lack these capabilities. It stands to reason, however - consuming events from a stream is one thing, but adding in all the more complex operations that a business requires is yet another.

These chapters aren't meant to compare the technologies, but rather to provide a generalized overview of how these frameworks work and to

showcase their capabilities. For the purposes of evaluating heavyweight frameworks, this chapter covers aspects of Apache Spark, Apache Flink and the Apache Beam model as examples of the sorts of technology and operations commonly provided.

> **WARNING**
>
> The first edition of this book also discussed Apache Storm and Apache Heron. They have been removed for the second edition, as their usage has greatly dwindled over time.

One defining characteristic of a heavyweight streaming framework is that it requires an independent cluster of processing resources to perform its operations. This cluster typically constitutes a number of shareable worker nodes, along with master nodes that schedule and coordinate work.

A second defining characteristic is that the heavyweight framework uses its own internal mechanisms for handling failures, recovery, resource allocation, task distribution, data storage, communication, and coordination between processing instances and tasks. This is in contrast to the lightweight framework, functions-as-a-service, and the basic producer/consumer implementations that rely heavily on the container management system (CMS) and the event broker to fulfil these needs.

A third characteristic is that the heavyweight framework provides its own automated mechanism for *shuffling* data between instances. This is a critical requirement for handling joins and aggregations at scale, ensuring that all data of a given key ends up in the correct processing instance.

These three characteristics are the main reasons why these frameworks are dubbed *heavyweight*. Having to manage and maintain additional clustered frameworks independently of the event broker and the CMS is no small task.

Heavyweight frameworks have one more notable property that separates them from lightweight frameworks: they're also capable of processing data in batch jobs, which speaks to their roots and historical origins. Let's take a

look at the history of these frameworks first, then get into an example to highlight their capabilities.

A Brief History of Heavyweight Frameworks

Heavyweight stream-processing frameworks are directly descended from their heavyweight batch-processing predecessors. Apache Hadoop, one of the most widely known processing engines, was released in 2006, providing open source big-data technologies for anyone to use. Hadoop bundled a number of technologies together to offer massive parallel processing, failure recovery, data durability, and internode communication, allowing users to access commodity hardware cheaply and easily to solve problems requiring many thousands of nodes (or more).

MapReduce was one of the first widely available means of processing extremely large batches of data (aka big data). It has since been overtaken by many more performant options, but for a time it was the dominant processing mechanism for large batches of data. The size of big data has also increased over time; although workloads of hundreds (or thousands) of gigabytes were common in the early days, workloads today have scaled to sizes in the terabyte and petabyte range. As these data sets have grown so has the demand for faster processing, more powerful options, simpler execution options, and solutions that can provide near-real-time stream-processing capabilities.

This is where Flink, Spark, and Beam come in, to name just the open-source and freely available Apache options. These solutions were developed to process big data scale workloads and provide actionable results much sooner than those provided by batch-based MapReduce jobs. Spark, for example, started as a high-performance replacement for MapReduce, but was largely limited to batch processing. Stream processing came later, as the technology evolved. Others, like Flink, were capable of processing streams from the start.

These technologies are undoubtedly familiar to most big-data aficionados and are likely already being used to some extent in the data science and analytics branches of many organizations. In fact, this is how many

organizations start dabbling in event-driven processing, as these teams convert their existing batch-based jobs into streaming-based pipelines.

Instead of going deeper under the hood, let's instead take a look at an example first to illustrate the utility of heavyweight streaming frameworks.

Example: Session Windowing of Clicks and Views

Imagine that you are running a simple online advertising company. You purchase ad space across the internet and resell it to your own customers. These customers want to see their return on investment, which in this case is measured by the click-through rate of users who are shown an advertisement.

Additionally, user engagements can only be billed on a per-session basis, with a session defined as continuous user activity without a break longer than 30 minutes. After 30 minutes of inactivity, the session is closed, and any new use activity will create a new session.

In this example there are two event streams: user product clicks, as shown in Example 10-1, and user advertisement views, as shown in Example 10-2. The goal is to aggregate these two streams into session windows and emit them once an *event time* (not wall-clock time) of 30 minutes has passed without new user event. Refer to Chapter 8 for a refresher on stream time and watermarks.

Example 10-1. The `Click` schema in Protobuf

```
message Click {
  int32 userId = 1;
  string productId = 2;
  Timestamp createdEventTime = 3;
}
```

Example 10-2. `AdView` schema with Protobuf

```
message Click {
  int32 userId = 1;
  string adId = 2;
  string productId = 3;
```

```
  Timestamp createdEventTime = 4;
}
```

The event-driven microservice must perform the following operations:

1. Group all events of the same `userId` key together, such that all events for a given user are local to a processing instance.
2. Window the events together with a 30-minute session timeout. Events that occur after more than 30 minutes of inactivity will create a new session window.
3. Within each session window, compute the `AdView` events that have a corresponding `ProductClick`, mark these as an `AdEngagement`, and emit the computed list downstream.

An `AdView` is only generated when an advertisement is successfully shown to the user. A `ProductClick` is only generated when that user clicks on the product. Note that a user may click on a product even if they did not see an advertisement, such as when browsing the website or looking at search results.

The abbreviated Apache Flink source code in Example 10-3 shows the microservice's code using its MapReduce-style syntax:

Example 10-3. Snippet of Flink code showing the stream processing operations

```
//Create streams of click and view events (super type is UserEvent)
DataStream<Either<ClickEvent, AdViewEvent>> eitherClicks = ...
DataStream<Either<ClickEvent, AdViewEvent>> eitherViews = ...

eitherClicks
        .union(eitherViews)
        .keyBy(t -> {
                if (t.isRight())
                        return t.right().getProductId();
                else
                        return t.left().getProductId();
        })
        .window(EventTimeSessionWindows.withGap(Duration.ofMinutes(30)))
        .process(new ProcessClickViewWindowFunction())
        .addSink(new KafkaTopicSink(...));
```

The process function shown in Example 10-4 does a lot of the heavy lifting. It iterates through the windowed grouping of clicks and views and pairs them up according to ad id. The list of engaged ads is then emitted downstream, where it can be written to a sink, such as another Kafka topic.

Example 10-4. The innards of the ProcessClickViewWindow that matches ad views with clicks

```
public class ProcessClickViewWindowFunction extends
        ProcessWindowFunction<Either<ClickEvent, AdViewEvent>, List<String>, Long,
TimeWindow> {

    @Override
    public void process(Long key,
                        Context context,
                        Iterable<Either<ClickEvent, AdViewEvent>> input,
                        Collector<List<String>> out) {

        //Map<ProductId, Tuple2<AdId, WasClicked>>
        Map<Long, Tuple2<String, Boolean>> map = new HashMap<>();
        List<String> adEngagements = new ArrayList<>();

        //  First, get all the Advertisements that have both a click and a view.
        // Note that we're ignoring the event time in this example, but
        // logically it's very hard to click on an advertisement that
        // you've never seen on your screen.
        for (Either<ClickEvent, AdViewEvent> event: input) {
            //Build or get a tuple representing the associated product click/view
            Tuple2<String, Boolean> adEngage =
                map.getOrDefault(key, new Tuple2<>("", false));

            if (event.isLeft()) //A Click Event - Add Clicked=true to Tuple2
                map.put(key, new Tuple2<>(adEngage.f0, true));
            else //An AdView Event - Add AdId to Tuple2
                map.put(key, new Tuple2<>(event.right().getAdId(), adEngage.f1));
        }

        for (Map.Entry<Long, Tuple2<String, Boolean>> clickViews: map.entrySet()) {
            if (clickViews.getValue().f0.length() > 0 && clickViews.getValue().f1) {
                adEngagements.add(clickViews.getValue().f0);
            }
        }
        //key=String (userId), value=List of adEngagements (List<String>)
        out.collect(adEngagements);
    }
}
```

The `List<String> adEngagements` collection stores the results of which `AdViews` are associated with `ProductClicks`, and can be considered an engagement. The output adheres to the following format:

Key	Value
String userId	List<String> adEngagements

A visual representation of this topology is shown in Figure 10-1, with a parallelism of 2 (note the two separate instances).

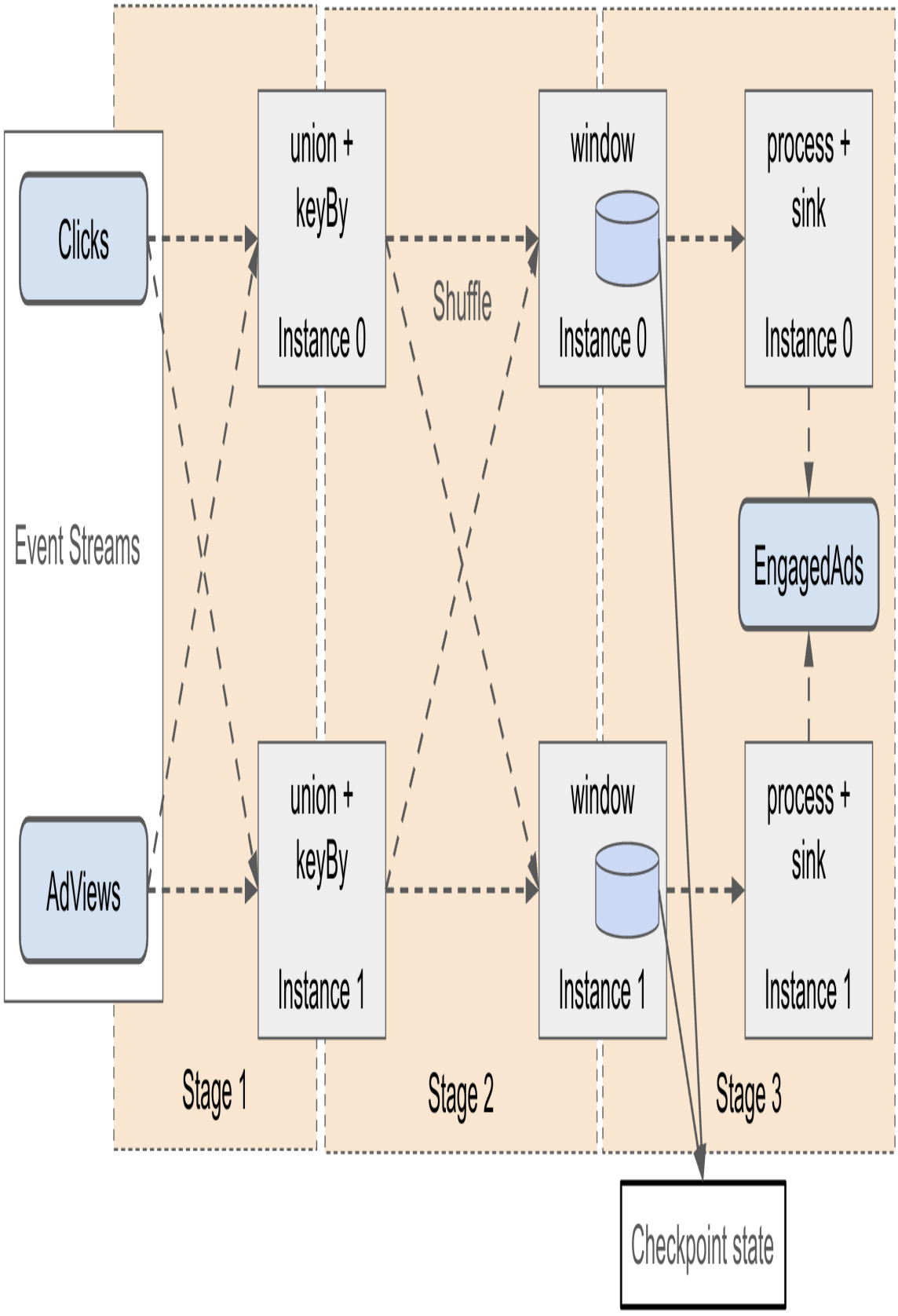

Clicks
AdViews
Event Streams
union + keyBy
Instance 0
union + keyBy
Instance 1
Shuffle
window
Instance 0
window
Instance 1
process + sink
Instance 0
process + sink
Instance 1
EngagedAds
Stage 1
Stage 2
Stage 3
Checkpoint state

Figure 10-1. Session processing topology from user Clicks and AdViews

Stage 1

The executors for each instance are assigned their tasks, which are in turn assigned the input event stream partitions for processing. Both the click and view streams are unioned into a single stream logical stream, and then grouped by the `userId` key.

Stage 2

The `keyBy` operator, in conjunction with the downstream `window` and `process` operators, requires shuffling the now-merged events to the correct downstream instances. All events for a given key are consumed into the same instance, providing the necessary data locality for the remaining operations.

Stage 3

In stage 3, the session windows are processed, emitting the list of adEngagements per user. Each list is written as its own event to the `EngagedAds` output event stream.

> **TIP**
>
> Streaming frameworks provide many additional controls over windowing and time-based aggregations. This can include retaining sessions and windows that have closed for a period of time, so that late-arriving events can be applied and an update emitted to the output stream.

Next, Figure 10-2 illustrates the effects of scaling down to just a single degree of parallelism. Assuming no dynamic scaling, you would need to halt the stream processor before restoring it from a checkpoint with the new parallelism setting. Upon startup, the service reads the stateful keyed data

back from the last known good checkpoint and restores the operator state to the assigned partitions. Once state is restored, the service can resume normal stream processing.

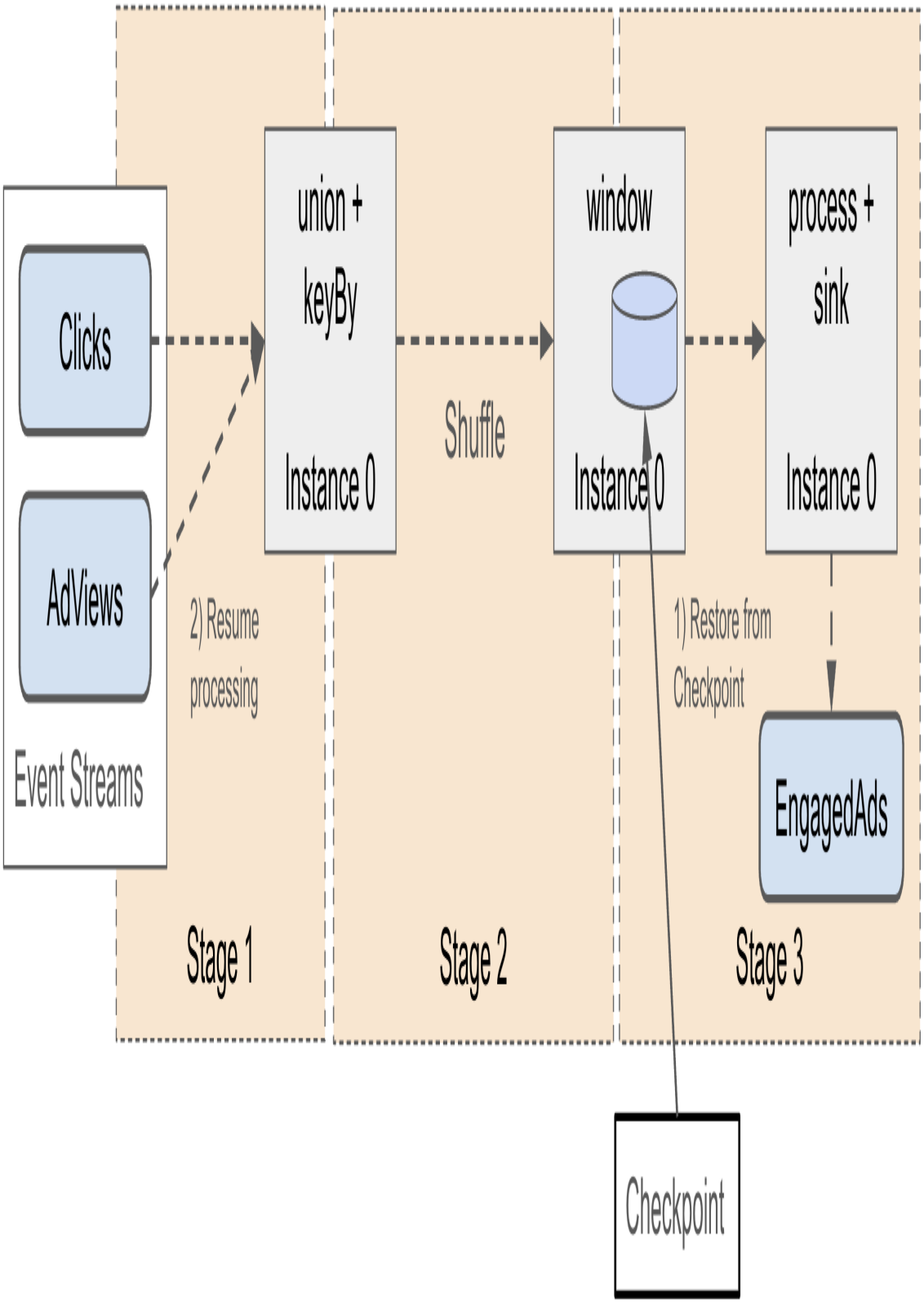

Clicks
AdViews
Event Streams
union +
keyBy
Instance 0
Shuffle
window
Instance 0
process +
sink
Instance 0
2) Resume
processing
1) Restore from
Checkpoint
EngagedAds
Stage 1
Stage 2
Stage 3
Checkpoint

Figure 10-2. Restoring the application with just a single processing instance

Stage 1 operates as before, though in this case instance 0 is consuming and processing every partition. It will still group and shuffle data, though the source and destination remain the same instance. The last stage of the topology, stage 3, still windows, processes, and emits the `adEngagements` to the output stream.

Heavyweight stream processing frameworks provide a lot of powerful options. They treat streams and events as building block primitives, which in turn lets you construct powerful declarative applications to group, aggregate, join, process, and store your data. We've only scratched the surface on their capabilities, but now, it's time to look under the hood to see what's going on.

The Inner Workings of Heavyweight Frameworks

The aforementioned Apache Spark and Apache Flink frameworks operate similarly to one another at a coarse-grained level. This is *not* to say that they are identical, but rather that they operate using similar modes of development, deployment, scaling, storage, and failure handling.

Proprietary solutions, like Google's Dataflow, which executes applications written using Apache Beam's API, probably *also* operate similarly. But this is only an assumption given that the source is closed and the backend is not described in detail.

Therefore, the challenge to describing the under the hood operations of heavyweight frameworks is that each has its own operational and design nuances. Full coverage of each framework is far beyond the scope of this chapter, and you will have to consult the documentation for your selected heavyweight framework.

A heavyweight stream processing *cluster* is a grouping of dedicated processing and storage resources. The resources in the cluster are organized according to two primary roles:

- The *master node* which prioritizes, assigns, and manages workers and tasks performed by the workers. It also has backup nodes to fail over to in case it crashes or becomes unavailable.
- *Worker (or executor) node*, which execute the business logic contained in its assigned tasks. It is the worker node resources, such as CPU, memory, and disk, that process the streaming application's business logic.

The tasks themselves are automatically generated based off of the topology of the stream processing application. They connect directly to the event broker and consume and produce to and from event streams. Figure 10-3 shows a rough breakdown of how this works.

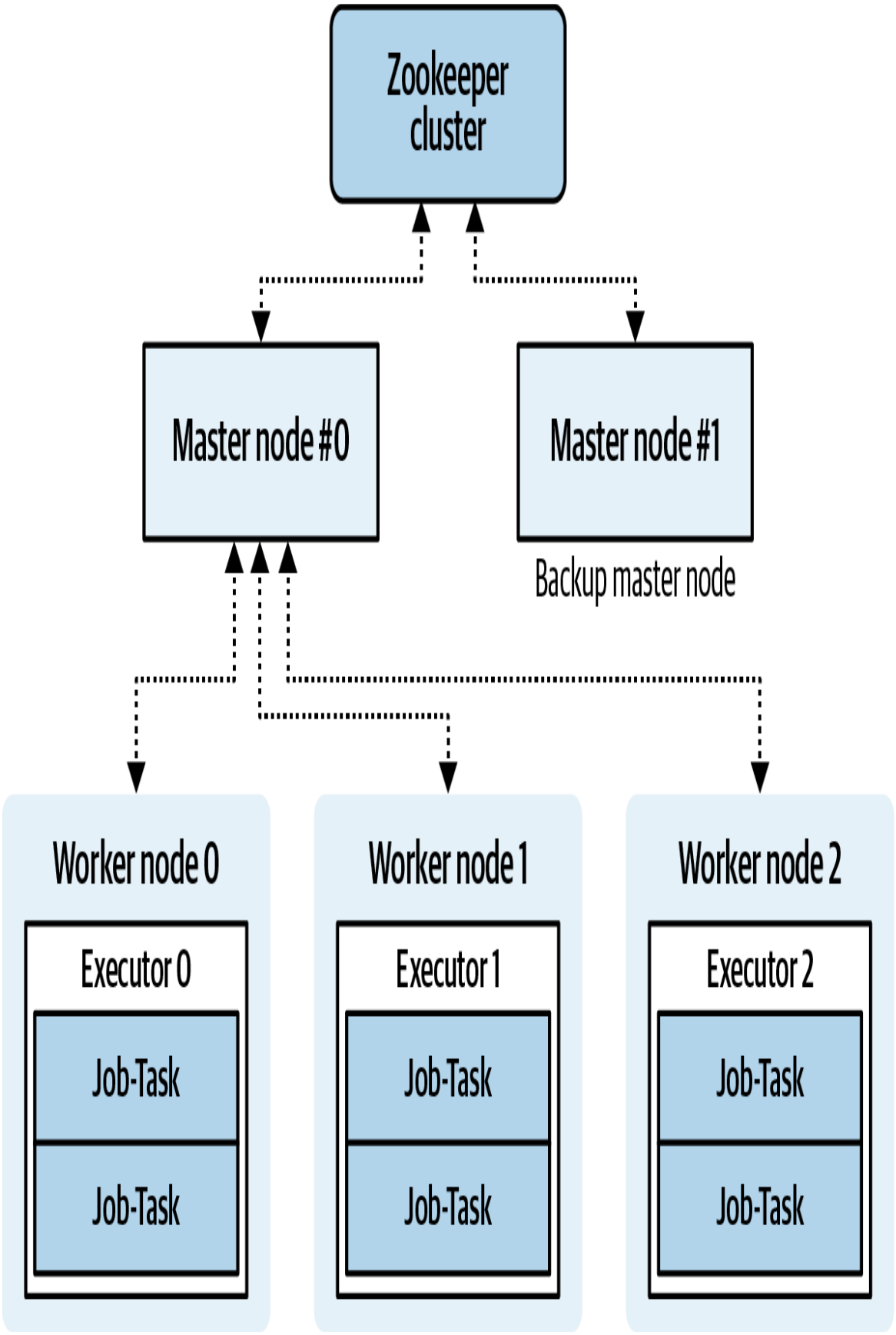

Zookeeper cluster
Master node #0
Master node #1
Backup master node
Worker node 0
Executor 0
Job-Task
Job-Task
Worker node 1
Executor 1
Job-Task
Job-Task
Worker node 2
Executor 2
Job-Task
Job-Task

Figure 10-3. A generic view of a heavyweight stream-processing framework using Zookeeper

A *job* is a stream-processing *application* that uses the framework's software development kit (SDK). The terminology is often used interchangeably, though calling an application *a job* is a leftover from batch-based data processing. Either way, a job is an application, and a streaming job is a streaming application that runs indefinitely just like any other microservice described in this book.

This figure also shows Apache Zookeeper, which plays a supporting role for this streaming cluster. Zookeeper provides highly reliable distributed coordination for determining which master node is in charge. Heavyweight frameworks are designed to be highly available and resilient to one or more failures, be it a worker node, master node, or Zookeeper node. Upon failure of a master node, Zookeeper helps decide which of the remaining masters is the new leader to ensure continuity of operations.

> **NOTE**
>
> Zookeeper has historically been a major component in providing coordination of distributed heavyweight frameworks. Newer frameworks may or may not use Zookeeper. In either case, distributed coordination is essential for reliably running distributed workloads.

Upon submission to a cluster, a stream processing application is broken down into tasks and assigned to the worker nodes. The task manager monitors the tasks and ensures that they are completed, restarting tasks and rebalancing tasks as necessary. Task managers are usually set up with high-availability, such that if the task manager itself fails, a backup manager steps in and take over. Otherwise, all your jobs would fail and it would be an expensive restart.

Figure 10-4 shows the process of submitting a job to the cluster via master node 1, which in turn is translated into tasks for processing by the executors. These long-running tasks establish connections to the event broker and begin to consume events from the event stream.

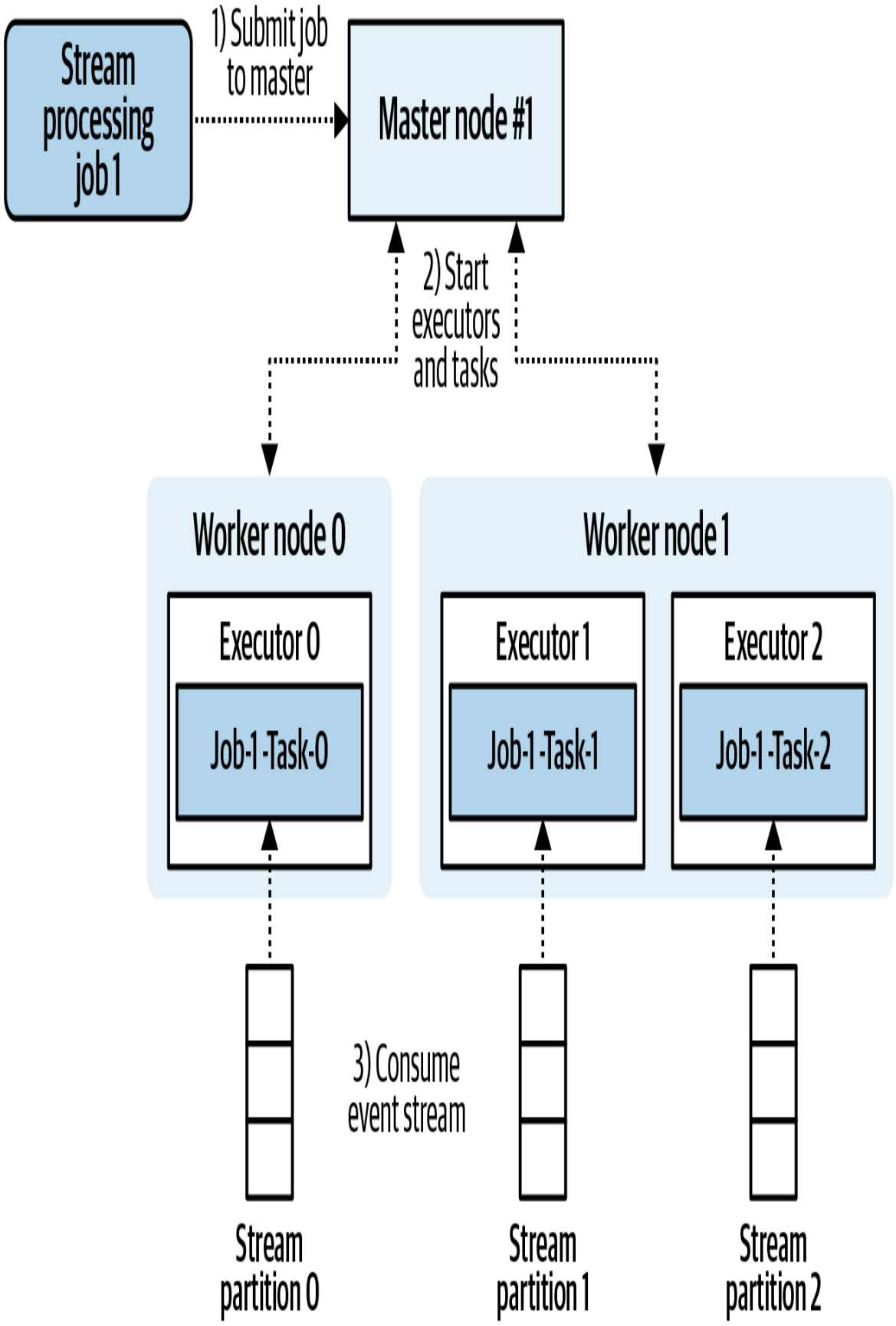

Stream processing job1
1) Submit job to master
Master node #1
2) Start executors and tasks
Worker node 0
Worker node 1
Executor 0
Job-1-Task-0
Executor 1
Job-1-Task-1
Executor 2
Job-1-Task-2
3) Consume event stream
Stream partition 0
Stream partition 1
Stream partition 2

Figure 10-4. Submitting a stream processing job to read from an event stream

Though this example shows a 1:1 mapping between tasks and stream partitions, you can configure the degree of parallelism for each application. One task can consume from all the partitions, or many tasks could consume from the same partition, say in the case of a queue.

Benefits and Limitations

The heavyweight frameworks discussed in this chapter have their roots and history in serving *analytical* use cases. They provide significant value around analyzing large volumes of events in near–real time to enable quicker decision-making. Some fairly common patterns of usage include the following:

- Extract data, transform it, and load it into a new data store (ETL)
- Perform session- and window-based analysis
- Detect abnormal patterns of behavior
- Aggregate streams and maintain state
- Perform any sort of stateless streaming operations

These frameworks are powerful and fairly mature, with many organizations using them and contributing back to their source code. There are numerous books and blog posts, excellent documentation, and many sample applications available to you. There are, however, several fairly significant shortcomings that limit, but don't completely preclude, microservice applications based on these frameworks.

First, these heavyweight frameworks were not originally designed with microservice-style deployment in mind. Deploying these applications requires a dedicated resource cluster beyond that of the event broker and CMS, adding to the complexity of managing large numbers of applications at scale. There are ways to mitigate this complexity, along with new

technological developments for deployment, some of which are covered in detail later in this chapter.

Second, these frameworks are JVM (Java Virtual Machine)-based, which limits the implementation languages you can use to create singular microservice applications. A common workaround is to use the heavyweight framework to perform transformations as its own standalone application, while another standalone application in another language serves the business functionality from the transformed state store.

Third, materializing an entity stream into an indefinitely retained table is not supported out of the box by all stream-processing frameworks. This can preclude creating table-table joins, stream-table joins, and computing long-running aggregations, limiting the scope of addressable business problems.

A wide range of streaming frameworks have emerged over the past few decades, though they are not all equal. A good acid test to a streaming framework's capabilities is its ability to serve joins, either as streams or as materialized tables. This is a challenging component to implement correctly, and proves to be a substantial barrier to its adoption and development.

Further, many stream frameworks focus heavily on time-based aggregations, with examples, blog posts, and advertisements emphasizing time-series analysis and aggregations based on limited window sizes. Some careful digging reveals that the leading frameworks provide a *global window*, which allows for the materialization of event streams into tables with indefinite retention. From here, you can implement your own custom join features, though I find that these are still far less well documented and exhibited than they should be, considering their importance in handling event streams at scale in an organization.

> **TIP**
>
> If you're investigating other streaming frameworks, check to see how (or if) they handle joins. If there is no support, it's a good indication that the streaming framework hasn't addressed some of the harder problems yet, and that you may want to look at other options.

While the majority of streaming frameworks originated for analytical workloads, there remain fully capable of powering operational use cases as event-driven microservices. Your next choice then becomes - build, or buy?

Cluster Setup Options and Execution Modes

There are a number of options when it comes to building and managing your heavyweight stream processing cluster, each with its own benefits and drawbacks.

Use a Hosted Service

The first and simplest way to manage a cluster is to just pay someone to do it for you. Just as there are a number of compute service providers, there are also providers who will be happy to host and possibly manage most of your operational needs for you. This option usually has the highest sticker price when compared to the just running your own free open-source software. But the former is a total cost, while the latter forgoes the costs of hardware, development, maintenance, upgrading, debugging, monitoring, and scaling. There is also an opportunity cost to consider, as all the time and effort you spend building your own streaming cluster is time not spent doing work that is critical to your business.

Confluent, founded by the co-creators of Apache Kafka, offers a fully managed Flink SQL alongside a Kafka offering. Amazon offers managed Flink and Spark services; Google, Databricks, and Microsoft offer their own bundling of Spark; and Google offers Dataflow, its own implementation of an Apache Beam runner. There are of course many other options, but you will have to research them for yourself.

> **WARNING**
>
> Note that not all managed services are created equally. Some are entirely serverless, while others require varying degrees of hands-on maintenance and monitoring. The general trend seems to be continually moving toward a full serverless-style approach, where the entire physical cluster is invisible to you as a subscriber. This may or may not be acceptable depending on your security, performance, and data isolation needs. Be sure that you understand what is and is not offered by these service providers, as they may not include the same features of an independently operated cluster.

Build and Run Your Own Cluster

Heavyweight frameworks historically have their own dedicated clusters composed of commodity hardware. Figure 10-3 showed a basic overview of what that may look like from a high level.

Running your own cluster is a challenge, but this isn't to say that you shouldn't do it. Rather, you simply need to be prepared. This book doesn't go into depth on this subject because it's highly related to the streaming technologies you choose, and their relationship with the other technologies you already have.

> **WARNING**
>
> "How hard could it be?" is a popular sentiment when setting up your own heavyweight streaming framework. It can be very easy to get a basic proof of concept working, but hardening it into a reliable production-ready service can be very challenging and expensive.

Very large companies with deep pockets and adequate staffing may find it cost acceptable to build, deploy, run, manage, scale, monitor, debug, and update their own cluster(s). Smaller companies may find it more reasonable to choose ready-to-go managed services instead, and reevaluate optimizing for costs once they've found a product-market fit.

A cluster can also be created in conjunction with the CMS. One mode of operation involves simply deploying the cluster on CMS-provisioned resources. The second mode mode involves leveraging the CMS itself as the

means of scaling and deploying individual microservices. This latter mode has become increasingly more popular alongside adoption of containerization. Using the CMS to deploy and manage applications simplifies your workflows and reduces overall system complexity.

Mode One: Deploying and running the cluster using the CMS

Deploying the heavyweight cluster using the CMS has many benefits. The master nodes, worker nodes, and Zookeeper (if applicable) are brought up within their own container or virtual machines. These containers are managed and monitored like any other container, providing visibility into failures as well as the means to automatically restart these instances.

> **TIP**
>
> You can enforce static assignment of master nodes and any other services that you require to be highly available, to prevent the CMS from shuffling them around as it scales the underlying compute resources. This prevents excessive alerts from the cluster monitor about missing master nodes.

Mode Two: Specifying resources for a single application using the CMS

Historically, the heavyweight cluster has been responsible for assigning and managing resources for each application instance. Modern CMSes can also perform these operations, but they can do it for any kind of microservice - regardless of the language and framework it is coded in.

Spark and Flink enable you to directly leverage Kubernetes for scalable application deployment beyond their original dedicated cluster configuration, where each application has its own set of dedicated worker nodes. For example, Apache Flink enables applications to run independently within their own isolated session cluster using Kubernetes. Apache Spark offers a similar option, allowing Kubernetes to play the role of the master node and maintain isolated worker resources for each application. A basic overview of how this works is shown in Figure 10-5.

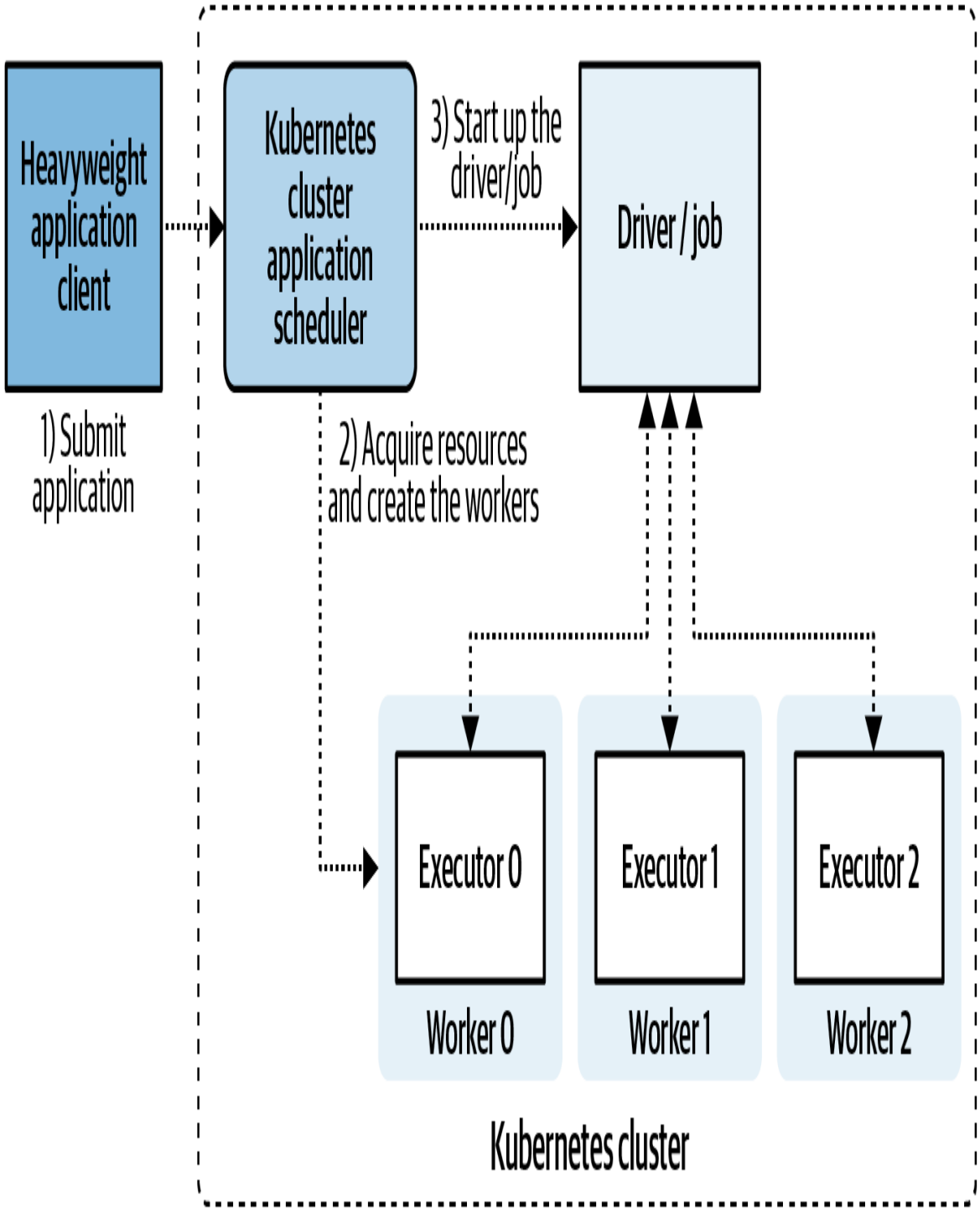


Figure 10-5. Single job deployed on and managed by Kubernetes cluster

> **TIP**
>
> This deployment mode is nearly identical to how you would deploy non-heavyweight microservices and merges lightweight and BPC deployment strategies.

There are several advantages to this deployment pattern:

- It leverages the CMS resource acquisition model, including for scaling.
- There is complete isolation between jobs.
- You can use different frameworks and different versions.
- Heavyweight streaming applications can be treated just like microservices, using the same deployment processes.

And of course, there are also several disadvantages:

- Support is not available for all leading heavyweight streaming frameworks.
- Integration is not available for all leading CMSes.
- Features available in full cluster mode, such as automatic scaling, may not yet be supported.

Application Submission Modes

Applications can be submitted to the heavyweight cluster for processing in one of two main ways: driver mode and cluster mode.

Driver Mode

Driver mode is supported by Spark and Flink. The *driver* is simply a single, local, standalone application that helps coordinate and execute the application, though the application itself is still executed within the cluster

resources. The driver coordinates with the cluster to ensure the progress of the application and can be used to report on errors, perform logging, and complete other operations. Notably, termination of the driver will result in termination of the application, which provides a simple mechanism for deploying and terminating heavyweight streaming applications. The application driver can be deployed as a microservice using the CMS, and the worker resources can be acquired from the heavyweight cluster. To terminate the driver, simply halt it as if it were any other microservice.

Cluster Mode

In cluster mode, the entire application is submitted to the cluster for management and execution, whereupon a unique ID is returned to the calling function. This unique ID is necessary for identifying the application and issuing orders to it through the cluster's API. With this deployment mode, commands must be directly communicated to the cluster to deploy and halt applications, which may not be suitable for your microservice deployment pipeline.

Handling State and Using Checkpoints

Stateful operations may be persisted using either internal or external state (Chapter 7), though most heavyweight frameworks favor internal state for its high performance and scalability. Stateful records are kept in memory for fast access, but are also spilled to disk for data durability purposes and when state grows beyond available memory. Using internal state does carry some risks, such as state loss due to disk failure, node failures, and temporary state outages due to aggressive scaling by the CMS. However, the performance gains tend to far outweigh the potential risks, which can be mitigated with careful planning.

Checkpoints, snapshots of the application's current internal state, are used to rebuild state after scaling or node failures. A checkpoint is persisted to durable storage external to the application worker nodes to guard against data loss.

You can checkpoint with any datastore that is compatible with the framework, such as Hadoop Distributed File System (HDFS, a common option) or a highly available external data store. Each partitioned state store can then restore itself from the checkpoint, providing both full restoration capabilities in the case of a total application failure, and partial restoration capabilities in the case of scaling and worker node failures.

> **TIP**
>
> The Hadoop Distributed File System (HDFS) was the precursor to many modern cloud storage systems today, like Azure Blob Storage and Amazon S3, just to name a few. While HDFS itself isn't in common usage anymore, its API, and its influence, lives on in many products.

There are two main states that the checkpointing mechanism must consider when consuming and processing partitioned event streams:

Operator state

> The pairs of `<partitionId, offset>`. The checkpoint must ensure that the internal key state (see next item) matches up with the consumer offsets of each partition. Each `partitionId` is unique among all input topics.

Key state

> The pairs of `<key, state>`. This is the state pertaining to a keyed entity, such as during aggregations, reductions, windowing, joins, and other stateful operations.

The checkpointing mechanism synchronously records both the operator and keyed state, such that it accurately represents the processing of all the events marked as consumed by the operator state. A failure to do so may result in events either not being processed at all or being processed multiple times. An example of this state as recorded into a checkpoint is shown in Figure 10-6.

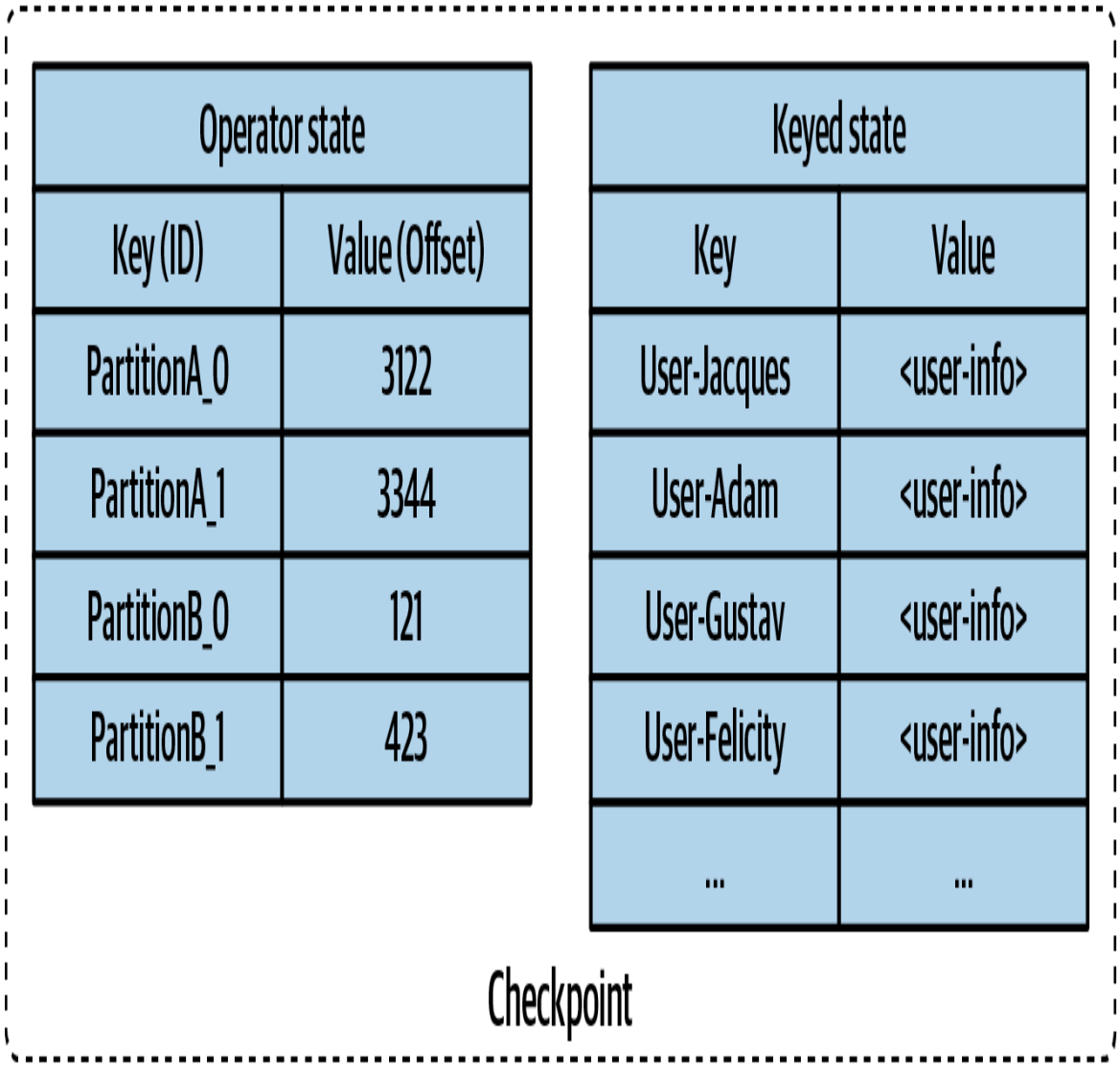


Figure 10-6. A checkpoint with operator and key state

> **TIP**
>
> Restoring from a checkpointed state is functionally equivalent to using snapshots to restore external state stores, as covered in "Recovery Using Snapshots or Checkpoints".

The state associated with the application task must be completely loaded from the checkpoint before you can process any new data. The heavyweight framework must also verify that the operator state and the associated keyed state match for each task, ensuring the correct assignment of partitions among tasks. Each of the major heavyweight frameworks discussed at the start of

this chapter implements checkpoints in its own way, so check the documentation.

Scaling Applications and Handling Event Stream Partitions

The maximum parallelism of a heavyweight application is typically constrained by the input stream partition counts. It can only read data into its framework as quickly as it can iterate through the input streams, particularly as it processes the data in the order that it is written. Because heavyweight processing frameworks are particularly well suited for computing massive amounts of user-generated data, it is quite common to see cyclical patterns with significant computational requirements during the day and very few in the middle of the night. An example of a daily cyclical pattern is shown in Figure 10-7.

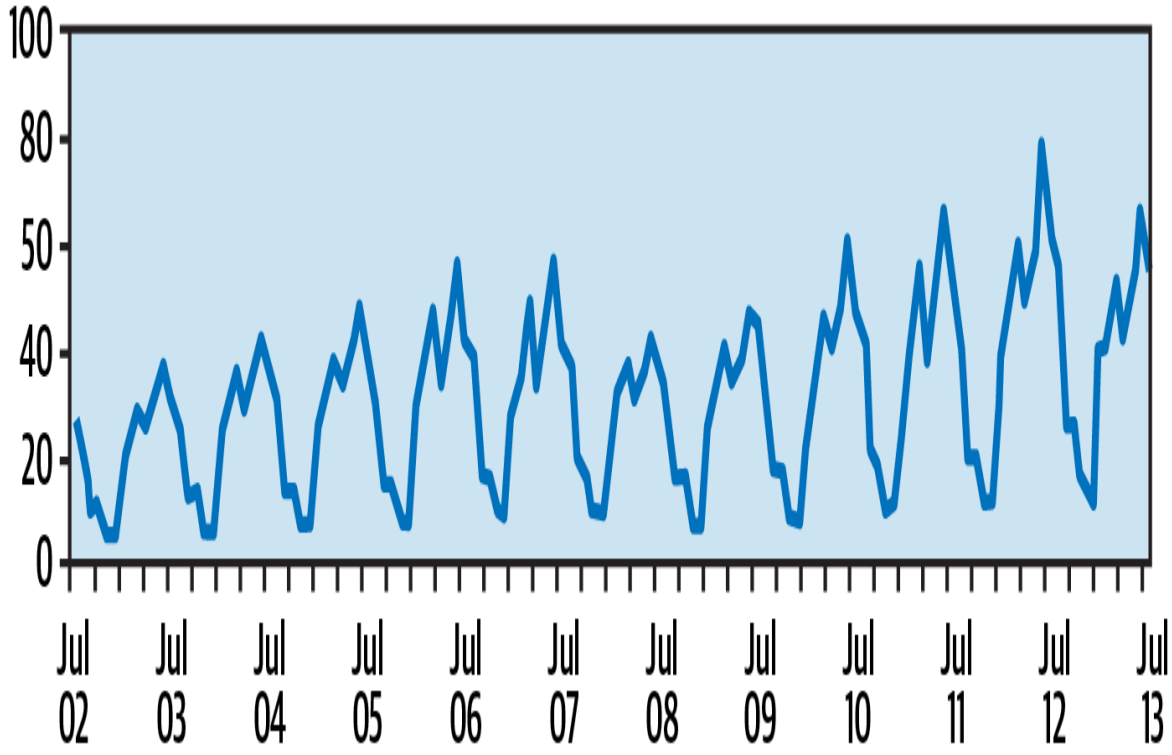


Figure 10-7. Sample of daily cyclical data volume

Applications that process such data benefit greatly from the ability to scale up with increasing demand and down with decreasing demand. Proper scaling can ensure that the application has sufficient capacity to process all events in a timely manner, without wasting resources by overprovisioning. Ideally, the latency between when an event is received and when it is fully processed should be minimized, though many applications are not that sensitive to temporarily increased latency.

> **WARNING**
>
> Scaling an application is separate from scaling a cluster. All scaling discussed here assumes that there are sufficient cluster resources to increase parallelism for the application. Refer to your framework's documentation for scaling of cluster resources.

Stateless streaming applications are very easily scaled up or down. New processing resources for an application can simply join or leave the consumer group, upon which resources are rebalanced and streaming is resumed. Stateful applications can be more difficult to handle; not only does state need to be loaded into the workers assigned to the application, but the loaded state needs to match the input event stream partition assignments.

There are two main strategies for scaling stateful applications, and while the specifics vary depending on the technology, they share a common goal of minimizing application downtime.

Scaling an Application by Restarting It

The first strategy is quite simple. You pause the application, checkpoint any state, and then stop it. Next, you restart the application with the new parallelism, reloading the stateful data from the checkpoints before resuming processing from where it was left off. Scaling by restart is supported by all heavyweight streaming frameworks, and speaks to the roots of these technologies in batch-based analytical processing.

But why must the application be paused, checkpointed, and restarted to scale up or down? There are a two main problems that make dynamic scaling difficult, but not impossible (as covered in the next section):

- *State redistribution* to the scaled instances is essential for correctness, but it takes time to store the data from one instance and reload it in another.
- *Data shuffling* is another factor, as scaling down a node that is in the middle of shuffling data can cause data loss. Additionally, adding a node without changing the shuffler's destination at precisely the right moment can also cause data loss or duplication of data.

Figure 10-8 shows a regular shuffle, where each downstream reduce operation sources its shuffled events from the upstream `groupByKey` operations. If one of the instances were abruptly terminated, the reduce nodes would no longer know where to source the shuffled events from, leading to a fatal exception. Thus, you need to pause the application, let the shuffles complete, and checkpoint the state to maintain an accurate accounting of the application's progress.

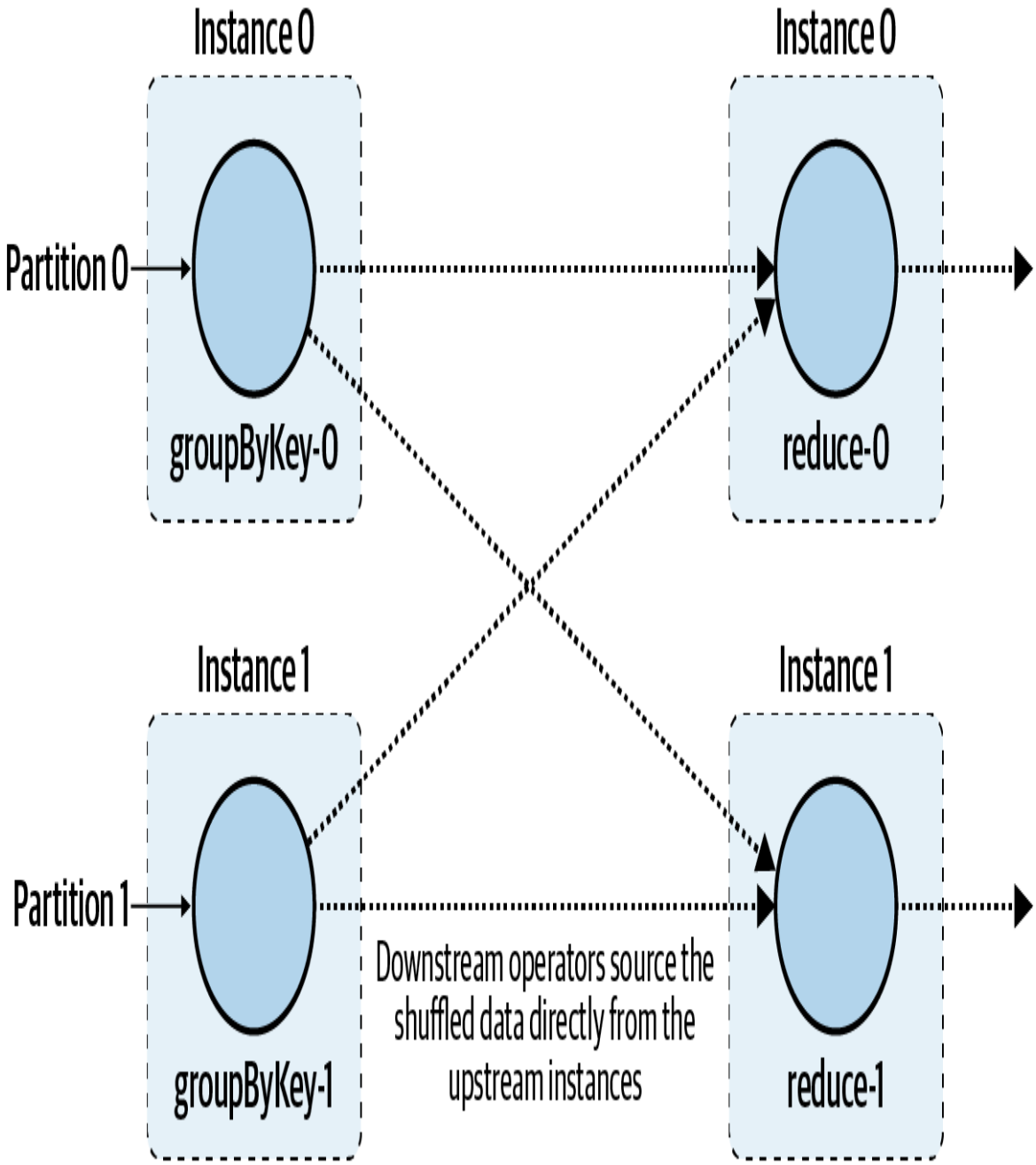


Figure 10-8. Logical representation of a regular shuffle

Scaling by restart is a safe and simple way to change your application's parallelism. You'll need to consider if the downtime is acceptable, particularly as the downtime will relate to the size of your checkpointed state. Note that scaling by restart is also an excellent way to validate your disaster recovery setup, since it's effectively equivalent to recovering your application from the last known good state.

The good news is that frameworks like Flink and Spark also offer the means to scale your application while it is running.

Scaling an Application While It Is Running

This second strategy allows you to remove, add, or reassign application instances without stopping the application. It is available only in some heavyweight streaming frameworks, as it requires careful handling of both state and shuffled events. Just like in scale by restart, adding and removing instances requires redistributing assigned stream partitions and reloading state from the last checkpoint.

The exact mechanism of scaling while running varies from framework to framework. For example, Spark's dynamic resource allocation allows for dynamic scaling while the application is running. However, it requires using coarse-grained mode for cluster deployment and using an *external shuffle service* (ESS) as an isolation layer.

The ESS receives the shuffled events from the upstream tasks and stores them for consumption by the downstream tasks, as shown in Figure 10-9. The downstream consumers access the events by asking the ESS for the data that is assigned to them.

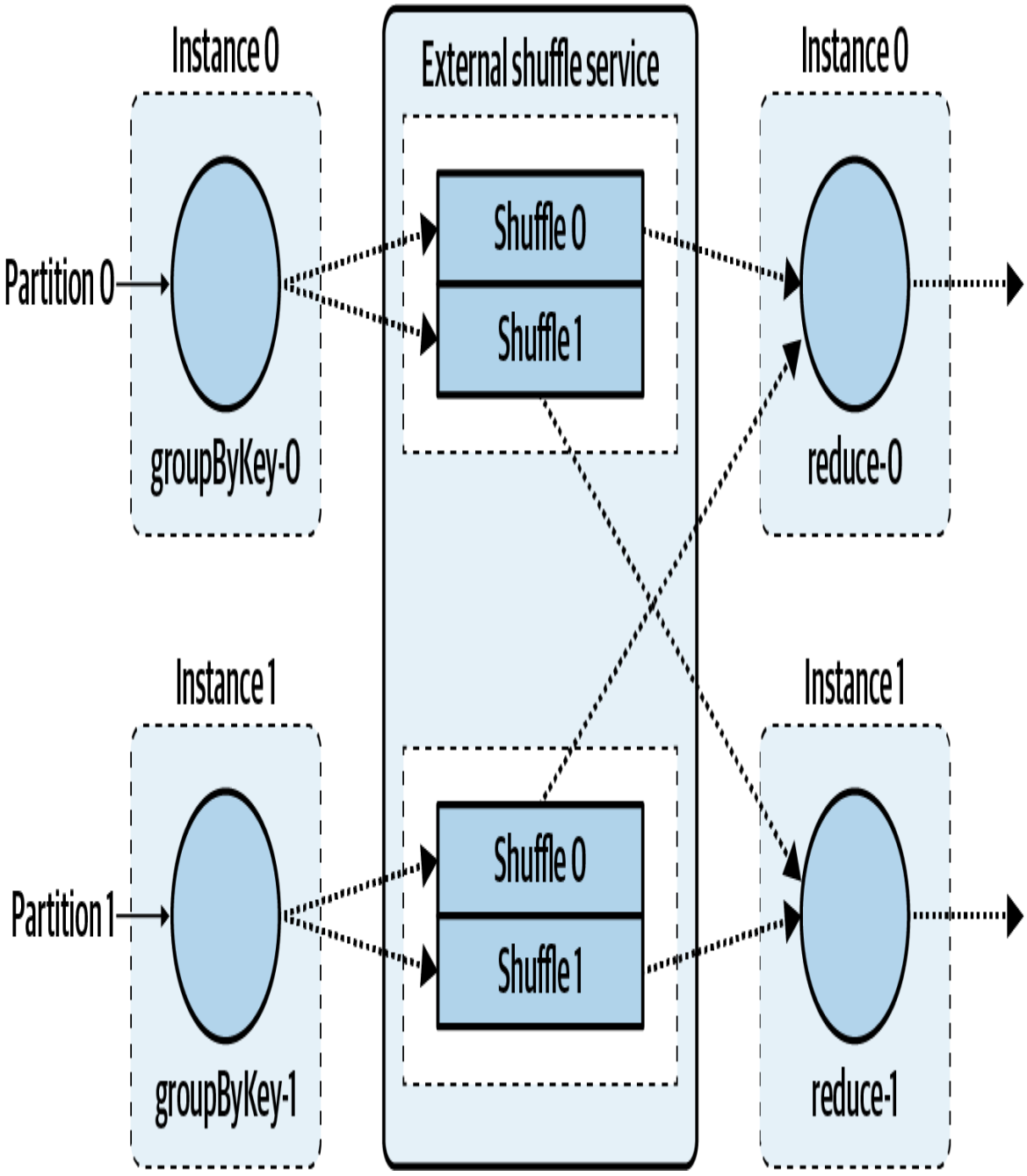


Figure 10-9. Logical representation of a shuffle using an external shuffle service

Task instances can be scaled down on demand since the downstream operations are no longer dependent on a specific upstream instance. The shuffled data remains within the ESS, and a scaled-down service, as shown in Figure 10-10, can resume processing. In this example, instance 0 is the only remaining processor and takes on both partitions, while the downstream operations seamlessly continue processing via the interface with the ESS.

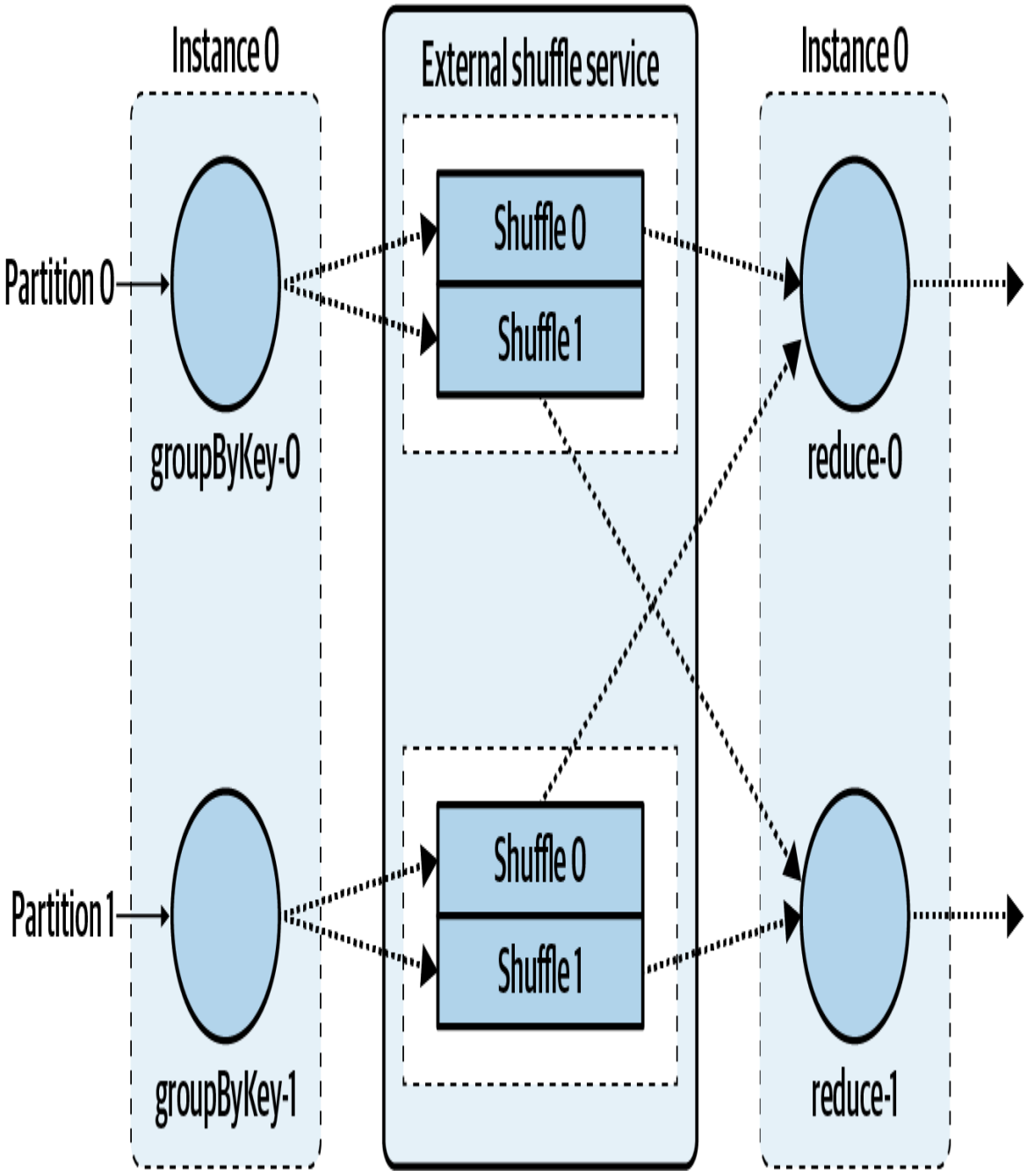


Figure 10-10. Downscaled application using an external shuffle service (note instance 1 is gone)

Spark has also provided dynamic scaling functionality *without* the use of an ESS, via *shuffle tracking* and via *shuffle block decomissioning*. The former works by tracking the stages that generate shuffle files, and keeping executors that generate that data alive while downstream jobs that use them are still active. The latter relies on the block manager to migrate the shuffled blocks

(the data) during a graceful shutdown of the instance, to provide continuity for the services.

Apache Flink provides an *adaptive scheduler* that permits the dynamic scaling of Flink applications. It relies on Flink's *declarative resource management* that abstracts resources from the job, and streamlines up- and down-scaling of resources.

Active autoscaling for live applications remain an area of active development for both Spark and Flink. This is an area that will continue to see more development, especially as data volumes and the demand for event-driven processing increase. Resource efficiency, low latency, and accurate and consistent results require reliable autoscaling mechanisms.

Autoscaling Applications

Autoscaling is the process of automatically scaling applications in response to specific metrics. These metrics may include processing latency, consumer lag, memory usage, and CPU usage, to name a few. Some frameworks may have autoscaling options built in, such as Google's Dataflow engine and Spark Streaming's dynamic allocation functionality. Others may require you to collect your own performance and resource utilization metrics and wire them up to the scaling mechanism of your framework, such as the lag monitor tooling discussed in [[Link to Come]].

Recovering from Failures

Heavyweight clusters are designed to be highly tolerant to the inevitable failures of long-running jobs. Failures of the master nodes, worker nodes, and Zookeeper nodes (if applicable) can all be mitigated to allow applications to continue virtually uninterrupted. These fault-tolerance features are built into the cluster framework, but can require you to configure additional steps when deploying your cluster.

In the case of a worker node failure, the tasks that were being executed on that node are moved to another available worker. Any required internal state

is reloaded from the most recent checkpoint along with the partition assignments. Master node failures should be transparent to applications already being executed, but depending on your cluster's configuration you may be unable to deploy new jobs during a master node outage. High-availability mode backed by Zookeeper (or similar technology) can mitigate the loss of a master node.

> **TIP**
>
> Make sure you have proper monitoring and alerting for your master and worker nodes. While a single cluster node failure won't necessarily halt processing, it can still degrade performance and prevent applications from recovering from successive failures.

Multitenancy Considerations

Aside from the overhead of cluster management, you must account for multitenancy issues as the number of applications on a given cluster grows. Specifically, you should consider the priority of resource acquisition, the ratio of spare to committed resources, and the rate at which applications can claim resources (i.e., scaling). For instance, a new streaming application starting from the beginning of time on its input topics may request and acquire the majority of free cluster resources, restricting any currently running applications from acquiring their own. This can cause applications to miss their service-level objectives (SLOs) and create downstream business issues.

Here are a couple of methods to mitigate these challenges:

Run multiple smaller clusters

> Each team or business unit can have its own cluster, and these can be kept fully separate from one another. This approach works best when you can requisition clusters programmatically to keep operational overhead low, either through in-house development work or by using a third-party service provider. This approach may incur higher

financial costs due to the overhead of running a cluster, both in terms of coordinating nodes (e.g., master and Zookeeper nodes) and monitoring/managing the clusters.

Namespacing

A single cluster can be divided into namespaces with specific resource allocation. Each team or business group can be assigned its own resources within their own namespace. Applications executed within that namespace can acquire only those resources, preventing them from starving applications outside of the namespace through aggressive acquisition. A downside to this option is that spare resources must be allocated to each namespace even when they're not needed, potentially leading to a larger fragmented pool of unused resources.

Languages and Syntax

Heavyweight stream processing frameworks are rooted in the JVM languages of their predecessors, with Java being the most common, followed by Scala. Python is also commonly represented, as it is a popular language among data scientists and machine learning specialists, who make up a large portion of these frameworks' traditional users. MapReduce-style APIs are commonly used, where operations are chained together as immutable operations on data sets. Heavyweight frameworks are fairly restrictive in the languages their APIs support.

Choosing a Framework

Choosing a heavyweight stream-processing framework is much like selecting a CMS and event broker. You must determine how much operational overhead your organization is willing to authorize, and if that support is sufficient for running a full production cluster at scale. This overhead

includes regular operational duties such as monitoring, scaling, troubleshooting, debugging, and assigning costs, all of which are peripheral to implementing and deploying the actual applications.

Software service providers may offer these platforms as a service, though the options tend to be more limited than selecting providers for your CMS and event broker. Evaluate the options available to you and choose accordingly.

Lastly, the popularity of a framework will inform your decision. Apache Flink is one of the leading streaming frameworks, and has seen significant growth and adoption since the first version of this book was released. Spark's structured streaming is also quite popular for building streaming applications. Apache Beam is also popular, but primarily as a means of running on Google's Dataflow. It hasn't seen the same adoption and usage growth that Spark and Flink have.

> **WARNING**
>
> Keep in mind that a heavyweight streaming framework is not reasonably capable of implementing *all* event-driven microservices. Verify that it is the correct solution for your problem space before committing to it.

Summary

This chapter introduced heavyweight stream processing frameworks, including a brief history of their development and the problems they were created to help solve. These systems are highly scalable and allow you to process streams according to a variety of analytical patterns, but they may not be sufficient for the requirements of some stateful event-driven microservice application patterns.

Heavyweight frameworks operate using centralized resource clusters, which may require additional operational overhead, monitoring, and coordination to integrate successfully into a microservice framework. Recent innovations in cluster and application deployment models have provided better integration with container management solutions such as Kubernetes, allowing for more

granular deployment of heavyweight stream processors similar to that of fully independent microservices.

In the next chapter, we'll shift gears just a bit to look at the lightweight frameworks, and how they different from the heavyweight options for building event-driven microservices.

Chapter 11. Lightweight Framework Microservices

> **A NOTE FOR EARLY RELEASE READERS**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 13th chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

Lightweight frameworks differ from heavyweight frameworks in that they do not require a dedicated processing and resource management cluster. Instead, they rely solely on the event broker and the container management system (CMS) to scale, manage state, and recover from failures.

Historically speaking, there have been only a few lightweight frameworks that caught on enough to mention here. Apache Kafka Streams and Apache Samza were the only two lightweight streaming frameworks identified in the first edition of this book. In the intervening years, it has become clear that Kafka Streams remains the only one in widespread common use, particularly as it remains a core component of the Apache Kafka project. Apache Samza's usage has diminished, as has its release cycle frequency, and I as a consequence I wouldn't recommend starting a new project with it over Kafka Streams.

Like Samza, other lightweight frameworks have emerged and faded away over the years. The open source BSD licensed python Faust framework is

another example. It isn't that there is nobody who uses these frameworks nowadays, it's simply that the critical social mass to keep them up to date with event broker evolutions and changes is no longer present. The reality is that building a full-featured, durable, reliable, and scalable framework is *hard*, and when there are other off-the-shelf options available, it may simply be too hard to rally enough developers and users to your cause.

This chapter explores lightweight frameworks with a particular slant towards Apache Kafka Streams. It remains not only the biggest lightweight streaming framework in use today, but perhaps the only one that will still remain in the developer toolkit in yet another five years, thanks to its embedding into the core Apache Kafka project.

First, let's look at an example.

Example: Joining Products with Brand Data on a Foreign Key

Consider an example where you've set up a connector to pull both the brand and product data from your ecommerce database, as shown in Figure 11-1.

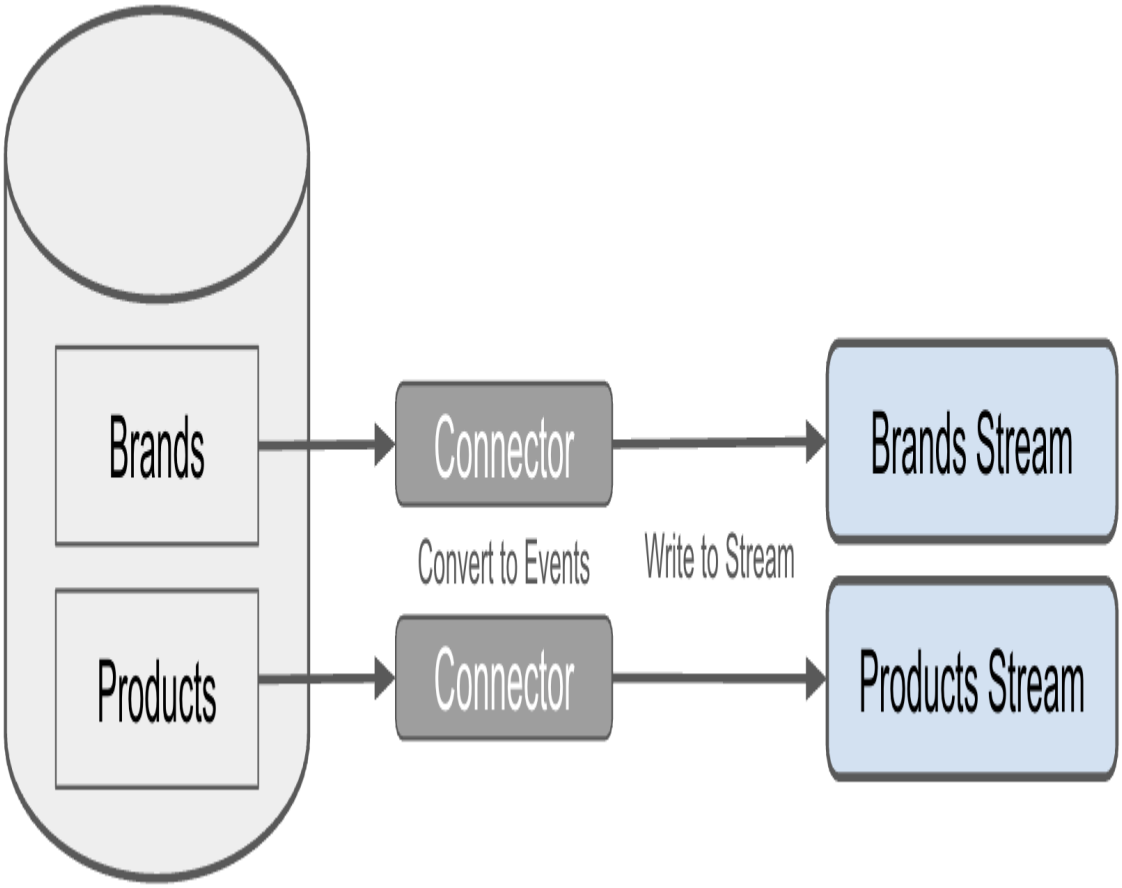


Figure 11-1. Creating brand and product event streams from a database source via connectors

The product information contains everything about the product - what it is, a plain text description, the price, and a foreign-key reference to the brand (e.g. `brandId`). Meanwhile, the brand information contains things like the official brand name and a link to its logo for use in advertising.

The lightweight Kafka Streams framework provides the means to join the brand and merchant data together on the `brandId` foreign key, producing an `EnrichedProduct` stream by denormalizing with the brand data. A slightly-concatenated version of the code can be seen in Example 11-1.

Example 11-1. A Kafka Streams microservice joining and enriching `Product` with `Brand` data

```
public Topology buildTopology(Properties envProps) {
    // Configuration code not shown for brevity
```

```
    // 1) Create the Kafka topic Serializer/Deserializers, aka Serdes
    BrandSerde brandSerde = new BrandSerde(...);
    ProductSerde productSerde = new ProductSerde(...);
    EnrichedProductSerde enrichedProductSerde = new EnrichedProductSerde(...);

    // 2) Create the stream builder
    KStreamBuilder builder = new KStreamBuilder();

    // 3) Declare two KTables.
    KTable<Long, Brand> brandTable =
        builder.table(Serdes.Long(), brandSerde, "brand-topic");
    KTable<Long, Product> productTable =
        builder.table(Serdes.Long(), productSerde, "product-topic");

    // 4) Declare the join, providing the joiner function
   productTable
     .join( brandTable,
            Product::getBrandId,
            new ProductToBrandJoiner())

    // 5) Convert back to a stream and write to the output
     .toStream()
     .to("enriched-product-topic",
         Produced.with(Serdes.Long(), enrichedProductSerde));

   // 6) Build and return the declared streaming topology
    return builder.build();
}
```

The code example first shows the declaration of the serdes (1). These are the serializer / deserializers that convert the byte array records from Kafka into Avro, Protobuf, or JSON Schema objects. These are then mapped to plain old Java objects, at least for this Kafka Streams example. Brand information is mapped to `Brand` objects while product information is mapped to `Product` objects.

Next, the code declares the stream builder (2), which is used to declare the streaming topology of the application. In step (3), two `KTables` are declared, one for holding `Brand` information materialized from the `brand-topic`, with the other being a `Product` table containing the materialization of the `product-topic`. Both are declared using the stream builder.

In step (4), the `productTable` is joined with the `brandTable` in a table-to-table join. The `.join` function declares the join, including the `ProductToBrandJoiner` as shown in Example 11-2. It specifies how to join the two entities together by extending the ValueJoiner class

Example 11-2. The ProductToBrandJoiner definition used by the joiner service to populate the EnrichedEcomItem

```
public class ProductToBrandJoiner implements
    ValueJoiner<Product, Brand, EnrichedProduct> {
    public EnrichedProduct apply(Product p, Brand b) {
        return EnrichedProduct
                .newBuilder()
                .setId(p.getId())
                .setName(p.getName())
                .setDescription(p.getDescription())
                .setPrice(p.getPrice())
                .setBrandName(b.getBrandName())
                .setBrandLogoURI(b.getBrandLogoURI())
                .build();
    }
}
```

Step (5) converts the results of the join back into a stream, which is then written out to a Kafka topic named `enriched-product-topic`. The `enrichedProductSerde` converts the data from plain-old Java objects to byte array entries for writing to Kafka.

Finally, step (6) tells the Kafka Streams application to build the topology, which is returned to the calling class which then sets up the topology and begins streaming data.

If you'd like to see more about foreign-key joins for Kafka streams, or about joins in general, check out the official documentation.

Lightweight frameworks offer stream processing features comparable to heavyweight frameworks, They tend to be of interest to those who want the more advanced features of heavyweight frameworks, without having to invest into managing and running their own clusters. For example, access to declarative streaming topologies, table materializations, foreign- and

primary-key joins, groupings, windowings, and aggregations are all features lacking from a basic producer/consumer.

Under the Hood of Lightweight Frameworks

Lightweight frameworks are defined by their lightweight deployment model. You can deploy lightweight microservices like any other event-driven application by relying on native CMS functions, increasing and decreasing parallelism by simply adding or removing instances.

Lightweight framework microservices do not rely on a secondary resource cluster residing somewhere else (like Apache Flink and Apache Spark) to provide them with durable state storage, failure recovery, and scalability. Instead, they rely primarily on the event broker, and to a lesser extent the CMS.

Event Shuffling and Repartitioning

Event shuffling in lightweight framework microservices relies on the internal event streams hosted by the event broker. The producer shuffles events to their corresponding partitions using an internal event stream, whereupon they are consumed by the downstream processing instances of the same application.

Data of the same key must be local to a given processing instance for any key-based operations, such as a `join`, or a `groupByKey` and `reduce`/`aggregation`. These shuffles involve sending the events through an internal event stream, with each event of a given key written into a single partition (see "Copartitioning Event Streams").

The internal event stream acts as an external shuffle service similar to the ESS described in heavyweight frameworks ("Scaling an Application While It Is Running"). Figure 11-2 illustrates the basic lightweight model, including an internal event stream used to shuffle data between instances.

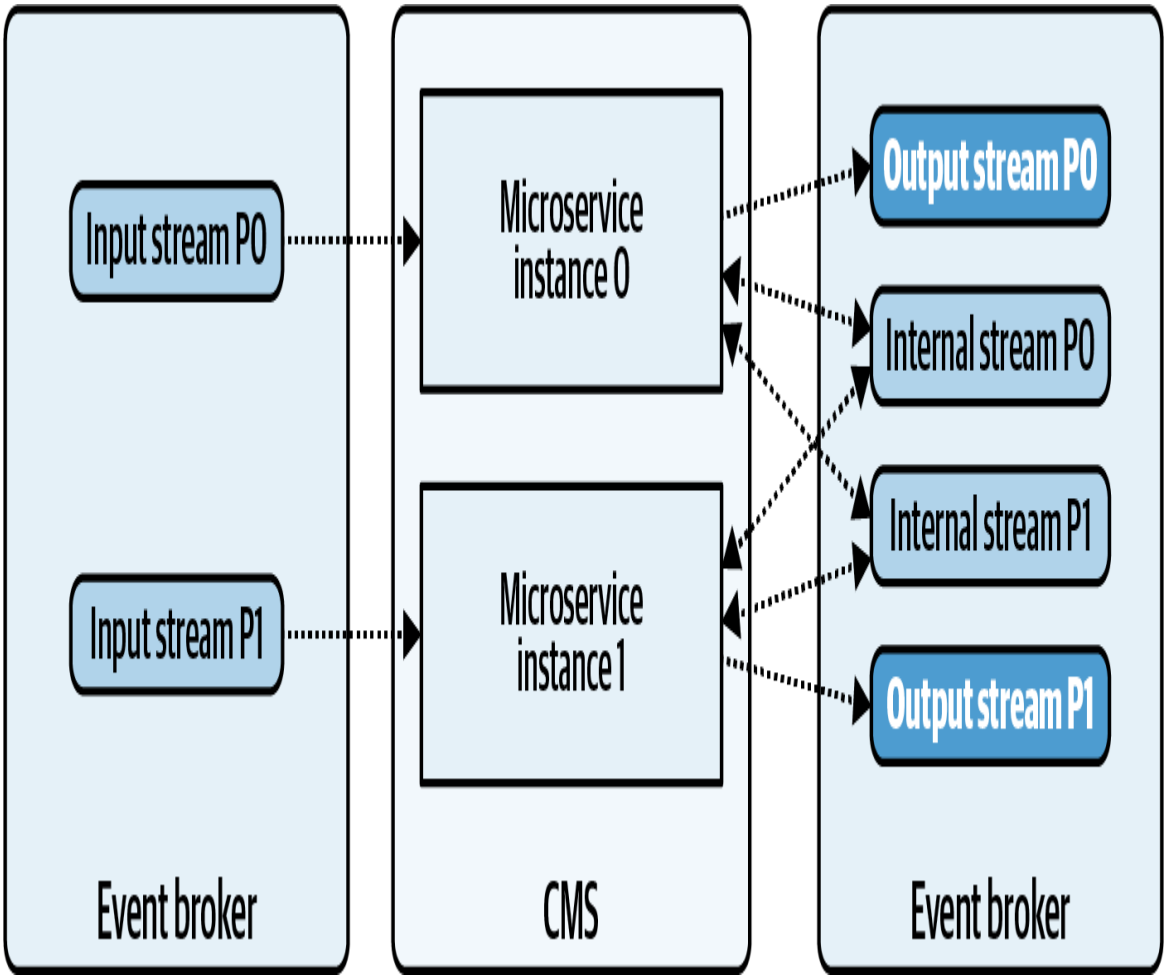


Figure 11-2. Internal event streams for repartitioning and shuffling data

> **CAUTION**
>
> Internal streams are private and are not used by any other microservice. Do not couple microservices together on internal streams.

Relying on internal event streams for event shuffling enables the lightweight application to scale dynamically since any in-flight events are durably retained to the event broker. The biggest tradeoff is an increase in latency, which is unavoidable as there aren't any other event shuffle options. Kafka Streams, for example, is bound entirely to Apache Kafka.

Handling State and Using Changelogs

In-memory and disk-based key-value stores are the most common form of state store for the lightweight framework, particularly since the entire domain-specific language (DSL) of lightweight frameworks is based on manipulating and storing keys and values. Lightweight frameworks, by default, use internal state stores backed by changelog streams stored in the event broker (see more in "Materializing State to an Internal State Store").

> **TIP**
>
> Since every lightweight application is fully independent of the others, one application could request to run on instances with very high-performance local disk, while another could request to run on instances with extremely large, albeit perhaps much slower, hard-disk drives.

You can extend your lightweight frameworks to plug in different storage modes, provided you implement the appropriate storage interface (for example, Kafka Streams StateStore Interface). Implementing your own state store allows you to use external state stores and alternative storage and querying models. Instead of using an internal key-value store, for example, you could choose to Amazon's DynamoDB as the backing state store instead.

Scaling and Recovering from Failures

Scaling a microservice and recovering from a failure are effectively the same thing. Adding an application instance, due to intentional scaling of a long-running process or due to a failed instance recovering, requires partition assignment and corresponding state assignment. Similarly, removing an instance, deliberately or due to failure, requires reassigning the partitions and state to another live instance to continue processing uninterrupted.

One of the main benefits of the lightweight framework model is that applications can dynamically scale. There is no need to restart an application just to change parallelism, though there may be a delay in processing due to consumer group rebalancing and rematerialization of state from the changelog. Figure 11-3 illustrates the process of scaling an application up.

The assigned input partitions are rebalanced (including any internal streams) and the state is restored from the changelogs prior to continuation of work.

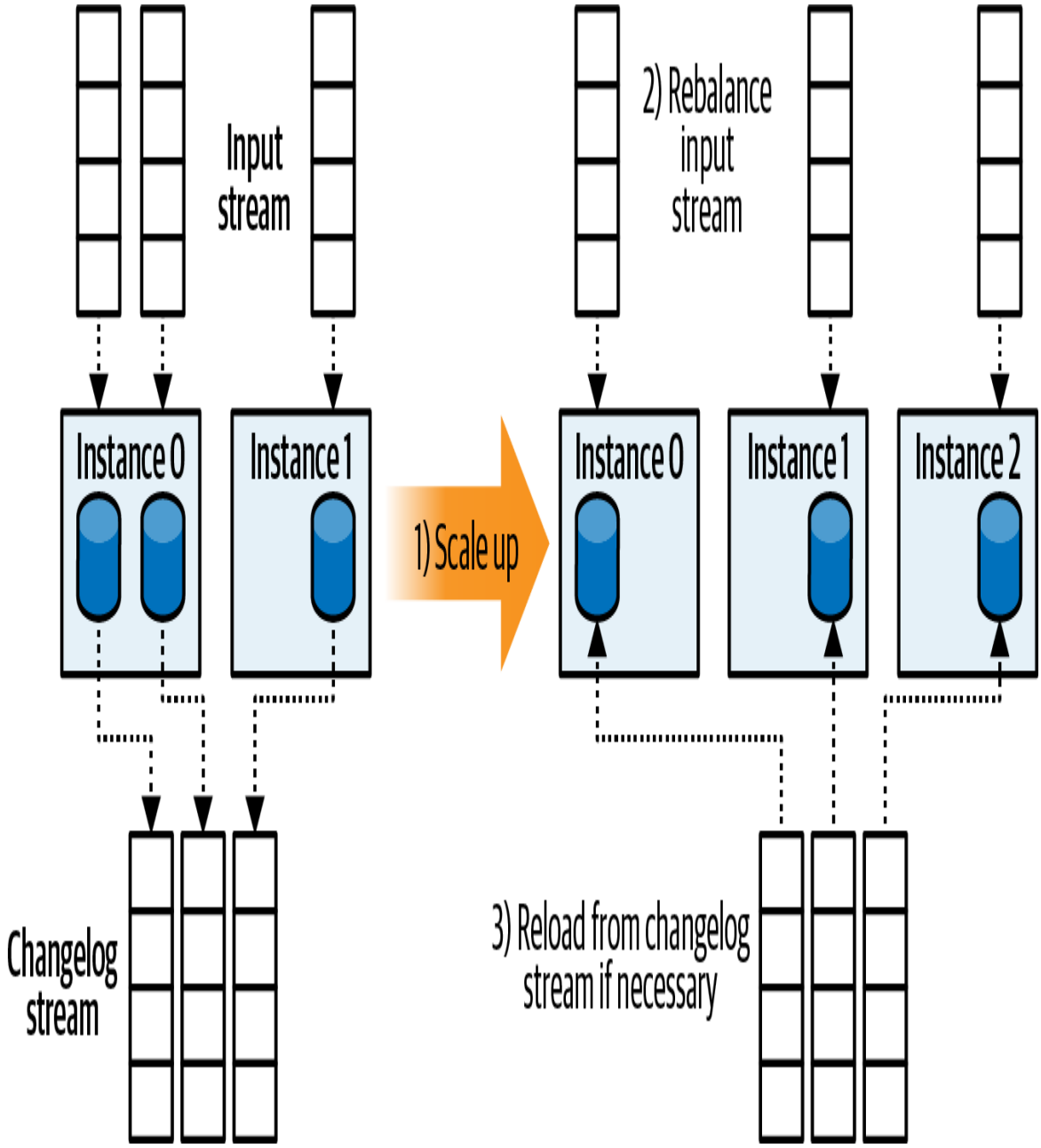


Figure 11-3. Scaling up a lightweight microservice

State Assignment

An instance with new internal state assignments must load the data from the changelog before processing any new events, similar to how checkpoints are

loaded from durable storage in heavyweight solutions. The operator state (the mappings of `<partitionId, offset>`) for all event stream partitions, both input and internal, is stored within the consumer group for the individual application. The keyed state (pairs of `<key, state>`) is stored within the changelog for each state store in the application.

When reloading from a changelog, the application instance must prioritize consumption and loading of all internal stateful data prior to processing any new events. This is the state restoration phase, and any processing of events before state is fully restored risks creating nondeterministic results. Once state has been fully restored for each state store within the application topology, consumption of both input and internal streams may be safely resumed.

State Replication and Hot Replicas

A hot replica, as introduced in "Using hot replicas", is a copy of a state store materialized off of the changelog. It provides a standby fallback for when the primary instance serving that data fails, but can also be used to gracefully scale down stateful applications. When an instance is terminated and a consumer group is rebalanced, partitions can be assigned to leverage the hot replica's state and continue processing without interruption. Hot replicas allow you to maintain high availability during scaling and failures, but they do come at the cost of additional disk and processor usage.

Similarly, you can use hot replicas to seamlessly scale up the instance count without having to suffer through processing pauses due to state rematerialization on the new node. One of the current issues facing lightweight frameworks is that scaling an instance up typically follows the current workflow:

1. Start a new instance.
2. Join the consumer group and rebalance partition ownership.
3. Pause while state is materialized from the changelog (this can take some time).

4. Resume processing.

One option is to populate a replica of the state on the new instance, wait until it's caught up to the head of the changelog, and then rebalance to assign it ownership of the input partitions. This mode reduces outages due to materializing the changelog streams and only requires extra bandwidth from the event broker to do so. This functionality is currently under development for Kafka Streams.

Summary

This chapter introduced lightweight stream processing frameworks, including their major benefits and tradeoffs. Lightweight frameworks do not require their own independent resource cluster, but instead rely on the event broker and the CMS to provide the necessary scaling, shuffling, and data durability functionality.

Kafka Streams is a highly scalable processing frameworks that relies extensively on integration with the event broker to perform large-scale data processing. It remains the single most popular lightweight framework still in use today. In contrast, Apache Samza used to be an alternative option, but its popularity, usage, and project activity has dwindled since the first edition of this book was released.

In the next chapter, we'll take a look at going even further up the abstraction layer by writing microservices using streaming SQL.

Chapter 12. Streaming SQL

> **A NOTE FOR EARLY RELEASE READERS**
>
> With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.
>
> This will be the 14th chapter of the final book. Please note that the GitHub repo will be made active later on.
>
> If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

Streaming SQL is a *declarative* mechanism for building microservice-like queries. It's a handy way to reduce the overhead on getting started with stream processing, and is accessible to those of you who are familiar with plain old SQL. You tell the SQL API what it is you want the outcome to look like, and it does all the work to implement it using the underlying engine.

Streaming SQL is most commonly implemented as a layer on top of a full-featured streaming framework, as introduced in "The Streaming SQL Query". As examples, both Apache Flink and Apache Spark provide their own streaming SQL layers on top of their lower level APIs. Flink provides Flink SQL, which Apache Spark provides Spark SQL (in conjunction with its structured streaming product). While there are other projects available out there that offer SQL APIs on top of streaming frameworks, this chapter focuses solely on these two open-source Apache projects.

That being said, the principles of the streaming SQL remain largely the same regardless of the engine or framework you choose. The syntax and some of the lower-level details may differ, but the general idea remains the same: A

SQL-syntax stream processing API that lets you avoid the deeper abstraction layers of the streaming framework.

It's important to point out that there is no definitive SQL streaming standard at the moment (ANSI or otherwise). It's a bit of a wild west, with different frameworks offering differing syntaxes and semantics from one another. When we talk of streaming SQL then, it's important to note that we're talking about dialects in general - the syntax of one framework won't necessarily match the syntax of the other. But they're also not going to be that wildly different, and there's plenty of supporting documentation to help clear things up if you get stuck.

The Basics of the Continuous Query

SQL is most commonly associated with table-based operations, and for good reason. SQL databases have been around for decades, and you're likely to have written your first SQL queries against a table in a relational database. Streaming SQL is simply a repurposing for use with event streams, with streams-as-tables forming the basic data abstraction.

Figure 12-1 shows an example of a streaming SQL query, where all events with a value less than or equal to 5 are filtered out. The remaining events are returned by the SQL query into the next stage, which can include either another SQL query, a table materialization, or even writing out to another event stream.

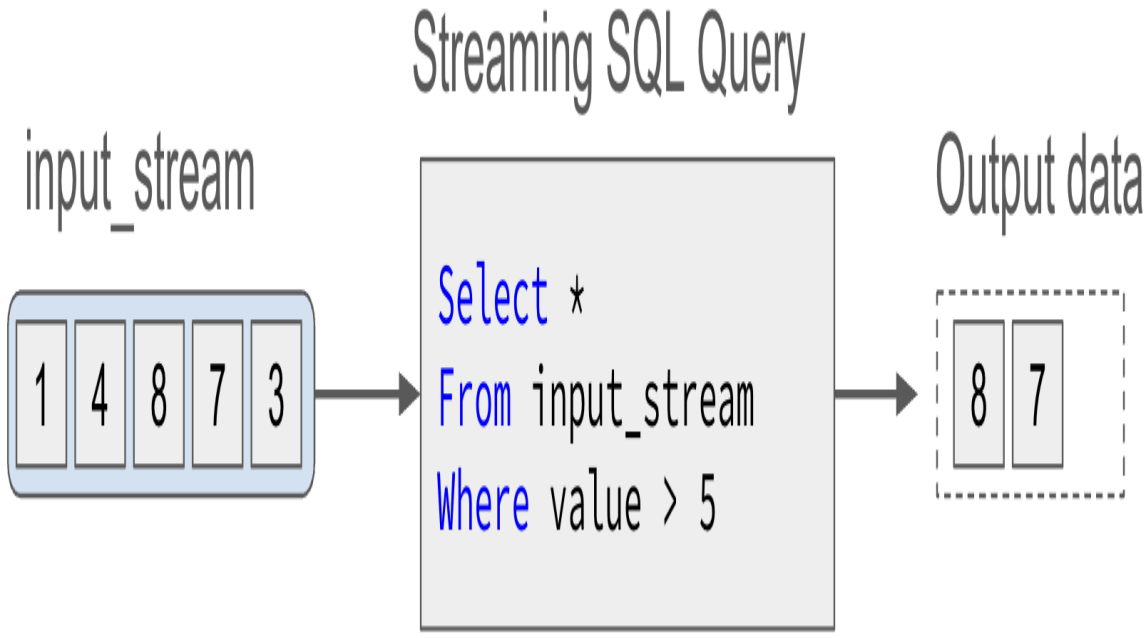


Figure 12-1. A simple SQL query filtering out events with a value less than or equal to 5

Streaming SQL queries run indefinitely on the stream of data, applying the query logic to each event as it's received. In contrast, a query on a conventional relational database operates on the data that was present at the time it was executed. While a streaming query runs indefinitely, a conventional query will terminate once it has iterated across its bounded data set.

Streaming SQL is, however, quite similar to conventional queries in that they both operate on *tables*. While Figure 12-1 shows a valid SQL query, it omitted the necessary step of declaring a table on top of the stream.

Turning Streams into Tables for Streaming SQL Queries

The first step in working with streaming SQL is to turn your stream into a table. You create the tables by *declaring* a definition of the table on top of the streaming data, which you can then write your queries against.

There are three types of tables that you can expect to see in a streaming SQL framework:

Upsert Tables

> These require a primary key. Each event is an upsert or a delete such that a new record of the same key overwrites an old record of the same key. Upsert tables are identical to a materialized table as described in "Materializing State from Entity Events".

Append-Only Tables

> Does not require a primary key. Append-only tables insert every record, maintaining a complete history of all previous data. All records are immutable and cannot be deleted, which makes an append-only table very much like the append-only log backing the event stream itself.

Retraction Tables

> A slightly more advanced type that does not necessarily appear in all streaming frameworks. A retraction table supports inserts, updates, and deletions, but with the entire row acting as a key. For example, you insert a row into the table via a `+I["Gary", 123]` event, but remove it using a `+D["Gary",123]` event. Retraction tables primarily reflect the results of upstream computations, such as when emitting events from a Flink dynamic table

```
Check your documentation to ensure your SQL streaming framework supports both append
and upsert tables, at the least. Some frameworks are limited to only supporting
append-only tables, which will significantly limit your processing options.
```

Upsert and append-only table thypes are common between most streaming frameworks. Most tables declared off of event streams will either be upsert or append-only, and will form the basis for your streaming SQL queries.

Let's take a look at an enrichment example using Apache Flink SQL.

Example: Enriching Data Using Flink SQL

First, declare a table over top of the event stream. For example, here is a Flink SQL declaration of a `TABLE` from a Kafka topic named `Photographs`:

```
CREATE TABLE Photographs (
  id BIGINT PRIMARY KEY NOT ENFORCED,
  photographer_id BIGINT NOT NULL,
  camera_id BIGINT NOT NULL,
  photo_url STRING NOT NULL,
  height INT,
  width INT
) WITH (
  'connector' = 'upsert-kafka',
  'topic' = 'Photographs',
  'key.format' = 'raw',
  'value.format' = 'json',
  'properties.bootstrap.servers' = 'broker.url:9092',
  'properties.group.id' = 'Photographs_group_id'
);
```

The streaming framework that underpins the SQL interface interprets this table declaration. It first creates a `Photographs` table by registering the table declaration metadata into a *metadata catalog*, allowing other streaming SQL engines to also query the data. The table uses the upsert-kafka connector to generate the materialized table, where new events are upserted into the table (any old values of the same key are overwritten).

> **TIP**
>
> The primary key in the Photographs table is set to `NOT ENFORCED` because it does not own the data. The enforcement of the key is the responsibility of the producer service that is writing to the Photographs topic.

Metadata catalog compatibility may vary from streaming SQL engine to streaming SQL engine. Flink provides support for several catalog types,

including both JDBC-based and Apache Hive, a very popular and common catalog implementation. Meanwhile, Apache Spark provides both in-memory (local) catalog and Hive support. If you're running streaming SQL jobs in production, you're likely to need either Hive or a Hive-compatible catalog for maintaining durable metadata, to allow your queries to see all available tables.

Back to the example. Create another table to represent the photographers who took the photographs:

```
CREATE TABLE Photographers (
  kafka_key_id BIGINT PRIMARY KEY NOT ENFORCED,
  first_name STRING,
  last_name STRING,
  signup_date TIMESTAMP WITH TIME ZONE
) WITH (
  'connector' = 'upsert-kafka',
  'topic' = 'Photographers',
  'properties.bootstrap.servers' = 'broker.url:9092',
  'properties.group.id' = 'Photographers_group_id',
  'key.format' = 'raw',
  'key.fields' = 'kafka_key_id',
  -- Schema registry URL for deserializing the Avro value
  'value.avro-confluent.url' = 'http://schema.registry.url:8082',
  'value.format' = 'avro-confluent',
  'value.fields-include' = 'EXCEPT_KEY',
);
```

You may have noticed some differences from the `Photographs` table. This table uses the `avro-confluent` value format instead of a plain old JSON value, and specifies the URL for the Confluent Kafka schema registry (as covered in "The Role of the Schema Registry"). In short, the Flink table job automatically deserializes the Avro data from the Kafka topic using schema information from the schema registry, converting it into plain old Java objects for Flink to process.

The next step creates an `Enriched_Photographs` table to store photograph data joined with the photographer's information:

```
CREATE TABLE Enriched_Photographs (
  id BIGINT PRIMARY KEY NOT ENFORCED,
  photographer_id BIGINT NOT NULL,
  photo_url STRING NOT NULL,
  height INT,
  width INT,
  first_name STRING,
  last_name STRING
) WITH (
  --Omitting some of the table details for brevity.
  --It's similar to the previous examples
  connector' = 'upsert-kafka',
  'topic' = 'Enriched_Photographs',
  ...
)
```

Finally, join the photographs with their respective photographers by using an INNER join on the `photographer_id` foreign key:

```
INSERT INTO Enriched_Photographs
 SELECT pics.id,
        pics.photographer_id,
        pics.camera_id,
        pics.photo_url,
        pics.height,
        pics.width,
        people.first_name,
        people.last_name,
 FROM Photographs as pics
 INNER JOIN Photographers as people
 WHERE pics.photographer_id = people.kafka_key_id
```

The joined results are combined into a single row per photograph, and emitted to the `Enriched_Photographs` topic. Note that the photograph and photographer information is retained indefinitely within the Flink job, mirroring the key-space of the source topics. Any tombstones issued on either source topic will cause deletion of the corresponding photographs, which will also propagate through to the `Enriched_Photographs`.

While SQL tasks are relatively easy enough to write, this chapter hasn't yet covered anything on how the code itself is actually deployed. Let's take a look at that now.

Executing Streaming SQL Code

There several ways to execute your streaming SQL code.

Executing SQL Code From The Command Line Interface

The command line interace (CLI) is one option for executing your code, as provided by both Flink SQL and Spark SQL. The most common use of the CLI is for testing and local development work, though you could plug it into the production clusters (security issues notwithstanding).

Executing Flink's `./bin/sql-client.sh` or Spark's `./bin/spark-sql` brings up a command line prompt for you to write (or copy and paste) your SQL code. If you wanted to find my information, you would do write something like this:

```
flink-2.0.0 ./bin/sql-client.sh
Flink SQL>
> SELECT *
> FROM Photographers
> WHERE first_name = "Adam"
> AND last_name = "Bellemare";
```

Executing a streaming SQL query the CLI requires that it remain running indefinitely. If you terminate the CLI, you terminate the job, though you could of course run it in a container like any other microservice. However, you may want to look at the next modes for better production options.

Executing SQL Code From Within The Application

You can also embed the SQL query code right into the native application code. For example, if you were to declare and query the `Photographs` Flink Table through a Java application, the main function would contain code that looks like the following:

```
// Create the Flink table environment.
TableEnvironment tableEnv = TableEnvironment.create(...);
```

```
// Specify the external Hive catalog and choose the database
tableEnv.useCatalog("hive_ext");
tableEnv.useDatabase("photo_db");

// Create the Photographs table using SQL DDL.
// The table metadata is written into the Hive catalog
tableEnv.executeSql(
    "CREATE TABLE Photographs (id BIGINT ...) WITH (...)");

// Query the Photographs table using SQL DDL
tableEnv.executeSql(
    "SELECT * from Photographs where first_name = "Alice");
```

Indeed, you are limited to the languages supported by the framework (Java, Python, etc). The benefit is that you can still get away with far less coding in it by programming most of the business logic with embedded SQL queries. A templated code generator to can create all the overhead boilerplate and dependency inclusions, so that you can spend your efforts primarily on writing the SQL code.

Embedding SQL in the application dovetails well with microservice deployment pipelines, including testing and operational needs like scaling and monitoring.

Executing SQL Code From A Notebook

A notebook is an interactive computing device that allows you to combine executable code, rich text, equations, visualizations, and other components. Data analysts, data scientists, and data engineers have historically been the primary users of notebooks, particularly in the analytics domain. Many of the original notebook use cases centered around data exploration and transformations, though deployment of notebooks into production data pipelines soon followed.

Examples of popular free open-source notebook frameworks for writing streaming code include Apache Zeppelin and Jupyter. Meanwhile,

Notebooks have proven to be extremely popular for exploring data, building data transformations, and executing code without having to build an actual application. There are several core benefits to notebooks:

Interactive development

Users can write, execute, and modify code in small chunks (called cells) and see the results immediately. This is particularly useful for data exploration and visualization.

Visualizations and documentation

Users can combine code with narrative text (using Markdown), equations (using LaTeX), and visualizations. This makes it easier to document the logic and process of data analysis or computational research.

Container-like isolation

You can include code, environment details, and even hard-coded data to allow your notebook to run in other deployments and environments.

Multi-language support

Notebook frameworks can support multiple languages, including Python, Java, Scala, and R.

Shareable

You can download and share notebooks just like any other file.

> **TIP**
>
> Notebooks are not just for SQL. You can write many types of event-driven microservices inside a notebook using non-SQL languages.

Interpeters provide interoperability between the notebook and the underlying processing engine. For examples, the Apache Zeppelin interpreter for Flink.

Depending on your framework, you may need to install interpreters separately, or they may be integrated into the notebook engine itself.

There are also some drawbacks to using notebooks:

Version control complexity

> Notebooks can be difficult to integrate with version control systems like Git due to large file sizes and binary data. However, many provide plugins to improve the experience, such as Jupyter's nbdime or Jupytext.

Performance limitations

> For large-scale data processing tasks, notebooks might not be the most efficient solution. They may struggle to provide the necessary scale for you event-driven microservice.

Testing limitations

> Notebooks are not well suited for unit and integration testing. For business critical applications, you should extract the logic into a dedicated microservice project complete with its own unit and integration testing frameworks.

There are also some notebook-like options with stripped down functionality, which are generally limited to hosted solutions. The code is still embedded into a notebook-like UI, but you cannot download the notebook and are generally limited to just a single language. These notebook-like solutions are primarily to act as a graphic user interface for submitting code to a SQL Gateway, as covered in the next section.

Executing SQL Code Via a Gateway

A SQL gateway is a service that enables multiple clients to execute their SQL code concurrently and remotely. The client submits the job to the gateway, where it is then executed within its own session. Multiple independent sessions can run within the service, relying on the processing

cluster for executing the streaming operations. Figure 12-2 shows two clients connecting to the gateway, one to submit a job and the other to get the status of an existing job.

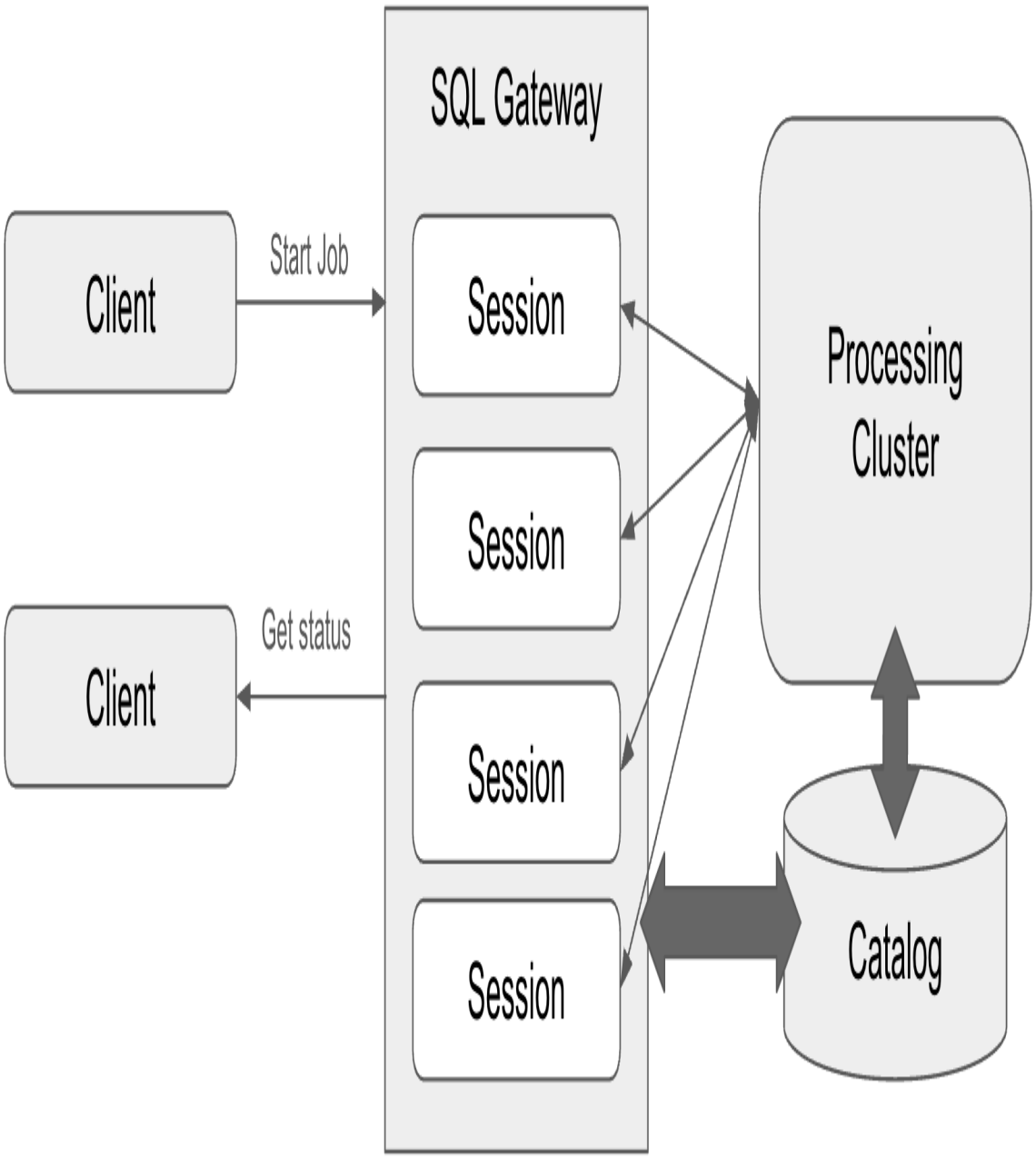


Figure 12-2. A SQL gateway for submitting, running, and evaluating jobs

Spark distributed SQL engine and Flink's SQL gateway are two examples of this option.

The main benefit of using a gateway is that you can cut out the overhead of creating an application for building your service. In fact, you can even use the CLI to submit the jobs directly to the gateway. Ideal candidates for gateway submission include simple SQL queries that won't change over time are, such as joiner services for denormalizing relational data.

SQL as a Sidecar

SQL can also provide powerful sidecar capabilities ("Integration with Existing Systems Using the Sidecar Pattern") to do work that may otherwise be too difficult to do in your microservice. For instance, many basic producer/consumer frameworks lack the abilities to do joins and other complex aggregations. SQL can easily fill that gap.

Figure 12-3 shows an example of a hybrid application, where a dedicated SQL query materializes and joins the streams together. The `Enriched_Photographs` topic created earlier in this project is joined with `Cameras` topic to bring in information relating to the camera used to take the photograph. This combined event is then written to a new topic for dedicated usage by the downstream microservice.

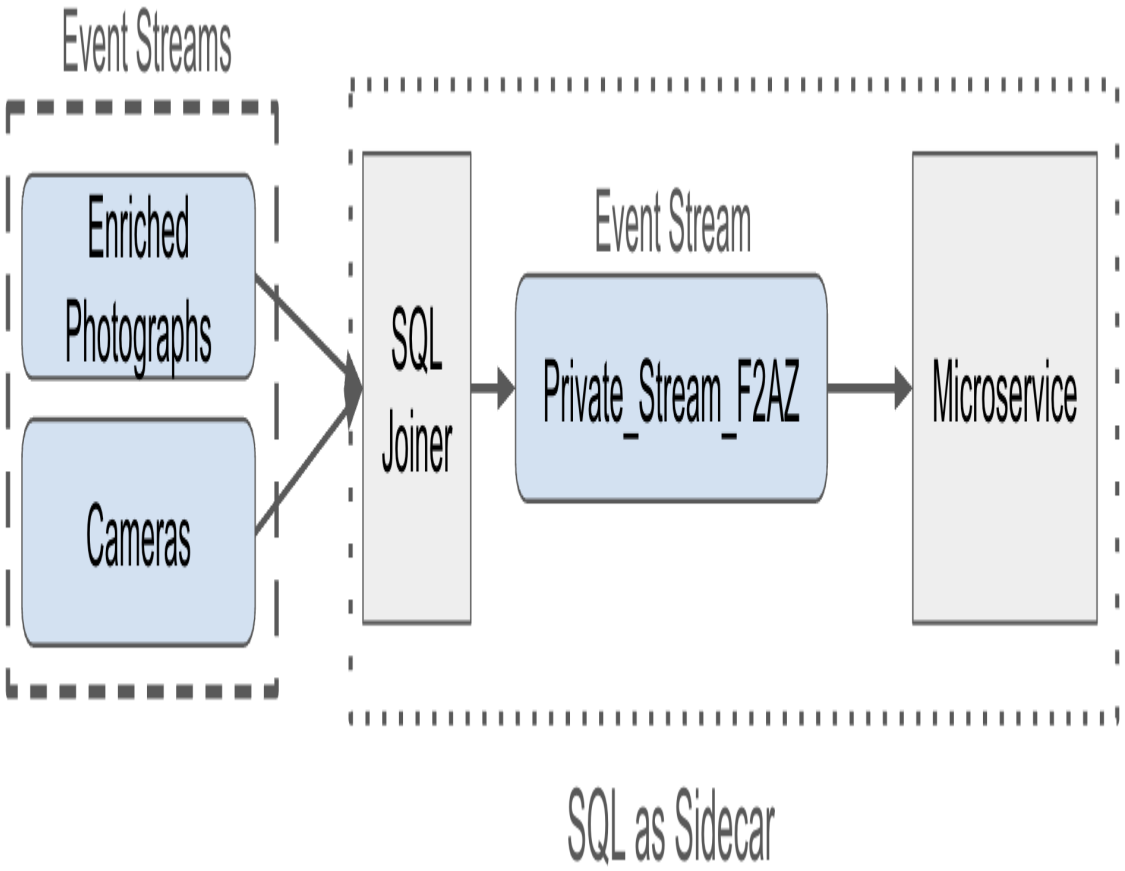


Figure 12-3. A SQL sidecar for performing operations that may not be possible for the microservice's framework

Notice that the output event stream is named `Private_Stream_F2AZ` - a private placeholder name. This is an example of a stream that is purpose-built explicitly for usage by that downstream microservice. The SQL joiner and the microservice are both within the single bounded context, and that stream, joiner, and service's internal boundary remains private to the outside world.

A SQL sidecar can give you the best of both worlds, allowing you to use whatever libraries, languages, and frameworks that are compatible with basic event consumption and production, while harnessing powerful stream processing frameworks. Joins, windowing, aggregations, and complex event-driven state machines are offloaded to the SQL engine, while the remaining business logic can reside inside the microservice.

> **WARNING**
>
> When terminating the microservice, ensure that the SQL job is also terminated to ensure that no zombine processes remain. It is important to keep the SQL query and microservice components tightly coupled throughout their lifecycle.

User-Defined Function Calls

Streaming SQL frameworks also allow you to make user-defined function (UDF) calls inline with your SQL code. A UDF is a custom function that provides an operation that isn't easily accomplished in SQL syntax, written in the native language of the underlying SQL framework. UDFs are supported by both Flink and Spark. Consider an example of a simple Flink UDF that appends `_modified` to a string value:

```
import org.apache.flink.table.functions.ScalarFunction;

public class AppendModifiedUDF extends ScalarFunction {
    public String eval(String input) {
        if (input == null) {
            return null;
        }
        // Append '_modified' to the input string
        return input + "_modified";
    }
}
```

This Flink code illustrates a scalar UDF, where one record in results in one record out. There are three main types of UDFs, primarily divided by the relationship between records in to records out:

- Input one record and return a one record as the result
- Input multiple record and return one aggregated record as the result
- Input one record and return multiple records as the result

You can use UDFs to not only modify values, but build aggregations and calculate complex state machines based on record data. You can also use

UDFs to make calls to external systems, allowing you to integrate with third party services and things like GenAI models.

Summary

Streaming SQL simplifies the building of microservice-like queries through a declarative mechanism familiar to those proficient in traditional SQL. By specifying the desired outcome, the SQL API undertakes the implementation using the underlying streaming engine, making stream processing more accessible and reducing the initial overhead. Both Apache Flink and Apache Spark offer streaming SQL capabilities on top of their lower-level APIs, serving as prominent examples of how streaming frameworks incorporate a SQL API to enable broader uptake.

The fundamental concept of streaming SQL revolves around treating streams as tables, where a continuous query runs indefinitely, applying logic to each incoming event. The transition from streams to tables is a critical step in streaming SQL, enabling familiar SQL operations on dynamically updating data. This approach supports various table types, including upsert and append-only tables, which cater to different data processing needs and constraints within a streaming context.

Streaming SQL code execution include command line interfaces, embedded SQL in applications, interactive notebooks, and submitting jobs to a SQL gateway. Each method has its own set of advantages, catering to different stages of development, from testing and local development to production deployments. Moreover, streaming SQL frameworks facilitate user-defined functions (UDFs), allowing custom logic beyond built-in SQL capabilities. This flexibility further bridges the gap between the straightforwardness of SQL and the complexity of stream processing, making it a powerful tool for building queries as part of a event-driven architecture.

Part IV. The Rest (to be classified)

Chapter 13. Eventual Consistency

A NOTE FOR EARLY RELEASE READERS

With Early Release ebooks, you get books in their earliest form—the author's raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 16th chapter of the final book. Please note that the GitHub repo will be made active later on.

If you'd like to be actively involved in reviewing and commenting on this draft, please reach out to the editor at ccorbin@oreilly.com.

NOTE

An earlier version of this chapter previously appeared in *Building an Event-Driven Data Mesh*.

Eventual consistency is often a primary concern with distributed systems. For event-streams, eventual consistency concerns tends to lie mostly in the consumer side, as each consumer reads and processes events at their own rate.

Fortunately for us, there are some who have been looking at, working on, and thinking about eventual consistency for quite a long time. Pat Helland is just such a person and has written an excellent piece that collates insights and opinions from numerous thought leaders on the subject.

> *Since Doug [Terry] coined the phrase eventual consistency in the Bayou paper in 1995, I was interested in his perspective. When he defined eventual consistency, it meant that for each object in a collection of objects, all the replicas of each object will eventually have the same value. Then, he said: "Yeah, I should have called it eventual convergence."*
>
> —Pat Helland

This quote is significant in that many often confuse *consistency* as something other than *convergence*. For event-driven systems, we're very interested in ensuring that services are convergent, though they may never actually be consistent since they each consume and process events at their own rate.

Helland goes on to discuss a definition by Peter Alvaro, from his 2015 PhD thesis "Data-Centric Programming for Distributed Systems":

> *A system is convergent or "eventually consistent" if, when all messages have been delivered, all replicas agree on the set of stored values.*
>
> —Peter Alvaro

Both Terry and Alvaro converged (ha!) on the same definition of eventual consistency, putting the focus on independent replicas eventually converging on the same set of stored values. We'll keep using the *eventual consistency* terminology, but keep in mind we're really talking about *convergence of data.*

A consumer that is continually materializing an event stream can easily provide you information on what data it *does have* in its data store, but it can't tell you what it *doesn't have*. Because the data store is continually converging, it may tell you it doesn't have a piece of data when queried, but then immediately receive and process that data in the very next clock cycle. However, a consumer can at least tell you if it is *caught up* to a *given offset*, and we can use this knowledge when resolving questions about convergence with other systems.

Many of the questions and concerns regarding eventual consistency in the event-driven world stem from a concern that bad things will happen because

of it. It's often used as a threatening term, listed in the cons section of an architectural comparison article. But it doesn't have to be a con, because there are only a few big things to watch out for.

There are two main reasons why two services may have not yet converged:

A consumer service is lagging on event consumption

> A service is lagging behind on its consumption, processing, and storage of the data in comparison to another service.

The producer service is lagging on event production

> A producer service may encounter a bug or an error that prevents it from writing new updates to the event stream. In this case, the internal state of the producer service is up to date (converged), but the external representation written to the event stream is not. Consumers of this stream may have converged with each other, but will still lack convergence with the producer since the data is simply missing from the stream.

Note that these two options are not mutually exclusive. It is entirely possible that one or more consumers' services are lagging behind *and* the producer itself has failed to publish data to the stream.

The thing is, eventual consistency isn't as threatening as you may have been led to believe. The vast majority of time your event-driven services will be up to date with the latest events, unless it's trying to catch up from the beginning (such as a new or reset application).

One of the reasons people tend to be apprehensive about eventual consistency is that they're concerned that they may not know if their services are sufficiently converged to give accurate results. After all, it may be hard to predict when your services start to lag, and if they do, how to detect it and what to do about it.

The crux of the matter is that eventual consistency is mostly only an issue when one independent context issues a *direct query* to another independent context, ignorant to the state of their mutual convergence. Independent asynchronous event processing means that there are no guarantees that their internal data sets have converged, and so you can end up with wildly different answers depending on when these queries take place.

Let's take a deeper look at how contexts, event time, and boundaries relate to convergence, and how to avoid getting into problematic situations.

Converging on Consistency, One Event at a Time

Each event-driven processing instance effectively exists in its own time bubble, with its internal time based solely on the event timestamps that it has consumed and integrated into its state (see "Timestamps"). The vast majority of event streams provide data in an incrementing offset and timestamp order, though certainly the events can also be out of order (more on this later in the chapter). Thus, while the consumer service is free to look at the wall-clock time, its own internal time is based completely on the timestamps of the events that it has consumed.

Take Figure 13-1, which shows two independent consumers reading from a single event stream. Each service is fully independent in processing the events, applying business logic, and saving the data in state.

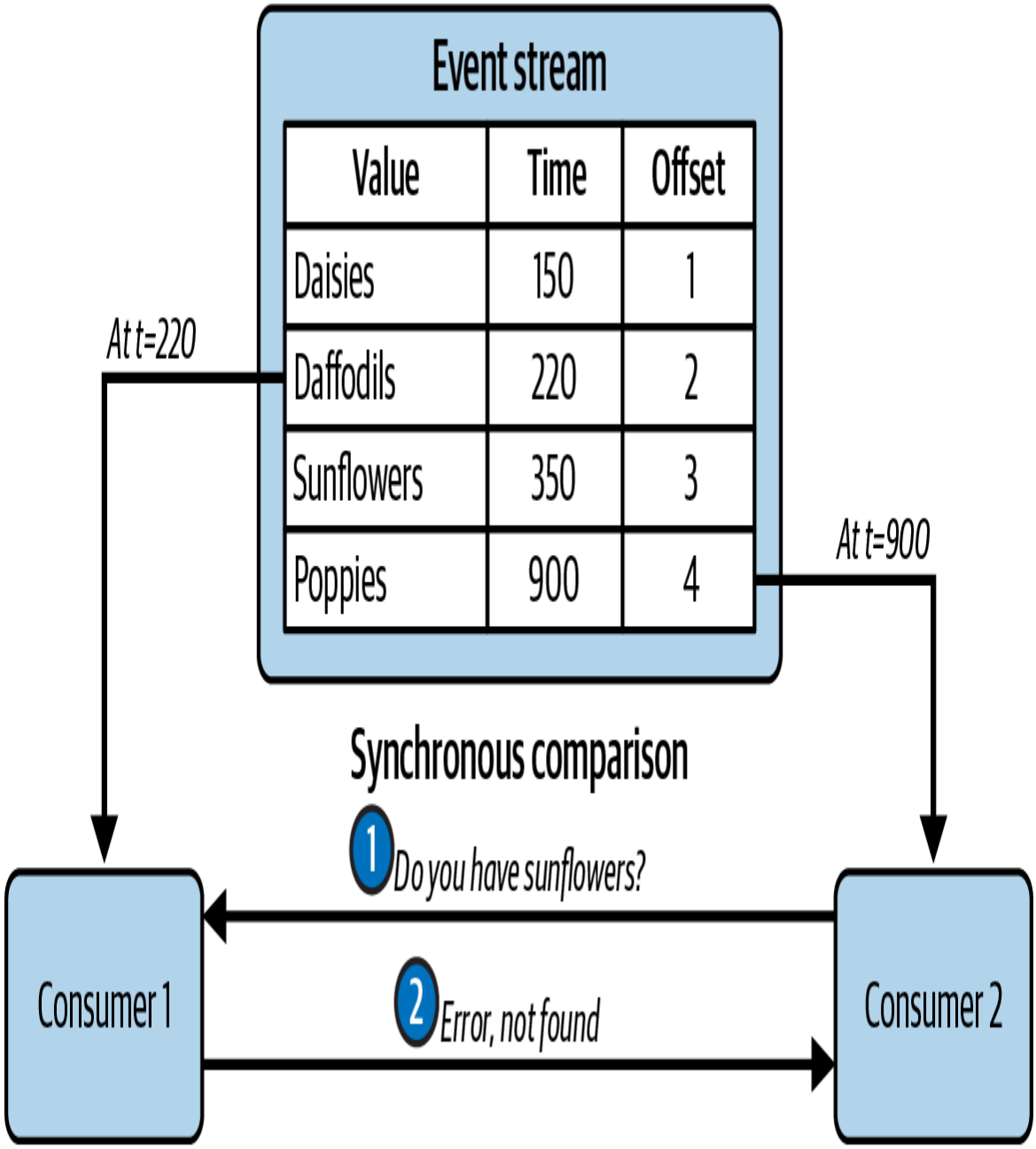


Figure 13-1. The basic problem with eventual consistency

The results that each service provides to the outside world, be it by request-response API, an output event stream, or other means, are very unlikely to be precisely synced with similarly materialized data in another service. This is where discrepancies and confusing results can creep in. Consumer 2 at time `t=900` asks Consumer 1 at `t=220` for its copy of `Sunflowers` data. Having never even heard of `Sunflowers`, all Consumer 1 can do is reply with `Error, not found`.

> **TIP**
>
> In addition to using offsets or incrementing event IDs, you may choose to use the event time, representing when the event occurred, to account for convergence. In cases where events are created via CDC, event time is typically defined as the time the data was created in the source database.

In contrast, think of two synchronous services that communicate over request-response APIs and that own and store all of their data within their own services. When one of these services issues an API call to another, it's not thinking, "I wonder what time it is over there." The assumption is that these two services are in the *same time bubble* and have the same wall-clock time or are close enough that we don't care.

This assumption is largely true, because a synchronous service doesn't typically rely on a stream of events to populate its state store. Rather, it handles or fails the requests immediately and returns the most up-to-date data that it has available, representing the current wall-clock time.

Let's take a look at a simple shipping and delivery company. In this model, a driver is an employee who can drive a truck. We need a driver for each truck we want to send out for deliveries, otherwise that vehicle isn't going anywhere. How would the assignment of a driver to a truck be affected in synchronous and asynchronous systems?

Figure 13-2 shows a set of synchronous services on the left residing in a single time bubble. All data is maintained in a fully consistent state, such that a point-in-time query will return a complete list of both trucks and drivers at that instant.

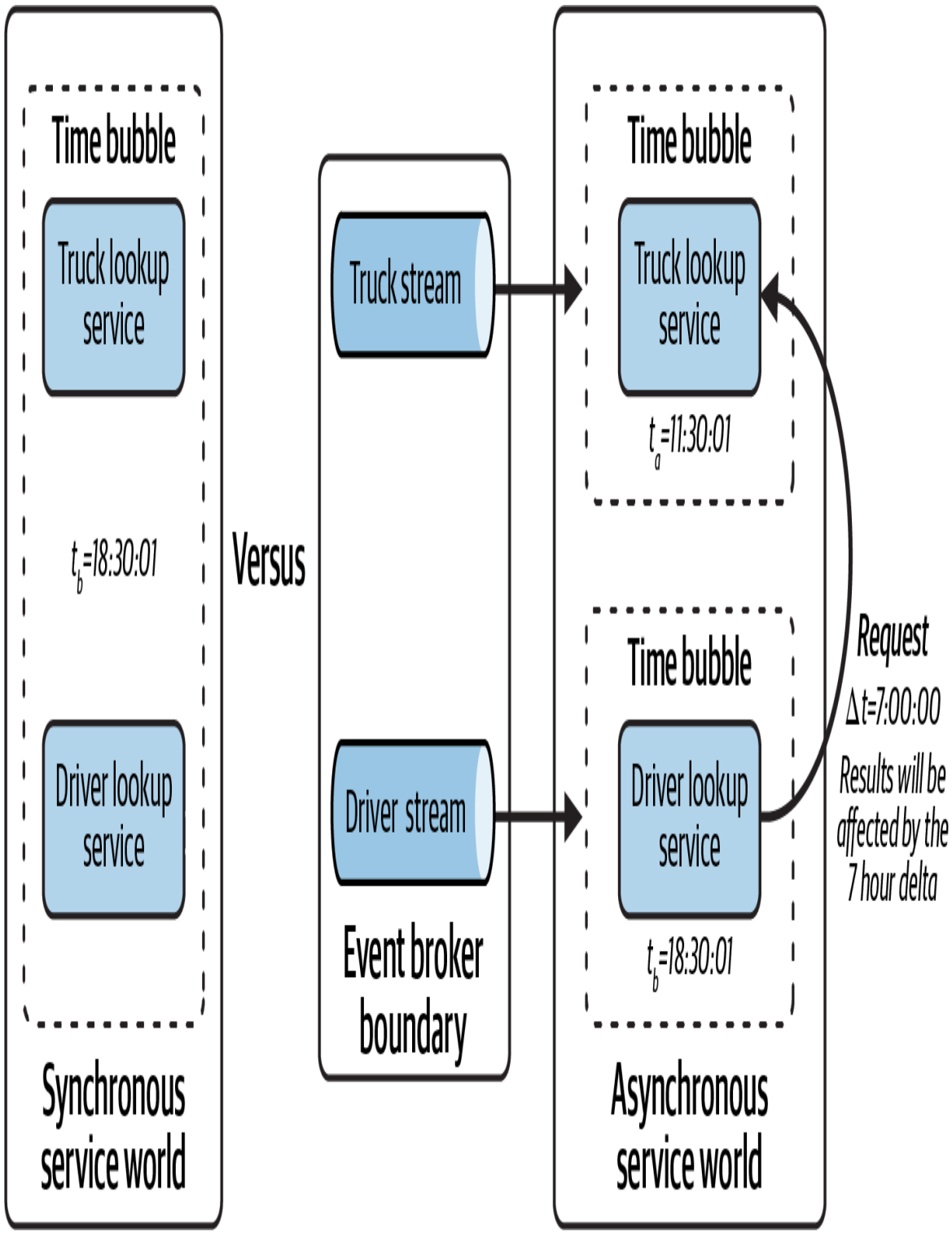


Figure 13-2. The internal time of a service is determined by the union of its input offsets

Meanwhile, the services on the right are sourced from event streams. In this example, notice that there is a time lag of 7 hours for the truck lookup service, as it has fallen behind in materializing its incoming events. If the

driver service asks the truck service to provide a new truck for assignment, it may not have any available, despite new trucks having been published to the event stream. The service must *catch up*; otherwise, it'll give nondeterministic results.

It may seem tempting to require that all drivers and trucks be registered together in a single atomic event. In fact, this is a good technique to use *when possible*, as it enforces a consistency within the event itself. However, it's not necessarily always possible for all business use cases. There are many professional drivers who do not own their own truck and there are companies that lease trucks but do not provide drivers. Each must be registered independently.

> **TIP**
>
> System times are largely synchronized to within milliseconds, thanks in large part to frequent synchronizations with Network Time Protocol servers (see "Synchronizing Distributed Timestamps"). If you require perfect time alignment between two events, you should refactor your domain to put the critical time-sensitive data into the same singular event. Otherwise, you'll have to plan to handle eventual consistency.

While a lagging service is one source of convergence issues, a second source is an event stream that is not yet updated despite all consumers being fully caught up. This is especially problematic when the events in one stream are related to the data in the other stream, such as by a foreign or primary key.

Let's do away with lag, latency, and processing time and just pretend for a moment that you have a service that can instantly consume and materialize any number of events, from any number of streams. In the case of Figure 13-3, we have a `Flower Pot Builder` service that is consuming and joining data from two streams to determine what's the best soil to put in the pot for each flower type.

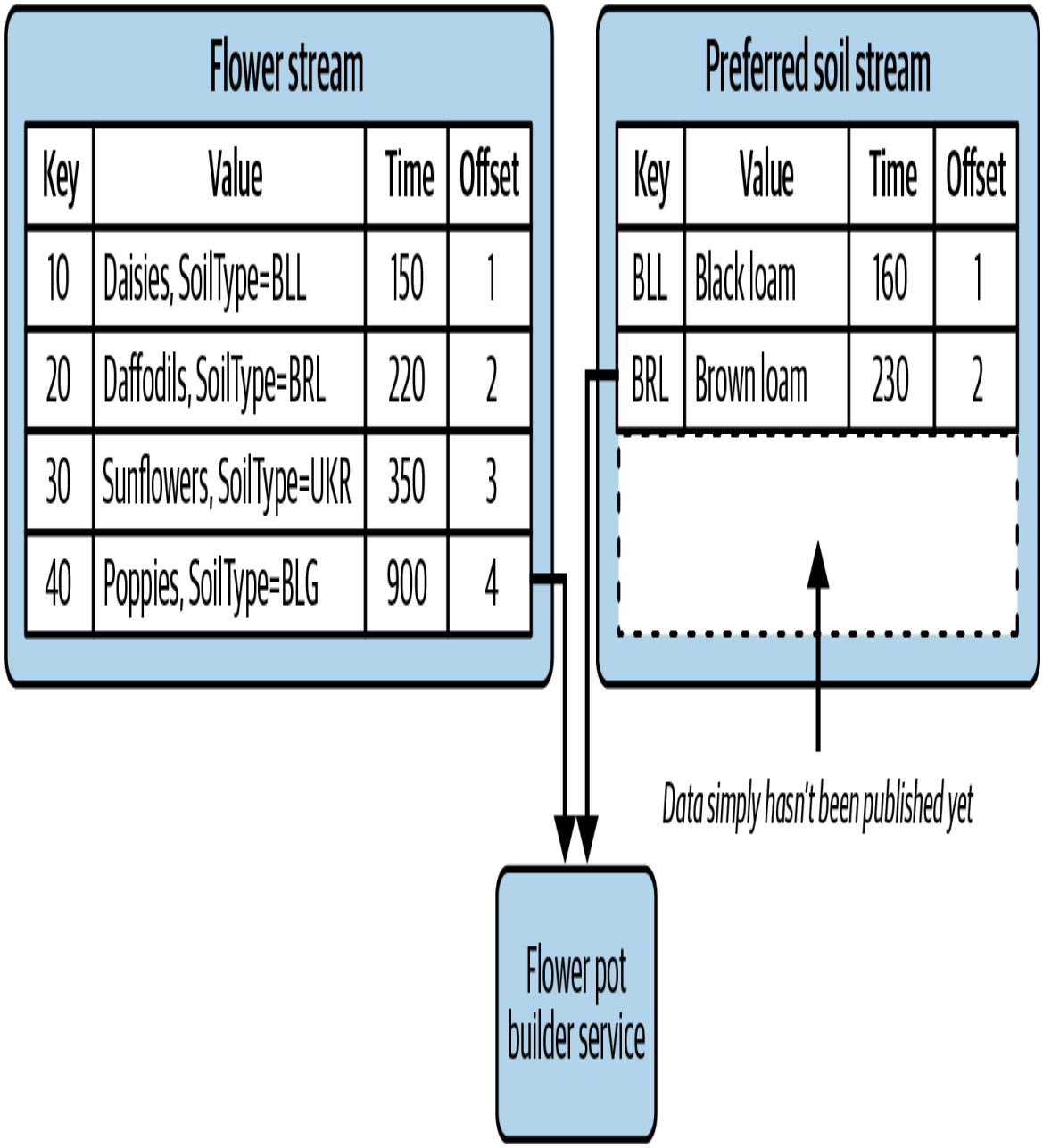


Figure 13-3. The data in the source event streams hasn't yet converged

Although the service is fully up to date with each event stream, there are no matching records in the `Preferred Soil` stream for `Sunflowers` and `Poppies`. The `Flower Pot Builder` service will need to wait until it can obtain the matching `Preferred Soil` events for those flowers. There are many reasons why these two streams may be out of sync: they may be sourced from different domains, they may have different SLAs, the producer

for `Preferred Soil` may be down, the network may be partitioned, or the event broker may be unavailable, to name just a few.

While it's possible that the data may simply not exist *at all, anywhere*, in many cases we expect data to exist based on certain business rules and properties. For example, `Preferred Soil` has a foreign key relationship with `Flowers`, so we can expect any `Flower` record with a populated `Soil Type` field to have a corresponding `Preferred Soil` record. The records for both `Sunflowers` and `Poppies` are missing, however. This data may yet show up, but as this example shows, even fully up-to-date consumers of the existing event streams may be inconsistent with upstream systems through no fault of their own.

The eventual consistency issues we face basically boil down to consumers that have not yet converged and event streams that have not yet converged with their source. Next, let's take a look at a few more detailed practical scenarios and some strategies for dealing with eventual consistency.

Strategies for Dealing with Eventual Consistency

You have two main options when dealing with eventual consistency, either between services or within a single service.

The first option is to simply wait for the state to become consistent, such as waiting for the event that completes the join, ends the session, or finalizes an aggregation. This option works equally well when querying an external service that gives you an inconsistent answer—you can simply wait and retry the query again at a later time. You may also choose to output an incomplete result that indicates a lack of consistency, but you will need to update it with the final results when you receive the appropriate data to act.

The second option is to give up after a certain period of time. *Giving up is final*. If the missing event you were waiting for shows up a split-second after you give up, it's still too late to do anything with it. If the server you were

querying finally has the result you need, it doesn't matter as you won't be notified or sent a follow-up request.

Prevent Failures to Avoid Inconsistency

There are a whole variety of technologies, clouds, brokers, connectors, and other types of tools that you may choose to build event-driven microservices. So instead of trying to unsuccessfully iterate through the entire list, let's just assume that there will be failures. These may include, but are not limited to:

- Services crashing
- Network connectivity issues
- Broker connectivity issues
- Cloud storage connectivity issues

You can reduce your chances of failure through good DevOps practices, monitoring, resource scaling, and testing. It's also important to ensure your consumer services have sufficient resources to scale up and stay up to date with the latest events and that any outages or failures on your producer side are quickly identified and fixed.

In short, anything you can do to keep your consumers and producers reading and writing without lag will contribute towards reducing the effects of eventual consistency.

Use State Event Types

Accidental divergence is always possible due to a misunderstanding of the events inside an event stream. Delta event types, for example, tend to be more challenging for the consumer to reconcile the data into a format that is consistent with other consumers. While it is still entirely possible to reach convergence via delta events, the added layer of complexity in reconciling delta events makes it just that much more likely that you'll end up in inconsistent states (See Chapter 4 for more details).

State events tend to be much simpler for consumers to read, process, and integrate into their state stores. Consumers are less likely to make mistakes in how they process and interpret the data, leading them to converge to the same results as other consumers.

Expose Eventual Consistency in the Server Response

You've probably seen this strategy employed before. Ever book a flight, a hotel, or rent a car online, and see the little spinning icon saying "Please wait to confirm, do not hit refresh"? Exposing the eventually consistent nature of a system is common practice in the world of UIs, and we can adopt this strategy for use in server-to-server communication.

There are a few options for this strategy:

Halt serving requests when lag exceeds threshold

> The queried service monitors its own consumer lag of its input event stream offsets, and only serves data if the lag is lower than the threshold. Instead of returning the data, your service returns a message indicating that it is not ready, such as an HTTP 503 (Service Unavailable). You may also choose to return a Retry-After response indicating when the service should be ready based on typical throughput processing. If the consumer lag is less than the threshold, the service will serve the queries and provide a response as normal.

Provide stale data to requester

> Your service can provide a response to the requester regardless of how stale the data is. You can include a response in the payload indicating that the data is stale and make it clear that it's up to the requesting client to choose how to proceed. In some cases a client doesn't much care about stale data—in other cases it's critical, and the client may choose to hold off further processing until it can have its request served with up-to-date data.

Provide a callback API

> Clients can register to have their request handled when your service is no longer lagging and receive a callback with the requested data. This strategy is more complicated to implement because either the client will have to block and wait for the callback or it'll need to implement context switching logic to work on other tasks until the callback occurs. Additionally, your service will need to buffer and handle the callbacks, plus provide SLAs for its users.

But what about event time? Can you use time since your last new event to detect if you're lagging? For some cases you can, but in many others you cannot. Let's consider an example.

Say you have an event stream that contains user `click` events, such as a share or like on a social media site. For the most popular websites, you could expect to see hundreds or thousands (or more) of events per second. Based on historical trends, if more than 10 seconds go by without a new `click` event, you can surmise that something is wrong in the pipe. If the event broker insists your consumer is up to date, then the problem likely lies in one of the other services further upstream.

Alternately, consider a stream that provides event-carried state transfer for a slowly-updating stream. In this case, a `user entity`, which is only updated when a new user is registered, when a field in their identity is updated (e.g. email address), or when they are deleted. There could be many hours (or days) between events, the data in the stream remains valid and any service that has consumed the stream remains converged and up to date. It just simply hasn't received any new events (because there are none!), and so cannot tell you any more information than the event time of the last event.

You may accidentally infer that your consumer is lagging by using the event time of the last-processed event alone. And while you *can* use event time to infer lag in cases of high-frequency updates, it remains unsuitable for many

other use cases. You would do well to rely on offsets to detect and expose lag whenever possible.

Summary

Eventual consistency can be a bit tricky to manage, but by and large it's about becoming aware of *how* it can be introduced and figuring out *if* it needs to be mitigated at all. Event streams offer consumers the means to converge on the same final state, though each consumer must uphold its own end of the bargain by correctly processing and integrating the data into its own domain.

In the event-driven space, a lagging consumer service may be inconsistent with another service or data set. Comparing data between a lagging service and a non-lagging service will likely show inconsistencies, as both are operating within their own frame of relative time. Avoiding direct request-response queries between asynchronous services allows you to avoid having to deal with data inconsistencies. In the event that you must deal with these inconsistencies, it's best to account for the services time frames and incorporate them into the inter-service communication.

Eventual-consistency issues can also arise when data in one event stream references data in another stream that may not yet have been produced. A lagging producer is often the culprit, and the consumers must wait for the producer to eventually produce the missing data. It's important to design your applications to accommodate late-arriving data, while also taking into account any end-to-end latency metrics that may require earlier action with incomplete data.

Next up, we're going to take a look at how to incorporate request-response patterns into event-driven microservices.

About the Author

Adam Bellemare is a staff engineer for the data platform at Shopify. He's held this position since 2020. Previously, he worked at Flipp from 2014 to 2020 as a staff engineer. Prior to that, he held a software developer position at BlackBerry, where he first got started in event-driven systems.

His expertise includes DevOps (Kafka, Spark, Mesos, Kubernetes, Solr, Elasticsearch, HBase, and Zookeeper clusters, programmatic building, scaling, monitoring); technical leadership (helping businesses organize their data communication layer, integrate existing systems, develop new systems, and focus on delivering products); software development (building event-driven microservices in Java and Scala using Beam, Flink, Spark, and Kafka Streams libraries); and data engineering (reshaping the way that behavioral data is collected from user devices and shared within the organization).